claude-windows / d1586dfe-adae-4d40-a85d-ed0bb23738b8

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\d1586dfe-adae-4d40-a85d-ed0bb23738b8.jsonl`
- SHA-256: `e0c377afee1084f661daf66d562e802983bfdb030fa1b792434df1e2835097fb`
- Source modified: `2026-07-01T02:50:44+00:00`
- Imported at: `2026-07-05T16:48:20+00:00`
- project: `C--Users-avidu-Projects-muneem`
- session_id: `d1586dfe-adae-4d40-a85d-ed0bb23738b8`

Transcript

1. user (2026-06-10T16:07:23.996Z)

Hello. Can you take deep read of this repository and its sister repository gmail-attachments-gdrive and identify any issues in code, architecture and roadmaps? Also want you to do a product market fit to analyse whether the direction is reasonable. Also identify key areas for improvements in both short term (code and components which you can fix right away) and longer horizon (product definitions, architectural evolution etc.) ? I am looking to decide whether it is better to focus on the grocery billing consolidation or the finance angle as a first step or maybe there are others which can be an easier/promising start.

2. assistant (2026-06-10T16:07:44.894Z)

I'll do this in phases: first scout both repos to scope the work, then run a multi-agent workflow for the deep review (code, architecture, security, roadmap), plus a market-landscape pass for the PMF question, and synthesize a recommendation at the end.

Let me start by checking the session handoff and the shape of both repos.

3. user (2026-06-10T16:07:45.893Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\muneem.

4. user (2026-06-10T16:07:46.742Z)

C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\description
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\applypatch-msg.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\commit-msg.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\fsmonitor-watchman.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\post-update.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\pre-applypatch.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\pre-commit.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\pre-merge-commit.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\pre-push.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\pre-rebase.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\pre-receive.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\prepare-commit-msg.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\push-to-checkout.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\sendemail-validate.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\hooks\update.sample
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\info\exclude
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\objects\pack\pack-112d4eba9cdc2a5df0efeb14af0da29c956eb96f.idx
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\objects\pack\pack-112d4eba9cdc2a5df0efeb14af0da29c956eb96f.pack
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\objects\pack\pack-112d4eba9cdc2a5df0efeb14af0da29c956eb96f.rev
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\logs\refs\remotes\origin\HEAD

```
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\packed-refs
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\refs\remotes\origin\HEAD
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.clasp.json
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.claspignore
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\config
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\HEAD
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\logs\HEAD
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\logs\refs\heads\master
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\refs\heads\master
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.github\workflows\ci.yml
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.gitignore
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\appsscript.json
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\docs\ARCHITECTURE.md
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\docs\VISION.md
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\esbuild.config.mjs
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\eslint.config.mjs
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\LICENSE
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\PRIVACY.md
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\README.md
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\ROADMAP.md
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\html\index.html
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\jest.config.js
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\package-lock.json
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\package.json
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\config.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\contentTypes.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\drive.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\entry.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\gmailHistory.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\historySync.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\logger.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\naming.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\notify.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\processed.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\query.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\quota.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\state.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\stats.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\sync.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\types\gmailAdvanced.d.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\ui.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\auditability.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\contentTypes.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\drive.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\gmailHistory.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\historySync.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\naming.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\notify.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\processed.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\query.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\quota.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\.git\index
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\setup.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\stats.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\sync.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\syncIncremental.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\ui.test.ts
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tsconfig.json
```

5. user (2026-06-10T16:07:47.230Z)

```
.venv\Scripts\python.exe
.venv\Scripts\pythonw.exe
.git\description
.git\hooks\applypatch-msg.sample
```

```
.git\hooks\commit-msg.sample
.git\hooks\fsmonitor-watchman.sample
.git\hooks\post-update.sample
.git\hooks\pre-applypatch.sample
.git\hooks\pre-commit.sample
.git\hooks\pre-merge-commit.sample
.git\hooks\pre-push.sample
.git\hooks\pre-rebase.sample
.git\hooks\pre-receive.sample
.git\hooks\prepare-commit-msg.sample
.git\hooks\push-to-checkout.sample
.git\hooks\sendemail-validate.sample
.git\hooks\update.sample
.git\info\exclude
.git\refs\remotes\origin\HEAD
testdata\promo-emails\dataquest.pdf
testdata\promo-emails\makemytrip-2.pdf
testdata\promo-emails\airasia-1.pdf
testdata\promo-emails\airasia-2.pdf
testdata\promo-emails\airasia-3.pdf
testdata\promo-emails\flipkart.pdf
testdata\promo-emails\makemytrip-1.pdf
testdata\promo-emails\swiggy.pdf
testdata\promo-emails\unacademy.pdf
testdata\promo-emails\tataneu.pdf
CLAUDE.md
testdata\promo-emails\README.md
docs\TESTING.md
.venv\.gitignore
.venv\pyvenv.cfg
.venv\Scripts\activate
.venv\Scripts\activate.fish
.venv\Scripts\Activate.ps1
.venv\Scripts\activate.bat
.venv\Scripts\deactivate.bat
.venv\Lib\site-packages\pip\__init__.py
.venv\Lib\site-packages\pip\__main__.py
.venv\Lib\site-packages\pip\py.typed
.venv\Lib\site-packages\pip\__pip-runner__.py
.venv\Lib\site-packages\pip\_internal\__init__.py
.venv\Lib\site-packages\pip\_internal\build_env.py
.venv\Lib\site-packages\pip\_internal\cache.py
.venv\Lib\site-packages\pip\_internal\configuration.py
.venv\Lib\site-packages\pip\_internal\exceptions.py
.venv\Lib\site-packages\pip\_internal\main.py
.venv\Lib\site-packages\pip\_internal\pyproject.py
.venv\Lib\site-packages\pip\_internal\self_outdated_check.py
.venv\Lib\site-packages\pip\_internal\cli\__init__.py
.venv\Lib\site-packages\pip\_internal\wheel_builder.py
.venv\Lib\site-packages\pip\_internal\cli\autocompletion.py
.venv\Lib\site-packages\pip\_internal\cli\base_command.py
.venv\Lib\site-packages\pip\_internal\cli\cmdoptions.py
.venv\Lib\site-packages\pip\_internal\cli\command_context.py
.venv\Lib\site-packages\pip\_internal\cli\index_command.py
.venv\Lib\site-packages\pip\_internal\cli\main.py
.venv\Lib\site-packages\pip\_internal\cli\main_parser.py
.venv\Lib\site-packages\pip\_internal\cli\parser.py
.venv\Lib\site-packages\pip\_internal\cli\progress_bars.py
.venv\Lib\site-packages\pip\_internal\cli\req_command.py
.venv\Lib\site-packages\pip\_internal\cli\spinners.py
.venv\Lib\site-packages\pip\_internal\cli\status_codes.py
.venv\Lib\site-packages\pip\_internal\commands\cache.py
.venv\Lib\site-packages\pip\_internal\commands\__init__.py
.venv\Lib\site-packages\pip\_internal\commands\check.py
.venv\Lib\site-packages\pip\_internal\commands\completion.py
```

```
.venv\Lib\site-packages\pip\_internal\commands\configuration.py
.venv\Lib\site-packages\pip\_internal\commands\debug.py
.venv\Lib\site-packages\pip\_internal\commands\download.py
.venv\Lib\site-packages\pip\_internal\commands\freeze.py
.venv\Lib\site-packages\pip\_internal\commands\hash.py
.venv\Lib\site-packages\pip\_internal\commands\help.py
.venv\Lib\site-packages\pip\_internal\commands\index.py
.venv\Lib\site-packages\pip\_internal\commands\inspect.py
.venv\Lib\site-packages\pip\_internal\commands\install.py
.venv\Lib\site-packages\pip\_internal\commands\list.py
.venv\Lib\site-packages\pip\_internal\commands\lock.py
.venv\Lib\site-packages\pip\_internal\commands\search.py
.venv\Lib\site-packages\pip\_internal\commands\show.py
.venv\Lib\site-packages\pip\_internal\commands\uninstall.py
.venv\Lib\site-packages\pip\_internal\commands\wheel.py
.venv\Lib\site-packages\pip\_internal\distributions\__init__.py
.venv\Lib\site-packages\pip\_internal\distributions\base.py
.venv\Lib\site-packages\pip\_internal\distributions\installed.py
.venv\Lib\site-packages\pip\_internal\distributions\sdist.py
.venv\Lib\site-packages\pip\_internal\distributions\wheel.py
.venv\Lib\site-packages\pip\_internal\index\__init__.py
.venv\Lib\site-packages\pip\_internal\index\collector.py
.venv\Lib\site-packages\pip\_internal\index\package_finder.py
.venv\Lib\site-packages\pip\_internal\index\sources.py
.venv\Lib\site-packages\pip\_internal\locations\__init__.py
.venv\Lib\site-packages\pip\_internal\locations\_distutils.py
.venv\Lib\site-packages\pip\_internal\locations\_sysconfig.py
.venv\Lib\site-packages\pip\_internal\locations\base.py
.venv\Lib\site-packages\pip\_internal\metadata\_json.py
.venv\Lib\site-packages\pip\_internal\metadata\__init__.py
.venv\Lib\site-packages\pip\_internal\metadata\base.py
(Results are truncated. Consider using a more specific path or pattern.)
```

6. user (2026-06-10T16:07:59.150Z)

```
.githooks\pre-commit
.githooks\pre-push
.githooks\precommit_check.py
.github\workflows\ci.yml
.mypy_cache\3.11\cache.0.db
.mypy_cache\3.11\cache.1.db
.mypy_cache\3.11\cache.10.db
.mypy_cache\3.11\cache.11.db
.mypy_cache\3.11\cache.12.db
.mypy_cache\3.11\cache.13.db
.mypy_cache\3.11\cache.14.db
.mypy_cache\3.11\cache.15.db
.mypy_cache\3.11\cache.2.db
.mypy_cache\3.11\cache.3.db
.mypy_cache\3.11\cache.4.db
.mypy_cache\3.11\cache.5.db
.mypy_cache\3.11\cache.6.db
.mypy_cache\3.11\cache.7.db
.mypy_cache\3.11\cache.8.db
.mypy_cache\3.11\cache.9.db
.mypy_cache\3.13\cache.0.db
.mypy_cache\3.13\cache.1.db
.mypy_cache\3.13\cache.10.db
.mypy_cache\3.13\cache.11.db
.mypy_cache\3.13\cache.12.db
.mypy_cache\3.13\cache.13.db
.mypy_cache\3.13\cache.14.db
.mypy_cache\3.13\cache.15.db
.mypy_cache\3.13\cache.2.db
.mypy_cache\3.13\cache.3.db
```

.mypy_cache\3.13\cache.4.db
.mypy_cache\3.13\cache.5.db
.mypy_cache\3.13\cache.6.db
.mypy_cache\3.13\cache.7.db
.mypy_cache\3.13\cache.8.db
.mypy_cache\3.13\cache.9.db
.mypy_cache\.gitignore
.mypy_cache\CACHEDIR.TAG
.pytest_cache\v\cache\lastfailed
.pytest_cache\v\cache\nodeids
.pytest_cache\.gitignore
.pytest_cache\CACHEDIR.TAG
.pytest_cache\README.md
.ruff_cache\0.15.15\12765879588015877657
.ruff_cache\0.15.15\13428912160643764434
.ruff_cache\0.15.15\4373846357469854159
.ruff_cache\0.15.15\5647337317940576954
.ruff_cache\0.15.15\9380428756468918620
.ruff_cache\0.15.15\9919018729046908979
.ruff_cache\.gitignore
.ruff_cache\CACHEDIR.TAG
docs\CURRENT_STATE.md
docs\DEPENDENCIES.md
docs\encryption-at-rest.md
docs\feedbackJune2026.md
docs\ingesting-statements.md
docs\onboarding-a-bank.md
docs\parser-architecture.md
docs\prior-art-email-mining.md
docs\research-open-banking-india.md
docs\ROADMAP.md
docs\schema.md
docs\TESTING.md
src\muneem\ingestion__init__.py
src\muneem\ingestion\base.py
src\muneem\ingestion\gmail.py
src\muneem\ingestion\imap.py
src\muneem\ingestion\outlook.py
src\muneem\parsers__init__.py
src\muneem\parsers\au.py
src\muneem\parsers\axis.py
src\muneem\parsers\base.py
src\muneem\parsers\clustering.py
src\muneem\parsers\concepts.py
src\muneem\parsers\engine.py
src\muneem\parsers\hdfc.py
src\muneem\parsers\profiles.py
src\muneem\parsers\savings.py
src\muneem\parsers\validation.py
src\muneem__init__.py
src\muneem\cli.py
src\muneem\config.py
src\muneem\db_key.py
src\muneem\db.py
src\muneem\errors.py
src\muneem\extract.py
src\muneem\log.py
src\muneem\migrations.py
src\muneem\ocr.py
src\muneem\offers.py
src\muneem\password_rules.py
src\muneem\recovery.py
src\muneem\redaction.py
src\muneem\secrets.py
src\muneem\statement_unlock.py

```
src\muneem\unlock.py
src\muneem.egg-info\dependency_links.txt
src\muneem.egg-info\entry_points.txt
src\muneem.egg-info\PKG-INFO
src\muneem.egg-info\requires.txt
src\muneem.egg-info\SOURCES.txt
src\muneem.egg-info\top_level.txt
testdata\aubank-statements\Statement_2024MTH07_XXXXX859_146.pdf
testdata\aubank-statements\Statement_2026MTH01_XXXXX859_721.pdf
testdata\aubank-statements\Statement_2026MTH03_XXXXX859_108.pdf
testdata\aubank-statements\Statement_2026MTH04_XXXXX859_849.pdf
testdata\axis-statements\AXIS BANK  Statement for August 2024.pdf
testdata\axis-statements\AXIS BANK  Statement for August 2025.pdf
testdata\axis-statements\AXIS BANK  Statement for January 2026.pdf
testdata\axis-statements\AXIS BANK  Statement for June 2025.pdf
testdata\axis-statements\AXIS BANK  Statement for May 2025.pdf
testdata\axis-statements\AXIS BANK  Statement for October 2023.pdf
testdata\axis-statements\AXIS BANK  Statement for October 2025.pdf
testdata\credentials\client_secret_207319377548-
j0vbfnqoqak4cq6kqa4s9ikq6l5sgore.apps.googleusercontent.com.json
testdata\hdfc-statements\Avi_Dullu_28022026_183854522.pdf
testdata\hdfc-statements\Avi_Dullu_30042026_001858675.pdf
testdata\hdfc-statements\Avi_Dullu_30062025_105046061.pdf
testdata\hdfc-statements\Avi_Dullu_30092025_094157457.pdf
testdata\hdfc-statements\Avi_Dullu_30112025_110905081.pdf
testdata\hdfc-statements\Avi_Dullu_31082025_205409615.pdf
testdata\hdfc-statements\HDFC_Avi_Dullu_31072024_150007641.pdf
testdata\promo-emails\airasia-1.pdf
testdata\promo-emails\airasia-2.pdf
testdata\promo-emails\airasia-3.pdf
testdata\promo-emails\dataquest.pdf
testdata\promo-emails\expected.yaml
testdata\promo-emails\flipkart.pdf
testdata\promo-emails\makemytrip-1.pdf
testdata\promo-emails\makemytrip-2.pdf
testdata\promo-emails\README.md
testdata\promo-emails\swiggy.pdf
testdata\promo-emails\tataneu.pdf
testdata\promo-emails\unacademy.pdf
testdata\snapshots\April Credit Card Statement.json
testdata\snapshots\lder Credit Card Statement.json
tests\ingestion\test_gmail.py
tests\ingestion\test_providers.py
tests\parsers\test_au.py
tests\parsers\test_axis.py
tests\parsers\test_clustering.py
tests\parsers\test_engine.py
tests\parsers\test_hdfc.py
tests\parsers\test_local_au.py
tests\parsers\test_local_axis.py
tests\parsers\test_parse_errors.py
tests\parsers\test_registry.py
tests\parsers\test_savings.py
tests\parsers\test_validation.py
tests\conftest.py
tests\synthpdf.py
tests\test_cli.py
tests\test_db_crypto_ops.py
tests\test_db_crypto.py
tests\test_db_invariants.py
tests\test_db.py
tests\test_dependency_licenses.py
tests\test_errors.py
tests\test_eval_cli.py
tests\test_extract.py
```

```
tests\test_log.py
tests\test_migrations.py
tests\test_ocr.py
tests\test_offers.py
tests\test_password_rules.py
tests\test_precommit_hook.py
tests\test_promo_regression.py
tests\test_provenance.py
tests\test_recovery.py
tests\test_redaction.py
tests\test_regression_synth.py
tests\test_repo_hygiene.py
tests\test_secrets.py
tests\test_smoke.py
tests\test_statement_unlock.py
tests\test_unlock.py
tests\test_verifier_properties.py
tools\gmail_auth.py
tools\imap_auth.py
tools\outlook_auth.py
.coverage
.gitattributes
.gitignore
CLAUDE.md
DESIGN.md
Makefile
pyproject.toml
README.md
```

7. user (2026-06-10T16:08:35.109Z)

```
1       # Current State
2
3       > **Last Updated:** 2026-06 (concept-core engine + issuer refocus: HDFC / Axis / AU
Bank)
4       > Summarizes current capabilities, completed features, and near-term next steps for
`muneem`.
5       > See [`ROADMAP.md`](./ROADMAP.md) for milestone tracking and [`parser-
architecture.md`](./parser-architecture.md) for the parser design.
6
7       > **Scope reminder ? muneem is a _whole-finance_ bookkeeper, not a bank tool.** Bank &
credit-card
8       > statements are the first vertical slice; **equities, mutual funds, ETFs, and bonds are
core
9       > targets**, parsed into the same local ledger, with transaction/instrument-event
classification to
10      > follow. The parser core is deliberately **family-agnostic** (see §2), so adding an
instrument
11      > family is *new data + a reconciliation rule*, not a new pipeline.
12
13      ## 1. Core Ingestion Pipeline
14      The ingestion pipeline supports a generic `SearchCriteria` interface and multiple
providers (Track B plumbing):
15
16      * **Gmail** (`muneem.ingestion.gmail`): Full OAuth2 read-only + search + attachment
download implemented.
17      * **IMAP** (`muneem.ingestion.imap`): Basic auth + search implemented; `get_headers` /
attachment methods are NotImplemented (experimental).
18      * **Outlook** (`muneem.ingestion.outlook`): Device-flow auth + basic search implemented;
attachment/header/download NotImplemented (experimental).
19
20      Companion one-time auth tools live in `tools/`. No high-level `muneem fetch` CLI command
yet (see Roadmap B.4).
21
22      ## 2. Statement Parsers (concept-core architecture)
```

Deterministic parsers live under `muneem.parsers`, registered via decorator and selected by `get_parser` on text heuristics + sender. Submodules are **auto-loaded on package import** (`_autoload_parsers`), so the registry is populated in the production path (not just under tests).

The parser core is **concept-first and verification-first** (see [`parser-architecture.md`](./parser-architecture.md)): an extractor maps an issuer's columns onto a bank-agnostic **concept schema** (`concepts.py`); a generic **engine** (`engine.py`) does the column?concept mapping (L0 header lexicon ? profile column-order ? L2 content search); and a shared **verifier** (`validation.py`) decides trust by *arithmetic* ? per-row balance walk + total reconcile + SUMMARY bookend + a recorded confidence verdict. The engine is **family-agnostic**: parameterized by a `FamilySpec` + an injected reconciliation oracle, so a future **instrument family** (equity/MF/ETF/bond holdings + buy/sell/dividend/fee events) reuses the same control flow with its own concepts + oracle. Issuer knowledge lives in `profiles.py` as **data, not code**. Bank **savings** parsers share one engine-driven path ? `savings.py` (per-table + coordinate-clustered candidates ? engine ? oracle), so adding a savings issuer is mostly a profile; HDFC keeps bespoke coordinate clustering (the documented `extract_tables` exception).

Active issuers ? we have real statements + passwords, so these get validated on real data:

* **HDFC Bank** (`hdfc.py`): Savings / transaction-account statements via **coordinate clustering** (deliberately *not* `extract_tables()` ? HDFC's table is semi-ruled). **Self-validating:** running-balance reconciliation + a cross-check against the bank's own SUMMARY (Dr/Cr counts, opening/closing balance); records `documents.status` = `reconciled`/`unreconciled`. Refactored onto the concept core as the **reference implementation**. ? **Verified on the user's 6 real statements** ? all reconcile *and* bookend, without inspecting any value.
* **Axis Bank** (`axis.py`): Savings + Credit Card. Savings runs the shared engine-driven path (`savings.py`) ? per-table + coordinate-clustered candidates ? engine mapping ? reconciliation oracle ? plus a **balance-delta reconstruction**: when columns don't split into a clean debit/credit (a merged amount column, an interleaved value-date, a sparse deposit column), it derives direction from the running-balance sign and the oracle cross-checks `amount == |?balance|`. Oracle-gated, so it never persists unreconciled rows. ? **6/7 real statements reconcile + persist**; the 1 miss (May 2025) has a transaction line the clusterer drops ? incomplete ledger ? the oracle quarantines it (stores **nothing**). Credit-card has no running balance (count only).
* **AU Bank** (`au.py`): engine-driven savings via the shared `savings.py` path ? same recipe, just `AUBANK_PROFILE`. ? **4/4 real statements reconcile + persist** (the cleanest issuer so far).

> _Dropped: **ICICI** ? no statements on hand to validate against, so it was removed to focus effort on the issuers we can actually verify (HDFC/Axis/AU). It can return as a profile when data exists._

All balance-bearing parsers share the verifier and split parsing from DB-insert, so row logic is unit-testable without a PDF/DB and a parse is validated before it's trusted. **Concept-core steps 1?3 have landed** (the verifier + concept schema; the generic mapping engine + profiles; the shared savings path with Axis + AU Bank on it). Parsers use `pdfplumber` (with optional password). **Password resolution is now wired into `muneem ingest --doc-type bank_statement`** ? the hybrid unlock (keychain-cached + `password_rules.yaml` candidates ? interactive prompt; `unlock.py` + `statement_unlock.py`) runs before extraction and forwards the password to the parser, in-memory only. See [ingesting-statements.md](./ingesting-statements.md).

See `password_rules.py` (prefers user config dir) and `cli.py:ingest`.

3. Database & CLI

* Schema v4 with `documents`, `offers`, and per-bank txn tables (`axis_*`, `hdfc_transactions`) ? documented in [`schema.md`](./schema.md). Dedup on sha256. **Versioned migrations** via a hand-rolled runner (`muneem/migrations.py`): the v4 schema is the frozen baseline, forward migrations layer on (recorded in `schema_migrations` with checksums), drift is rejected ? see [`schema.md`](./schema.md) and feedbackJune2026 P6. A unified `transactions` table (plus `holdings` for instruments) with per-row provenance/confidence is the planned direction once the design decision lands (parser-architecture.md §9; feedbackJune2026 P3).

```
41      * **The on-disk ledger is encrypted at rest** (decision D.2) ? **SQLCipher** transparent
AES-256 via `sqlcipher3`, opened through `db.connect_encrypted()`; the 256-bit key lives in the
OS keychain. SQL queries are unaffected. `:memory:` and the CI fixtures stay on plaintext stdlib
`sqlite3`. See [encryption-at-rest.md](./encryption-at-rest.md).
42      * CLI: `ingest <pdf>` (promo, or `--doc-type bank_statement` ? **auto-unlocks encrypted
PDFs**, see [ingesting-statements.md](./ingesting-statements.md)), `offers list`, `txns list
<bank>`, `eval <folder>` (reconcile-rate report ? the seed?golden onboarding loop, see
[onboarding-a-bank.md](./onboarding-a-bank.md)), and `db` encryption ops (`status`, `encrypt`,
`rotate-key`, `backup`, `setup-recovery`, `recover` ? see [encryption-at-rest.md](./encryption-
at-rest.md)).
43      * Config uses platformdirs for local dirs; secrets (OAuth tokens, cached statement
passwords, **the DB encryption key**) via keyring.
44
45      ## 4. Testing & Hygiene
46      * Strong offline regression on `testdata/promo-emails/` + synthetic parser/engine
fixtures (default CI).
47      * Security guards in `test_repo_hygiene.py` + license allowlist test.
48      * **DB invariant tripwires** (`test_db_invariants.py`, default CI): a **schema lock**
(pins `SCHEMA_VERSION` + every table/column shape ? an accidental drift fails the merge; a
deliberate change is "explicitly allowed" by updating the lock in the same PR) plus **encryption
guards** (production ingest must write an *encrypted* ledger; a keyed open never silently
downgrades to plaintext; foreign keys enforced + cascade; `documents.sha256` dedup).
49      * **Local git hooks** (`.githooks/`, enable via `make hooks`): `pre-commit` blocks
staging sensitive files; `pre-push` runs ruff + the offline suite (incl. the invariant
tripwires) so only green code reaches GitHub. The pre-push gate is the **local** stand-in for
server-side branch protection, which GitHub gates behind Pro / public repos (muneem stays
private). Both are bypassable with `--no-verify` (the conscious "explicitly allowed" escape).
50      * Local-only tests for real statements (gitignored data; passwords via env/keyring,
never committed). The suite never touches the real OS keychain (autouse fixture in
`tests/conftest.py`) ? headless-CI safe.
51
52      ## 5. Next Steps
53      Concept-core **steps 1?3 are done**, and **password unlock is now wired into `muneem
ingest`** (C.1 ? hybrid candidates ? prompt, keychain-cached; see [ingesting-
statements.md](./ingesting-statements.md)). Real-statement precision: **HDFC 6/6, AU 4/4, Axis
6/7** (the 1 Axis miss stores nothing ? oracle-gated). In rough order from here:
54
55      * **Recover the May 2025 vintage**: the clusterer drops one transaction line ?
incomplete ledger; needs smarter line recovery. Isolated and safe (it currently stores nothing;
June 2025 was recovered via balance-delta reconstruction).
56      * **Step 4 ? L3 local-vision fallback** (pending the local-VLM deep-research run) for
layouts the deterministic ladder can't reconcile.
57      * **Step 5 ? unify storage** (`transactions` + `holdings`) and begin the **instrument
family** (equity/MF/ETF/bond ? holdings + buy/sell/dividend/fee classification) ? the next big
piece of the whole-finance vision.
58      * **Regression CI for statements** (decided 2026-06, *deferred ? not yet built*): a
smart mix of synthetic full-pipeline fixtures in cloud CI + a self-hosted runner exercising the
real HDFC/AU statements (which stay local, off GitHub). Grounded plan + rationale in
[ROADMAP.md](./ROADMAP.md) (§ "Planned ? Regression CI for statement parsing").
59
60      Parked: DESIGN §8.3 (local LLM for sensitive categories), §8.5 (merchant
canonicalization). _(Encryption-at-rest timing, D.2, is now resolved & implemented ? SQLCipher;
see [encryption-at-rest.md](./encryption-at-rest.md).)_
61
62      The project stays **local-first**, secrets in the OS keychain, promo corpus safe for
CI/LLM experiments.
63
```

8. user (2026-06-10T16:08:35.274Z)

```
1       # Vision ? Email as a smart way to organize your life
2
3       > Status: living document. The overall product direction. Architectural detail
4       > and the decision log live in [`ARCHITECTURE.md`](./ARCHITECTURE.md).
5
```

```
## North star

Turn a person's email into a **high-fidelity, structured, queryable record of
their life** ? receipts, bank statements, flights & travel, coupons, shopping ?
built automatically, privately, and reliably. Email becomes the source of truth
that organizes the rest.

## What this repo is ? and is not

This repository (`gmail-attachments-to-gdrive`) is the **extraction / ingest
layer**. It is the reliable, quota-aware "front door" that:

- watches a user's Gmail **in their own account**,
- archives raw artifacts (attachments) ? still valuable on its own,
- **classifies** messages and **extracts** a clean, normalized signal,
- **redacts/minimizes** that signal, and **hands it off** to downstream
  processors.

It is **not** the domain parser, the datastore, or the analytics engine. That
logic is deliberately **isolated in separate downstream services**:

| Concern | Owner |
| --- | --- |
| Gmail watching, archiving, classification, extraction, redaction, handoff | **this
repo** |
| Bank-statement & financial parsing, indexing, encrypted storage, analytics |
**Muneem** (`avidullu/muneem`) |
| Grocery / receipts, coupons, travel processing | future siblings of Muneem |

The boundary is intentional: **extraction (this repo) is decoupled from
processing (downstream).** This repo never imports domain-parsing logic;
downstream services never read Gmail. They meet only at a versioned handoff
contract.

## Foundation first

Attachment archiving is the **hardened core**. Everything else is built on its
proven strengths, which we preserve and extend, never bypass:

- per-user isolation (`executeAs: USER_ACCESSING`),
- a per-user lock and self-healing trigger fan-out,
- the hybrid `history.list` + date-window sync,
- "mark a message done only on clean completion,"
- pure-logic / GAS-shell separation with heavy unit testing,
- ruthless respect for Apps Script quotas and the 6-minute boundary.

## Audience & rollout

- We **build for public** ? the long-term goal is a wide launch.
- **For now, stay under 100 users** (OAuth "Testing" mode) to harden the
  system, learn the value proposition, and make an *educated* call on whether to
  financially invest (the annual CASA security assessment, hosted infra) **once
  there is a substantial user base**.
- Until that call, we stay in Google's no-verification zone (see
  [`../README.md`](../README.md) ? Multi-user deployment / publishing).

## Privacy north star

- **Minimize egress.** Only **non-PII** signal ever leaves the user's Google
  account, and only to **our own isolated, hosted service** ? **never third
  parties**.
- **Scrub + encrypt** as required, applied *before* anything leaves.
- Continuously **reduce** what leaves while maintaining product health.
- The in-account baseline needs **zero** external dependency; external
  processing is a deliberate, **separately-consented** upgrade with a privacy
```

```
69        policy that names exactly what is sent, where, and why.
70
71      ### The auditability invariant
72
73      **This repo is the only code that ever touches a user's Gmail, and it stays
74      auditable and self-deployable by a power user at any time.** It is intended to be
75      open-sourced precisely so that a privacy-conscious user can read ? and run ? the
76      exact code that reads their mail. Muneem and every downstream service operate
77      only on minimized, non-PII signal and never hold the Gmail grant. This is a
78      binding constraint on all future work; the enforceable form (and a per-change
79      checklist) lives in [`ARCHITECTURE.md`](./ARCHITECTURE.md#auditability-invariant-
binding).
80
81      ## First verticals
82
83      1. **Grocery receipts** ? high volume, forgiving fidelity bar; proves the whole
84         ingest ? classify ? extract ? index pipeline cheaply.
85      2. **Bank statements** ? high-fidelity path; output feeds **Muneem**, proving the
86         redact ? handoff ? downstream-processing seam.
87
88      ## Success criteria (proposed)
89
90      - The user's index auto-populates with correct entities at **high precision**;
91        low-confidence items are **surfaced for review**, never silently dropped.
92      - Re-running an improved extractor over history **corrects past entities without
93        duplicates** and without re-copying blobs.
94      - Egress stays **minimal and non-PII**; the in-account baseline works with no
95        external service available.
96
97      ## Phased plan (high level)
98
99      Detailed status is tracked in [`../ROADMAP.md`](../ROADMAP.md); the technical
100     shape of each phase is in [`ARCHITECTURE.md`](./ARCHITECTURE.md).
101
102     0. **Ingest model** ? a rich, stable `EmailMessage`; the copier consumes it
103        (no behavior change).
104     1. **Classify + extract + index (GAS-only, in-account)** ? classifier, pluggable
105        extractors, normalized entities, a user-owned index. No egress.
106     2. **Coverage + re-extraction** ? more extractors, body/OCR handling, extraction
107        cursors & status, "re-extract for new rules," review queues.
108     3. **Handoff to hosted services** ? redact/minimize, then feed Muneem & siblings;
109        separate consent + privacy update; extraction/egress budgets.
110     4. **Surfaces & ops** ? dashboards, Calendar/Tasks automation, a richer review &
111        configuration UI, Shared Drives, schema migration.
112
```

9. user (2026-06-10T16:08:35.083Z)

```
testdata/promo-emails/README.md
testdata/promo-emails/airasia-1.pdf
testdata/promo-emails/airasia-2.pdf
testdata/promo-emails/airasia-3.pdf
testdata/promo-emails/dataquest.pdf
testdata/promo-emails/expected.yaml
testdata/promo-emails/flipkart.pdf
testdata/promo-emails/makemytrip-1.pdf
testdata/promo-emails/makemytrip-2.pdf
testdata/promo-emails/swiggy.pdf
testdata/promo-emails/tataneu.pdf
testdata/promo-emails/unacademy.pdf
---- gmail repo ----
a764030 Codify the auditability invariant (Option A) + enforce no egress (#14)
90c85ba Merge pull request #13 from avidullu/docs/vision-and-architecture
4975254 docs: capture extraction-pivot vision and architecture
172b3b3 Validate setup input + traceable attachment-copy errors (#12)
```

964b58c Portable attachment names + robust content-type matching (#11)
5e934b1 Make the web app multi-user-ready for small-group / org deployment (#10)
e4362a3 Add hybrid history.list incremental sync (opt-in) (#9)
6d0b10e ci: trigger deploy to verify CLASPRC_JSON (v2 format)
098ddfa ci: validate clasp creds shape and pin clasp in deploy (#8)
c41630e ci: bump checkout/setup-node to v5 on Node 24 (#7)

10. user (2026-06-10T16:08:35.108Z)

```
1       # feedbackJune2026 ? Architectural Review: Response & Incorporation Plan
2
3       > **Status: Approved direction (2026-06).** Avi has signed off on this analysis/plan.
The §5 open
4       > decisions (flagged **? DECISION**) remain to be settled before *their* phases are
implemented and
5       > must not be silently locked by code. The low-risk Phase-A items that carry **no** open
decision
6       > (Windows CI, `docs/schema.md`, marking Outlook/IMAP experimental) are being
implemented now; the
7       > rest waits on §5.
8       >
9       > Author's note (attribution): claims about current code are grounded in the repo as of
`main`
10      > after PRs #34?#40. Where I infer or recommend, I say so.
11
12      ---
13
14      ## 0. Framing ? what "commercial-grade" means *here*
15
16      muneem is **paid, but single-user, local-first, and lifelong**: one person's entire
financial life,
17      on their own machine, for years. So the bar is **correctness · zero-leak · auditability
·
18      recoverability · maintainability-by-one**. It is **not** multi-tenant SaaS. That
distinction drives
19      several push-backs below: the right move is to be *bulletproof at single-user scale*,
not to import
20      enterprise machinery (heavy ORMs, distributed anything, a web tier) that adds surface
without
21      serving the actual risk. "High bar" = fewer moving parts, each provably correct and
recoverable.
22
23      Two non-negotiables carry over from CLAUDE.md and stay load-bearing in everything below:
24      1. **No sensitive data leaves the machine** without explicit per-category, per-session
approval.
25      2. **No open architectural decision gets silently locked** by code. This doc proposes;
Avi disposes.
26
27      ---
28
29      ## 1. Corrections ? three premises in the review are already stale
30
31      The review is strong, but three items were **done in the recent code-quality pass** and
should be
32      re-scoped from "needed" to "extend / verify":
33
34      | Review claim | Reality (as of `main`) | What actually remains |
35      |---|---|---|
36      | "currently zero logging in src/" | **False now.** `muneem/log.py` (`get_logger` +
`RedactingFilter`) + `muneem/redaction.py` landed in **#36**, wired into `cli`, with masking
tests (`test_redaction`, `test_log`). | Structured (JSON/logfmt) output + **broad adoption**
across modules + per-stage leak tests. The *foundation* exists. |
37      | "muneem db backup (proper Online Backup API, not naive cp)" | **Done in #39.**
`backup_database()` uses the SQLite Online Backup API (WAL-safe), destination keyed ? encrypted
backup. | A documented restore runbook; optional *recovery-key* backups (see §2). |
```

| "key rotation scaffolding (the deferred passphrase path)" | **Rotation done in #39.** `rekey_database()` + `muneem db rotate-key` (keychain-first with rollback). | The **passphrase path** specifically ? but reframed as **recovery**, not just "stronger" (see §2). |

Also already in place (so the review's "table-stakes" list is largely satisfied): SQLCipher encryption-at-rest (#34), schema-lock + DB-invariant tripwires (#34), pre-commit secret guard, license allowlist, **mypy** + **ruff** + **80% coverage gate** in CI and the pre-push hook (#38/#40), verifier property-fuzz (#40). The engineering-hygiene floor is commercial-grade *today*.

---

2. ? The biggest risk the review *under-weights*: key-loss / disaster recovery (P0)

Encrypting the ledger at rest (correctly!) created a **catastrophic single point of failure** that none of the ten bullets names directly:

> The 256-bit DB key lives only in the OS keychain. **`muneem db backup` encrypts the backup with the same key.** So if the keychain is lost ? OS reinstall, profile/Credential-Manager corruption, dead machine, new laptop ? the ledger **and every backup** are *permanently unrecoverable*.

For a tool whose entire pitch is "hold my whole financial life for years," this outranks most of the listed features. It must be solved **before** we accumulate real, irreplaceable history.

The right way (reframe the deferred "passphrase mode" from security ? recovery):
- Keep the random DB key (fast, full-entropy) as the working key.
- Add a **recovery wrapper**: encrypt the DB key under a user passphrase (Argon2id, OWASP params) and store that wrapped blob in a `recovery.json` the user backs up **off-machine** (password manager / printed). Losing the keychain ? restore the key from passphrase + recovery file.
- And/or `muneem db export-key` ? prints the raw key **once**, with loud warnings, for the user's password manager. Simpler; less safe-by-default.
- Optionally let `db backup --recovery-key` write a backup under the passphrase-derived key, so a backup is restorable even with the keychain gone.
- Ship a **`docs/recovery.md` runbook**: "keychain lost," "move to a new machine," "verify a backup."

? DECISION: which recovery model ? (a) passphrase-wrapped recovery file *(recommended)*, (b) one-time key export, (c) both. Pick before storing real numbers long-term.

---

3. Point-by-point: verdict · the right way · dependencies

Verdicts: **? Accept** · **? Accept-with-changes** · **? Push back** · **?? Largely done**.

P1 ? Structured, redacted logging (D.1) · ? Accept-with-changes
Premise stale (see §1). **Right way:** build on `log.py`/`redaction.py`; add a structured formatter (logfmt or JSON ? ? minor decision), adopt `get_logger(__name__)` across `parsers/`, `db`, `unlock`, `ingestion` at sensible levels; keep `RedactingFilter` as the *backstop*, not the primary control

(code passes already-redacted context). **Tests:** feed synthetic sensitive values through *each
stage* and assert masking (extend the #36 tests). The other half of D.1 ? *idempotent, resumable
re-runs* ? is a **pipeline** concern (staging dir + `documents.status`), partly separable; don't
conflate. **Priority: high, low-risk ? Phase A.**

P2 ? Two-layer regression CI · ? Accept (already the plan)
Layer 1 (synthetic full-pipeline via `synthpdf.py`) exists (`test_regression_synth.py`); expand it
for more HDFC/Axis/AU header variants + bookend drift (this also absorbs the deferred "adversarial
synthpdf fixtures" follow-up from the code pass). Layer 2 (self-hosted runner over real, gitignored
statements; passwords from `password_rules.yaml`, **never** GitHub secrets) is decided/deferred.
Caveat to record: a self-hosted runner executes CI on Avi's personal machine ? fine for a
no-external-contributor repo, but note it runs only when the machine is on, and never enable it for
forked PRs. **? DECISION:** confirm self-hosted-runner acceptance. **Phase D.**

P3 ? Storage shape + provenance/confidence · ? DECIDED (2026-06)
Decision (Avi): keep **flat, per-issuer source-of-truth tables** ? each parser writes its own
isolated table (today's `axis_*` / `hdfc_*` / `au_*`, extended per issuer/instrument). Rationale:
regression isolation (a bad parse refreshes *one* table), trivial backfill/rollback, parser-faithful
storage. Cross-source querying is served by a **derived serving layer** ? a SQL **view / materialised
table** now, graduating to a separate query-optimised "serving DB" only when latency/cost demand it;
the flat tables remain the rollback-able **source of truth**. This **supersedes** the earlier "core +
per-family canonical table" sketch, and fits the reconcile-oracle philosophy *better* (each source
stays independently verifiable). Endorsed ? no strong reason to override the flat-truth + serving-
layer (bronze?silver) split.

Provenance + confidence are first-class but NOT per-row (Avi): recorded in a **separate
`provenance` table keyed by `document_id`** ? `source_type` (pdf/aa/sms/manual), `parser`,
`parser_version`, `mapping_layer`, `confidence`. Every txn row already FKs `documents(id)`, so it
reaches its provenance by join with **zero redundancy**. The reconciliation oracle treats every
source (PDF / AA / SMS) as a **hint to validate**, never ground truth. **Phase B = the `provenance`
table (the first forward migration) + a serving view; instrument-family tables follow the same
per-issuer-flat pattern.**

P4 ? L3 vision fallback + golden evals · ? Accept-with-changes (sequence!)
Decision stands (DeepSeek-OCR-3B, MIT, 4-bit+SDPA on the 2070S, reconcile-gated, self-consistency
resampling). **Push-back on ordering:** L3 should come **after D.3** ? the eval writeup that
characterizes where the deterministic ladder actually breaks. Building a fallback before we know
its target failure modes is guessing. So: **D.3 eval first ? then L3.** Golden sets (local +

synthetic now; cloud eval-only, approval-gated) extend `muneem eval`. **Phase E (after D.3).**

P5 ? Resolve/time-box parked decisions (§8.3 LLM locality, §8.5 merchant canon) · ? Accept

These stay parked until a milestone *forces* them, and surface to Avi with the hardware/latency constraints first. **Hard rule restated:** no cloud path for bank/CC/broker data, even experimentally, without per-session approval + redaction + a test gate. Merchant canon (§8.5) is downstream of P3+P8. **Governance item ? not a build; revisit at the phase boundary.**

P6 ? Versioned, tested schema migrations + `docs/schema.md` · ? Accept ? but ? **not Alembic**

Accept the need; **reject Alembic** (it drags SQLAlchemy ? heavy, ORM-shaped ? against muneem's deliberate raw-`sqlite3`, stdlib-light design). **Right way:** a tiny hand-rolled runner ? numbered `migrations/NNN_*.py|sql` applied in order inside the keyed connection, recorded in a `schema_migrations` table, each with an **idempotency/up-checksum**, gated by the `user_version` pragma. Pair with the existing schema-**lock** test (drift tripwire) ? versioned + tested + enforced.

Add a living `docs/schema.md`. Note: migrations run *inside* the encrypted connection ? fine, but the runbook must cover "migrate then re-verify reconciliation." **Phase A (precedes P3's migrations).**

P7 ? Ingestion completeness + `muneem fetch` · ? Accept ? but ? re-sequence

Push-back on "before heavy parser work": the parser core is already *proven* (HDFC 6/6, AU 4/4, Axis 6/7) and the user supplies statements manually today. `fetch` is **automation/convenience, not correctness** ? it doesn't unblock parser accuracy or the whole-finance vision; **storage unification (P3) does.** So: **P3 before P7.** What *is* cheap and should happen immediately: explicitly mark Outlook/IMAP attachment paths **experimental/deferred** in code + docs (the mypy override already treats them as such) so no one mistakes scaffolding for support. Then build `fetch` (incremental `historyId`/`deltaLink`, sender allowlist + "financial document" heuristics, staging ? `documents` rows as `locked`/`unparsed`, robust dedup) as **Phase F**. The fast bit (mark experimental) is **Phase A**.

P8 ? Tier-2 transaction semantics + merchant canonicalization · ? Accept (downstream)

Strictly **after P3** (needs the canonical schema) and needs **real receipt data** to expose the mess (don't design merchant canon in the abstract). Counterparty/merchant extraction + fuzzy match + a user-editable **override table**; itemized receipts (Swiggy/Amazon) likewise. **Phase G.**

P9 ? Observability / ops surface · ? Accept (partly done; pull recovery forward)

Done: `db backup` (Online Backup API), `db rotate-key`, `db status` (#39). **Right way for the rest:** `muneem verify` (re-run reconciliation over *stored* rows ? a cheap, high-value integrity audit that also exercises provenance), `export` (CSV/JSON for the user's own analysis ? explicitly **redaction-exempt** since it's the user's own data to their own disk, but never to stdout/logs), `txns` filters

(date/amount/account/status), light analytics. **Crucially, the recovery model (§2) belongs here but is P0 ? pull it into Phase A, not "later."** Document backup/restore + "keychain lost." **Phase A (recovery) + Phase H (the rest).**

P10 ? Packaging / distribution / reproducibility · ? Accept-with-changes

- **Lockfile** (uv.lock or pip-tools `requirements*.txt`): ? yes ? reproducibility is real for a tool with a compiled dep (sqlcipher3). **Phase A.**
- **Cross-platform CI matrix** (Windows + macOS, not just ubuntu): ? **high value, under-rated** ? the dev box is Windows, and the secret backends genuinely differ (DPAPI vs Secret Service vs Keychain) and sqlcipher3 wheels are per-OS. This directly de-risks "works on my machine" for the one machine that matters. **Phase A.**
- **SBOM in CI** (CycloneDX): ? reasonable for paid software but **time-box / low-urgency** ? it serves a distribution/audit story we don't have yet. Do it near first external distribution, not now. **Phase H+.**
- **Local web UI:** ? defer ? CLI stays authoritative; a web tier is new attack surface + scope for no current need. Revisit only if a real UX gap appears.

---

4. Proposed sequencing (dependency-ordered)

> Rule of thumb: **recoverability + foundation first; then the one big schema decision; then
> everything that depends on it.** Each phase is shippable and individually PR-able.

- **Phase A ? Foundation & recovery (low-risk, high-leverage):** §2 **recovery model (P0)** · finish structured-logging adoption (P1) · hand-rolled migration runner + `docs/schema.md` (P6) · lockfile + cross-platform CI matrix (P10) · mark Outlook/IMAP experimental (P7-fast).
- **Phase B ? Provenance table (first forward migration) + serving view (P3, DECIDED).** Flat per-issuer source of truth stays; add the separate `provenance` table + a cross-source serving view. Unblocks the instrument family.
- **Phase C ? Instrument family** (equity/MF/ETF/bond events) on the unified schema + new `FamilySpec`/oracle (P3 cont.).
- **Phase D ? Regression CI Layer 2 (self-hosted) + the D.3 eval writeup** (P2) ? also tells us where L3 is actually needed.
- **Phase E ? L3 vision fallback + golden evals (P4),** informed by D.3.
- **Phase F ? `muneem fetch` + ingestion completeness (P7).**
- **Phase G ? Tier-2 semantics + merchant canonicalization (P8),** on real receipt data.
- **Phase H ? Remaining ops surface (verify/export/filters/analytics) (P9) + SBOM (P10).**
- **Parked/governance throughout:** §8.3, §8.5 (P5) ? surface at their forcing milestones.

---

5. Decisions ? settled vs still open

Settled (2026-06):

1. **Recovery / key-loss model** ? **passphrase-wrapped recovery file** (Argon2id + AES-GCM). ? shipped (#43).

2. **Storage shape** ? **flat per-issuer source-of-truth tables + a derived serving layer; provenance/
   confidence in a separate `provenance` table** (P3). Endorsed; Phase B implements the provenance table.
3. **Migration mechanism** ? **hand-rolled runner** (not Alembic). ? shipped (#44).
8. **Open Banking / Account Aggregator** ? **AA deferred; PDF-first**, with explicit re-adopt triggers
   (see §7 and [`research-open-banking-india.md`](./research-open-banking-india.md)).

Still open:
4. **Structured log format** (P1) ? logfmt vs JSON.
5. **Self-hosted runner** (P2) ? accept the run-CI-on-my-machine trade-off?
6. **macOS first-class** (P10) ? Windows + Linux are first-class now (CI matrix, #42); macOS best-effort?
7. **Parked:** §8.3 sensitive-doc LLM locality, §8.5 merchant canonicalization ? confirm still parked.

---

6. Push-back summary (explicit, with reasons)

- **Alembic ? hand-rolled migrations.** Dep-minimization + raw-`sqlite3` design + the schema-lock
  already exists; Alembic's ORM weight buys nothing here.
- **"Ingestion before heavy parser work" ? storage unification (P3) first.** Parser core is proven;
  `fetch` is convenience, not correctness. (But mark Outlook/IMAP experimental *now* ? cheap.)
- **L3 before D.3 ? reversed.** Characterize the deterministic ladder's failure modes *first*; don't
  build a fallback blind.
- **SBOM / web UI ? time-box / defer.** Low urgency vs correctness + recoverability; web UI is
  unjustified surface for a single-user CLI tool.
- **Don't over-engineer for scale muneem doesn't have.** Single-user, local-first. Bulletproof beats
  big.
- **And the addition the review missed: key-loss recovery is P0** ? above most of the feature list.

---

7. Open Banking / Account Aggregator (researched 2026-06) ? decided: **defer, PDF-first**

Full writeup + sources: [`research-open-banking-india.md`](./research-open-banking-india.md). Summary:

- **Verdict:** a single-user, local-first, no-paid-API tool **cannot** use India's Account Aggregator
  (AA) today. **FIU status requires a regulated entity** (no individual/hobbyist path); the only
  reachable integration rides under a licensed FIU via a **paid TSP that decrypts your whole feed in
  its cloud**. So AA-primary is both *impossible* and *undesirable* under muneem's principles.
- **On the security fear:** AA is *safer in transit* (E2E-encrypted, data-blind aggregator, no
  credential sharing) ? but **"self-custody" is not an available AA mode**; decryption is institutional
  (FIU/TSP cloud). For muneem's threat model (one local honeypot, zero third parties) that's a **net
  privacy regression** ? so the wariness is justified, for a subtle reason (a new institutional cloud
  tier, not weak crypto).

```
248     - **Fidelity:** AA wins on live bank rows + demat equity/MF, but **can't cover CC,
bonds, broker
249       detail, or non-demat MF** (muneem's exact scope), caps history ~2y, and still needs
muneem's
250       enrichment/oracle layer. A boost on a *subset*, not a replacement.
251     - **Stance:** **PDF-first core; AA a deferred, opt-in *complement* (import from a
regulated FIU's
252       export, oracle-validated, provenance-tagged) ? never primary.** Re-adopt triggers: a
TSP documents
253       **true on-device, self-custody-key decryption** (the key gate); CC/bonds go live on
AA; a free
254       personal tier appears; or muneem's PDF coverage saturates.
255     - **New near-term candidate it surfaced ? local SMS/email transaction-alert parsing
(P11):** RBI
256       mandates per-transaction bank SMS; mature OSS parsers exist; **zero third party, fully
local,
257       near-real-time.** Limits: no history/balances/holdings, brittle formats, no
investments ? so a
258       *freshness complement* to PDF, not a replacement, and it violates **no** muneem
principle (echoes
259       the predecessor's email-mining DNA). Slots into Phase A (optional) / Phase F.
260
261     ---
262
263     *Next step: §5's settled decisions are being implemented (recovery #43, migrations #44,
Windows CI
264     #42; provenance table next). Remaining open: log format, self-hosted runner, macOS. One
PR per item,
265     CI green and ?80% coverage throughout.*
266
```

11. user (2026-06-10T16:08:35.737Z)

```
1       # Roadmap
2
3       Next steps for the modernised project, grouped by effort and grounded in the
4       current Apps Script / Gmail platform constraints.
5
6       ## Platform constraints that shape the design
7
8       For consumer (gmail.com) accounts:
9
10      - **6 minutes** max per execution; **90 minutes/day** total trigger runtime;
11        **20 triggers** per script per user; ~**20,000** Gmail read calls/day.
12        ([Apps Script quotas](https://developers.google.com/apps-
script/guides/services/quotas))
13      - There is **no clearly documented daily Drive file-create quota**, so
14        `MAX_FILES_ADD_PER_DAY` (240) is a self-imposed heuristic, not an API limit.
15      - Gmail's
[`users.history.list`](https://developers.google.com/workspace/gmail/api/guides/sync)
16        with a stored `startHistoryId` is the purpose-built incremental sync (~2 quota
17        units vs 5 for a list call) and returns only changes. A `historyId` can expire
18        (HTTP 404 ? fall back to full sync), so it complements the date-window
19        backfill rather than replacing it.
20      - clasp can deploy from CI using a `CLASPRC_JSON` secret (contents of
21        `~/.clasprc.json`); service accounts do **not** work with the Apps Script API,
22        so use a dedicated owner account's token.
23        ([clasp #225](https://github.com/google/clasp/issues/225))
24
25      ## Quick wins
26
27      - [x] **Cover the orchestration shells with tests.** `sync.ts` / `ui.ts` had no
28        coverage ? the trickiest logic (trigger fan-out, quota reset, catch-up loop,
29        `processInput` state reset). *(done in this branch; see `tests/sync.test.ts`,
30        `tests/ui.test.ts`)*
```

```
- [x] **Raise the per-run wall-clock budget.** Was capped at 1 minute against a
  6-minute ceiling, making backfill ~6× slower than necessary. Now
  `MAX_RUNTIME_MS` (5 min) in `src/config.ts`, threaded through `hasTime`.
- [x] **Guard against trigger leakage.** A `LockService` script lock prevents
  concurrent runs from stacking triggers, deletion is scoped to the sync
  handler, and a cap prunes any excess. *(done in this branch; `src/sync.ts`)*
- [x] **Add a Jest `coverageThreshold`** so CI fails if coverage regresses.
  *(done; `jest.config.js`, gated at 85% stmts/lines/funcs, 75% branches,
  `entry.ts` bundler glue excluded)*

## Medium

- [x] **CI/CD auto-deploy on merge to `master`.** A gated `deploy` job writes
  `CLASPRC_JSON` ? `~/.clasprc.json` and runs `clasp push` / `clasp deploy`
  against the committed `.clasp.json`. *(done; `.github/workflows/ci.yml`.
  Requires only the `CLASPRC_JSON` repo secret; skips cleanly if unset.)*
- [x] **Cross-run dedup by message/attachment ID.** A bounded FIFO of processed
  Gmail message ids (`src/processed.ts`, capped at `MAX_PROCESSED_IDS`) is
  persisted in user properties; `updateDrive` skips messages already in the set
  and reports newly-completed ids back to `sync.ts`. A message is only marked
  done when all its attachments were copied cleanly (not truncated by the daily
  cap or a transient error), so re-scanned windows skip the Drive
  `getFilesByName` probe. *(done; `src/processed.ts`, `src/drive.ts`,
  `src/sync.ts`, `src/state.ts`)*
- [x] **Observability.** Level-prefixed structured logging (`src/logger.ts`),
  per-day run stats accumulator (`src/stats.ts` ? runs/files/errors/last error,
  persisted via `PROP_RUN_STATS`), and an opt-in daily summary
  (`src/notify.ts`) logged on each new-day rollover. The summary email is OFF by
  default; enabling it needs `ENABLE_SUMMARY_EMAIL` + the `script.send_mail`
  scope. *(done)*

## Larger / architectural

- [x] **Hybrid incremental sync.** Keep the date-window scan for the initial
  backfill; once caught up (`isToday`), switch the steady state to the Gmail
  advanced service + `history.list`/`startHistoryId`, storing `historyId` in
  properties and falling back to full sync on 404. *(done; pure logic in
  `src/historySync.ts`, advanced-service shell in `src/gmailHistory.ts`, wired
  into `src/sync.ts`. Gated behind `ENABLE_INCREMENTAL_SYNC` (OFF by default)
  for gradual rollout over the proven backfill. The Gmail advanced service is
  declared in `appsscript.json`; no new OAuth scope ? `gmail.readonly` covers
  history reads. Backfill captures the mailbox `historyId` before processing so
  mail arriving mid-run is still caught; expiry (HTTP 404) clears the cursor and
  rewinds to a bounded date-window re-scan.)*
- [ ] **More structure options & Shared Drive support** ? by-sender, nested
  `Year/Month`, label-driven folders; currently My Drive only.

## Open-source & auditability readiness

Concrete prep so the [auditability invariant](docs/ARCHITECTURE.md#auditability-
invariant-binding)
is real, not aspirational, before/when the repo is opened up.

- [x] Root `LICENSE` file (MIT, matching the source headers).
- [x] Egress guard ? a test that fails CI if `src/` makes any network call
  (`tests/auditability.test.ts`). Update it deliberately when the user-endpoint
  egress lands (assert the allowlisted destination, don't remove the guard).
- [ ] `SECURITY.md` ? how to report issues + the current data-flow guarantees
  (today: reads Gmail, writes the user's own Drive, **no egress**).
- [ ] **"Deploy your own from source"** guide ? self-host the GAS project end to
  end, so the trust claim is "run the code you read."
- [ ] **Reproducible / verifiable build** ? pin the toolchain and document how to
  diff a deployed `dist/Code.js` against a build of a given commit.
- [ ] `CONTRIBUTING.md` carrying the auditability PR checklist.
```

12. user (2026-06-10T16:13:27.715Z)

```
Workflow launched in background. Task ID: wctcr4nab
Summary: Deep review of muneem + gmail-attachments-to-gdrive: code, security, architecture,
roadmap, and market fit
Transcript dir: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-
muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\subagents\workflows\wf_27df53c8-8e0
Script file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-
muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\workflows\scripts\muneem-deep-review-
wf_27df53c8-8e0.js
(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath:
"C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-
muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\workflows\scripts\muneem-deep-review-
wf_27df53c8-8e0.js"} to iterate without resending the script.)
Run ID: wf_27df53c8-8e0
To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\.claude\projects\C--
Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\workflows\scripts\muneem-deep-
review-wf_27df53c8-8e0.js", resumeFromRunId: "wf_27df53c8-8e0"}) ? completed agents return
cached results.

You will be notified when it completes. Use /workflows to watch live progress.
```

13. user (2026-06-10T16:13:35.719Z)

```
1       # muneem ? Design Document
2
3       > Status: **draft, pre-implementation**. This document captures the vision, the pipeline
shape, the security posture, and the open decisions to resolve before code lands. It is
intentionally opinionated where the author has a strong view, and explicitly undecided where the
trade-offs need a human call.
4
5       ---
6
7       ## 1. What muneem is
8
9       **muneem** is a personal-finance bookkeeper that ingests financial documents from the
user's email accounts, unlocks the password-protected ones, parses them into a normalized
schema, stores them in a local database, and surfaces analytics across the user's full financial
life.
10
11      The name is the Hindi word for the traditional household/business bookkeeper ? a trusted
figure who maintained the family ledger by hand. This project is the same role, automated.
12
13      **Single user. Local-first. Privacy-critical.**
14
15      ## 2. Why
16
17      The user's financial life today is fragmented across many silos:
18
19      - 3?5 bank accounts (statements arrive monthly as password-protected PDFs)
20      - Multiple credit cards (monthly bills, also password-protected PDFs)
21      - Brokerage and mutual-fund accounts (quarterly statements, portfolio snapshots)
22      - Grocery and shopping receipts (Swiggy, Zomato, Amazon, Flipkart, BigBasket, Blinkit ?
mostly plain HTML/PDF in email body)
23      - Utility bills, insurance premiums, EMI schedules
24      - Promotional offers and coupons (often forgotten)
25
26      Each silo has its own portal, login, and format. There is no consolidated view. **The
inbox is the only place where they already converge.**
27
28      Concrete use-cases the user wants once this exists:
29
30      1. **Grocery & shopping price intelligence** ? track what was bought, where, and at what
price, over time. Detect when a merchant is no longer competitive on staples.
31      2. **Investment portfolio timeline** ? a single time-series of holdings across brokers,
```

so allocation drift and rebalancing decisions are obvious.
32   3. **Bank/card comparison** ? fees paid, interest earned, reward redemption efficiency,
surfaced as objective numbers.
33   4. **Item catalog** ? every significant purchase linked to its receipt/invoice, so
warranty windows, return windows, and replacement cycles are queryable.
34   5. **Forgotten-deals reminder** ? surface the unused coupons and offers already sitting
in the inbox (this is the original goal of the predecessor project; see § 11).
35
36   ## 3. Origin
37
38   muneem is the successor to `~/Projects/Email-Mining`, a small prototype the user built
earlier that extracted promo codes from manually-saved email PDFs using PyMuPDF + Gemini for
image OCR. Same insight ("the inbox contains structured value I'm forgetting"), generalized
scope.
39
40   See [`docs/prior-art-email-mining.md`](./docs/prior-art-email-mining.md) for what it
did, what to inherit, and what not to.
41
42   ## 4. The pipeline (five stages)
43
44   Each stage is independent enough to develop and test in isolation. They communicate
through well-defined data boundaries (raw files on disk ? unlocked files on disk ? structured
records in DB ? derived views).
45
46   ### Stage 1 ? Email ingestion
47
48   **Goal:** pull messages and attachments from Gmail and Outlook into a local staging
area.
49
50   - **Gmail:** [Gmail API](https://developers.google.com/gmail/api) over OAuth2. Read-only
scope (`gmail.readonly`). Supports server-side search queries (`has:attachment filename:pdf
from:hdfc newer_than:90d`) which is far cheaper than pulling everything and filtering locally.
Has good rate limits for personal use.
51   - **Outlook:** [Microsoft Graph API](https://learn.microsoft.com/en-
us/graph/api/resources/mail-api-overview) over OAuth2. Read-only scope (`Mail.Read`). Similar
search capability.
52   - **Why not IMAP:** worse search, no labels, harder OAuth, and we lose Gmail's powerful
query syntax. Use the native APIs.
53
54   Mechanically straightforward. The main design questions are: **how is "financial
document" detected** (sender allowlist? subject patterns? attachment heuristics? hybrid?), and
how is incremental sync state tracked (Gmail `historyId` / Graph `deltaLink`).
55
56   ### Stage 2 ? Attachment unlocking
57
58   **Goal:** decrypt password-protected PDFs.
59
60   The decryption itself is trivial ? every mainstream PDF library handles the standard
security handler. In Go, `github.com/pdfcpu/pdfcpu`; in Python, `pikepdf` or `pypdf`.
61
62   The real design question is **where the passwords come from.** Indian bank and CC
statement passwords are almost always derived from user attributes:
63
64   - "First 4 letters of name (uppercase) + DDMM of DOB" ? common at HDFC, Axis
65   - "PAN number" ? common at some brokers
66   - "Last 4 digits of card number" ? common for CC statements
67   - "Customer ID" ? some banks
68   - "Mobile number" ? utilities
69
70   Three plausible approaches, each with trade-offs:
71
72 | Approach | Pro | Con |
73 |---|---|---|
74 | **Per-sender rule template** the user configures once (e.g., `from:alerts@hdfcbank.net
? first4(name)+ddmm(dob)`) | Deterministic, no guessing, auditable | Requires upfront setup;

rules must be maintained as banks change formats |
| **Candidate dictionary** (try a small list of derived strings per PDF) | Zero config; "just works" | Brute-forceable feel; fails noisily on unknown senders |
| **Interactive prompt** (ask the user the first time, cache for that sender) | Safe, simple | Friction during initial backfill of years of statements |

A hybrid is likely best: candidate dictionary first, fall back to rule template, fall back to interactive prompt. **Decision parked.** See § 8.

Stage 3 ? Document parsing

This is the hard 80% of the project. Every issuer's PDF layout is different and changes over time.

The space of approaches, in increasing robustness and cost:

1. **Per-issuer regex/template parsers.** Precise; brittle. One parser per format, breaks when the issuer redesigns. Realistic for the top ~5 senders that dominate volume.
2. **Text-layer extraction + structural heuristics.** Pull text with `pdfminer.six` / `pdfplumber` (both MIT); rely on coordinates and font runs to recover tables. Works well for "born digital" PDFs (most bank statements); fails on scanned/image PDFs. (We avoid PyMuPDF/`fitz` ? it is AGPL; see § 5 rule 8.)
3. **OCR (Tesseract) for image-based PDFs and embedded raster content.** Slower; need layout reconstruction.
4. **LLM-assisted structured extraction.** Hand page text (+ optionally page rasters) to a model with a target JSON schema. Generalizes across layouts; expensive per page; **must be local for sensitive document categories**. Options: Ollama-hosted models (Llama 3, Qwen, Mistral), or a small purpose-built model.

In practice this will be **a hybrid**: deterministic template parsers for the top ~5 high-volume senders (where they pay for themselves), LLM-assisted extraction as the long-tail fallback. Promo emails and other non-sensitive categories can use cloud LLMs; bank/card/investment statements **must not** without explicit per-category approval.

Embedded image deduplication (Email-Mining used MD5 hash + SSIM) is a real win here: bank statements repeat the same logo/letterhead on every page, and we don't want to OCR that 30 times per document.

Stage 4 ? Normalization & storage

Goal: map heterogeneous parser outputs into one schema, persist locally.

Storage: **SQLite**. Single file, no server, easy to back up, easy to encrypt at rest (SQLCipher), good enough performance for years of personal data. The schema needs to span:

- `documents` ? every ingested file with sender, date, type, hash, path
- `accounts` ? bank accounts, cards, broker accounts, identified by issuer + last-4 or similar
- `transactions` ? line items from statements, normalized (date, amount, currency, counterparty, category, source_document)
- `holdings` ? point-in-time snapshots of investment positions per account
- `merchants` ? canonicalized merchant identities (so "SWIGGY*BANGALORE", "Swiggy Instamart", and "SWIGGY BNGLR" collapse to one)
- `items` ? line-items from itemized receipts (grocery, shopping)
- `offers` ? promotional offers/coupons with expiry, source, redemption status

The merchant canonicalization step is non-trivial ? it deserves its own thought (fuzzy match + manual override table is the usual answer).

Stage 5 ? Analytics & insights

Goal: turn the normalized DB into the use-cases in § 2.

Once the data is in the schema, the analytics are mostly SQL + light application logic. Likely surface: a local CLI initially, possibly a small local web UI later. Notebook-style

exploration is fine for prototyping queries before they're promoted to first-class views.

---

5. Security posture (first class)

The threat model deserves to be stated plainly: **muneem aggregates the user's entire financial life ? every account, every transaction, every holding ? into one local store. That store is a honeypot.** A leak would be catastrophic.

Standing rules:

1. **All raw statement data stays on the local machine.** No cloud sync of the staging dir or the DB.
2. **No cloud LLM APIs for sensitive document categories** (bank, CC, broker, MF, insurance) without explicit per-category user approval discussed in the current session. The default extraction path for these is local (Tesseract + local LLM via Ollama, or deterministic parsers).
3. **Non-sensitive categories** (promo emails, generic newsletters, public receipts) may use cloud LLMs after a privacy review of the specific content type.
4. **DB encrypted at rest** ? *implemented* (decision D.2, 2026-05) with **SQLCipher**: transparent AES-256 page encryption via the permissive `sqlcipher3` binding, so SQL queries work unchanged. The 256-bit key is random, lives only in the OS keychain (never in the repo, config, args, or logs), passed in raw-key form. OS full-disk encryption (BitLocker/LUKS) is a recommended complement, not a substitute. See [`docs/encryption-at-rest.md`](./docs/encryption-at-rest.md).
5. **OAuth tokens** stored in the OS keychain (Windows Credential Manager / macOS Keychain), not on disk.
6. **Logs must not contain extracted statement text.** Sanitize aggressively; redact account numbers, balances, line-items.
7. **Nothing sensitive ever in git.** `.gitignore` blocks real PDFs, the DB file, `.env`, and password files. Do not weaken it.
8. **Commercial-license safety.** Only dependencies under permissive, commercial-safe licenses (MIT, BSD, Apache-2.0, ISC, HPND). No copyleft (GPL/LGPL/**AGPL**) or source-available licenses. **PyMuPDF/`fitz` is AGPL-3.0 and is banned** ? use pdfminer.six (MIT) / pypdfium2 (Apache/BSD) instead. Enforced by `tests/test_dependency_licenses.py`; tracked in [`docs/DEPENDENCIES.md`](./docs/DEPENDENCIES.md).
9. **Minimize paid dependencies.** Prefer local / self-hosted / open-source (and, where needed, own-trained) models and tools. Paid APIs ? cloud LLMs, paid OCR/vision ? are a **fallback only**, gated behind explicit per-use approval. This extends rule 2 from privacy to cost.

6. Non-goals (for v1)

- **Multi-user / SaaS / hosted product.** Single user, single machine.
- **Cloud sync of raw data.** Period.
- **Cracking unknown passwords.** muneem decrypts PDFs the user is authorized to read, using passwords the user knows or can derive.
- **Browser scraping bank portals.** Email APIs only. Portal scraping is a fragile, ToS-fraught rabbit hole; we deliberately avoid it.
- **Real-time alerting / push.** Batch-pull on a schedule is sufficient.
- **Tax preparation, investment advice, or anything regulated.** muneem surfaces data; the user decides.

7. Recommended first vertical slice

Before generalizing, prove the end-to-end pipeline on the smallest meaningful scope:

> **Gmail ? one specific bank's monthly statements ? unlocked ? parsed into `transactions` ? stored in SQLite ? a CLI command that lists last month's transactions.**

This exercises every stage exactly once, on a real input the user cares about, with no detours. It will surface the actual hard parts (which we can only guess at now) and force the open decisions in § 8 to get made on real evidence.

Concretely: pick **one** Indian bank statement format the user has many examples of (HDFC, given the user's geography and statement history), build the parser for that one format only, and put the rest of the pipeline around it. Then evaluate before generalizing.

8. Open architectural decisions

These were parked for explicit user discussion (**don't pick silently**). Items **8.1, 8.2, and 8.4 are now resolved** ? see the inline resolution notes below and [`docs/ROADMAP.md`](./docs/ROADMAP.md) § 1. Items **8.3 and 8.5 remain open**.

8.1 Language

Option	Pro	Con
Go only	Single binary, easy distribution, great concurrency for the ingestion side, the user already uses Go for the badminton-highlight-indexer project	Weak PDF parsing / OCR / ML ecosystem; rolling our own is painful
Python only	Best-in-class PDF, OCR, and ML libraries (pdfplumber, PyMuPDF, Tesseract bindings, Ollama clients, every parser under the sun); fastest to prototype	Heavier deployment story; less ergonomic for the long-running ingestion daemon
Go ingestion + Python parsing	Plays to each language's strengths	Two-language project complexity; inter-process boundary to maintain

Recommended default: **Python-only for v1**, accept the deployment-story cost in exchange for moving fast on the hard parts. Revisit if the ingestion side outgrows Python.

> **Resolved (2026-05-30): Python-only for v1.** Adopted the recommended default. See [`docs/ROADMAP.md`](./docs/ROADMAP.md) § 1.

8.2 Password sourcing strategy

See § 4 Stage 2. The hybrid (dictionary ? template ? prompt) is the natural answer, but the design of the rule-template config format and the candidate dictionary needs a user call.

> **Resolved (2026-05-30): Hybrid ? candidate dictionary ? per-sender rule template ? interactive prompt**, caching the winning method per sender; password material never in the repo or logs. The rule-template config format and dictionary design are deferred to implementation (ROADMAP Milestone C.1). See [`docs/ROADMAP.md`](./docs/ROADMAP.md) § 1.

8.3 LLM locality, per document category

The default is "local for sensitive, cloud allowed for non-sensitive after review." But which models? Ollama with Llama 3 / Qwen / Mistral on the user's hardware? A small fine-tuned model? Cloud (Anthropic / OpenAI / Gemini) with strict redaction for the non-sensitive categories?

This decision is closely coupled to the user's hardware capacity (GPU? RAM?) and willingness to wait on inference latency.

8.4 First-slice scope confirmation

§ 7 proposes "Gmail + one bank ? CLI". Confirm with user before building.

> **Resolved (2026-05-30): Build both tracks in parallel** ? the promo-corpus parse?store?query spine *and* the Gmail ingestion plumbing, as two decoupled tracks that converge at the first bank slice; first bank = **HDFC**. This refines § 7 (which proposed a single end-to-end bank slice first). See [`docs/ROADMAP.md`](./docs/ROADMAP.md) §§ 1 and 4.

8.5 Merchant canonicalization approach

Fuzzy string matching has well-known failure modes on merchant names. Do we use a curated rules file (user maintains)? A library? A learned model? Defer until real data exposes the actual mess.

8.6 Ingestion source ? Open Banking (Account Aggregator) vs PDF parsing ? **RESOLVED

(2026-06): PDF-first, AA deferred**

    Researched in depth ([`docs/research-open-banking-india.md`](./docs/research-open-banking-india.md)). India's "open banking" is the RBI **Account Aggregator** framework. A single-user, local-first, no-paid-API tool **cannot** consume AA today ? **FIU status requires a regulated entity** (no individual path), and the only reachable integration rides under a licensed FIU via a **paid TSP that decrypts the entire feed in its cloud** (on-device/self-custody-key decryption is *not* an available AA mode). AA is *safer in transit* than emailed PDFs (E2E-encrypted, data-blind aggregator), but for muneem's threat model (one local honeypot, zero third parties) the realistic integration is a **net privacy regression**. AA also can't cover muneem's full scope (no credit cards / bonds / broker detail / non-demat MF) and still needs the enrichment/reconcile layer. **Decision:** PDF-first core; AA only ever a **deferred, opt-in complement** (import from a regulated FIU's export, oracle-validated, provenance-tagged), with explicit re-adopt triggers (chiefly: a TSP offering true on-device decryption). A new local-only candidate surfaced ? **SMS/email transaction-alert parsing** ? as a freshness complement to PDF.

    ### 8.7 Storage shape ? **RESOLVED (2026-06): flat per-issuer source of truth + derived serving layer**

    Each parser writes its own **flat, isolated per-issuer table** (the source of truth ? regression-isolated, trivially backfilled/rolled back). Cross-source querying is a **derived serving layer** (a SQL view/materialised table now; a separate serving DB only when latency/cost demand). **Provenance + confidence** are first-class but live in a **separate `provenance` table keyed by `document_id`** (not duplicated per row): source_type (pdf/aa/sms), parser, parser_version, mapping_layer, confidence. The reconciliation oracle treats every source as a hint to validate. See [`docs/schema.md`](./docs/schema.md) and [`docs/feedbackJune2026.md`](./docs/feedbackJune2026.md) P3.

    ## 9. Proposed (not built) file layout

    This is a *suggestion* for once the language decision is made. Don't materialize it prematurely.

    ```
    muneem/
    ??? README.md
    ??? CLAUDE.md
    ??? DESIGN.md
    ??? docs/
    ?   ??? prior-art-email-mining.md
    ?   ??? (future: parser-notes-per-issuer.md, schema.md, threat-model.md)
    ??? testdata/
    ?   ??? promo-emails/         # non-sensitive corpus (in repo)
    ?   ??? statements/           # GITIGNORED ? real statements, never committed
    ??? (source tree TBD based on § 8.1)
    ??? (DB file, GITIGNORED)
    ```

    ## 10. Glossary

    - **Muneem** (?????) ? traditional Indian household/business bookkeeper.
    - **Issuer** ? the bank, card network, broker, or merchant that produced a document.
    - **Statement** ? a periodic financial summary from an issuer (bank statement, CC bill, broker statement, MF report).
    - **Holding** ? a position in an investment account at a point in time.
    - **Line item** ? a single transaction or itemized purchase row within a statement or receipt.
    - **Born-digital PDF** ? a PDF generated from a software source with a real text layer (most bank statements). Contrast with scanned/image PDFs.

    ## 11. Vision continuity from Email-Mining

    The predecessor project (`~/Projects/Email-Mining`) extracted promo codes and expiry dates from manually-saved promo email PDFs, using PyMuPDF for text+image extraction and Gemini

for image OCR and summarization. Its scope was just "find the coupon I forgot about."
231
232   muneem **subsumes** that scope. The deals/coupons feature survives as **one of several extraction targets** ? one of the analytics surfaces, fed by the same five-stage pipeline that handles bank statements and everything else. The promo-email PDFs from that project are preserved in [`testdata/promo-emails/`](./testdata/promo-emails/) as the first regression corpus.
233
234   What was learned there and worth keeping: the PDF text+image extraction *shape* is right (but PyMuPDF/fitz itself is AGPL-3.0 ? muneem uses pdfminer.six (MIT) instead; see § 5 rule 8); image dedup by hash + SSIM is a real win for repeated chrome; the shape "page text + per-image OCR ? unified text ? structured extraction" generalizes well. What we change: programmatic Gmail/Outlook fetching instead of manual save-as-PDF; local extraction by default for sensitive categories; persistent normalized DB instead of one-shot stdout.
235
236   Full details in [`docs/prior-art-email-mining.md`](./docs/prior-art-email-mining.md).
237

14. user (2026-06-10T16:13:42.948Z)

```
1       # Architecture
2
3       > Status: living document. Captures the **target architecture** for the
4       > extraction pivot and the **decisions** behind it. Product framing is in
5       > [`VISION.md`](./VISION.md). This supersedes the original "just copy
6       > attachments" mental model; the copier survives as one extractor at the leaf.
7
8       ## Auditability invariant (binding)
9
10      **This repository is the _only_ component that reads or processes a user's
11      Gmail, and it must remain auditable and self-deployable by a power user at any
12      time.** This is the product's core privacy guarantee and a deliberate constraint
13      on every future decision (see ADR-008). It is why the project is built to be
14      open-sourced: *don't trust us ? read, and run, the code.*
15
16      Every change MUST preserve all of the following. A change that cannot is, by
17      definition, a decision to break the invariant and needs explicit owner sign-off.
18
19      1. **Sole Gmail surface.** Gmail is read/processed *only* here, in the user's own
20         account (`executeAs: USER_ACCESSING`). No other component ? Muneem included ?
21         ever holds the Gmail OAuth grant or sees raw mail. Muneem may *orchestrate*
22         this app; it is never *itself* the Gmail reader.
23      2. **Egress == the audited boundary.** Nothing derived from a user's mail leaves
24         their Google account except through the in-repo **REDACT/MINIMIZE** stage,
25         carrying only minimized, non-PII signal, only to the user's own hosted
26         service, never to third parties. The set of egress destinations is explicit
27         and reviewable in source.
28      3. **Run-your-own-from-source.** A power user can build and deploy their own
29         instance from the published source and obtain an equivalent app; the build is
30         reproducible and the verification steps are documented ? "trust by running the
31         code you read," not merely "read the code."
32      4. **No secrets in source.** Credentials exist only as deploy-time secrets; the
33         repository contains nothing sensitive.
34      5. **Transparent over clever** at the read and egress/redaction boundaries ?
35         prefer obviously-correct, reviewable code even at some cost to brevity.
36
37      **PR review checklist (apply to every change):**
38
39      - [ ] Gmail access stays entirely within this repo, in-account?
40      - [ ] If any data leaves the account, does it pass through REDACT/MINIMIZE and go
41            only to the user's own service (non-PII)?
42      - [ ] Build still reproducible and self-deployable from source?
43      - [ ] No new secrets, third-party endpoints, or network calls? (Default: none ?
44            enforced by `tests/auditability.test.ts`.)
45
```

```
## Principles (preserved from today's codebase)

1. **Shells around pure functions.** GAS-facing code is a thin shell; all logic
   is pure and unit-tested.
2. **Quota discipline.** Respect the 6-minute execution limit, the 90-min/day
   trigger budget, and per-user caps. Self-healing trigger fan-out; per-user
   lock; "mark done only on clean completion."
3. **Per-user isolation.** Runs as the accessing user; state is per-user.
4. **Versioning & provenance everywhere.** Every persisted shape carries a
   `schemaVersion`; every extracted entity carries where it came from and which
   extractor (and version) produced it.

## The pipeline

This repo owns the left half; downstream services own the right half. They meet
at the **handoff contract** only.

```
                ?????????????????????????? this repo ??????????????????????????   ?? downstream
??
 Gmail ???  INGEST ??? CLASSIFY ??? EXTRACT ??? REDACT/MINIMIZE ??? HANDOFF ???  Muneem
/
                (rich      (type +     (normalize  (strip PII,         (sink)        grocery
/
                EmailMsg)  confidence) light)      encrypt)                          coupon
svc
                   ?                                                                 ??
parse,
                   ???? RAW ARCHIVE (Drive folders ? optional per type later)
encrypted
                   ???? LOCAL INDEX (Store interface; Sheets now, remote later)
store,

analytics
```

- **INGEST** builds a durable `EmailMessage` **once** per message (body, subject,
  labels, attachment metadata ? via the Gmail advanced service). Extraction
  re-runs over this record without re-touching Gmail or re-copying blobs.
- **CLASSIFY** assigns a type + confidence (sender/subject/body rules now; richer
  models downstream later).
- **EXTRACT** is a registry of pluggable, versioned extractors. The current
  attachment copier becomes the `RawArchive` extractor.
- **REDACT/MINIMIZE** enforces the privacy north star before any egress.
- **HANDOFF** routes a normalized payload to the right downstream **sink**.

Data model

All pure shapes, versioned from day one.

```
EmailMessage {                          // INGEST output ? the stable substrate
  schemaVersion, id, threadId, historyId?,
  from, to[], cc[], subject, date, labels[],
  snippet, plainBody?, htmlBody?,
  attachments: [{ index, name, contentType, sizeBytes,
                  attachmentId/partId, sha256?, driveFileId? }],
}

ExtractedEntity {                       // EXTRACT output ? the queryable payload
  schemaVersion, entityKey,             // stable dedup key, e.g.
                                        //   receipt:{merchant}:{total}:{date}
  type, subtype, confidence,
  fields,                               // typed per kind (receipt, statement, ?)
  provenance: { sourceMsgIds[], attachmentIndex?,
```
```

```
                    extractor, extractorVersion, extractedAt, model? },
      status: pending|extracted|low_confidence|needs_review|corrected,
      links: { driveFileId?, ? },
    }

    HandoffPayload {                        // REDACT/MINIMIZE output ? what leaves
      schemaVersion, entityKey, type, extractorVersion,
      nonPiiFields,                         // minimized, scrubbed signal only
      // encrypted in transit; PII stripped or tokenized
    }
    ```

    - **Entity keys** give content-stable dedup (re-mailed receipts; cross-source
      fusion of an email confirmation + a PDF ticket ? one entity).
    - **Attachment `sha256`** enables content-addressable dedup of re-sent files.
    - **Provenance + `extractorVersion`** make "this extraction is stale ? re-run"
      possible and auditable.

    ## The extraction ? processing boundary

    - This repo produces a normalized, classified, **redacted** `HandoffPayload` per
      detected document/entity and routes it to a downstream **sink**.
    - Downstream services (Muneem for finance; siblings for groceries/coupons) own
      domain parsing, indexing, **encrypted storage**, and analytics.
    - **Contract is owned jointly, defined from our side first.** The exact
      Muneem-facing schema and transport are **TBD and aligned with that repo**
      (this repo's GitHub tooling is scoped to itself, so the contract is specified
      here and reconciled with Muneem separately).
    - Hard rule: this repo imports **no** domain-parsing logic; downstream reads
      no Gmail.

    ## Storage abstraction (decision: extensible, not baked-in)

    A single `Store` / `Sink` interface; code depends only on the interface.

Impl	When	Properties
`GoogleSheetsStore` (user-owned "LifeIndex" sheet)	v1	in-account, queryable, zero
infra, doubles as a first dashboard
`RemoteStore` (our hosted DB) / Muneem-API sink	later	the durable system of record

    `PropertiesService` stays only for small per-user **control state** (cursors,
    flags, counters) ? never the entity index.

    ## Privacy & egress controls

    - **Egress only to our own isolated, hosted service. No third parties.**
    - A **Redactor/Minimizer** stage runs before any egress: strip/tokenize PII,
      keep only non-PII signal, encrypt in transit (encrypted at rest downstream).
    - North star: continuously **shrink** what leaves while keeping the product
      healthy.
    - The in-account tier (Phases 0?2) has **no egress** and needs no privacy
      change. The external tier (Phase 3) is **flag-gated + separately consented**,
      and updates the privacy policy to name exactly what is sent and where.
    - **Cascade to be aware of:** sending restricted-scope Gmail data outside Google
      is what makes Google's **CASA security assessment mandatory for a public
      launch**. Deferred ? consistent with "stay <100 until the user base justifies
      the investment."
    - **Open tension to resolve (Phase 3 design):** bank-statement and receipt
      signal (merchant, amounts, account identifiers) is often itself sensitive.
      "Non-PII only" needs a concrete per-vertical definition of what is sent vs.
      tokenized vs. kept fully in-account.

    ## State & cursors
```
```

```
166
167     - Keep the hybrid `history.list` + date-window skeleton; **extend, don't
168       duplicate.**
169     - Add an **extraction-aware cursor** ("fully extracted up to X with
170       extractor-set V") distinct from the copy cursor, plus per-message extraction
171       status ? so a "re-extract window for new rules" reprocesses without
172       re-copying (content hash) or duplicating entities (entity keys).
173
174     ## Quota, cost & pacing
175
176     - **Separate budgets:** raw-copy cap (existing) vs. extraction cap vs.
177       egress/handoff cap.
178     - Per-item time budgets, **prioritization** (recent + high-confidence/trusted
179       senders first), and **deferral** of heavy items.
180     - Heavy ML/PDF work happens **downstream**, not in Apps Script ? GAS stays the
181       lightweight, reliable watcher/archiver/handoff.
182
183     ## Observability
184
185     - Per-**classifier** and per-**extractor** success / failure / ambiguity rates ?
186       not just "files added."
187     - Surfaced **review queues:** "matched no extractor" and "low confidence."
188     - Structured logs carrying `messageId`; extraction & handoff metrics.
189
190     ## Platform spectrum
191
192     1. **Pure-GAS conservative extractors** (regex, known formats `.ics`/`.csv`,
193        native Drive OCR via Doc conversion) for the easy ~70%, **in-account**.
194     2. **Hosted, isolated service** (Muneem pattern) for heavy/ML extraction,
195        storage, and analytics ? fed only the minimized, non-PII handoff.
196
197     ## Decision log
198
199     | # | Decision | Rationale |
200     | --- | --- | --- |
201     | ADR-001 | Pipeline (ingest ? classify ? extract ? redact ? handoff), not a copy loop |
Treat mail as structured data; make extraction re-runnable over a durable record |
202     | ADR-002 | Extraction (this repo) is decoupled from processing (Muneem & siblings) |
Isolate domain logic + encrypted storage + analytics downstream; clean contract seam |
203     | ADR-003 | Storage via a `Store`/`Sink` interface ? Sheets v1, remote/Muneem later |
Extensible by design; never baked into the code |
204     | ADR-004 | Egress: non-PII only, to our own isolated hosted service, no third parties;
scrub + encrypt; minimize | Privacy north star; preserves trust; in-account baseline needs no
egress |
205     | ADR-005 | Audience: <100 (Testing) now, public later; CASA deferred | Harden + learn
value before investing in assessment/infra |
206     | ADR-006 | First verticals: grocery receipts + bank statements (? Muneem) | Volume +
the high-fidelity handoff path |
207     | ADR-007 | Preserve current strengths (pure/shell, quotas, triggers, per-user,
versioning/provenance) | The recent rewrite is exactly why this pivot is feasible |
208     | ADR-008 | **Option A:** this repo is the sole Gmail-touching component ? in-account,
auditable, self-deployable; Muneem orchestrates but never holds the Gmail grant. Codified as the
binding [auditability invariant](#auditability-invariant-binding). | Verifiable privacy is the
product's moat and shrinks future verification scope; it concentrates the trust-critical surface
in one open, in-account place. |
209
210     ## Open items for the next iteration
211
212     - **Muneem handoff contract:** schema + transport (e.g. `UrlFetch` to a signed
213       endpoint vs. write-to-a-shared-store) ? define jointly with `avidullu/muneem`.
214     - **Per-vertical "non-PII" definition** for receipts and bank statements (what is
215       sent vs. tokenized vs. kept in-account).
216     - **First extractor slice** to build in Phase 1 (recommend a narrow
217       "receipt-smelling" detector) and its golden-email test fixtures.
218
```

15. user (2026-06-10T16:14:09.966Z)

Next wakeup scheduled for 22:10:00 (in 1550s). Nothing more to do this turn ? the harness re-invokes you when the wakeup fires or a task-notification arrives.

16. user (2026-06-10T16:32:42.521Z)

<task-notification>
<task-id>wctcr4nab</task-id>
<tool-use-id>toolu_014Kjje7FJhK2YXaUkpAonwG</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\tasks\wctcr4nab.output</output-file>
<status>completed</status>
<summary>Dynamic workflow "Deep review of muneem + gmail-attachments-to-gdrive: code, security, architecture, roadmap, and market fit" completed</summary>
<result>{"code":[{"key":"muneem-core","summary":"Deep review of muneem's 16 core (non-parser) modules plus their tests. The crypto design is fundamentally sound ? SQLCipher raw-key open with key-before-any-access ordering and verification, no silent plaintext fallback, fresh salt+nonce per recovery-file wrap with sane Argon2id parameters, and value-free keychain error messages ? but the key LIFECYCLE has serious atomicity gaps: `db rotate-key` destroys the only durable copy of the current key before rekeying and its rollback misses non-EncryptedDatabaseError failures (critical, permanent-lockout window), and a transient keychain read failure causes `get_or_create_key` to mint a new key over the real one (high). `db backup` unlinks the destination before validating it is not the live ledger, so `muneem db backup &lt;ledger-path&gt;` silently replaces the ledger with an empty DB (high). Medium items include un-validated key material spliced into PRAGMA strings (non-hex keys silently fall back to SQLCipher passphrase-KDF semantics), forward-migration checksums that are recorded but never re-verified, a legacy-DB backfill that stamps partial/older schemas as current without DDL, an Axis-CC listing bug printing '+Cr' instead of the amount, redaction patterns that miss PAN and spaced card/Aadhaar formats, and rotation silently invalidating the recovery file and prior backups. Low findings cover Windows chmod being a no-op, ATTACH path quoting, recovery-file KDF param validation, a missing mkdir in unlock's plaintext path, and the redaction filter covering only cli.py's logger.","verified":[{"title":"Key rotation overwrites the only copy of the current key before the file is rekeyed; rollback misses realistic failures","severity":"critical","area":"crypto / key lifecycle","file":"src/muneem/cli.py","line":"438-446","description":"`db rotate-key` writes the fresh key into the keychain FIRST (`db_key.set_key(fresh)`), destroying the only durable copy of the current key (it survives only in process memory as `current`), and only then rekeys the file. Two realistic failure modes permanently lock the ledger: (1) a crash/power loss/Ctrl-C between `set_key(fresh)` and a completed `PRAGMA rekey` leaves keychain=fresh, file=old-key, old key gone; (2) the rollback `except` only catches `dbmod.EncryptedDatabaseError`, but `rekey_database` (src/muneem/db.py:274-279) can raise `sqlcipher3.OperationalError` (e.g. 'database is locked' ? very plausible on Windows with another process/AV holding the file) or any other sqlcipher3 error from `PRAGMA rekey`, which escapes uncaught, skips the rollback, and exits with keychain=fresh / file=old. Unless the user happens to have a recovery file (which wraps the OLD key), the entire financial ledger and all same-key backups are unrecoverable. The inline comment ('Keychain-first with rollback: if the file rekey fails, the keychain still matches the file') asserts the opposite of what the code does ? keychain-first guarantees a mismatch window.","evidence":"# Keychain-first with rollback: if the file rekey fails, the keychain still matches the file.\n   db_key.set_key(fresh)\n   try:\n   dbmod.rekey_database(db_path, current, fresh)\n   except dbmod.EncryptedDatabaseError as exc:\n   db_key.set_key(current) # roll back; the file is still under the old key","suggestion":"Never have a moment where only one key exists in durable storage: write the fresh key under a secondary keychain name (e.g. 'db-key-pending') first, rekey the file, then promote it to 'db-key' and delete the pending entry; on open failure, fall back to trying the pending key. At minimum: rekey the file BEFORE touching the 'db-key' entry, broaden the except to catch all exceptions (re-raising after rollback), and stash the old key under 'db-key-prev' before overwriting.","confidence":0.92,"verdict":{"confirmed":true,"votesReal":3,"votesTotal":3,"adjustedSeverities":["high","high","critical"],"notes":["CONFIRM: Traced cli.py:438-446, db.py:266-280 (rekey_database), db_key.py, secrets.py, errors.py, and tests/test_db_crypto_ops.py. Confirmed: the keychain's single 'db-key' entry is overwritten with the fresh key BEFORE the file rekey; the rollback catches only EncryptedDatabaseError (a MuneemError subclass) while PRAGMA rekey/commit raise raw sqlcipher3 errors that escape it; crash/Ctrl-C in the window also bypasses rollback ? all leaving keychain=fresh/file=old with no durable copy of the old key (backups share the same key, so they're lost too). Adjusted critical?high: the trigger is a rare

user-initiated op with a narrow failure window, an optional recovery file (PR #43) wraps the old key, and the ledger is re-derivable from the user's retained source PDFs; also the inline comment is accurate for the one caught path, so 'asserts the opposite' mildly overstates.","CONFIRM: Confirmed in code: cli.py:440 overwrites the keychain 'db-key' (the only durable copy of the current key) before the file rekey, violating the explicit docstring contracts in db.py:271-272 and db_key.py:45, and the inline comment asserts the opposite of the behavior. The uncaught-exception path is real and reachable: EncryptedDatabaseError is a MuneemError unrelated to sqlcipher3 exceptions, and 'database is locked' from a concurrent SHARED-lock holder surfaces exactly at PRAGMA rekey (db.py:276, only try/finally) because the earlier verification SELECT succeeds ? so the rollback is skipped and the process exits with keychain=fresh/file=old; Ctrl-C and disk-full during the page-rewrite window behave the same. I downgrade critical?high because the catastrophe requires a failure during a rare manual operation, the shipped P0 recovery file (db setup-recovery, PR #43) wraps the OLD key and fully recovers a failed rotation if the user set it up, and source statements remain re-ingestable ? but the lockout is deterministic (no crash timing needed) for the locked/disk-full cases, so it stays high.","CONFIRM: Verified in code: cli.py:440-442 overwrites the sole keychain copy of the old key before PRAGMA rekey (keyring.set_password replaces in place, no versioning), and the rollback at cli.py:443 only catches EncryptedDatabaseError while rekey_database (db.py:274-279) leaves PRAGMA rekey/commit unwrapped, so sqlcipher3.OperationalError (e.g. database-is-locked) escapes uncaught and skips rollback. I searched for mitigations and found none: tests cover only the happy path, no pending/prev-key mechanism exists, and the opt-in recovery file is neither required nor refreshed by rotate-key (and goes stale after a successful rotation). The implementation also contradicts the explicit file-first contracts in rekey_database's and set_key's own docstrings, and docs/encryption-at-rest.md repeats the inverted comment ? critical stands given total ledger+backup loss, tempered only slightly by rotate-key being a rare command."]}},{"title":"Transient keychain read failure mints a new key and silently overwrites the only DB key","severity":"high","area":"crypto / key lifecycle","file":"src/muneem/db_key.py","line":"24-31","description":"`secrets.get_secret` (src/muneem/secrets.py:25-30) swallows every `KeyringError` and returns `None`, deliberately treating 'backend unavailable' the same as 'key absent'. `get_or_create_key()` then interprets that `None` as 'no key has ever been minted', generates a fresh key, and `set_secret` OVERWRITES the existing 'db-key' entry. If a read fails transiently (locked/temporarily failing Credential Manager session, keyring backend hiccup) while the subsequent write succeeds, the only copy of the real ledger key is destroyed; the next `connect_encrypted` fails with 'wrong key' and the data is gone unless a recovery file exists. `connect_encrypted` (src/muneem/db.py:207) calls `get_or_create_key()` unconditionally ? even when an encrypted ledger file already exists on disk, which is exactly the case where minting is never the right answer.","evidence":"def get_or_create_key() -> str:
... (truncated 232067 chars, full result in C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\tasks\wctcr4nab.output)</result>
<usage><agent_count>35</agent_count><subagent_tokens>2567029</subagent_tokens><tool_uses>680</tool_uses><duration_ms>1154787</duration_ms></usage>
</task-notification>

17. user (2026-06-10T16:33:01.152Z)

=== CODE REVIEWERS ===

18. user (2026-06-10T16:33:13.018Z)

LENGTH: 258056
{
  "summary": "Deep review of muneem + gmail-attachments-to-gdrive: code, security, architecture, roadmap, and market fit",
  "agentCount": 35,
  "logs": [
    "Launching 4 code/security reviewers (with verification), 3 doc/test reviewers, and the market-research agent",
    "muneem-security: 10 fi
...
ures/email-receipts/",
        "WellyBox ? automatic email receipt collection ? https://www.wellybox.com/expense-tracker-software/",
        "Zoho Expense for Gmail ? Workspace Marketplace listing ? https://workspace.google.com/marketplace/app/zoho_expense_for_gmail/927860167710"

```
      ]
    }
  }
}
```

19. user (2026-06-10T16:33:27.201Z)

```
=== LOGS ===
Launching 4 code/security reviewers (with verification), 3 doc/test reviewers, and the market-
research agent
muneem-security: 10 findings (0 high/critical to verify)
muneem-core: 14 findings (3 high/critical to verify)
gmail-code: 19 findings (3 high/critical to verify)
muneem-parsers: 12 findings (3 high/critical to verify)
Verification done: 9 high/critical findings confirmed, 0 refuted
=== CODE REVIEWERS ===
```

muneem-core

SUMMARY: Deep review of muneem's 16 core (non-parser) modules plus their tests. The crypto design is fundamentally sound ? SQLCipher raw-key open with key-before-any-access ordering and verification, no silent plaintext fallback, fresh salt+nonce per recovery-file wrap with sane Argon2id parameters, and value-free keychain error messages ? but the key LIFECYCLE has serious atomicity gaps: `db rotate-key` destroys the only durable copy of the current key before rekeying and its rollback misses non-EncryptedDatabaseError failures (critical, permanent-lockout window), and a transient keychain read failure causes `get_or_create_key` to mint a new key over the real one (high). `db backup` unlinks the destination before validating it is not the live ledger, so `muneem db backup <ledger-path>` silently replaces the ledger with an empty DB (high). Medium items include un-validated key material spliced into PRAGMA strings (non-hex keys silently fall back to SQLCipher passphrase-KDF semantics), forward-migration checksums that are recorded but never re-verified, a legacy-DB backfill that stamps partial/older schemas as current without DDL, an Axis-CC listing bug printing '+Cr' instead of the amount, redaction patterns that miss PAN and spaced card/Aadhaar formats, and rotation silently invalidating the recovery file and prior backups. Low findings cover Windows chmod being a no-op, ATTACH path quoting, recovery-file KDF param validation, a missing mkdir in unlock's plaintext path, and the redaction filter covering only cli.py's logger.

-- VERIFIED high/critical --
[critical -> confirmed=True votes=3/3 adj=high,high,critical] Key rotation overwrites the only copy of the current key before the file is rekeyed; rollback misses realistic failures | src/muneem/cli.py:438-446
[high -> confirmed=True votes=3/3 adj=medium,low,medium] Transient keychain read failure mints a new key and silently overwrites the only DB key | src/muneem/db_key.py:24-31
[high -> confirmed=True votes=3/3 adj=high,high,high] `db backup` deletes the destination before any validation ? `muneem db backup <ledger-path>` destroys the ledger | src/muneem/db.py:292-295

-- UNVERIFIED high/critical (overflow) --

-- MEDIUM/LOW --
[medium] Key is spliced into PRAGMA strings with no 64-hex validation ? non-hex keys silently switch SQLCipher to passphrase-KDF mode, and quotes break out of the SQL literal | src/muneem/db.py:109-111
[medium] Forward-migration checksums are recorded but never verified ? only the baseline is drift-checked | src/muneem/migrations.py:133-139
[medium] Legacy/partial-DB backfill records the baseline without running DDL ? older or partially-created DBs are stamped current with missing tables | src/muneem/migrations.py:124-131
[medium] `txns list` prints '+Cr' instead of the amount for Axis credit-card credit rows | src/muneem/cli.py:356-359
[medium] RedactingFilter misses PAN, spaced/hyphenated card numbers, and spaced Aadhaar formats | src/muneem/redaction.py:14-17
[medium] `db rotate-key` silently invalidates the recovery file and all prior backups | src/muneem/cli.py:428-447
[low] encrypt_database: ATTACH path spliced into a SQL literal breaks on apostrophes, and a failed final rename deletes the verified encrypted copy | src/muneem/db.py:234-235, 255-262
[low] unlock(): plaintext-copy path skips parent-directory creation that the encrypted path

performs | src/muneem/unlock.py:69-71
[low] unwrap_key parses attacker-influenced KDF params outside the malformed-payload guard, breaking the RecoveryError contract | src/muneem/recovery.py:90-96
[low] _harden_permissions / chmod 0o600 is a no-op on Windows, the primary platform | src/muneem/db.py:127-134
[low] Redacting logger is opt-in per module and only cli.py opts in; third-party and future module loggers bypass it | src/muneem/log.py:34-43

muneem-parsers

SUMMARY: The parsing core's architecture (concept schema + arithmetic oracle gating persistence) is genuinely strong, and I verified the sensitive testdata dirs (statements, snapshots, credentials) are not git-tracked in any commit. However, deep review found the safety guarantee has real holes exactly where arithmetic cannot see: the savings reconcile-oracle short-circuits on the per-row walk and never consults the printed opening/closing bookends, so a statement missing its leading or trailing transactions persists as 'reconciled' (empirically confirmed); stripping Cr/Dr suffixes without sign semantics lets the L2 mapping search endorse a fully direction-inverted parse of an overdraft statement as verified (empirically confirmed); and HDFC persists all rows even when unreconciled while `txns list` never filters by status. The tolerance equals the currency quantum with a strict-greater comparison, so ~42% of exact one-paisa corruptions pass the walk (measured). Routing is substring-based with alphabetical first-match precedence (an HDFC statement containing a '@aubank' UPI handle routes to AUParser ? confirmed), and sha256 dedup permanently blocks re-ingesting a quarantined document after a parser fix. The Axis credit-card path persists with no verification at all; imap/outlook providers are confirmed experimental and unreachable from the CLI (no fetch command; only manual tools/ scripts), and Gmail uses the minimal read-only scope with keychain token storage.

-- VERIFIED high/critical --
[high -> confirmed=True votes=3/3 adj=high,high,high] Savings oracle ignores available opening/closing bookends when the balance walk passes ? missing leading/trailing transactions persist as 'reconciled' | src/muneem/parsers/savings.py:78-84, 194-205
[high -> confirmed=True votes=3/3 adj=high,high,high] Cr/Dr suffix sign-stripped in parse_amount ? an overdraft (Dr-balance) statement reconciles with every debit/credit INVERTED and persists as verified | src/muneem/parsers/validation.py:32, 45
[high -> confirmed=True votes=3/3 adj=high,high,high] HDFC parser persists all rows even when the parse is UNRECONCILED, and `txns list` never filters by document status | src/muneem/parsers/hdfc.py:435-458

-- UNVERIFIED high/critical (overflow) --

-- MEDIUM/LOW --
[medium] Reconciliation tolerance equals the currency quantum with strict-greater compare ? ~42% of exact one-paisa corruptions pass the balance walk | src/muneem/parsers/validation.py:70-71, 91, 118
[medium] Parser routing by bare substring + alphabetical first-match: a statement mentioning 'aubank' in a narration routes to AUParser | src/muneem/parsers/au.py:33-37
[medium] sha256 dedup permanently blocks re-ingesting a quarantined statement after a parser fix | src/muneem/cli.py:121-124
[medium] Axis credit-card parser persists with zero verification, fixed column indices, sign-stripping, and silent 0.0 coercion | src/muneem/parsers/axis.py:127-149, 169-201
[medium] Dates are never parsed, validated, or normalized ? raw locale strings persisted; arithmetic cannot catch date-column or date-content errors | src/muneem/parsers/engine.py:37-40, 261-263
[medium] Clustering drops whole transactions (the Axis May-2025 mechanism) and all narration continuation lines | src/muneem/parsers/clustering.py:30-40, 82-89
[low] Gmail provider: refresh-failure crash path, unbounded in-memory attachments, sender-controlled filenames returned for future path joins | src/muneem/ingestion/gmail.py:33-37, 92-115
[low] Outlook provider replays a cached access token forever (no expiry/refresh) and silently drops the newer_than_days filter; experimental status of imap/outlook confirmed | src/muneem/ingestion/outlook.py:39-44, 66-67
[low] Dead `safe_amount` helper that maps '0.00' to None is still exported from the parsers package | src/muneem/parsers/base.py:8-21

gmail-code

SUMMARY: Deep correctness review of gmail-attachments-to-gdrive (Apps Script/TS). The date-window backfill core is well-designed (deliberate ±1-day window overlap absorbs timezone skew; fan-out-before-delete trigger replacement is genuinely self-healing; the FIFO + getFilesByName probe prevents duplicate copies). However, the opt-in incremental history sync has several real data-loss/wedge paths: the historyId cursor is advanced even when messages were truncated by the daily cap or failed transiently, GmailApp.getMessageById throws (rather than returning null) for deleted messages which wedges the incremental loop until cursor expiry, pagination truncation at 20 pages silently skips history, and no spam/trash/draft filtering is applied (unlike the backfill query). In backfill mode, the 'mark done only on clean completion' invariant is undermined because lastSynced advances past a window even when attachment-level errors occurred, so failed messages are never retried. Additional defensible issues: getOrCreateFolderInFolder iterates ALL folders in the user's Drive (DriveApp.getFolders() is not 'children of root'), state is persisted only at run end so a 6-minute hard kill undercounts the daily cap, setup re-runs race in-flight syncs (no lock in processInput), CI creates a brand-new deployment per push instead of updating one in place, and execution logs containing Gmail message ids land in developer-visible Cloud Logging, in tension with the privacy policy's 'author has no access' claim.

-- VERIFIED high/critical --
[high -> confirmed=True votes=3/3 adj=high,medium,medium] Incremental sync advances historyId past messages that were not fully processed (silent data loss) | src/sync.ts:175-189
[high -> confirmed=True votes=3/3 adj=medium,medium,medium] GmailApp.getMessageById throws on deleted messages, wedging incremental sync until the cursor expires | src/gmailHistory.ts:70-77
[high -> confirmed=True votes=3/3 adj=high,high,high] Backfill advances lastSynced past windows that had attachment-level errors, so failed messages are never retried | src/sync.ts:213-218

-- UNVERIFIED high/critical (overflow) --

-- MEDIUM/LOW --
[medium] getOrCreateFolderInFolder(DriveApp, ...) iterates ALL folders in the user's Drive, not root children | src/drive.ts:87, 12-15, 34-44
[medium] No runtime-budget check inside updateDrive; all bookkeeping persisted only at run end, so the 6-minute hard kill loses it | src/sync.ts:199-203, 233-244
[medium] Incremental path applies no spam/trash/draft/sent filtering, diverging from the backfill query | src/gmailHistory.ts:37-42
[medium] History pagination truncated at MAX_HISTORY_PAGES silently skips changes (cursor jumps to current mailbox id) | src/gmailHistory.ts:12, 37-48
[medium] History-expiry recovery rewinds a fixed 30 days; lastSynced is never advanced during incremental mode, so longer outages lose mail | src/sync.ts:170-173
[medium] Setup re-run races an in-flight sync: processInput takes no lock, so the running sync's end-of-run writes clobber fresh setup state | src/ui.ts:58-72
[medium] Distinct attachments sharing a filename are silently dropped while the message is marked done | src/drive.ts:128, 148, 177-180
[medium] Execution logs (Gmail message ids, raw error text) flow to developer-visible Cloud Logging, in tension with PRIVACY.md claims | src/logger.ts:8-10
[medium] CI creates a brand-new Apps Script deployment on every push instead of updating the existing one | .github/workflows/ci.yml:54
[medium] Unpaged GmailApp.search caps results (~500 threads); denser 30-day windows can permanently skip mail | src/sync.ts:208
[low] getFilesAddedToday parses JSON unguarded, outside the run's try/catch ? a malformed value causes a permanent crash-loop | src/state.ts:49-55
[low] logError severity claim is wrong: Logger.log always lands at INFO in Cloud Logging | src/logger.ts:4-9, 24-27
[low] Backfill query 'to:me' misses cc/bcc-received mail, and diverges from the incremental path | src/query.ts:23
[low] Properties-wipe recovery path re-copies up to a year of attachments as duplicates into the root folder | src/sync.ts:96-100
[low] SUMMARY_EMAIL_RECIPIENT is a global constant: in a multi-user deployment every user's summary is mailed (as them) to one hardcoded address | src/config.ts:71-72
[low] Single script timeZone (America/Los_Angeles) governs day boundaries, cap resets and summaries for all users | appsscript.json:2

muneem-security

SUMMARY: muneem's security posture is unusually strong for a personal project: git ground truth

is verified clean (nothing sensitive tracked now or ever in history, including the full --all log over statements, credentials, snapshots, env and DB patterns), the SQLCipher encryption seam is enforced by tested invariants that make plaintext-ledger regressions a failing build, secrets live exclusively in the OS keychain with value-free error messages, and CI uploads no artifacts and physically cannot see real statements. The real residual risk is concentrated in two places. First, information-at-rest inside the repo working directory: real statement PDFs, parser snapshots containing real credit-card rows, and a real OAuth client secret all sit unencrypted in testdata/ ? untracked and hook-blocked, but exposed to sync/backup/malware and one forced add away from GitHub, and the strongest guard layer (the pre-commit hook) is opt-in and undocumented for the Windows dev platform, with the CI backstop firing only after a bad commit has already reached GitHub. Second, supply chain: every dependency is unpinned with no lock or hash checking, including the native sqlcipher3 wheel that provides the at-rest encryption itself. Specific smaller gaps: client_secret_*.json escapes all three guard layers once outside testdata/, the recovery passphrase floor is 8 characters on an offline-attackable file, local-statement tests write real parsed rows to plaintext temp SQLite files, and the redaction backstop misses spaced card numbers and unredacted filenames in eval output. All findings have concrete, low-effort fixes; none indicate sensitive data has actually leaked.

-- VERIFIED high/critical --

-- UNVERIFIED high/critical (overflow) --

-- MEDIUM/LOW --
[medium] Real-statement derivatives sit unencrypted in the repo working dir (snapshots, statements, OAuth client) | tests/conftest.py:27-48
[medium] client_secret_*.json escapes every guard layer once outside testdata/ | .githooks/precommit_check.py:25,48-51
[medium] Pre-commit guard is opt-in and undocumented for Windows; CI backstop fires only after the secret is already on GitHub | Makefile:20-22
[medium] Supply chain: no version pinning, no lock file, no hash checking; native crypto binaries pulled unpinned | pyproject.toml:13-28
[low] Recovery passphrase floor of 8 characters on an offline-attackable file | src/muneem/recovery.py:41,61-62
[low] local_statements tests persist real parsed transactions into plaintext SQLite files in pytest temp dirs | tests/parsers/test_local_au.py:39-44
[low] Redaction backstop has pattern gaps; eval output prints unredacted statement filenames | src/muneem/redaction.py:15-17
[low] unlock.write_decrypted is an unused footgun that writes decrypted statement copies to disk | src/muneem/unlock.py:41-53
[low] Transient keychain read failure can mint and store a divergent DB key (availability, not confidentiality) | src/muneem/db_key.py:24-31
[low] Strengths: layered hygiene with verified-clean history, hard encryption seam, value-free diagnostics, sterile CI | tests/test_repo_hygiene.py:

20. user (2026-06-10T16:33:38.816Z)

<persisted-output>
Output too large (66.9KB). Full output saved to: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\tool-results\bxoew9pc1.txt

Preview (first 2KB):
########## muneem-tests-ci ##########
SUMMARY: muneem's test/CI architecture is genuinely strong for a pre-1.0 personal project: the offline suite (390 tests) passes in 6.35s at 86.17% coverage against the 80% CI gate, CI really does run ruff+mypy+pytest on a Linux 3.11-3.13 + Windows 3.13 matrix, the schema-lock and plaintext-downgrade tripwires exist and work, the license allowlist is enforced as a test, and sensitive paths are defended in depth (gitignore default-deny, tested pre-commit guard, repo-hygiene backstop; git history verified clean). The biggest gaps: the key-rotation CLI's failure/rollback path has zero test coverage and its "keychain-first" ordering contradicts its own docstrings (a crash mid-rotation orphans the ledger key); real parsed credit-card transactions sit in plaintext JSON in testdata/snapshots/, written by a now-dead fixture; the real-statement test layer has no CI story at all today (Layer-2 self-hosted runner is design-only, local tests skip silently without passwords); synthetic full-pipeline fixtures cover only 2 of 4 parser families with idealized 3-row single-page layouts; and Google's default OAuth filename pattern (client_secret_*.json) is not blocked by any guard outside testdata/.

MAP: ARCHITECTURE (verified by reading + running):
- CI (.github/workflows/ci.yml): job "test" on push/PR/dispatch ? matrix ubuntu {3.11,3.12,3.13}
+ windows-latest 3.13; steps: pip install -e .[dev], ruff check, ruff format --check, mypy,
pytest --cov=muneem --cov-fail-under=80. Job "ocr" (Tesseract, pytest -m ocr) runs ONLY on
workflow_dispatch.
...
</persisted-output>

21. user (2026-06-10T16:33:44.045Z)

<system-reminder>[Truncated: PARTIAL view ? showing lines 1-133 of 163 total (25884 tokens, cap 25000). Call Read with offset=134 limit=133 for the next page, or Grep to find a specific section. Do NOT answer from this page alone if the answer may be further in the file.]</system-reminder>

1
2 ########## muneem-tests-ci ##########
3 SUMMARY: muneem's test/CI architecture is genuinely strong for a pre-1.0 personal project: the offline suite (390 tests) passes in 6.35s at 86.17% coverage against the 80% CI gate, CI really does run ruff+mypy+pytest on a Linux 3.11-3.13 + Windows 3.13 matrix, the schema-lock and plaintext-downgrade tripwires exist and work, the license allowlist is enforced as a test, and sensitive paths are defended in depth (gitignore default-deny, tested pre-commit guard, repo-hygiene backstop; git history verified clean). The biggest gaps: the key-rotation CLI's failure/rollback path has zero test coverage and its "keychain-first" ordering contradicts its own docstrings (a crash mid-rotation orphans the ledger key); real parsed credit-card transactions sit in plaintext JSON in testdata/snapshots/, written by a now-dead fixture; the real-statement test layer has no CI story at all today (Layer-2 self-hosted runner is design-only, local tests skip silently without passwords); synthetic full-pipeline fixtures cover only 2 of 4 parser families with idealized 3-row single-page layouts; and Google's default OAuth filename pattern (client_secret_*.json) is not blocked by any guard outside testdata/.
4
5 MAP: ARCHITECTURE (verified by reading + running):
6 - CI (.github/workflows/ci.yml): job "test" on push/PR/dispatch ? matrix ubuntu {3.11,3.12,3.13} + windows-latest 3.13; steps: pip install -e .[dev], ruff check, ruff format --check, mypy, pytest --cov=muneem --cov-fail-under=80. Job "ocr" (Tesseract, pytest -m ocr) runs ONLY on workflow_dispatch.
7 - Local gates (.githooks/, active on the dev box: core.hooksPath=.githooks): pre-commit runs precommit_check.py (blocks staging PDFs, anything under testdata/ except promo-emails|synthetic, DBs, key material, password_rules.*, .env, recovery files); pre-push runs ruff+format+mypy+full offline pytest (no coverage gate). Hooks are the substitute for branch protection (private repo, none available). Makefile is Linux/macOS-only convenience.
8 - pytest config (pyproject): addopts deselect markers ocr/llm/local_statements ? default suite is offline/deterministic/secretless. mypy covers src/muneem but ingestion.* has ignore_errors=True (documented as temporary scaffolding).
9 - Test tiers: (1) offline default ? unit + synthetic full-pipeline (tests/test_regression_synth.py + tests/synthpdf.py: stdlib hand-rolled born-digital PDFs with Helvetica metrics for right-aligned amount columns, run through real pdfminer/pdfplumber?parser?reconcile; covers HDFC savings + AU savings layouts only); (2) ocr marker ? promo corpus T2, manual-dispatch CI only; (3) local_statements marker ? tests/parsers/test_local_axis.py, test_local_au.py, plus local tests in test_hdfc.py and test_unlock.py run real encrypted statements from gitignored testdata/{axis,aubank,hdfc}-statements/; they skip unless a password is available (password_rules.yaml or MUNEEM_TEST_*_PW env); never in CI; ROADMAP Layer-2 self-hosted runner is designed but NOT implemented; (4) llm marker ? never in CI.
10 - Tripwires: tests/test_db_invariants.py pins EXPECTED_SCHEMA_VERSION=5 + exact table/column shapes (explicit-allow = edit the constants in the same PR), plus security invariants: CLI ingest must write an encrypted ledger (is_plaintext_sqlite check), keyed open must raise (never plaintext-fallback) when sqlcipher3 missing, FK enforcement/cascade, sha256 dedup unique. tests/test_repo_hygiene.py asserts git ls-files contains no sensitive paths/PDFs outside promo-emails. tests/test_dependency_licenses.py allowlists every pyproject dep (MIT/BSD/Apache/Zlib/HPND) and bans GPL/AGPL substrings.
11 - Verifier fuzz: tests/test_verifier_properties.py ? 180 seeded-random cases (consistent ledgers accepted; single perturbation past tolerance caught by balance walk and total oracle). Deterministic, no hypothesis dep.
12 - Isolation: tests/conftest.py autouse _isolate_keychain replaces muneem.secrets

get/set/delete with an in-memory dict ? the real OS keychain (incl. DPAPI on the Windows runner) is NEVER touched by any test anywhere. conftest also defines a `snapshot` fixture (writes/reads testdata/snapshots/*.json, UPDATE_SNAPSHOTS=1 to regenerate) that NO test currently uses.
13   - RUN RESULT (C:/Users/avidu/Projects/muneem/.venv, python 3.13.13, CI invocation): 390 passed, 21 deselected (ocr + local_statements incl. parametrized real-PDF ids), 6.35s, coverage 86.17% (gate 80% passed). Per-module: cli.py 70%, ingestion/outlook.py 38%, ingestion/imap.py 50%, ocr.py 57%, parsers/base.py 58%; parsers/db/migrations/validation all 92-100%.
14   - Sensitive-data ground truth (existence/tracking only): testdata/{aubank,axis,hdfc}-statements (4/7/7 PDFs), testdata/credentials/ (one real client_secret_*.apps.googleusercontent.com.json), testdata/snapshots/ (two JSONs that ARE real parsed CC transactions ? list of rows keyed account_number/amount/merchant_category/particulars/txn_date/type; 12 and 28 rows) ? ALL untracked, gitignored, and `git log --all` over those paths is empty (never committed). Only promo-emails PDFs are tracked.
15
16   STRENGTHS:
17   * The offline suite is real and fast: 390 tests, 6.35s, genuinely offline/deterministic/secretless (keychain autouse-isolated, no network, seeded RNG, tmp_path everywhere) ? verified by running it; 86.17% coverage clears the 80% CI gate.
18   * CI matrix matches the claim: Linux 3.11/3.12/3.13 + Windows 3.13 (the dev platform, exercising the sqlcipher3 win wheel and Windows paths), with ruff lint+format, mypy, and the coverage gate all actually wired.
19   * Defense-in-depth on sensitive data actually holds: .gitignore default-denies testdata/* with explicit allowlist, the pre-commit guard is itself unit-tested (32 tests via PRECOMMIT_TEST_FILES injection, portable, no git mutation), test_repo_hygiene.py is the post-commit backstop, and git history is verifiably clean of statements/credentials/snapshots.
20   * The DB tripwires are well designed: schema lock with an explicit-allow mechanism (edit EXPECTED_SCHEMA in the same PR), plus always-hold invariants ? CLI ingest writes a genuinely encrypted ledger end-to-end, keyed opens never silently downgrade to plaintext, FK cascade and sha256 dedup enforced.
21   * tests/synthpdf.py is a clever zero-dependency solution: hand-rolled valid PDFs with real Helvetica metrics so right-aligned amount columns land in the clusterer's bands, letting the full pdfminer/pdfplumber ? clustering ? engine ? reconcile pipeline run in cloud CI with no committed statements.
22   * The verifier (the project's truth oracle) gets 180 seeded property-fuzz cases including single-perturbation detection ? proportionate rigor for the component whose failure means silently wrong financial data.
23   * Local-only real-statement tests assert the right invariant (store-nothing or store-reconciled, surfacing only counts/booleans/parser names, never statement content), and pytest output discipline keeps contents out of logs.
24   * Leak-prevention is tested as behavior, not policy: redaction filters, ParseError wrapping (no path/PII digits in messages), encryption key never in error text, recovery file leaks neither key nor passphrase.
25   * License allowlist enforced as a failing test synced to pyproject (every dep must be allowlisted; GPL/AGPL substrings banned), directly encoding the commercial-safety memory rule.
26   * The pre-push hook runs the full local gate (ruff, format, mypy, offline suite) as a deliberate substitute for unavailable branch protection, with a documented --no-verify escape hatch, and it is actually enabled on the dev machine.
27
28   ISSUES:
29   [high] Key-rotation failure path has zero test coverage and the implementation contradicts its own documentation
30   muneem db rotate-key sets the NEW key in the keychain FIRST, then rekeys the file. The cli.py comment claims 'Keychain-first with rollback: if the file rekey fails, the keychain still matches the file' ? backwards: between set_key(fresh) and a successful rekey, the keychain does NOT match the file. A crash, Ctrl-C, or any exception other than EncryptedDatabaseError in that window leaves the ledger under the old key with only the new key stored ? locked out unless the recovery file exists. db_key.set_key's docstring says it is used 'after the file has been rekeyed', which the code does not do. Coverage confirms the except/rollback branch (cli.py lines 443-446) is never executed by any test; only the happy path (test_db_rotate_key_cli) and the no-ledger refusal are tested. The rollback set_key(current) call can itself fail (keychain error) with no defined behavior.
31   [medium] Real parsed credit-card transactions sit in plaintext JSON on disk, written by a now-dead snapshot fixture
32   testdata/snapshots/ contains two JSON files that are lists of real parsed CC

transaction rows (12 and 28 rows; keys: account_number, amount, merchant_category, particulars, txn_date, type ? values not inspected). They are gitignored, untracked, and verifiably never committed, so there is no repo leak ? but they are cleartext on the same disk the project encrypts the ledger to protect, directly undermining the at-rest posture. The `snapshot` fixture in tests/conftest.py that produced them is no longer used by ANY test (grep confirms only conftest references), yet it remains as a loaded footgun: UPDATE_SNAPSHOTS=1 writes real parsed data to a cwd-relative path with no redaction.
33 [medium] No CI story for real statements today; local_statements tests skip silently and nothing tracks freshness
34 The Layer-2 self-hosted runner (ROADMAP § Layer 2) is design-only ? no .github/workflows/real-statements.yml exists. The local_statements tests skip unless a bank password is resolvable, the default pytest addopts exclude them, and the pre-push hook runs the default suite only ? so a parser regression against real statement vintages (the product's actual job) is detected only if the developer remembers to run `pytest -m local_statements` manually. There is no record of when they last passed, and a skip looks identical to never-run.
35 [medium] Migration runner has an acknowledged-but-untested partial-failure window
36 migrations.py applies each migration via executescript (which commits implicitly) and records it in schema_migrations afterwards; the module docstring admits a crash between DDL and recording 'could require manual cleanup' and defers per-migration transactions. No test exercises a migration failing mid-script (partial DDL applied, nothing recorded) or failure between apply and record ? test_migrations.py covers baseline, idempotency, back-fill, forward apply, drift detection, and encrypted connections, all happy-path or clean-rejection. With v5 (provenance) now shipped and more migrations planned (unified schema P3), this window is live on an encrypted production ledger where 'manual cleanup' is harder.
37 [medium] Synthetic Layer-1 fixtures are far cleaner than real statements and cover only 2 of 4 parser families
38 test_regression_synth.py builds single-page PDFs with exactly 3 perfectly-aligned rows. There is no multi-page table (repeated headers, page-break row splits), no wrapped/multi-line narration, no merged or interleaved columns at the real-PDF level (those are tested only on hand-built cell matrices via pdfplumber mocks in test_savings/test_axis), no password-protected PDF through the real pipeline (unlock is tested separately with pypdf-made blanks; CLI ingest password flow is monkeypatched), and no Axis savings or Axis CC synthetic fixture at all ? only HDFC and AU layouts route through the full pdfplumber path in CI. The real-world failure the suite itself names (the Axis May 2025 dropped-row case) is reproduced only at matrix level, not as a born-digital PDF.
39 [medium] OCR tier is effectively unmonitored: the CI job only runs on manual dispatch
40 The ocr job in ci.yml is gated on `github.event_name == 'workflow_dispatch'`, so the T2 image-code tier (about half the promo corpus per docs/TESTING.md § 2) and ocr.py (57% line coverage in the default suite) get no automatic regression signal ? a Tesseract-interaction or ocr.py regression ships silently unless someone remembers to trigger the job. No schedule trigger exists.
41 [medium] Guard rails miss Google's default OAuth client filename pattern outside testdata/
42 The real OAuth client file on disk is named client_secret_<id>.apps.googleusercontent.com.json ? Google's default download name. Inside testdata/ it is protected by the blanket testdata rules, but if it were copied to the repo root or src/ (a common tutorial pattern), nothing blocks it: .gitignore lists only credentials.json/token.json/token.pickle/oauth_token*, precommit_check.py SECRET_NAMES has only 'credentials.json', and test_repo_hygiene FORBIDDEN_NAMES likewise. A `git add -A` of a root-level client_secret_*.json would pass all three guards. (Also minor: FORBIDDEN_PREFIXES in test_repo_hygiene references stale dirs testdata/statements/ and testdata/private/ that do not exist, though the broader noncorpus-testdata rule compensates.)
43 [low] Aggregate 86% coverage masks weak spots: CLI error paths 70%, ingestion 38-76% and double-exempted
44 The 80% gate measures the right package (--cov=muneem resolves to src/muneem via the editable install ? verified in the run output) but is aggregate-only. Untested regions include the rotate-key rollback (above), db backup error branch (cli.py 462-464), recovery CLI partial paths (486-497, 528-536), offers OCR-merge branch (258-270), offers/txns list formatting (307-361), and the unlock interactive paths. ingestion/outlook.py (38%) and imap.py (50%) are simultaneously exempt from mypy (ignore_errors=True) ? scaffolding with neither type nor behavioral pressure, yet its lines inflate denominator dynamics of the single gate. Erosion in cli.py can be offset by well-covered parser code without tripping anything.
45 [low] The real OS keychain backend is never exercised anywhere ? including on the Windows CI runner
46 The autouse _isolate_keychain fixture (correct for hygiene) means no test on any

platform ever touches a real keyring backend; test_secrets.py drives the wrappers by mocking the keyring library. So the production path ? DPAPI via keyring on the dev box, get_or_create_key minting on first run, keychain-unavailable behavior on headless setups ? has zero integration coverage; its first real failure will be discovered with real data at stake. The Windows CI matrix cell, justified in ci.yml partly by 'DPAPI secret backend', does not actually test DPAPI.
47 [low] Pytest IDs of local tests embed real statement filenames
48 test_local_axis.py parametrizes with ids=lambda p: p.name, so collected test IDs include real statement filenames (bank + statement month). These appear in local pytest/collection output and would appear in any future Layer-2 runner logs. Filenames only ? no account data ? but the project's own redaction discipline (redact_filename masks such names in errors/logs) is bypassed by test IDs.
49
50 RECOMMENDATIONS:
51 [short-term] Test and harden the rotate-key failure window: Add CLI tests that monkeypatch rekey_database to raise (a) EncryptedDatabaseError ? assert the keychain is rolled back to the old key ? and (b) a generic exception/KeyboardInterrupt ? decide and assert the behavior. Fix the ordering or the docs: either rekey-the-file-first then set_key (matching db_key.set_key's docstring), or keep both keys reachable during rotation (e.g. store the outgoing key under a db-key-previous entry until the rekey verifies) so no crash window can orphan the ledger. Also define behavior when the rollback set_key itself fails.
52 [short-term] Remove the dead snapshot fixture and securely delete the real-data snapshots: Delete the unused `snapshot` fixture from tests/conftest.py (no test uses it) and remove the two real-transaction JSON files from testdata/snapshots/. If parse-snapshotting of real statements is wanted later, redesign it to store only non-sensitive summaries (counts, reconcile booleans, hashes) or write into the encrypted ledger, never cleartext JSON.
53 [short-term] Block the client_secret_*.json filename pattern everywhere: Add client_secret*.json (and *.apps.googleusercontent.com.json) to .gitignore, to a pattern check in .githooks/precommit_check.py (alongside SECRET_NAMES), and to tests/test_repo_hygiene.py FORBIDDEN_NAMES ? Google's default download name is the most likely real-world filename for this secret. While there, update the stale FORBIDDEN_PREFIXES to the directory names actually in use.
54 [short-term] Add a migration partial-failure test and per-migration transactions: Write a test with a two-statement migration whose second statement fails, asserting the DB is left in a defined state (ideally fully rolled back and unrecorded). Wrap each forward migration's DDL + schema_migrations insert in one transaction (the module docstring already flags this as the known limitation) before migration v6 lands ? closing it is cheap now and expensive after a production ledger hits the window.
55 [short-term] Broaden the synthetic Layer-1 corpus: Extend tests/test_regression_synth.py with: an Axis savings and an Axis CC born-digital fixture (so all four parser families route through real pdfplumber in CI), a multi-page statement with repeated headers, a wrapped two-line narration row, a dropped-row fixture reproducing the Axis May 2025 must-not-reconcile case at the PDF level, and a password-protected synthetic PDF pushed through the actual `muneem ingest` CLI (synthpdf could gain RC4/AES encryption via pypdf, already a dependency).
56 [short-term] Put the OCR job on a schedule: Add `schedule: cron` (e.g. weekly) to the ocr job's triggers in ci.yml so the T2 tier and ocr.py get automatic regression signal instead of relying on someone remembering workflow_dispatch.
57 [long-term] Implement the Layer-2 real-statement runner (or an interim freshness mechanism): Stand up the decided-but-deferred self-hosted runner (ROADMAP § Layer 2) running `pytest -m local_statements` on pushes. Until then, add a low-tech freshness signal: a local scheduled task or a `make test-local` habit that runs the local tier and writes a last-green timestamp the default suite can warn about (e.g. a non-failing report when real-statement tests haven't passed in N days), so silent skips can't quietly become months of untested real-vintage parsing.
58 [long-term] Split the coverage gate so the aggregate can't hide erosion: Keep the 80% global gate but add per-area floors ? e.g. fail-under on muneem.parsers+db+migrations (currently ~95%) at a high bar, a separate explicit (lower) expectation for cli.py error paths, and either exclude muneem.ingestion from coverage until the fetch command lands or, when it does, remove both the mypy ignore_errors override and the coverage slack in the same PR.
59 [long-term] Add an opt-in real-keychain integration tier: Introduce a `keychain` marker (mirroring ocr/local_statements) with a handful of tests that exercise the real keyring backend ? get_or_create_key mint/reuse, set/delete, rotation end-to-end against DPAPI ? run manually on the dev box and later on the self-hosted runner, using a dedicated test service name so the production db-key entry is never touched.
60 [long-term] Extend property fuzzing beyond the verifier: The seeded-fuzz pattern in test_verifier_properties.py is cheap and effective; apply it to the clustering and engine layers

(random column geometries, jittered x-positions, randomized row counts feeding synthpdf) so the layout-inference code ? the most likely source of silent mislabeling after the verifier ? gets the same adversarial pressure. Consider hypothesis (MIT, allowlist-compatible) if the hand-rolled generators grow unwieldy.
61
62 ########## muneem-docs ##########
63 SUMMARY: muneem's doc suite is unusually honest and mostly code-accurate at the operational layer (schema.md, ingesting-statements.md, encryption-at-rest.md match the code line-for-line; Outlook/IMAP are marked experimental in code exactly as promised; sensitive testdata is untracked and was never in git history). The debt is at the strategic layer: (1) sequencing authority is split between ROADMAP.md's Milestones/Tracks and feedbackJune2026 §4's Phases A?H, with colliding letter namespaces and neither doc fully owning the plan; (2) the designated "live status" doc (CURRENT_STATE.md) and parser-architecture.md both lag shipped reality (schema v5/provenance, Axis 6/7); (3) Phase C (instrument family) is the riskiest, least-specified phase ? no broker corpus evidenced, no instrument reconciliation oracle designed, sequenced before the regression-CI safety net the project itself decided it needs; (4) the roadmap has no phase delivering daily user value (query/reporting), no cash/manual-entry plan, no multi-year backfill/archive strategy, and a stale v1 definition of done; and (5) a head-on, unacknowledged contradiction with the sister repo gmail-attachments-to-gdrive, whose three-day-old "binding auditability invariant" declares muneem must never hold the Gmail grant ? while muneem already ships full Gmail OAuth ingestion and plans `muneem fetch` (Phase F), and bank-statement PDFs are PII that cannot flow through the sister's "non-PII minimized signal" contract anyway. The still-open decisions (log format, self-hosted runner, macOS) are genuinely still open in code ? no silent locks found, with one partial lapse around the L3 model choice never being back-recorded in DESIGN §8.3/ROADMAP §1.
64
65 MAP: PROJECT: muneem (C:/Users/avidu/Projects/muneem), Python 3.11+, single-user local-first personal-finance bookkeeper for Avi (Bangalore; Indian banks/brokers). Successor to ~/Projects/Email-Mining (promo-code extractor prototype).
66
67 VISION (DESIGN.md): ingest financial docs from email -> unlock password PDFs -> parse -> normalized local SQLite -> analytics over the whole financial life (groceries price-intel, portfolio timeline, bank/card comparison, item catalog, forgotten deals). Five-stage pipeline: (1) Email ingestion (Gmail/Outlook APIs, OAuth read-only), (2) Attachment unlocking (passwords derived from user attributes), (3) Document parsing ("the hard 80%": template parsers + text-layer heuristics + OCR + LLM fallback), (4) Normalization & storage (SQLite; documents/accounts/transactions/holdings/merchants/items/offers), (5) Analytics (CLI first). Security posture: everything local; no cloud LLM on sensitive categories without per-session approval; SQLCipher encryption at rest (implemented); permissive licenses only (PyMuPDF/AGPL banned); no paid APIs by default. Non-goals: SaaS, cloud sync, portal scraping, real-time.
68
69 OPEN DECISIONS (DESIGN §8 + feedbackJune2026 §5): 8.1 language=RESOLVED Python-only (2026-05-30). 8.2 passwords=RESOLVED hybrid dictionary->template->prompt (implemented in `muneem ingest`). 8.3 LLM locality for sensitive docs=OPEN (but the L3 vision model is researched-and-chosen in parser-architecture.md: DeepSeek-OCR-3B MIT, 4-bit+SDPA on RTX 2070S, build deferred ? not back-recorded in DESIGN §8.3, and ROADMAP §9 still says "unconfirmed"). 8.4 first slice=RESOLVED both tracks parallel, HDFC first. 8.5 merchant canonicalization=OPEN (parked until real receipt data). 8.6 Account Aggregator=RESOLVED 2026-06 deferred/PDF-first (research-open-banking-india.md: FIU regulatory gate; TSPs decrypt in cloud = net privacy regression; SMS-alert parsing surfaced as local freshness complement). 8.7 storage shape=RESOLVED 2026-06 flat per-issuer source-of-truth tables + derived serving layer + separate provenance table keyed by document_id. feedback §5 still-open: log format (logfmt vs JSON), self-hosted-runner acceptance, macOS first-class. Verified in code: none silently locked (log.py has no formatter; only ci.yml exists, ubuntu+windows matrix, no real-statements.yml, no macOS).
70
71 SEQUENCING DOCS: ROADMAP.md (created 2026-05-30) tracks Milestone 0 (done), Track A promo spine (done), Track B Gmail ingestion (OAuth+search+download done; `muneem fetch` B.4 NOT built), Milestone C HDFC slice (parsers+unlock done; unified-schema C.3 pending), Milestone D harden/evaluate (D.2 encryption done; D.1 logging and D.3 eval writeup pending), §6 generalize backlog, plus a "Planned ? Regression CI" section (two-layer: synthetic cloud CI [Layer 1 exists: tests/synthpdf.py + test_regression_synth.py, all-branch triggers live] + self-hosted runner over real statements [decided, NOT built]). feedbackJune2026.md §4 (approved 2026-06) defines a DIFFERENT phase plan: A foundation/recovery (recovery #43, migrations #44, Windows CI #42, Outlook/IMAP-experimental #41 done; lockfile and structured-logging NOT done) -> B provenance table + serving view (provenance table shipped #46; serving view NOT built) -> C

instrument family (equity/MF/ETF/bond) -> D regression CI Layer 2 + D.3 eval -> E L3 vision -> F `muneem fetch` -> G merchant canon/Tier-2 -> H ops surface (verify/export/filters) + SBOM. CURRENT_STATE.md is named the live status doc; its §3 still says "Schema v4" and calls a unified table "with per-row provenance/confidence ... the planned direction once the design decision lands" ? both superseded by the §8.7 decision and the shipped v5 provenance migration.
72
73   CODE STATE (verified): CLI = info, ingest (promo + --doc-type bank_statement with hybrid unlock, --ocr, --no-prompt), offers list, txns list {hdfc,axis,axis_cc,au}, eval <folder> (reconcile-rate report), db {status,encrypt,rotate-key,backup,setup-recovery,recover}. No fetch. db.py: SCHEMA_VERSION=4 baseline (documents, offers, axis/axis_cc/au/hdfc txn tables) + migrations.py v5 provenance (document_id UNIQUE FK, source_type/parser/parser_version/mapping_layer/confidence) ? schema.md mirrors this exactly. record_parse/record_provenance called from hdfc.py/axis.py/au.py. Parser core: concepts.py + engine.py (FamilySpec-parameterized L0 lexicon -> profile order -> L2 content search, oracle-selected) + profiles.py (AXIS/AUBANK/HDFC as data) + savings.py shared path + validation.py verifier ? matches parser-architecture.md §§3?6. Real-data precision per ROADMAP/CURRENT_STATE: HDFC 6/6, AU 4/4, Axis 6/7 (May 2025 quarantined, stores nothing); parser-architecture.md header/§10/§10a still say Axis 2/4 and HDFC 4/4 (stale). Axis credit-card parser unverified on real data. Ingestion: gmail.py full OAuth2 gmail.readonly (token in keychain) + tools/gmail_auth.py; imap.py/outlook.py carry "EXPERIMENTAL / INCOMPLETE" docstrings as promised. Hygiene: testdata/* deny-by-default gitignore (only promo-emails/ and synthetic/ whitelisted); real statement dirs (hdfc/axis/aubank-statements), snapshots/ (2 credit-card JSON snapshots), credentials/ (1 real OAuth client JSON) exist locally, are untracked, and `git log --all` over those paths is empty (never committed); precommit hook + test_repo_hygiene.py enforce (the hook docstring references an earlier statement-leak near-miss as motivation). testdata/synthetic/ contains no committed files ? synthetic fixtures are generated at test time, contradicting TESTING.md §7 / onboarding-a-bank.md §3 which say they are committed.
74
75   SISTER REPO (C:/Users/avidu/Projects/gmail-attachments-to-gdrive): Apps Script attachment archiver pivoting to an ingest->classify->extract->redact->handoff pipeline. docs/VISION.md + ARCHITECTURE.md (codified 2026-06-03, PR #14, "binding auditability invariant" / ADR-008): that repo is THE ONLY code that touches Gmail; "Muneem and every downstream service operate only on minimized, non-PII signal and never hold the Gmail grant"; Muneem is framed as "our own isolated, hosted service"; handoff contract/schema is explicitly TBD ("define jointly with avidullu/muneem") and handoff is its Phase 3 (unbuilt); per-vertical "non-PII" definition for bank statements is an acknowledged open item. muneem's docs contain ZERO references to this repo, the invariant, or any handoff.
76
77   PRIOR-ART LESSONS (prior-art-email-mining.md): keep the pipeline shape (page text + per-image OCR -> unified text -> structured extraction), image dedup MD5+SSIM, the promo corpus (now testdata/promo-emails/, 10 PDFs + expected.yaml answer key); drop manual save-as-PDF, cloud-Gemini-by-default, one-shot stdout, no-tests/no-schema; PyMuPDF is AGPL and banned (pdfminer.six/pypdfium2 instead). TESTING.md grounds tiers T1?T4 in actual corpus inspection (ligatures, image-only codes, relative expiry, filename?issuer, third-party recipient PII noted honestly).
78
79   STRENGTHS:
80   * Operational docs are code-accurate: schema.md matches db.py/migrations.py DDL column-for-column including the v5 provenance migration (PR #46); ingesting-statements.md matches cli.py behavior and exact error strings; encryption-at-rest.md matches the implemented db subcommands (status/encrypt/rotate-key/backup/setup-recovery/recover) and states the threat model honestly ('does not defend against malware running as the logged-in user').
81   * Promises kept in code: Outlook/IMAP are explicitly marked '?? EXPERIMENTAL / INCOMPLETE' in module docstrings exactly as feedbackJune2026 P7/Phase A required (PR #41); the migration runner is hand-rolled (not Alembic) per the explicit push-back; provenance is a separate document_id-keyed table per the §8.7 decision, not per-row.
82   * Open-decision discipline largely holds: log format (no formatter in log.py), self-hosted runner (no real-statements.yml workflow), and macOS (CI matrix is ubuntu+windows only) are all verifiably still open in code ? nothing silently locked.
83   * Sensitive-data hygiene is defense-in-depth and verified: real statement dirs, snapshots, and the OAuth client are untracked and absent from all git history; .gitignore is deny-by-default on testdata/*; a pre-commit hook plus test_repo_hygiene.py (which is actually stronger than TESTING.md describes ? it forbids ALL non-corpus testdata and ALL non-corpus PDFs) enforce it as failing builds.
84   * Honest-status culture: ROADMAP openly flags its own checkbox rot and points to

CURRENT_STATE; TESTING.md marks 6 of 10 answer-key rows 'unverified' rather than inventing ground truth and discloses the third-party recipient PII in the corpus; research-open-banking-india.md labels every claim FACT / note-sourced / INFERENCE.
85 * feedbackJune2026.md is a model review-response artifact: it corrects the reviewer's stale premises with PR citations, pushes back with reasons (Alembic, fetch-before-storage, L3-before-eval), and catches the genuinely highest-stakes gap itself (key-loss makes the ledger AND all backups unrecoverable ? fixed as P0 in #43).
86 * The verification-first parser architecture ('extraction proposes, reconciliation disposes') is both well-documented and faithfully implemented: engine.py/profiles.py/validation.py match parser-architecture.md §§3?6, and the oracle-gating claim ('stores nothing rather than mislabeled rows') is real in the Axis May-2025 case.
87
88 ISSUES:
89 [high] Unreconciled head-on contradiction with sister repo gmail-attachments-to-gdrive over who touches Gmail
90 The sister repo's binding auditability invariant (docs/VISION.md + ARCHITECTURE.md ADR-008, codified 2026-06-03) states it is THE ONLY code that ever touches a user's Gmail and that 'Muneem ... never hold[s] the Gmail grant', receiving only 'minimized, non-PII signal'. muneem already violates this as a statement of fact: src/muneem/ingestion/gmail.py implements full gmail.readonly OAuth (token cached in keychain), tools/gmail_auth.py drives consent against a real OAuth client in testdata/credentials/, ROADMAP Track B marks OAuth+search+download done, and 'muneem fetch' is planned (ROADMAP B.4 / feedback Phase F). Neither repo references the other's constraint ? muneem's docs contain zero mentions of the sister repo. Two further axes: (a) the sister VISION frames Muneem as a HOSTED service, while muneem's DESIGN declares 'Multi-user / SaaS / hosted product' a non-goal and forbids cloud sync of raw data, period; (b) bank-statement PDFs are inherently PII (names, account numbers, transactions), so they categorically cannot flow through the sister's 'non-PII minimized signal' egress contract ? the sister repo itself lists 'per-vertical non-PII definition for bank statements' as unresolved, and the handoff contract/transport is TBD (its Phase 3, unbuilt). Reconciliation options: (1) muneem keeps its own ingest ? costs: breaks the sister invariant (needs explicit scoping, e.g. 'the invariant binds the public product, not the owner's personal instance'), duplicates Gmail sync/dedup logic, two OAuth surfaces; benefits: zero cloud processing of statements, full raw-PDF fidelity, passwords stay local, no Apps Script quota fragility. (2) muneem consumes the handoff ? costs: blocked on an undefined contract several phases away, the PII problem above, statement classification running in Google's cloud runtime, and password-protected PDFs cannot be unlocked in-account without putting password material into Apps Script (a real regression); benefits: single auditable Gmail surface, muneem drops google-api deps. (3) The unexplored middle: the sister archives raw attachments to Drive in-account (no egress, invariant-intact ? archiving is already its hardened core), and muneem ingests from the locally-synced Drive folder; the Gmail grant stays solely with the sister, unlock stays local, no handoff contract needed ? at the cost of a Drive-sync dependency and replacing historyId sync with folder watching. Whichever is chosen, today the sister VISION is stale as a description of reality, and muneem's Phase F is in unflagged conflict with a document calling itself binding.
91 [high] Two competing sequencing sources of truth with colliding namespaces (ROADMAP Milestones/Tracks vs feedbackJune2026 Phases A-H)
92 ROADMAP.md declares 'this doc owns sequencing', but the actually-current plan is feedbackJune2026 §4's Phases A-H ? a different ordering vocabulary where the letters collide ('Track A' = promo spine vs 'Phase A' = foundation/recovery; ROADMAP 'Milestone C/D' = HDFC slice / harden vs feedback 'Phase C/D' = instrument family / regression CI). ROADMAP's checkbox lists are self-admittedly rotten (B.1-B.3 unchecked though done), its §6 'Generalize' backlog overlaps Phases C/E/F/G without mapping, and its §10 v1 definition of done predates the whole-finance rescope. For a project whose docs are explicitly the memory across sessions, a restart cannot tell which plan governs without reading four docs in the right order and mentally diffing them.
93 [high] Phase C (instrument family) is the riskiest, most under-specified phase ? and is sequenced before the regression-CI safety net
94 Phase C is the next big build after B, but: (a) no broker/MF/CAS corpus is evidenced anywhere ? testdata holds only hdfc/axis/aubank statements, while the bank parsers' entire credibility came from '6 real statements + passwords' validation; (b) the instrument reconciliation oracle is undesigned ? the engine is FamilySpec-parameterized in code, but no doc states what arithmetic identity replaces the balance walk for holdings statements (units walk? units x NAV = value? CAS statements often lack a clean per-row identity, and dividend/fee classification has no oracle at all), which is exactly the load-bearing component of the whole 'verification-first' philosophy; (c) which artifact is the source (CDSL/NSDL CAS vs broker statement vs tradebook export) is neither chosen nor parked as an open decision; (d) Phase D

(regression CI Layer 2 + the D.3 eval writeup) comes AFTER C ? yet feedbackJune2026 itself argued 'characterize failure modes first, don't build blind' to justify D.3-before-L3, and the same logic applies to building a whole new family before the safety net exists; (e) cheaper unfinished work is skipped over: the Axis credit-card parser is still unverified on real data and HDFC CC bills (the other half of the first slice) are absent.
95 [high] The roadmap delivers no daily user value: no query/reporting surface in any phase, and Phase B's serving view silently went missing
96 Every DESIGN §2 use-case (price intelligence, portfolio timeline, bank/card comparison, item catalog) is the product's point, yet the only read surface after ~46 PRs is 'muneem txns list <bank>' (last 50 raw rows per single bank) and 'offers list'. Cross-issuer querying was explicitly promised as part of Phase B ('Phase B = the provenance table + a serving view') ? the provenance table shipped in #46 but no SQL view exists anywhere in src/ or tests/, and no tracker records the gap; feedback §5 marks decision 2 as settled with 'Phase B implements the provenance table', quietly dropping the view. Filters/analytics/export are deferred to Phase H, dead last. For a single-user tool, a weekly-usable 'show me last month across all accounts' query is what makes the user keep feeding it data through the long Phase C-G middle.
97 [medium] Whole categories missing from the roadmap: cash/manual entry, multi-year backfill & raw-PDF archive strategy, quarantine UX, restore runbook, refreshed definition of done
98 (1) Cash/manual data entry: migration v5 already reserves source_type='manual', but no phase plans an entry/correction path ? a personal ledger without cash spends can never reconcile with reality. (2) Multi-year archive strategy: nothing covers backfilling years of Gmail history (query design, quota, ordering), what happens to documents.path when staged files move, or the retention policy for raw locked PDFs in the staging dir (SQLCipher protects the DB; the PDF staging area is protected only by optional full-disk encryption ? undiscussed). (3) Quarantine UX: parser-architecture §11's 'what does the CLI do with a quarantined statement' is the user-facing handling of the real 1/7 Axis failure and is unowned by any phase. (4) feedbackJune2026 §2 prescribed a docs/recovery.md runbook ('keychain lost', 'new machine', 'verify a backup') and P6 a 'migrate then re-verify' runbook ? neither exists; encryption-at-rest.md covers fragments. (5) ROADMAP §10's v1 'definition of done' is still the old HDFC-slice wording and no phase has exit criteria, so 'done' for the whole-finance vision is undefined. (6) SMS-alert parsing was surfaced as a decided-worthy candidate (P11) but is assigned vaguely to 'Phase A (optional) / Phase F' with no owner.
99 [medium] CURRENT_STATE.md ? the designated live-status doc ? is wrong about the DB layer it points restarts at
100 Its §3 says 'Schema v4' and that 'a unified transactions table ... with per-row provenance/confidence is the planned direction once the design decision lands'. Reality: the decision landed (2026-06, DESIGN §8.7: flat per-issuer tables are permanent by design, provenance is a separate table, explicitly NOT per-row) and partially shipped (migration v5 provenance, PR #46). A restart reading CURRENT_STATE would re-litigate a settled decision in the opposite direction and miss that the schema is at v5. Everything else in the doc (CLI list, parser precision, ingestion status) verifies as accurate against code.
101 [medium] parser-architecture.md lags shipped reality the most among design docs: stale precision numbers and a superseded data-model section
102 Header and §10 step 3 say 'Axis 2/4' and §10a says 'HDFC 4/4 and AU 4/4 qualify today' ? reality is HDFC 6/6, AU 4/4, Axis 6/7 (June 2025 recovered via balance-delta reconstruction, which this doc never mentions). §9 'Data-model evolution' still points at 'a unified transactions table ... plus provenance + confidence columns' ? superseded by the §8.7 decision (flat per-issuer is permanent; provenance is a separate table); §11's 'storage unification timing' open question is answered but unannotated. Since this is the doc the L3/instrument-family work will be built against, the stale direction in §9 is the kind of drift that gets silently re-implemented.
103 [medium] Open-decision discipline lapse around §8.3: the L3 model was decided but never back-recorded, leaving DESIGN and ROADMAP contradicting parser-architecture
104 DESIGN §8.3 (LLM locality/model for sensitive categories) carries no resolution note, and ROADMAP §9 still says 'Ollama + Llama/Qwen/Mistral is the likely shape, but unconfirmed' ? while parser-architecture.md §4 states 'The research resolved the model: DeepSeek-OCR-3B (MIT)' with hardware specifics, and the auto-memory records it as decided. This is a partial resolution (the model is chosen, the locality rule is unchanged, the build is deferred), but the project's own decision-log discipline (ROADMAP §7: 'append resolutions to §1 and mark the corresponding DESIGN.md §8 item resolved') was followed for 8.1/8.2/8.4/8.6/8.7 and skipped here. A restart obeying 'do not pick silently' would either re-research the model or wrongly treat DeepSeek as unapproved.
105 [low] Phase A declared 'foundation first' but is quietly incomplete while Phases B/C proceed

Of Phase A's five items, two are unfinished: no dependency lockfile exists (uv.lock / requirements*.txt absent from the repo, despite P10 marking it 'Phase A' and 'reproducibility is real for a tool with a compiled dep (sqlcipher3)'), and structured-logging adoption (P1) is stalled on the open log-format decision ? log.py is a redaction filter with no formatter and adoption beyond cli/parsers is thin. Neither gap is tracked anywhere as remaining Phase-A work; the closing note of feedbackJune2026 implies Phase A is done modulo the three open decisions.

[low] Synthetic-fixture story: docs say committed PDFs in testdata/synthetic/, reality is runtime generation ? three docs and a gitignore comment overstate

TESTING.md §7 ('Stored under testdata/synthetic/ (committed; clearly fake)'), onboarding-a-bank.md step 3 ('Commit it to testdata/synthetic/'), and the .gitignore comment ('committed synthetic fixtures are tracked') all describe committed fixture PDFs. In reality git tracks nothing under testdata/synthetic/ ? tests/test_regression_synth.py generates born-digital PDFs into tmp_path at test time via tests/synthpdf.py (which is arguably the better design, per ROADMAP's Layer-1 description 'generates ... at test time'). A new-bank onboarding session following onboarding-a-bank.md verbatim would try to commit PDFs into a directory pattern the hygiene tooling treats as a whitelisted-but-empty path. Minor related staleness: TESTING.md §9 describes the hygiene guard as checking testdata/statements/ and testdata/private/, while test_repo_hygiene.py actually enforces the stronger 'nothing under testdata/ except promo-emails/ and synthetic/' rule ? the docs understate the real guard. Also TESTING.md's golden-file layer (testdata/promo-emails/golden/) does not exist.

[low] README understates project status and asserts untested macOS support while macOS first-class is an explicitly open decision

README 'Status' says the skeleton/CLI/offline-CI 'are in place' ? three months of shipped substance (verified real-statement parsers, hybrid unlock, SQLCipher encryption + recovery, migrations, provenance) is invisible, which matters because README is the entry doc. It also states 'Runs on Linux, macOS, and Windows' while feedbackJune2026 §5 decision 6 leaves 'macOS first-class?' open and the CI matrix tests only ubuntu+windows ? the claim is plausible (keyring/platformdirs) but unverified, against the project's own attribution discipline. Cosmetic but same genre: cli.py's module docstring still says 'Pipeline subcommands (fetch, txns) land with their milestones' though txns has landed.

RECOMMENDATIONS:

[short-term] Write the cross-repo ADR: decide muneem-fetch vs sister-handoff vs Drive-folder middle path, and scope the auditability invariant: One page, referenced from both repos. Decide explicitly: (a) does the sister repo's binding invariant apply to Avi's personal muneem instance or only to the future public product? (b) does muneem keep direct Gmail ingest (then amend the sister VISION's 'only code that touches Gmail' claim or scope it), consume the future handoff (then resolve that statement PDFs are PII and cannot ride a 'non-PII' contract ? the per-vertical carve-out must be designed jointly), or adopt the middle path (sister archives attachments to Drive in-account, muneem ingests the locally-synced folder; Gmail grant stays single, unlock stays local, no handoff contract needed)? This blocks Phase F and is currently a silent contradiction between two 'binding' documents written in the same week.

[short-term] Collapse sequencing into one owner: fold feedbackJune2026 Phases A-H into ROADMAP.md (or formally demote ROADMAP's milestone sections to history): Rename to avoid the Track-A/Phase-A and Milestone-C/Phase-C letter collisions, mark each phase's real status (A: incomplete ? lockfile + structured logging pending; B: half-shipped ? serving view missing), give each phase an exit criterion, and refresh §10's definition of done for the whole-finance scope. Simultaneously fix the two stale status docs: CURRENT_STATE §3 (schema v5, provenance shipped, per-row direction superseded) and parser-architecture (Axis 6/7, HDFC 6/6, balance-delta recovery; annotate §9/§11 with the §8.7 resolution). These are hours of work that de-risk every future restart.

[short-term] Gate Phase C behind an instrument-family entry checklist, and consider swapping it with Phase D: Before building: (1) corpus in hand ? N real broker/MF/CAS statements with passwords, in a gitignored testdata/<issuer>-statements/ dir, same as the bank slice demanded; (2) a one-page oracle design ? what arithmetic identity verifies a holdings statement (units continuity, units x NAV = value, fee/dividend cross-checks) and what 'reconciled' means when there is no balance walk; (3) the artifact decision (CAS vs broker statement vs tradebook) parked as an explicit open decision. Apply the project's own D.3-before-L3 logic: land regression CI Layer 2 + the D.3 eval writeup BEFORE the biggest expansion of parser surface, and verify the Axis CC parser on real data first ? it is the cheapest unverified thing already written.

[short-term] Ship the missing Phase-B serving view plus a minimal daily query surface, ahead of schedule: The serving view was decided and promised alongside the provenance table ? implement the cross-issuer SQL view (normalizing the four flat tables into date/amount/direction/narration/issuer + provenance join) as migration v6, then a 'muneem txns

all --since/--month' and a one-screen monthly summary. This is days of work on settled decisions, makes the tool worth opening weekly during the long C-G middle, and exercises the provenance join for real. Track 'muneem verify' (re-run reconciliation over stored rows) as the next cheap ops win rather than leaving it in Phase H.
117 [long-term] Add the missing roadmap categories: cash/manual entry, backfill & archive strategy, quarantine UX, recovery runbook: (1) Manual entry/correction path using the already-reserved source_type='manual' (cash spends, fixing a mis-parsed row) ? without it the ledger can never be complete. (2) A multi-year backfill plan: Gmail history query design and ordering, documents.path lifecycle when staged files move, and an explicit retention/protection policy for raw locked PDFs in the staging dir (currently outside SQLCipher's protection, covered only by optional FDE). (3) Decide the quarantine UX (parser-architecture §11): what 'muneem txns'/'eval' shows for the 1-in-7 statement that stores nothing. (4) Write docs/recovery.md (keychain lost / new machine / verify a backup / migrate-then-reverify) as feedbackJune2026 §2 and P6 prescribed.
118 [long-term] Close the small decision-hygiene loops: Back-record the L3 model choice as a partial resolution in DESIGN §8.3 and ROADMAP §1's decision log (model: DeepSeek-OCR-3B decided; locality rule unchanged; build deferred). Resolve the three cheap open decisions when next forced: log format (logfmt vs JSON ? unblocks finishing Phase A's P1), self-hosted runner acceptance (unblocks regression CI Layer 2), macOS stance (then fix or keep README's claim). Commit the lockfile (P10, trivially overdue for a compiled-dep project). Align the synthetic-fixture wording in TESTING.md §7 / onboarding-a-bank.md / .gitignore comment with the actual (better) runtime-generation design, and update TESTING.md §9 to describe the stronger guard that actually exists.
119
120 ########## gmail-docs ##########
121 SUMMARY: gmail-attachments-to-gdrive has unusually disciplined docs for a side project ? a binding auditability invariant with a CI-enforced egress guard, an honest quota-constraints section, and a PRIVACY.md that matches the shipped code. But the vision layer is ahead of (and in one place contradicted by) reality: muneem already holds its own Gmail OAuth grant with an implemented fetch track, directly violating this repo's "sole Gmail surface" invariant; the "only non-PII signal leaves" rule is irreconcilable with a meaningful bank-statement handoff (and redaction of password-locked Indian bank PDFs is infeasible in Apps Script); the "versioned handoff contract" the whole decoupling rests on is defined in no artifact in either repo; and ROADMAP.md is a rear-view mirror that tracks none of the Phase 0/1 work or the grocery-receipts vertical the vision names first. Phases 0?1 are realistic on GAS; Phase 2 OCR and the EmailMessage durable store are under-specified; Phase 3 as written breaks either the privacy rule or the invariant. The 100-user Testing-mode posture is financially coherent but undercut by 7-day refresh-token expiry (background sync dies weekly for testers) and a hard wall at user 101 (CASA, both gmail.readonly and full drive being restricted scopes) ? with self-deployment, the distribution path most aligned with the invariant, existing only as an unchecked roadmap stub.
122
123 MAP: VISION (docs/VISION.md): north star = "turn a person's email into a high-fidelity, structured, queryable record of their life." This repo is the extraction/ingest front door only: watches Gmail in-account, archives attachments, classifies, extracts, redacts/minimizes, hands off. Domain parsing/storage/analytics live downstream: muneem (finance), future siblings (grocery/coupons/travel). First verticals: (1) grocery receipts, (2) bank statements ? muneem. PHASES: 0 ingest model (stable EmailMessage; copier consumes it) ? 1 GAS-only classify+extract+index (no egress; Sheets "LifeIndex") ? 2 coverage/re-extraction (more extractors, body/OCR, extraction cursors, review queues) ? 3 handoff to hosted services (redact/minimize, separate consent, egress budgets) ? 4 surfaces/ops. AUDITABILITY INVARIANT (ARCHITECTURE.md, "binding", ADR-008): this repo is the ONLY code that touches Gmail; muneem never holds the Gmail grant ("Muneem may orchestrate this app; it is never itself the Gmail reader"); egress only via in-repo REDACT/MINIMIZE, non-PII only, to "the user's own hosted service", never third parties; self-deployable/reproducible from source; enforced by tests/auditability.test.ts (regex ban on UrlFetchApp in src/*.ts) + a PR checklist. PRIVACY (PRIVACY.md, 2026-05-31): app runs entirely in user's account (executeAs: USER_ACCESSING, verified in appsscript.json), copies attachments Gmail?user's own Drive, no third-party transmission, "the author of the app has no access to your data", only bookkeeping state in PropertiesService. CURRENT IMPLEMENTED SURFACE (src/, verified): attachment copier only ? doGet setup form, syncUserDrives trigger, 30-day window backfill + flag-gated history.list incremental sync, per-user lock, processed-id dedup, structured Logger.log to Stackdriver (exceptionLogging: STACKDRIVER), opt-in MailApp daily summary (off by default). NO ingest model, classifier, extractor registry, Sheets store, redactor, or handoff code exists (zero grep hits for EmailMessage/classify/extractor/HandoffPayload). Scopes: gmail.readonly, full drive, script.scriptapp; webapp access ANYONE. ROADMAP.md: platform constraints documented (6 min/run, 90 min/day triggers, 20 triggers, ~20k Gmail reads/day); all "quick win"/medium/large items

checked done except: Shared Drive/structure options, SECURITY.md, "deploy your own" guide, reproducible build, CONTRIBUTING.md. No items for Phase 0/1, the handoff contract, or grocery receipts. PUBLISHING (README): deliberately in OAuth "Testing" no-verification zone ? ?100 manually-added test users, unverified-app warning, ~weekly re-consent (7-day refresh-token expiry); or Workspace Internal (org-only). Fully public requires brand + restricted-scope + domain verification + annual CASA (~$1?5k+, 6?16 weeks); README recommends narrowing drive?drive.file first; gmail.readonly stays restricted regardless. CONTRADICTION WITH MUNEEM (verified in muneem/DESIGN.md + docs/ROADMAP.md): muneem Track B "Gmail OAuth2 + search + attachment download ?" with gmail.readonly, `muneem fetch` CLI (B.4) pending; muneem's docs contain zero references to gmail-attachments-to-gdrive or any handoff contract; muneem is local-first (SQLCipher on the user's machine), not a hosted service.
124
125   STRENGTHS:
126   * The auditability invariant is codified as a binding constraint with a per-PR checklist AND a CI test (tests/auditability.test.ts) ? a privacy claim with actual enforcement teeth, which is rare.
127   * ROADMAP.md opens with a cited, accurate platform-constraints section (6 min/run, 90 min/day, 20 triggers, ~20k Gmail reads/day, historyId expiry semantics) ? the design is genuinely quota-aware, not aspirational.
128   * PRIVACY.md precisely matches what the code does today (verified: no UrlFetchApp anywhere in src/, executeAs USER_ACCESSING in the manifest, state only in PropertiesService).
129   * Clean phase gating: Phases 0?2 are explicitly in-account with no egress and no privacy-policy change; Phase 3 is flag-gated and separately consented ? correct sequencing of consent with capability.
130   * ARCHITECTURE.md honestly flags its own weakest point as an open tension ("bank-statement and receipt signal? is often itself sensitive? 'non-PII only' needs a concrete per-vertical definition") rather than hand-waving it.
131   * The pure-logic/GAS-shell separation with an 85% coverage gate, trigger-leak guards, and 'mark done only on clean completion' is a solid hardened core to build the pivot on ? ADR-007's claim is borne out by the source layout.
132   * README's publishing section is unusually clear-eyed about the CASA wall, the Testing-mode caveats, and the drive?drive.file narrowing ? the cost/process reality is documented, not discovered later.
133

22. user (2026-06-10T16:33:51.896Z)

134   ISSUES:
135   [critical] Muneem already violates the binding 'sole Gmail surface' invariant
136   ARCHITECTURE.md invariant #1 states no other component ? 'Muneem included ? ever holds the Gmail OAuth grant or sees raw mail', and VISION.md says 'downstream services never read Gmail'. But muneem has its own Gmail OAuth ingestion IMPLEMENTED (Track B: OAuth2 + search + attachment download done, `muneem fetch` B.4 pending) using gmail.readonly, and muneem's DESIGN/ROADMAP contain zero references to this repo or any handoff. The binding invariant is contradicted by shipped sibling code on day one, and neither repo records the conflict or a resolution path. 'Muneem may orchestrate this app' also has no defined mechanism (how does a local Python CLI orchestrate a GAS web app?).
137   [critical] 'Only non-PII signal leaves' cannot coexist with a meaningful bank-statement handoff to muneem
138   Muneem's entire pipeline requires the FULL raw statement: it unlocks password-protected Indian bank PDFs locally (name+DOB / PAN+DDMM conventions), parses every transaction row, and reconciles against the bank's summary. A redacted, 'non-PII' payload is useless to it ? merchant names, amounts, dates, and account identifiers ARE the signal and ARE sensitive/PII. Redaction in GAS is also infeasible for this vertical: Apps Script has no native libs, no PDF-decryption capability, and Drive's OCR-via-Doc-conversion cannot open password-locked PDFs at all, so the REDACT/MINIMIZE stage cannot even read the document it is supposed to scrub. ARCHITECTURE.md flags this as an 'open tension', but the phase plan builds Phase 3 on the unresolved premise, and VISION.md states the rule absolutely ('Only non-PII signal ever leaves'). The honest options are: raw-blob handoff under separate consent (breaking the rule), muneem keeping its own Gmail path (breaking the invariant ? the current de-facto state), or finance staying fully out of this pipeline. Also, VISION/ARCHITECTURE describe egress to 'our own isolated, hosted service', but muneem is a local-first desktop tool (SQLCipher on the user's machine), not hosted ? the destination category itself is mischaracterized.
139   [high] The 'versioned handoff contract' ? the seam the whole architecture rests on ? is defined in no artifact

VISION.md says the repos 'meet only at a versioned handoff contract'; ARCHITECTURE.md sketches a 3-field HandoffPayload shape and then says schema and transport are 'TBD and aligned with that repo' (UrlFetch to a signed endpoint vs write-to-shared-store undecided). No schema file, no contract doc, no version, no transport decision exists in either repo, and muneem doesn't know the contract is supposed to exist. Until this is a concrete artifact, the extraction?processing decoupling is prose, not architecture ? and Phase 3 cannot be designed, costed, or consent-scoped.

[high] Phase 2 (OCR/body handling) and the durable EmailMessage substrate are under-specified for GAS quotas

Phase 0's premise is 'extraction re-runs over this record without re-touching Gmail' ? but where the durable EmailMessage (with plainBody/htmlBody) LIVES is never stated. PropertiesService is explicitly excluded ('never the entity index') and is anyway capped (~9KB/value, ~500KB total); a Sheets row caps at 50k chars/cell and a spreadsheet at 10M cells, making body storage for tens of thousands of messages awkward and slow via SpreadsheetApp; Drive-hosted JSON shards are unmentioned. For Phase 2 OCR, the only named mechanism is 'native Drive OCR via Doc conversion' ? workable for images/simple PDFs but mediocre on statements, generates intermediate Docs needing cleanup, and is useless on password-locked PDFs. No quota math exists for a multi-year backfill under 6 min/run, 90 min/day triggers, and ~20k Gmail reads/day (a 50k-message mailbox at even 2s/message of OCR+write is weeks of trigger budget). 'Heavy ML/PDF work happens downstream' is the right instinct, but Phase 2's 'body/OCR handling' sits in the in-account tier without a feasibility analysis. Phases 0?1 (rich EmailMessage via the Gmail advanced service, regex/sender classification, .ics/.csv extraction, Sheets index) are realistic on GAS.

[high] Auditability is weaker than stated for centrally-deployed users; Cloud Logging is already gray-zone egress

Three gaps: (1) End users of the central web-app deployment cannot view the deployed source, and the owner can push new code that all users execute on their next trigger run with no re-consent (scopes unchanged) ? so 'auditable at any time' truly holds only for self-deployers, and the reproducible-build + deploy-your-own items that would make verification possible are still unchecked. (2) Execution logs and exceptions go to Stackdriver/Cloud Logging in the DEVELOPER's GCP project for a centrally deployed script: src/drive.ts logs message IDs and exception text ('Attachment copy failed (message ?): ' + e, which can embed attachment names), and ARCHITECTURE.md plans 'structured logs carrying messageId'. That is data derived from a user's mail leaving the user's control ? in tension with invariant #2 ('Nothing derived from a user's mail leaves their Google account') and with PRIVACY.md's absolute 'The author of the app has no access to your data'. (3) The egress guard is a single regex for the literal 'UrlFetchApp' over src/*.ts only ? it doesn't scan html/index.html (client-side fetch from the served page), dist/ output, MailApp (SUMMARY_EMAIL_RECIPIENT is config-settable to any address), or trivially obfuscated access. Fine as a tripwire today; not the 'enforceable form' the docs imply.

[medium] Testing-mode 7-day refresh-token expiry kills the always-on value prop for the very users the plan wants to learn from

The posture is 'stay under 100 users to harden the system and learn the value proposition'. But for External/Testing apps, refresh tokens expire after 7 days ? each test user's background sync silently dies weekly until they revisit and re-consent. The README lists this as a one-line caveat, but it is fatal to evaluating a set-and-forget background archiver with real users: what testers experience is a product that stops working every week. The plan has no mitigation (e.g., expiry detection + nudge email, or steering testers to self-deploy where they own the OAuth client). Conclusions drawn from Testing-mode retention will be noise.

[medium] User 101 is a hard wall and the bridge plan (self-deploy distribution) exists only as an unchecked stub

At user 101: the Testing-mode test-user list is full; the only sanctioned path is External+Production = brand verification + restricted-scope verification + domain verification + annual CASA (~$1?5k+/yr, 6?16 weeks) ? and BOTH requested scopes are restricted (gmail.readonly, and full drive, which README already concedes should narrow to drive.file). 'Decide later with data' is financially coherent (ADR-005), but the plan is silent on the bridge: the one distribution channel that scales past 100 with zero verification AND maximally honors the auditability invariant is 'every power user deploys their own instance from source' ? exactly the unchecked 'Deploy your own from source' + 'reproducible build' roadmap items. The posture and the invariant point at the same answer; the roadmap doesn't prioritize it.

[medium] ROADMAP.md does not track the pivot: no Phase 0/1 items, no handoff contract, no grocery-receipts vertical

The roadmap is almost entirely checked-done copier hardening plus a thin tail (Shared Drives, SECURITY.md, self-deploy guide, reproducible build, CONTRIBUTING.md). Nothing tracks the EmailMessage ingest model (Phase 0), the classifier/extractor registry or the Sheets

LifeIndex store (Phase 1), the per-vertical non-PII definition, the handoff contract, or grocery receipts ? the vertical VISION.md names FIRST and calls the cheap proof of the whole pipeline. The vision and the roadmap describe two different projects right now; ARCHITECTURE.md's 'Open items' section is the only forward-looking work list and it isn't mirrored into the tracked roadmap.
151 [low] Decision-log hygiene: undated, status-less ADRs with unrecorded alternatives
152 The eight ADRs are one-line table rows: no dates, no status (proposed/accepted/superseded), no alternatives-considered, no links to discussion. ADR-008 is literally labeled 'Option A', implying Options B/C were weighed but they are recorded nowhere ? the exact context a future maintainer (or the CASA assessor) will need. Minor today; compounds as the pivot generates real decisions (handoff transport, store choice, non-PII matrix). Also cosmetic but real: appsscript.json timeZone is America/Los_Angeles while the user is in Bangalore and Gmail before:/after: boundaries follow it (README tells deployers to fix it, but the committed default is wrong for the owner's own deployment).
153
154 RECOMMENDATIONS:
155 [short-term] Reconcile Gmail ownership with muneem NOW ? one ADR recorded in both repos: Decide explicitly: (a) muneem keeps its implemented Gmail track and this repo's invariant is rescoped (e.g. 'sole Gmail surface for the multi-user product; the owner's local tools are exempt'), or (b) muneem's Track B is frozen/deprecated and this repo becomes its ingest front door (which forces the raw-blob handoff question immediately). Today the binding invariant is contradicted by shipped code in the sibling repo and neither repo acknowledges it. Whichever way it lands, record it as a dated ADR in ARCHITECTURE.md here and in muneem's DESIGN.md decision log.
156 [short-term] Write the handoff contract v0 as a real artifact and pick a transport: Promote the HandoffPayload sketch into a versioned schema file (JSON Schema or a markdown contract doc) plus a transport decision. A pragmatic v0 consistent with both repos: a Drive 'handoff' folder + manifest JSON (write-to-shared-store) that muneem's local CLI polls ? no hosted service, no UrlFetch, no change to the egress guard, works with muneem being local-first. Explicitly state, per vertical, whether the payload is structured fields or the raw blob, because that choice IS the privacy decision.
157 [short-term] Replace 'non-PII only' with a per-vertical data-minimization matrix: Bank statements are PII through and through; the absolute rule in VISION.md is unsatisfiable for the finance vertical. Reframe the privacy north star as minimization + explicit consent: a per-vertical table of sent / tokenized / kept-in-account (already an ARCHITECTURE.md open item ? give it a roadmap entry). For finance, the honest options are raw-blob-under-separate-consent or finance-stays-local; say which. Update VISION.md's 'Privacy north star' wording to match.
158 [short-term] Add the Phase 0/1 work to ROADMAP.md, grocery receipts first: Mirror ARCHITECTURE.md's open items into tracked roadmap entries: (1) EmailMessage ingest model consumed by the copier (Phase 0, no behavior change); (2) the narrow 'receipt-smelling' classifier + golden-email fixtures; (3) a GoogleSheetsStore spike that answers, with numbers, where the durable EmailMessage substrate lives (Sheets cell/spreadsheet limits, SpreadsheetApp write cost per run, Drive-JSON alternative) before Phase 0 code locks it in; (4) the handoff contract doc. The grocery vertical VISION names first currently has zero tracked work.
159 [short-term] Close the auditability gaps that are cheap today: (1) Ship SECURITY.md and the deploy-your-own guide (already roadmap stubs ? they are the enforcement of the invariant, not nice-to-haves). (2) Strengthen tests/auditability.test.ts: scan dist/Code.js and html/ too, assert the manifest's oauthScopes against an allowlist, and flag MailApp/ContentService alongside UrlFetchApp. (3) Scrub or gate log payloads (messageId, exception text) OR amend PRIVACY.md's 'author has no access to your data' to disclose Cloud Logging visibility for the central deployment ? self-deployers escape this, which is one more argument for that path.
160 [long-term] Make self-deployment the primary public distribution channel: The 100-user Testing cap, the weekly re-consent decay, the CASA wall at user 101, and the auditability invariant all point to the same strategy: the product's public form is 'clone, build, deploy your own' (each user owns their OAuth client ? no verification, no token expiry, no developer-visible logs, perfect invariant compliance), with the centrally hosted deployment reserved for friends/family until CASA is consciously bought. Decide this as posture, not just a runbook section; it changes what 'launch' means and what Phase 4 ops work matters.
161 [long-term] Do the Phase 2 feasibility math before committing OCR to the in-account tier: Before Phase 2 scope is fixed: prototype Drive OCR-via-Doc-conversion on a real receipt corpus and measure per-doc latency and quality; compute backfill wall-time under 90 min/day; design intermediate-Doc cleanup; and explicitly classify password-locked PDFs as downstream-only (GAS cannot open them ? no decryption primitive exists). If the numbers say in-account OCR only works for incremental new mail, write that into ARCHITECTURE.md so Phase 2 promises match the platform.

162   [long-term] Adopt lightweight ADR hygiene before the pivot generates real decisions: Add date, status, and a one-line 'alternatives considered' to each decision-log row (start by recording what ADR-008's Options B/C were). Cross-link decisions that bind both repos (Gmail ownership, handoff contract) so muneem's docs and this repo's docs can't silently diverge again. This is also the paper trail a future CASA assessment will ask for.
163

23. assistant (2026-06-10T16:33:58.791Z)

Now the market research / PMF section:

24. user (2026-06-10T16:34:05.223Z)

SUMMARY:
The direction is sound as a personal system and as a niche open-source play, but not (yet) as a mainstream consumer product. The Indian PFM market has consolidated around RBI's Account Aggregator rails (Fold, Jupiter, INDmoney), which are cloud-first, lending-subsidized, and still have real coverage gaps (credit-card detail, bonds, joint accounts, historical depth) ? exactly the gaps muneem's reconciliation-gated local PDF parsing fills. The Gmail-attachment-to-Drive layer is fully commoditized (Digital Inspiration's add-on alone has 2M+ installs and already decrypts password-protected bank PDFs), so Repo A's value must come from the classification/extraction front-door vision, not archiving. Email-parsing consumer products historically survived only by (a) selling aggregated purchase data (Slice/Rakuten?NielsenIQ), (b) pivoting to bank-linking (Truebill?Rocket Money), or (c) owning a narrow high-pain vertical (TripIt for travel) ? a cautionary base rate for the 'extraction front door' as a business. Publishing a Gmail-reading app in 2026 costs roughly $540?$4,500/year in CASA assessments plus ~2 months of process, annually recurring; the self-deploy 'run the code you read' model legitimately sidesteps this via Google's personal-use exemption and is itself the cheapest PMF probe: count completed self-deployments. Recommended sequencing: finance remains the first vertical for personal utility (already proven on real HDFC/Axis/AU data); grocery is a good second/demo vertical but a weak public wedge; the India tax-season document pack is the most under-considered alternative wedge.

=== MARKET MAP ===
* [India AA-based PFM] Fold Money: Bengaluru startup (founded 2019, ~11 employees, $2.5M seed, ?4.99Cr revenue FY25 per Tracxn) that pulls bank/CC/investment/EPF/NPS data via the RBI Account Aggregator framework and focuses on cash-flow awareness; categorization engine built with Setu (AA TSP). | RELEVANCE: The closest spiritual competitor to muneem's consolidated-ledger vision, but cloud-based and AA-dependent. Its tiny revenue after 6 years signals how hard it is to get Indians to pay for pure PFM. Its AA dependence means it inherits AA's gaps ? which is muneem's opening.
* [India PFM superapp] INDmoney: Investment-led superapp tracking stocks/MF/US equities plus free credit-card bill and savings-account tracking with auto-categorized expenses. | RELEVANCE: Shows the dominant Indian PFM monetization pattern: tracking is a free funnel into brokerage/distribution revenue, not a paid product. Competing head-on as a paid tracker is structurally hard.
* [India neobank with AA tracking] Jupiter (Spend Insights): Neobank whose Spend Insights links external bank accounts and credit cards via AA and auto-categorizes spends, dues, and statements. | RELEVANCE: AA-based aggregation is now table-stakes in Indian consumer fintech; differentiation cannot be 'see all accounts in one place' ? it must be depth (item-level, historical, verified) or privacy posture.
* [SMS-parsing expense tracker ? lender] Axio (ex-Walnut/Capital Float): Walnut pioneered SMS-parsing expense tracking in India; merged into Capital Float, rebranded Axio (NBFC), and was acquired by Amazon for ~$200M in September 2025 with RBI approval, primarily for its lending license/BNPL stack. | RELEVANCE: The canonical Indian story: expense tracking was never the business ? it was underwriting-data acquisition for credit. Also the direct precedent for muneem's SMS-freshness idea: technically proven, but the surrounding Play Store policy regime has since hardened.
* [India email-statement parser (incumbent)] CRED Protect: CRED's feature that, with consent, reads users' Gmail for credit-card statement emails, parses password-protected PDF statements (using stored user data to unlock them), and surfaces dues/spend analysis. Free; monetized via cards ecosystem. | RELEVANCE: Proof that email-based statement parsing works at scale in India AND that a deep-pocketed incumbent gives it away free. muneem cannot win on convenience; it can win on locality (CRED processes in its cloud), full-ledger scope beyond cards, and user-owned data.

* [Infrastructure (RBI rails)] Sahamati / AA ecosystem: 180+ FIPs and 950+ FIUs live; AA-facilitated lending at ?1.47 lakh crore in H1 FY26. PFM is a recognized consent use case. No individual/self-custody access path: data flows only to regulated FIUs via NBFC-AAs; joint accounts excluded; FIP-AA connectivity is bilateral and fragmented; investment data tilts toward MF/depository holdings; credit-card FIP coverage remains the weakest data class. DPDP Rules 2025 create 'Consent Managers' modeled on AA but do not open raw data access to individuals. | RELEVANCE: Confirms (as of mid-2026) that muneem's AA-deferral decision still holds: nothing changed in 2025-26 to give individuals self-custody pulls. The coverage gaps (CC detail, bonds, history) define exactly where PDF parsing remains the only complete source.
* [Gmail?Drive add-on (Repo A's layer)] Digital Inspiration 'Save Emails and Attachments': Workspace Marketplace add-on with 2M+ installs; saves emails/attachments to Drive in native formats, free tier + premium, and notably already decrypts password-protected bank/CC PDFs. | RELEVANCE: The archiving layer is commoditized by a single dominant indie (Amit Agarwal) plus cloudHQ. Repo A cannot compete on 'save attachments'; its only defensible angle is the auditable self-deploy trust model and the downstream extraction/index vision.
* [Gmail?Drive/Sheets add-ons] cloudHQ (Save Emails / Export Emails to Sheets): Suite of Gmail export add-ons with freemium tiers (Free/Premium Backup/Starter/Plus), converting threads to PDF and attachments to Drive, plus email-to-spreadsheet extraction. | RELEVANCE: Second proof the layer is commoditized ? and that 'email ? structured rows in a sheet' is already a productized category, though not finance-specialized for India.
* [Email-parsing survivor (travel)] TripIt (SAP Concur): Parses forwarded/inbox-synced booking confirmation emails into master itineraries since 2008-2010; still alive and the category default. | RELEVANCE: The one durable consumer email-parsing success. Lesson: it won a narrow, high-pain, time-critical vertical where consolidation has obvious immediate utility ? a template for choosing wedges.
* [Email-receipt parsing (data-resale model)] Slice Intelligence / Rakuten Intelligence ? NielsenIQ: Parsed purchase receipts from users' inboxes (fed by Unroll.me, acquired 2014); the consumer app was a loss-leader for an e-receipt panel-data business sold to NielsenIQ in 2022. | RELEVANCE: Email receipt extraction at scale monetized by selling aggregated purchase data ? the exact model muneem's privacy thesis rejects. Warns that grocery/receipt data has B2B resale value but weak direct consumer willingness-to-pay.
* [Subscription-finder PFM (US)] Rocket Money (ex-Truebill): Started around inbox/statement scanning for forgotten subscriptions; pivoted fully to Plaid bank-linking; acquired by Rocket Companies; freemium with premium bill negotiation. | RELEVANCE: Shows (a) subscriptions/recurring-charge detection is the email-adjacent wedge consumers demonstrably pay for, and (b) successful players abandoned email parsing for bank-feed parsing once available ? AA is the Indian analogue of that gravity.
* [Platform-native purchase tracking] Gmail Purchases view (Google native): Google has long auto-extracted receipts into a Purchases page (myaccount.google.com/purchases, data back to 2012) and in 2025 shipped a refreshed 'Purchases' view in Gmail consolidating orders/shipping/delivery. | RELEVANCE: The platform owner already does generic receipt extraction for free, natively. Any public front-door product must do something Gmail won't: India-specific finance depth, user-owned structured export, local processing, cross-source ledger.
* [SMB email-receipt extraction] SparkReceipt / WellyBox / Zoho Expense Gmail add-on: Commercial tools that connect Gmail/IMAP, auto-detect receipt emails, and extract vendor/amount/tax line items via AI, targeting freelancers/SMB bookkeeping. | RELEVANCE: Receipt extraction from email is a solved, crowded SMB category globally. The unowned space is consumer-side, India-format-specific, privacy-first extraction ? narrower than it looks.
* [OSS local-first PFM] Actual Budget / Firefly III: Self-hosted budgeting/finance apps; ingestion is generic CSV/bank-sync (EU/US oriented); Firefly relies on a data importer plus user-contributed CSV import configs; no Indian PDF statement ingestion exists in either. | RELEVANCE: The OSS PFM world has UI/ledger/budgeting solved but has NO Indian ingestion story. muneem could be the missing ingestion layer (export to Actual/Firefly/beancount) rather than rebuilding serving ? a cheap distribution wedge into existing communities.
* [OSS India importers] beancount-importers-india (plain-text accounting): Community importers for ICICI/SBI/Zerodha CSV/XLS tradebooks into Beancount; HDFC absent; CSV-based, single-maintainer, no PDF parsing, no validation guarantees. | RELEVANCE: Direct evidence the Indian plain-text-accounting niche exists but is underserved: importers are CSV-only, stale, and trust-free. Reconciliation-gated PDF parsing is a genuine step up for this exact audience.
* [OSS hobby parsers] Indie Indian statement parsers (xaneem/hdfc-cc-statement-parser, joeirimpan/hdfc-cc-parser-rs, codito/bout, StmtForge, vishwaraja/hdfc-pdf-converter): Scattered single-bank or multi-bank PDF?CSV/QIF parsers for HDFC/ICICI/SBI/Axis statements; mostly unmaintained one-offs; StmtForge claims 9 banks, offline, Streamlit dashboard. | RELEVANCE: Validates demand among Indian devs (people keep rebuilding this) and validates the gap: none

offer arithmetic reconciliation gating, encrypted ledgers, or maintained multi-issuer coverage. Also: muneem has competitors for mindshare in 'parse my HDFC PDF' GitHub searches.
* [India receipt-scanning rewards] magicpin / Club Corra: magicpin (since 2015) pays points for uploaded bills including grocery; Club Corra runs OCR-verified receipt uploads for 40+ partner brands redeemable as UPI cashback. | RELEVANCE: The only Indian consumer receipt-collection businesses pay users for receipts (data/marketing value to brands) ? strong signal that Indian consumers won't proactively consolidate grocery receipts without being paid, undermining grocery as a public-product wedge.
* [B2B statement parsing] Precisa / bank-statement OCR vendors (B2B India): Commercial bank-statement analysis tools (Precisa, et al.) parsing Indian bank PDFs for lenders/CAs with fraud detection, volatility scores, AA integration. | RELEVANCE: Statement parsing IS a paid market in India ? but B2B (lending ops, CAs), not consumer. If muneem ever monetizes parsing itself, this is the adjacent proven buyer, and also the entrenched competition.
* [Grocery data sources] Blinkit/Zepto/Instamart invoicing: Blinkit issues downloadable GST tax invoices per order (in-app, My Orders); launched GSTIN invoicing for businesses in Oct 2024; government uses q-commerce price data for GST-cut monitoring. Itemized invoices exist but live primarily in-app rather than reliably as email attachments. | RELEVANCE: Grocery item-level data exists and is genuinely absent from AA/PFM apps, but the front door for it is only partially email ? an Apps Script-only ingestion strategy under-covers the vertical (inference from invoice-retrieval flows).

=== GROCERY WEDGE ===
PROS ? Personal utility: item-level basket data is absent from every AA-based PFM (AA shows the ?487 Blinkit debit, never the basket ? inference, but structurally certain since FIP data schemas don't carry merchant line items); for a Bangalore household concentrated on Blinkit/Zepto/BigBasket/Instamart it would unlock price-over-time, shrinkflation, and category analytics nobody else offers. Development-process fit is excellent: grocery receipts are low-sensitivity (no account numbers), so they can be a cloud-LLM-permitted corpus under muneem's own security rules, making parser iteration fast and safely testable ? same property that made the promo-email corpus work. PROS ? Public product: genuinely unowned space in India; no consumer product consolidates itemized grocery receipts across q-commerce platforms; demo appeal ('see your real grocery inflation') is high and shareable. CONS ? Personal: source coverage is the killer ? Blinkit's itemized GST invoice is primarily an in-app download from My Orders, not a guaranteed email attachment, and Zepto/Instamart behave similarly (in-app first, inconsistent email), so an email-only front door under-covers the vertical (inference from documented invoice-retrieval flows); q-commerce template churn is rapid; the payoff is interesting analytics, not money or correctness ? low recurring pull. CONS ? Public: the demand signal is actively negative: the only receipt-collection businesses anywhere (magicpin, Club Corra in India; Fetch, Slice in the US) PAY consumers for receipts because the value accrues to brands/panels, not the consumer ? and the data-resale monetization that makes receipts a business is precisely what the privacy thesis forbids. Google's native Gmail Purchases view also free-rides the generic version. VERDICT: Good second vertical and an excellent low-risk demo/test corpus for the classification front door; as the FIRST vertical it is wrong on both axes ? on personal utility it loses to finance (already built, higher pain), and as a public wedge it has no willingness-to-pay and incomplete email coverage. Build it after the finance spine exists, scoped to the user's own 2-3 platforms.

=== FINANCE WEDGE ===
PROS ? Personal utility: this is the strongest possible first vertical and it is already de-risked ? muneem parses HDFC/Axis/AU on real data with reconciliation gating, which no AA app or OSS tool matches. It directly fills documented AA gaps as of 2025-26: credit-card data remains the weakest FIP class (AA Phase 1 was CASA; investment data tilts to MF/depository), joint accounts are excluded from AA pulls, historical depth is limited, and there is still no individual/self-custody AA path ? so PDF statements remain the only complete, user-controlled source of one's own financial history. The reconciliation gate ('rows persist only if the balance walk matches the bank summary') is a real correctness differentiator over the dozen hobby parsers on GitHub that silently drop or duplicate rows. CONS ? Public product: the trust asymmetry is brutal ? an unknown indie tool asking for bank statements competes with CRED Protect, which already parses password-protected CC statement PDFs from Gmail, free, backed by a recognized brand; Fold/Jupiter/INDmoney offer one-tap AA linking with zero parsing friction; and the published-app path runs straight into gmail.readonly restricted-scope verification + annual CASA ($540?$4,500/yr + ~2 months process). Password-protected Indian statement PDFs force handling of PII-derived password rules, raising both engineering and liability surface. Parser maintenance across dozens of issuers is a permanent treadmill, and every quarter AA coverage improves, the wedge narrows. VERDICT: As a personal tool, unambiguously the right first vertical

? already proven, highest pain, fills real gaps. As a public product, viable ONLY in the self-deploy/auditable-OSS form ('run the code you read', personal-use exemption, no CASA) targeting the privacy-conscious Indian dev/finance-nerd niche ? realistically hundreds to low thousands of users (inference) ? not as a head-to-head consumer app against free AA-based incumbents.

=== OTHER WEDGES ===
* India FY tax-season document pack (STRONGEST alternative): auto-collect Form 16, interest certificates, capital-gains statements (Zerodha/CAMS/KFintech emails), AIS/26AS alerts, insurance 80C/80D receipts into one CA-ready bundle each April-July. Effort: moderate ? sources are a finite, slow-churning set of large issuers, mostly genuine email attachments (unlike grocery), and it aligns exactly with muneem's broker/MF-statement roadmap. Payoff: high ? acute seasonal pain every Indian filer feels, a natural annual-renewal pricing story, and the clearest answer to 'what would someone pay for'. Sensitive scope applies, but the self-deploy model covers it. This is arguably a better PUBLIC first wedge than either grocery or general finance.
* Warranties / durable-goods invoices + AMC dates: extract invoices for electronics/appliances from email (Amazon.in, Flipkart, Croma, brand stores), index purchase date, warranty period, serial numbers; remind before expiry. Effort: low-moderate ? invoice emails are common, formats moderately stable, low sensitivity (could even avoid full-content scopes in a self-deployed script). Payoff: moderate ? real but episodic utility; the Blinkit-GST-invoice-for-warranty pattern shows Indians already care about keeping invoices for claims. Underserved niche, weak monetization, good goodwill/demo vertical.
* Subscriptions, renewals & e-mandate alerts: detect recurring charges from renewal emails, SI/e-mandate notifications, and insurance premium reminders ? the Truebill origin story, India-localized (RBI's e-mandate regime generates a clean, semi-standardized email/SMS trail ? inference). Effort: low-moderate. Payoff: this is the one email-adjacent category with PROVEN consumer willingness-to-pay globally (Rocket Money's premium business). Risk: card-side and AA-side detection by incumbents commoditizes it; still, as a front-door feature it is the highest-WTP signal extractor per unit effort.
* Travel/bookings (TripIt-for-India): parse MakeMyTrip/Cleartrip/IRCTC/airline confirmation emails into itineraries. Effort: moderate (formats well-structured). Payoff: low as a standalone ? TripIt, Gmail's native parsing, and Google Wallet already cover most of it; only worth doing as a corpus-diversity exercise for the classifier, not as a wedge.
* Coupons/deals carryover (the Email-Mining predecessor): lowest sensitivity, fun, already partially built. Payoff: lowest ? Gmail's Promotions tab, CashKaro, and GrabOn commoditize it, and expiring-coupon value is small. Keep as a free delight feature inside the front door, never the wedge.

=== PMF ASSESSMENT ===
OVERALL: the direction is reasonable ? with the crucial clarification that today this is a personal system with product-grade engineering, and the honest near-term ambition should be 'category-best personal tool + niche OSS' rather than consumer product. PERSONAL-UTILITY AXIS: near-maximal. The two repos compose correctly (Apps Script = acquisition inside the user's own trust domain; muneem = local sensitive processing), and the architecture's constraints (no cloud LLMs on statements, reconciliation gating, encrypted local ledger) cost little personally while delivering something no Indian app offers: a complete, verified, user-owned financial history including credit cards, bonds, and pre-AA history. PUBLIC-PRODUCT AXIS: the moat question. 'Reconciliation-gated parsing of Indian PDFs, fully local' IS differentiated ? vs AA apps (cloud-resident data, lending-subsidized business models per the Walnut?Axio?Amazon arc, and persistent coverage gaps in CC detail/bonds/joint accounts/history), vs OSS PFMs (Actual/Firefly/beancount have zero Indian PDF ingestion; community importers are CSV-only and stale), and vs hobby parsers (single-bank, unmaintained, no correctness guarantees). But it is a maintenance moat, not a defensibility moat: the asset is the discipline (validation gates, provenance, regression corpora) plus the grind of tracking issuer template churn ? replicable by any funded team, and increasingly automatable with LLMs (which cuts both ways; muneem's local-VLM fallback is the right hedge). The deepest structural insight from the market history: every consumer email-parsing business either sold the data (Slice), pivoted to bank feeds (Truebill), or owned one narrow vertical (TripIt). A privacy-first product refuses the first path and India's AA is the second ? so a public muneem must win the third way: own a vertical where PDFs/email beat AA permanently (credit-card statement detail, tax-season documents, historical archives). WHAT MAKES IT PAYABLE: (1) the tax-season pack ? annual, acute, CA-shareable output; (2) 'verified import' as the brand promise ? every statement reconciled or flagged, which no competitor states; (3) a parser-definitions maintenance subscription on top of MIT code (the WordPress/Sidekiq pattern) for self-hosters; (4) optionally B2B (CAs/family offices) ? but Precisa-class vendors already serve lenders, so that is a different, crowded motion. THE 100-USER APPS SCRIPT PROBE: sensible and nearly free, with two concrete caveats: (a) a

centrally-published Testing-status OAuth client with restricted scopes issues refresh tokens that expire every 7 days (per Google's OAuth docs), forcing weekly re-auth that will contaminate retention signals; (b) the better-designed probe is the one already implied by 'run the code you read' ? per-user self-deployment keeps every install inside Google's personal-use/internal exemptions indefinitely, costs zero CASA, and the conversion rate from README-reader to completed-self-deploy IS the PMF metric. If self-deploys stall in the dozens, the verdict is 'excellent personal tool, no product' ? a perfectly good outcome. If they reach hundreds organically (the beancount-India and r/IndiaInvestments self-hosting crowds are the seedbed ? inference), that justifies spending the ~$1-5K/yr CASA toll to publish. Do not pay the toll before the signal.

=== PUBLISHING CONSTRAINTS ===
As of mid-2026, publishing an app that reads Gmail content (gmail.readonly is a RESTRICTED scope; there is no non-restricted substitute for reading message bodies/attachments) requires: (1) OAuth brand verification (typically 2-3 business days); (2) restricted-scope verification ? scope-by-scope justification, a demo video of the OAuth flow and scope usage, accurate consent screen, declared application type ? which Google says 'can potentially take several weeks'; and (3) a CASA (Cloud Application Security Assessment, App Defense Alliance framework) security assessment, repeated AT LEAST EVERY 12 MONTHS, for any app that can move restricted data through its own/third-party servers. CASA tiers and 2026 costs: Tier 2 lab assessments run roughly $540?$1,800 (the Tier 2 self-scan option has been removed ? an authorized lab is now mandatory, a 2025 tightening); Tier 3 (required for higher-risk apps, large user counts, and Marketplace badge scenarios) runs ~$4,500 (TAC Security's published example). Authorized labs: TAC Security, Leviathan, DEKRA, Bishop Fox. Realistic first-time timeline: 1-4 weeks of lab turnaround inside an overall ~2-month project including remediation; October 2025 Google policy updates made CASA unambiguous for restricted Gmail/Drive scopes. EXEMPTIONS that matter for this project: personal-use-only apps, apps in development/testing/staging, and internal/domain-wide-install apps are exempt from verification ? so (a) the current <100-user Testing mode is free and legal, but external Testing-status clients with restricted scopes get 7-DAY refresh-token expiry (weekly re-consent) per Google's OAuth documentation, and users see an unverified-app warning; (b) the 'each user deploys their own copy of the MIT code in their own account' model keeps every deployment inside the personal-use exemption permanently ? no verification, no CASA, no token expiry pathology (Apps Script running as the account owner), at the cost of onboarding friction. One design note: an Apps Script that only files attachments into the user's own Drive could in principle narrow Drive-side scope to drive.file (non-restricted), but the Gmail-reading side stays restricted regardless of vertical ? the toll is triggered by reading email, not by what is extracted. Practical 2026 budget if publishing: $1,000?$5,000/year recurring (CASA + incidentals) plus 1-2 months of calendar time for the first pass ? material for a solo developer, trivial for a funded competitor, which is itself strategic information.

25. assistant (2026-06-10T16:36:13.114Z)

```
---
name: deep-review-2026-06
description: Full findings of the 2026-06-10 deep review of muneem + gmail-attachments-to-gdrive
(code, security, architecture, roadmap, PMF) ? 35-agent workflow, all critical/high findings
adversarially verified 3/3.
metadata:
  type: project
---
```

Deep review ? muneem + gmail-attachments-to-gdrive (2026-06-10)

Method: 8 parallel reviewers (muneem core / parsers / security / tests-CI / docs-roadmap; gmail code / gmail docs; web market research), every critical/high code finding verified by 3 adversarial agents with distinct lenses. 9/9 confirmed, 0 refuted. Offline suite run live: 390 passed, 6.35s, 86.17% coverage.

Confirmed critical/high code findings

muneem

1. **[critical] rotate-key destroys the only durable key copy before rekeying** ? `cli.py:438-446` writes the fresh key into the keychain *first*, then rekeys; rollback only

catches `EncryptedDatabaseError` while `PRAGMA rekey` raises raw `sqlcipher3.OperationalError` (e.g. database-is-locked ? likely on Windows with AV). Crash/Ctrl-C/locked-DB in the window ? keychain=fresh, file=old, old key gone; ledger + all backups unrecoverable unless recovery file exists (which wraps the OLD key and goes stale after successful rotation). Inline comment asserts the opposite of the behavior; contradicts `rekey_database`/`set_key` docstrings. Fix: pending-key dance (write fresh under `db-key-pending`, rekey, promote, delete) or at minimum rekey-file-first + broad except + keep old key under `db-key-prev`. Zero test coverage on the failure path.
2. **[high] `db backup <ledger-path>` destroys the ledger** ? `db.py:292-295` unlinks the destination before validating it isn't the live DB ? silently replaces ledger with empty DB.
3. **[high?medium] transient keychain read failure mints a new key over the real one** ? `secrets.get_secret` swallows `KeyringError`?None; `get_or_create_key` (db_key.py:24-31) treats None as "no key ever" and overwrites. `connect_encrypted` calls it even when an encrypted ledger already exists on disk ? exactly when minting is never right.
4. **[high] savings oracle ignores opening/closing bookends when the balance walk passes** ? `savings.py:78-84,194-205` short-circuits; a statement missing leading/trailing transactions persists as `reconciled` (empirically confirmed).
5. **[high] Cr/Dr suffix sign-stripped in `parse_amount`** ? `validation.py:32,45`; an overdraft (Dr-balance) statement reconciles with every debit/credit INVERTED and persists as verified (empirically confirmed).
6. **[high] HDFC persists rows even when UNRECONCILED; `txns list` never filters by document status** ? `hdfc.py:435-458`. Violates the store-nothing-unverified philosophy the other parsers follow.

gmail-attachments-to-gdrive

7. **[high] backfill advances `lastSynced` past windows with attachment-level errors** ? `sync.ts:213-218`; failed messages are never retried ("mark done only on clean completion" invariant broken at window level).
8. **[high/med] incremental sync advances historyId past messages truncated by the daily cap or transient failures** ? `sync.ts:175-189`; silent data loss (opt-in flag, default off).
9. **[med] `GmailApp.getMessageById` throws on deleted messages** ? `gmailHistory.ts:70-77`; wedges incremental sync until cursor expiry.

Notable mediums (selected)

- muneem: reconcile tolerance == currency quantum w/ strict-greater ? ~42% of one-paisa corruptions pass (validation.py:70-118); substring+alphabetical parser routing ? '@aubank' in a narration routes an HDFC statement to AUParser (au.py:33-37); sha256 dedup permanently blocks re-ingesting a quarantined doc after a parser fix (cli.py:121-124); Axis CC parser persists with ZERO verification; dates never parsed/normalized (raw locale strings persisted); migration runner partial-failure window (DDL commits before recording; untested); forward-migration checksums recorded but never re-verified; RedactingFilter misses PAN/spaced-card/Aadhaar; key spliced into PRAGMA without 64-hex validation.
- muneem security (posture strong, history verified clean ? nothing sensitive ever committed): real parsed CC rows sit in plaintext `testdata/snapshots/*.json`, written by a now-dead `snapshot` fixture in conftest (no test uses it); `client_secret_*.json` (Google's default download name) escapes all three guard layers once outside testdata/; no lockfile/pinning/hash-checking incl. the sqlcipher3 native wheel; recovery passphrase floor 8 chars; pre-commit hook opt-in.
- gmail: `getOrCreateFolderInFolder(DriveApp,?)` iterates ALL Drive folders not root children; state persisted only at run end (6-min kill loses bookkeeping); setup re-run races in-flight sync (no lock in processInput); same-filename attachments silently dropped; Cloud Logging carries message ids + error text (tension with PRIVACY.md "author has no access"); committed timeZone America/Los_Angeles; `SUMMARY_EMAIL_RECIPIENT` global; CI creates a new deployment per push.

Architecture / roadmap issues

1. **Cross-repo contradiction (the #1 issue overall):** gmail repo's *binding* auditability invariant (ADR-008, 2026-06-03) says it is the ONLY Gmail-touching code and "Muneem never holds the Gmail grant / sees only minimized non-PII signal" ? while muneem ships full gmail.readonly OAuth ingestion and plans `muneem fetch` (Phase F). Bank statements are PII; they cannot ride a "non-PII" handoff; GAS cannot even open password-locked PDFs. Sister also mischaracterizes muneem as a "hosted service" (it's local-first by design). Neither repo references the other's constraint. Options: (a) muneem keeps ingest + invariant rescoped to the public product; (b) muneem consumes the (undefined) handoff ? blocked, PII-incoherent; (c) **middle path

(recommended): sister archives attachments to Drive in-account (already its hardened core), muneem ingests from the locally-synced Drive folder** ? single Gmail grant, unlock stays local, no handoff contract needed.
2. Two competing sequencing sources of truth: ROADMAP.md Milestones/Tracks vs feedbackJune2026 Phases A?H, colliding letter namespaces; ROADMAP checkbox rot; CURRENT_STATE.md says "Schema v4"/per-row provenance planned (stale: v5 shipped, separate-table decided); parser-architecture.md says Axis 2/4, HDFC 4/4 (stale: 6/7, 6/6) and §9 still points at the superseded unified-table direction.
3. Phase C (instrument family) is the riskiest, least-specified phase: no broker/MF/CAS corpus in hand, no oracle design (what replaces the balance walk for holdings?), artifact choice (CAS vs broker stmt vs tradebook) unparked ? and sequenced BEFORE regression CI Layer 2 + D.3 eval, contradicting the project's own "characterize failure modes first" logic. Gate behind an entry checklist; consider swapping C and D; verify Axis CC on real data first (cheapest unverified thing already written).
4. **No daily user value anywhere in the roadmap**: only `txns list <bank>`/`offers list` exist after ~46 PRs; the Phase-B serving view was decided+promised and silently dropped (provenance table shipped without it). Ship view + `txns all --since/--month` + monthly summary early ? it's what keeps the user feeding the tool through the C?G middle.
5. Missing roadmap categories: cash/manual entry (source_type='manual' already reserved), multi-year Gmail backfill + raw-PDF staging retention strategy (PDFs outside SQLCipher protection), quarantine UX, docs/recovery.md runbook, refreshed definition of done; SMS-alert parsing (P11) unowned. Phase A quietly incomplete (no lockfile; structured logging stalled on log-format decision).
6. gmail repo: vision far ahead of roadmap ? zero tracked items for Phase 0/1, the handoff contract, or the grocery vertical VISION names first; handoff contract exists in no artifact; Phase 2 in-account OCR not feasibility-checked (and impossible for password-locked PDFs); auditability weaker than stated for centrally-deployed users (Cloud Logging, no reproducible build/deploy-your-own guide yet); 7-day refresh-token expiry in Testing mode kills the always-on value prop and contaminates the <100-user probe; user 101 = CASA wall ($540?$4,500/yr + ~2 months, annually recurring; both gmail.readonly and full drive are restricted). Self-deploy ("run the code you read") is the distribution channel that scales with zero verification and maximal invariant compliance ? make it the primary public posture.

PMF (researched, mid-2026)

- India PFM consolidated on AA rails (Fold, INDmoney, Jupiter) ? cloud-first, lending-subsidized, free; coverage gaps persist exactly where muneem plays: CC detail, bonds, joint accounts, pre-AA history; still no individual/self-custody AA path (AA-deferral decision still holds).
- CRED Protect already parses password-protected CC statement PDFs from Gmail, free ? can't win on convenience; win on locality + whole-ledger + verified-import.
- The attachment-archiving layer is commoditized (Digital Inspiration 2M+ installs, already decrypts bank PDFs; cloudHQ) ? Repo A's value must be the classification/extraction front door + auditable self-deploy trust model, not archiving.
- Email-parsing consumer history: sell the data (Slice?NielsenIQ), pivot to bank feeds (Truebill?Rocket Money), or own one narrow vertical (TripIt). Privacy thesis forbids #1, AA is #2 ? must win #3: own verticals where PDFs/email beat AA permanently.
- Moat: "reconciliation-gated parsing of Indian PDFs, fully local" is genuinely differentiated vs AA apps, OSS PFMs (zero Indian PDF ingestion in Actual/Firefly/beancount), and hobby parsers ? but it's a maintenance moat, not a defensibility moat.
- Wedge verdicts: **finance = right first vertical personally (already proven)**; grocery = wrong first wedge (Blinkit/Zepto invoices live in-app not reliably in email; receipt-collection businesses must PAY consumers; Google's Gmail Purchases free-rides) but excellent SECOND vertical / front-door test corpus (low-sensitivity ? cloud-LLM-permitted ? fast iteration); **India FY tax-season document pack = strongest public wedge** (Form 16, interest certs, CG statements from CAMS/KFintech/Zerodha, 80C/80D ? acute seasonal pain, finite slow-churning issuer set, email-native, annual-renewal pricing, aligns with instrument-family roadmap); subscriptions/e-mandate renewal detection = highest proven WTP per effort; warranties/invoices = goodwill niche; travel/coupons = no (commoditized).
- The 100-user probe: sensible but use self-deploy conversions as the PMF metric (Testing-mode weekly re-consent contaminates retention signal). Don't pay CASA before organic self-deploys reach hundreds.

Run/verification facts

- Offline suite: 390 passed / 21 deselected, 6.35s, coverage 86.17% (gate 80). CI matrix real

(ubuntu 3.11?3.13 + windows 3.13).
- git history verified clean across both repos for sensitive paths
(statements/credentials/snapshots/env/DB ? never committed).

26. user (2026-06-10T16:36:13.135Z)

File created successfully at: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\memory\deep-review-2026-06.md (file state is current in your context ? no need to Read it back)

27. assistant (2026-06-10T16:36:35.754Z)

---
name: session-handoff
description: ? Start here ? living handoff for muneem (and its sister repo gmail-attachments-to-gdrive); current state, next steps, ramp-up kit, key decisions.
metadata:
  type: project

Session handoff ? muneem (+ gmail-attachments-to-gdrive)

> Last refreshed: 2026-06-10, after the cross-repo deep review.

1. You are here

- muneem `main` @ 98481ee (provenance table, PR #46). Parser precision on real data: HDFC 6/6, AU 4/4, Axis 6/7. Encryption + recovery + migrations shipped. Sister repo `gmail-attachments-to-gdrive` master @ a764030 (auditability invariant, PR #14).
- A 35-agent deep review of BOTH repos (code, security, architecture, roadmap, PMF) completed 2026-06-10. **All findings in [[deep-review-2026-06]]** ? 9 confirmed critical/high code bugs, the cross-repo Gmail-ownership contradiction, roadmap gaps, and a researched grocery-vs-finance-vs-tax-pack wedge assessment delivered to Avi. **Awaiting Avi's direction** on (a) which fixes to start, (b) the cross-repo seam decision, (c) the first-vertical/wedge choice.

2. Next steps / open threads

1. Avi to react to the review: pick wedge (recommendation: finance stays first personally; grocery as 2nd/front-door corpus; tax-season pack as the public wedge candidate).
2. If fixes approved, P0 order: rotate-key atomicity ? `db backup` destination guard ? savings-oracle bookend check ? Cr/Dr sign semantics ? HDFC unreconciled persistence ? gmail backfill retry. One PR per item (see [[pr-workflow-cadence]]).
3. Cross-repo ADR needed: who reads Gmail (recommended middle path: sister archives to Drive in-account; muneem ingests the locally-synced Drive folder; invariant scoped to the public product). Record in both repos.
4. Doc debt: consolidate sequencing into one owner (ROADMAP vs feedbackJune2026 Phases A?H); refresh CURRENT_STATE (schema v5) + parser-architecture (stale precision numbers, superseded §9); ship the dropped Phase-B serving view + `txns all`.
5. Hygiene quick wins: delete dead `snapshot` fixture + the two real-CC-row JSONs in testdata/snapshots/; block `client_secret*.json` everywhere; add lockfile.

3. Ramp-up kit

- [[deep-review-2026-06]] ? the full review findings (memory dir).
- muneem: `CLAUDE.md` ? `DESIGN.md` ? `docs/feedbackJune2026.md` (the governing phase plan) ? `docs/CURRENT_STATE.md` (stale re: schema v5 ? see review).
- Sister: `docs/VISION.md` ? `docs/ARCHITECTURE.md` (ADR-008 invariant) ? `ROADMAP.md`.
- Memory index `MEMORY.md` for standing decisions (licenses, local-first, storage shape, L3, AA-deferred).

4. Key decisions (don't relitigate)

- Python-only; PDF-first/AA-deferred; flat per-issuer tables + separate provenance table + (promised, unbuilt) serving view; hand-rolled migrations (no Alembic); SQLCipher + keychain + passphrase recovery; D.3 eval before L3; licenses MIT/BSD/Apache only (PyMuPDF banned); no cloud LLMs on sensitive docs; sister repo: <100-user Testing mode, CASA deferred.

- Still open (genuinely, verified in code): log format; self-hosted runner; macOS stance; the cross-repo Gmail seam (NEW ? surfaced by the review); instrument-family artifact choice (CAS vs broker stmt vs tradebook).

28. user (2026-06-10T16:36:35.764Z)

File created successfully at: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\memory\session-handoff.md (file state is current in your context ? no need to Read it back)

29. user (2026-06-10T16:36:44.124Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

30. user (2026-06-10T16:36:50.925Z)

<system-reminder>This memory is 6 days old. Memories are point-in-time observations, not live state ? claims about code behavior or file:line citations may be outdated. Verify against current code before asserting as fact.</system-reminder>
1 # muneem ? Memory Index
2
3 - [PR workflow cadence](pr-workflow-cadence.md) ? one PR per work item; verify locally, then squash-merge as you go.
4 - [Prefer local over paid APIs](prefer-local-over-paid-apis.md) ? self-hosted/local/own-trained first; paid APIs only as a fallback.
5 - [Commercial license safety](commercial-license-safety.md) ? only MIT/BSD/Apache deps; avoid (A)GPL. PyMuPDF (AGPL) is banned.
6 - [HDFC parser tech decision](hdfc-parser-tech-decision.md) ? hand-roll coordinate clustering on pdfplumber; camelot-py is the license-safe fallback.
7 - [Parser validation pattern](parser-validation-pattern.md) ? every parser: split parse/persist, reconcile via shared validation, bookend vs the bank's SUMMARY, record documents.status.
8 - [Scope: all personal finance](scope-all-personal-finance.md) ? muneem covers the WHOLE financial life; equity/MF/ETF/bond parsing + instrument-event classification are core near-term goals, not backlog. Don't over-fit the concept core to bank rows.
9 - [L3 vision + eval strategy](l3-vision-and-eval-strategy.md) ? L3 fallback = DeepSeek-OCR-3B (MIT), reconcile-gated; dev RTX 2070S (8GB/Turing) ? 4-bit + SDPA. Golden evals: local + synthetic first; cloud eval-only & approval-gated.
10 - [DB encryption at rest](db-encryption-at-rest.md) ? D.2 DECIDED & IMPLEMENTED: SQLCipher via `sqlcipher3` (transparent AES-256), 256-bit key in OS keychain; `db.connect_encrypted()`; `:memory:`/tests stay plaintext.
11 - [Open Banking + storage decisions](open-banking-and-storage-decisions.md) ? AA/open-banking DEFERRED (PDF-first; regulatory + cloud-decrypt barrier); storage = flat per-issuer source-of-truth tables + derived serving layer; provenance/confidence in a SEPARATE `provenance` table (not per-row); SMS-alert parsing is a local freshness candidate.
12

31. user (2026-06-10T16:36:58.908Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\memory\MEMORY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

32. assistant (2026-06-10T16:38:36.395Z)

Everything is verified, persisted to memory, and ready. Here is the full synthesis.

Deep review: muneem + gmail-attachments-to-gdrive

TL;DR: Engineering quality in both repos is genuinely high ? the review confirmed the hygiene claims are real, not aspirational (git history verifiably clean, 390 offline tests green in 6.4s at 86% coverage, invariant tripwires work). But it found **9 confirmed critical/high

bugs** (each independently verified by 3 adversarial agents; zero refuted), concentrated in exactly the two places you'd least want them: muneem's **encryption-key lifecycle** and the **arithmetic verification oracle that is the product's core promise**. The single biggest issue isn't code at all: the two repos' "binding" architecture documents **contradict each other head-on** about who reads Gmail. On direction: **finance is the right first step and it's already de-risked ? keep it. Grocery is the wrong *first* wedge but an excellent *second* vertical**, and the research surfaced a stronger public wedge than either: the **India tax-season document pack**. Details below; the full findings are saved to project memory ([deep-review-2026-06.md](C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\memory\deep-review-2026-06.md)).

---

1. Code issues (all confirmed by 3/3 adversarial verifiers)

muneem ? key lifecycle (the scariest cluster)

1. **`db rotate-key` can permanently lock you out of the ledger** ([cli.py:438](src/muneem/cli.py:438)). It overwrites the keychain with the *new* key **before** rekeying the file, and the rollback only catches `EncryptedDatabaseError` ? but `PRAGMA rekey` raises raw `sqlcipher3.OperationalError` (e.g. "database is locked", very plausible on Windows with antivirus holding the file), which escapes uncaught. Result: keychain holds the new key, file is still under the old key, old key is gone. Backups share the key, so they're lost too. The recovery file helps only if set up ? and it wraps the *old* key, so it silently goes stale after every successful rotation. The inline comment claims the opposite of what the code does, the failure path has **zero test coverage**, and docs/encryption-at-rest.md repeats the inverted claim.
2. **`muneem db backup <ledger-path>` destroys the ledger** ([db.py:292](src/muneem/db.py:292)) ? the destination is unlinked before any check that it isn't the live DB.
3. **A transient keychain read failure mints a fresh key over the real one** ([db_key.py:24](src/muneem/db_key.py:24)) ? `get_secret` swallows `KeyringError` as "no key", and `connect_encrypted` calls `get_or_create_key` even when an encrypted ledger already exists on disk, which is precisely the situation where minting is never the right answer.

These three share one theme: **the encryption is sound, but the key's lifecycle isn't atomic.** For a tool whose pitch is "hold my whole financial life for years," this is the same class of risk your own feedbackJune2026 §2 correctly named P0 ? the recovery file solved half of it; rotation/backup/mint re-opened the other half.

muneem ? the verification oracle has holes exactly where arithmetic can't see

4. **The savings oracle never checks the printed opening/closing bookends when the per-row balance walk passes** ([savings.py:78](src/muneem/parsers/savings.py:78)) ? a statement missing its leading or trailing transactions persists as `reconciled`. Empirically confirmed by the reviewer.
5. **`parse_amount` strips Cr/Dr suffixes without sign semantics** ([validation.py:32](src/muneem/parsers/validation.py:32)) ? an overdraft statement (Dr balances) reconciles perfectly with **every debit and credit inverted** and persists as verified. Also empirically confirmed. This is the worst-case failure mode your architecture exists to prevent: wrong rows that *pass* the oracle.
6. **The HDFC parser persists rows even when unreconciled**, and `txns list` never filters by document status ([hdfc.py:435](src/muneem/parsers/hdfc.py:435)) ? the reference implementation violates the store-nothing-unverified philosophy that Axis/AU follow.

Worth knowing too: the reconcile tolerance equals the currency quantum with a strict-greater compare, so ~42% of exact one-paisa corruptions pass the walk (measured by the reviewer); parser routing is bare-substring + alphabetical, so an HDFC statement containing an `@aubank` UPI handle in a narration routes to the AU parser; sha256 dedup permanently blocks re-ingesting a quarantined statement after you fix the parser; the Axis credit-card parser persists with **no verification at all**; and dates are never parsed/normalized ? raw locale strings go into the ledger.

gmail-attachments-to-gdrive

The backfill core is well-designed (the ±1-day window overlap, the trigger fan-out, the FIFO dedup all check out), but:

7. **Backfill advances `lastSynced` past windows that had attachment-level errors** (`src/sync.ts:213`) ? failed messages are never retried. The "mark done only on clean completion" invariant holds per-message but not per-window.
8. **The opt-in incremental sync advances the historyId cursor past messages truncated by the daily cap or transient failures** (`src/sync.ts:175`) ? silent data loss in steady-state mode.
9. **`GmailApp.getMessageById` throws for deleted messages** (`src/gmailHistory.ts:70`) ? one deleted email wedges incremental sync until the cursor expires.

Plus mediums that matter for the product story: `getOrCreateFolderInFolder(DriveApp, ?)` iterates **every folder in the user's Drive** (not root children); state persists only at run end, so the 6-minute hard kill loses bookkeeping; execution logs carry Gmail message ids into developer-visible Cloud Logging ? in tension with PRIVACY.md's "the author has no access to your data"; and the committed `timeZone` is America/Los_Angeles, which governs day boundaries for a Bangalore owner.

Security posture (muneem)

Genuinely strong ? and the review verified the ground truth: **nothing sensitive is tracked now or was ever committed** in either repo's full history. The residual risks are local: two JSON files in `testdata/snapshots/` contain **real parsed credit-card transactions in plaintext** on the same disk the ledger encrypts (written by a `snapshot` fixture in conftest that no test uses anymore ? a loaded footgun); the real `client_secret_*.json` is protected only by being inside `testdata/` ? Google's default download filename is not blocked by `.gitignore`, the pre-commit hook, or the hygiene test if it ever lands anywhere else; and there is **no lockfile or version pinning at all**, including for the `sqlcipher3` native wheel that provides the at-rest encryption itself.

---

2. The architecture issue that outranks all code findings

The gmail repo's **binding auditability invariant** (ADR-008, codified June 3) declares: *this repo is the only code that ever touches Gmail; muneem never holds the Gmail grant and receives only minimized, non-PII signal.* Meanwhile muneem **ships** full `gmail.readonly` OAuth ingestion and plans `muneem fetch` (Phase F). Neither repo references the other's constraint. And it's worse than a stale doc ? the contract itself is incoherent for the finance vertical:

- Bank statements **are PII through and through**; merchant names, amounts, and account identifiers *are* the signal. A "non-PII minimized" payload is useless to muneem.
- Apps Script **cannot open password-protected PDFs** (no decryption primitive), so the REDACT/MINIMIZE stage can't even read the document it's supposed to scrub.
- The sister repo frames muneem as "our own isolated, **hosted** service" ? but muneem's DESIGN declares hosted/cloud-sync a non-goal, period.
- The "versioned handoff contract" the entire decoupling rests on **exists in no artifact in either repo**.

There's a clean reconciliation neither doc names: **the sister repo archives raw attachments to the user's own Drive in-account (already its hardened core ? that's not egress), and muneem ingests from the locally-synced Drive folder.** Single Gmail grant, unlock stays local, no handoff contract needed, invariant intact. The pragmatic interim is to scope the invariant explicitly ("binds the public product; the owner's personal local tools are exempt"). Either way, this needs a one-page cross-repo ADR recorded in both repos ? right now two documents written the same week, both calling themselves binding, contradict each other silently, and it **blocks Phase F**.

---

3. Roadmap issues

muneem:
- **Two competing sources of truth** ? ROADMAP.md's Milestones/Tracks vs feedbackJune2026's

Phases A?H, with colliding letter namespaces ("Track A" ? "Phase A", "Milestone C" ? "Phase C"). A restart can't tell which plan governs. CURRENT_STATE.md ? the designated live-status doc ? still says "Schema v4" and describes per-row provenance as the planned direction, both superseded; parser-architecture.md still says Axis 2/4 and HDFC 4/4.
- **Phase C (instrument family) is the riskiest, least-specified phase**: no broker/MF/CAS corpus in hand (the bank parsers' credibility came precisely from real-statement validation), no oracle design (what arithmetic identity replaces the balance walk for holdings? dividends/fees have none), the source artifact (CDSL/NSDL CAS vs broker statement vs tradebook) not even parked as a decision ? and it's sequenced *before* the regression-CI safety net, contradicting your own "characterize failure modes before building" logic from the D.3-before-L3 push-back. Cheapest unverified thing already written: the Axis CC parser. Validate that first.
- **Nothing in the roadmap delivers daily user value.** After ~46 PRs the only read surface is `txns list <bank>` (50 raw rows, one bank). The Phase-B serving view was decided, promised alongside the provenance table ? and silently dropped when #46 shipped. Filters/analytics sit in Phase H, dead last. For a single-user tool, a weekly-usable "show me last month across all accounts" is what keeps you feeding it data through the long C?G middle.
- **Missing categories entirely:** cash/manual entry (the schema already reserves `source_type='manual'`), multi-year Gmail backfill + retention policy for raw PDFs in the staging dir (which sit *outside* SQLCipher's protection), quarantine UX for the 1-in-7 statement that stores nothing, the prescribed `docs/recovery.md` runbook, and a refreshed definition of done. Phase A is also quietly incomplete: no lockfile, structured logging stalled on the open log-format decision.

gmail repo: the vision and roadmap describe two different projects. ROADMAP.md is a rear-view mirror of completed copier hardening; **zero tracked items** exist for Phase 0 (EmailMessage ingest model), Phase 1 (classifier/extractors/Sheets index), the handoff contract, or the grocery vertical the VISION names *first*. Phase 2's in-account OCR has no feasibility math (and is impossible for locked PDFs). And two product-killing operational facts are documented as one-line caveats: **Testing-mode refresh tokens expire every 7 days** ? your <100 test users experience a background archiver that dies weekly, so retention signal from the probe will be noise ? and **user 101 is a hard CASA wall** (~$540?$4,500/yr recurring + ~2 months process; both `gmail.readonly` and full `drive` are restricted scopes). The distribution channel that scales past 100 with zero verification and *maximal* invariant compliance ? self-deployment ? exists only as unchecked roadmap stubs.

---

4. Product-market fit: is the direction reasonable?

Yes ? as a personal system with product-grade engineering, and as a niche open-source play. Not (yet) as a mainstream consumer product. The market research (mid-2026, sourced) says:

- **India PFM has consolidated on Account Aggregator rails** (Fold, INDmoney, Jupiter) ? cloud-first, free, lending-subsidized (the Walnut?Axio?Amazon arc shows tracking was always underwriting-data acquisition). Fold's tiny revenue after 6 years shows Indians don't pay for pure PFM. But AA's gaps are persistent and structural: weak credit-card detail, no bonds, no joint accounts, ~2y history, and **still no individual/self-custody path** ? your AA-deferral decision holds, and those gaps are exactly where reconciliation-gated PDF parsing is the only complete source.
- **CRED Protect already parses password-protected CC statement PDFs from Gmail, free.** You cannot win on convenience; you can win on locality, whole-ledger scope, and "every row verified or quarantined" ? a promise no competitor makes.
- **The attachment-archiving layer is fully commoditized** ? Digital Inspiration's add-on alone has 2M+ installs *and already decrypts password-protected bank PDFs*. Repo A's value is the classification front door and the auditable self-deploy trust model, never archiving itself.
- **The history of consumer email-parsing is a warning:** survivors either sold the data (Slice?NielsenIQ ? your privacy thesis forbids it), pivoted to bank feeds (Truebill?Rocket Money ? AA is that gravity in India), or owned one narrow high-pain vertical (TripIt). A public muneem must win the third way.
- **The moat is real but it's a maintenance moat** (validation discipline + issuer-template grind + regression corpora), not a defensibility moat. The OSS angle is underexploited: Actual/Firefly/beancount have **zero Indian PDF ingestion**; the beancount-India importers are CSV-only and stale. muneem as *the* verified ingestion layer for that ecosystem is a cheap distribution wedge into an existing community.

- **The 100-user probe is nearly free and sensible ? but measure self-deploy conversions, not Testing-mode retention** (which the 7-day token expiry contaminates). If organic self-deploys stall in the dozens: excellent personal tool, no product ? a fine outcome. If they reach hundreds, *then* the CASA toll is justified.

---

5. Grocery vs finance vs something else ? my recommendation

The question is partly a false binary, because the two repos answer it at different layers:

- **Finance stays first ? and it's not really a choice anymore: it's already built and proven** (HDFC 6/6, AU 4/4, Axis 6/7 on real data, with an architecture nothing in the Indian market matches). The marginal cost of continuing is low, the personal pain it solves is the highest, and it fills documented AA gaps. Abandoning a de-risked vertical to start a new one would be strictly worse.
- **Grocery is the wrong *first* consolidation target.** The killer is source coverage: Blinkit/Zepto/Instamart itemized GST invoices live primarily **in-app**, not reliably as email attachments ? an email-only front door under-covers the vertical from day one. The public demand signal is actively negative (the only receipt-collection businesses anywhere ? magicpin, Club Corra, Fetch ? must *pay* consumers for receipts; the value accrues to brands, monetized by exactly the data-resale your thesis forbids; Google's Gmail Purchases free-rides the generic version). **But grocery is an excellent *second* vertical and the right first *test corpus* for the gmail repo's classifier** ? receipts are low-sensitivity, so they're cloud-LLM-permitted under your own rules, which makes extractor iteration fast, exactly like the promo corpus did for muneem.
- **The strongest wedge you haven't considered: the India FY tax-season document pack.** Auto-collect Form 16, bank interest certificates, capital-gains statements (Zerodha/CAMS/KFintech), AIS/26AS alerts, and 80C/80D receipts into one CA-ready bundle every April?July. The sources are a finite, slow-churning set of large issuers that genuinely arrive by email; the pain is acute, seasonal, and universal among Indian filers; it has a natural annual-renewal pricing story; and it aligns exactly with the instrument-family work already on muneem's roadmap.
Subscriptions/e-mandate renewal detection is the runner-up ? the one email-adjacent category with globally proven willingness-to-pay (Rocket Money), and RBI's e-mandate regime generates a clean notification trail.

So: **muneem keeps grinding finance; the gmail repo's Phase-1 extractor targets grocery receipts as its proof-of-pipeline (which its own VISION already says, for the right reason ? forgiving fidelity bar); and if/when you want a public wedge, lead with the tax pack, distributed self-deploy-first.**

---

6. Improvement plan

Short-term (concrete, fixable now ? roughly in priority order):
1. **rotate-key atomicity** ? pending-key dance (or rekey-file-first + broad except + keep old key under `db-key-prev`), plus failure-path tests. Also: have rotation refresh/invalidate-with-warning the recovery file, which currently goes silently stale.
2. **`db backup` destination guard** (refuse the live ledger path) and **never-mint-when-ledger-exists** in `get_or_create_key`.
3. **Oracle integrity**: bookend check even when the walk passes; Cr/Dr sign semantics in `parse_amount`; stop persisting unreconciled HDFC rows (or make `txns list` status-aware); widen the one-paisa tolerance comparison.
4. **Hygiene**: delete the dead `snapshot` fixture + securely delete the two real-CC-row JSONs; block `client_secret*.json` in all three guard layers; add a lockfile (overdue for a compiled-dep project).
5. **gmail repo**: retry windows with attachment errors; don't advance historyId past unprocessed messages; try/catch `getMessageById`; fix the folder-iteration and timezone defaults.
6. **Docs (hours, high leverage)**: collapse sequencing into one owner; refresh CURRENT_STATE + parser-architecture; back-record the L3 decision in DESIGN §8.3.
7. **Ship the dropped Phase-B serving view + `muneem txns all --month`** ? days of work on a settled decision, and it's the first thing that makes the tool worth opening weekly.

Longer horizon:
- **Write the cross-repo ADR** (Gmail seam ? recommend the Drive-folder middle path; scope the invariant) and a handoff-contract v0 artifact.
- **Gate Phase C** behind an entry checklist (real broker/CAS corpus + a one-page holdings-oracle design + the artifact decision), and consider swapping it with Phase D (regression CI + D.3 eval) per your own sequencing logic. Verify the Axis CC parser on real data first.
- **Add the missing roadmap categories**: manual/cash entry, backfill + staging-dir retention policy, quarantine UX, recovery runbook, refreshed definition of done.
- **Make self-deployment the primary public distribution posture** for the gmail repo (it resolves the token-expiry, CASA, and auditability problems simultaneously), and measure README?completed-deploy conversion as the PMF metric.
- **Product sequencing**: finance (continue) ? grocery as front-door corpus ? tax-pack as the public wedge probe ? subscriptions as the WTP feature.

Everything above, with file:line evidence and verifier notes, is persisted in the project memory; the session handoff now points at it. If you want, I can start on the fix list ? the natural first PR is the rotate-key atomicity fix with its missing failure-path tests, one PR per item as usual.

33. user (2026-06-10T16:40:00.482Z)

Check on workflow wctcr4nab (deep review of muneem + gmail-attachments-to-gdrive). If complete, synthesize the final report answering: issues in code/architecture/roadmap, PMF assessment, short-term vs long-term improvements, and the grocery-vs-finance first-step recommendation.

34. assistant (2026-06-10T16:40:10.737Z)

Already done ? the wakeup was just the fallback heartbeat. Workflow `wctcr4nab` completed (35 agents, 9/9 critical/high findings confirmed by adversarial verification), and the full synthesis was delivered in my previous message: code/architecture/roadmap issues, the PMF assessment, short-term vs long-term improvement plans, and the recommendation (finance stays first; grocery as the gmail repo's second/test vertical; the India tax-season pack as the strongest public wedge).

The findings are also persisted durably: [deep-review-2026-06.md](C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\memory\deep-review-2026-06.md) holds the full detail, and the [session handoff](C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\memory\session-handoff.md) points at it with the proposed next steps.

Nothing is pending on my side ? the next move is yours: pick which fixes to start (my suggested first PR is the `rotate-key` atomicity fix plus its missing failure-path tests), and weigh in on the cross-repo Gmail-seam decision and the wedge choice.

35. user (2026-06-10T16:58:51.527Z)

Can you make a plan of first fixing what is checked in and making it bulletproof while I deeply scan all your amazing findings and refresh my brain cache as to how such inconsistencies arose? Please show me a plan of maximum fixes or improvements and then you can go and help make this repo more civilized

36. assistant (2026-06-10T17:00:44.582Z)

Plan first, then I'll get to work. Scope note: #48/#49 already closed the serving-view and CURRENT_STATE items, and #48's deprecation trigger (retire the sister archiver once `fetch` ships) means the gmail repo gets the data-loss fixes only, last ? it's still running daily for you today, so they're worth having, but muneem is the focus. Open decisions (log format, self-hosted runner, macOS, the cross-repo seam, instrument family) are **excluded** ? those are yours.

The "make it civilized" plan ? maximum fixes, one PR each, squash-merged as verifie

Track A ? the ledger can never be lost (P0)

PR	Fix
A1	`rotate-key` atomicity: pending-key protocol (write fresh under `db-key-pending`, rekey file, promote, keep old under `db-key-prev`), rollback catches *all* exceptions, failure-path tests (incl. raw `sqlcipher3` errors + interrupt), fix the inverted comment + [encryption-at-rest.md](docs/encryption-at-rest.md), and warn that the recovery file is stale after rotation
A2	`db backup` guard: refuse when destination resolves to the live ledger (or any existing non-backup file) *before* unlinking; tests
A3	Key-mint guard: `KeyringError` no longer masquerades as "no key"; `connect_encrypted` never mints when an encrypted ledger already exists on disk ? loud recovery-pointing error instead

Track B ? the oracle keeps its promise (correctness of the core guarantee)
PR	Fix
B1	Savings oracle checks opening/closing bookends *always*, not only when the walk fails (kills the missing-leading/trailing-rows false "reconciled")
B2	`parse_amount` honors Cr/Dr suffix semantics ? overdraft statements can no longer reconcile inverted
B3	HDFC parity: quarantine on unreconciled (store nothing, like Axis/AU) + `txns list` becomes status-aware
B4	Tighten reconcile tolerance so an exact one-paisa corruption cannot pass the walk
B5	Parser routing anchored to header/sender evidence, not bare substring anywhere in the text (`@aubank` in a narration no longer hijacks routing)
B6	Status-aware dedup: a quarantined document can be re-ingested after a parser fix
B7	Axis CC verification floor: use whatever arithmetic the bill offers (summary cross-check), else persist with an explicit unverified status ? never silently "fine"
B8	Date normalization at the engine boundary (DD/MM/YYYY ? ISO, validated)

Track C ? security hygiene
PR	Fix
C1	Delete the dead `snapshot` fixture + the two real-CC-row JSONs in `testdata/snapshots/` (regenerable from your PDFs); block `client_secret*.json` / `*.apps.googleusercontent.com.json` in `.gitignore` + pre-commit hook + hygiene test; fix stale `FORBIDDEN_PREFIXES`
C2	Dependency pinning: constraints lockfile + CI installs against it (pip-native, no new tooling)
C3	Migrations: wrap each forward migration's DDL + record in one transaction; re-verify recorded checksums; partial-failure test
C4	Small-fixes bundle: 64-hex validation before the key PRAGMA splice; ATTACH path quoting; redaction patterns for PAN / spaced cards / Aadhaar + filter on the root logger; `unlock` mkdir parity; recovery KDF params inside the malformed-payload guard

Track D ? test depth
PR	Fix
D1	Synthetic corpus: Axis savings + Axis CC born-digital fixtures, multi-page + wrapped-narration + dropped-row-must-quarantine cases, password-protected PDF through the real CLI
D2	OCR CI job on a weekly cron; redact real filenames from local-test pytest ids

Track E ? docs that won't mislead a restart
PR	Fix
E1	parser-architecture.md refresh (6/6, 4/4, 6/7, balance-delta documented; §9/§11 annotated with the §8.7 resolution); back-record the L3 model in DESIGN §8.3 + ROADMAP §1; synthetic-fixture wording in TESTING.md/onboarding-a-bank/.gitignore; README status + macOS claim
E2	`docs/recovery.md` runbook (keychain lost / new machine / verify a backup / migrate-then-reverify) ? prescribed by feedbackJune2026 §2/P6
E3	Sequencing consolidation: ROADMAP.md becomes the single owner of Phases A?H, letter collisions renamed, phase statuses + exit criteria recorded (mechanical, no decisions changed)

Track F ? sister repo (last; lives until the deprecation trigger fires)
F1 backfill retries windows with attachment errors · F2 incremental-sync trio (historyId not advanced past unprocessed, `getMessageById` guarded, spam/trash filter) · F3 small fixes (folder iteration, `Asia/Kolkata` default, per-user summary recipient).

Order of execution: A ? B ? C ? D ? E ? F, gate green (`ruff` + `mypy` + full offline suite)
before each merge. Starting now with A1.

37. user (2026-06-10T17:00:45.084Z)

```
1       """The ledger's encryption key ? minted once, kept in the OS keychain.
2
3       muneem's local ledger is **encrypted at rest** with SQLCipher (DESIGN.md §5.4 rule 4).
4       The 256-bit key is generated on first use and stored in the OS keychain via
5       :mod:`muneem.secrets` (Windows Credential Manager / macOS Keychain / Secret Service) ?
6       never on disk in plaintext, never in the repo, never logged.
7
8       It is handed to SQLCipher in **raw-key form** (``PRAGMA key = "x'<64 hex>'"``), which
uses
9       the 256-bit key directly and *skips* the passphrase KDF. That is correct here because
the
10      key is already full-entropy random; KDF stretching only buys security for low-entropy
human
11      passphrases (an opt-in we may add later ? see ROADMAP D.2 phased plan).
12      """
13
14      from __future__ import annotations
15
16      import secrets as _stdlib_secrets
17
18      from muneem import secrets as _keychain
19
20      # Keychain entry name (under the shared "muneem" service) holding the 64-hex-char DB
key.
21      _DB_KEY_NAME = "db-key"
22
23
24      def get_or_create_key() -> str:
25          """Return the ledger's 256-bit key as 64 hex chars, minting+storing one on first
use."""
26          existing = _keychain.get_secret(_DB_KEY_NAME)
27          if existing:
28              return existing
29          key = _stdlib_secrets.token_bytes(32).hex()
30          _keychain.set_secret(_DB_KEY_NAME, key)
31          return key
32
33
34      def peek_key() -> str | None:
35          """Return the stored key, or ``None`` if none has been minted yet. Never creates
one."""
36          return _keychain.get_secret(_DB_KEY_NAME)
37
38
39      def new_key() -> str:
40          """Generate a fresh random 256-bit key as 64 hex chars (not stored)."""
41          return _stdlib_secrets.token_bytes(32).hex()
42
43
44      def set_key(key: str) -> None:
45          """Overwrite the stored DB key ? used by key rotation, **after** the file has been
rekeyed."""
46          _keychain.set_secret(_DB_KEY_NAME, key)
47
```

38. user (2026-06-10T17:00:45.502Z)

```
1       """Secret storage via the OS keychain.
2
3       OAuth tokens, cached statement passwords, and the DB encryption key are stored here
```

```
(Windows
4       Credential Manager / macOS Keychain / Secret Service), never in the repo, on disk in
plaintext, or
5       in logs. See DESIGN.md section 5.
6
7       External-call discipline (every keyring call is wrapped with attribution):
8       - **read** (:func:`get_secret`) is *graceful* ? a missing or unavailable backend returns
``None``
9         (absence is a valid state; this also keeps headless CI happy).
10      - **write** (:func:`set_secret`) is *loud* ? a failure raises
:class:`~muneem.errors.KeychainError`,
11        because silently losing a write (e.g. the DB key) would be dangerous. The secret
*name* is
12        included for diagnosis; the value never is.
13      """
14
15      from __future__ import annotations
16
17      import keyring
18      from keyring.errors import KeyringError, PasswordDeleteError
19
20      from muneem.errors import KeychainError
21
22      _SERVICE = "muneem"
23
24
25      def get_secret(name: str) -> str | None:
26          """Return the stored secret, or ``None`` if absent or the keychain is
unavailable."""
27          try:
28              return keyring.get_password(_SERVICE, name)
29          except KeyringError:
30              return None
31
32
33      def set_secret(name: str, value: str) -> None:
34          """Store (or replace) a secret in the OS keychain. Raises KeychainError on
failure."""
35          try:
36              keyring.set_password(_SERVICE, name, value)
37          except KeyringError as exc:
38              # Name only ? never the value ? in the message (it may be a password or the DB
key).
39              raise KeychainError(f"could not store secret {name!r} in the OS keychain") from
exc
40
41
42      def delete_secret(name: str) -> None:
43          """Delete a secret if present; a no-op if it is not (or the keychain is
unavailable)."""
44          try:
45              keyring.delete_password(_SERVICE, name)
46          except (PasswordDeleteError, KeyringError):
47              pass
48
```

39. user (2026-06-10T17:00:52.790Z)

```
1       """Local SQLite store: the ``documents`` and ``offers`` slice of the schema.
2
3       This is the minimal slice the promo spine needs (ROADMAP A.2). The full schema
4       (accounts, transactions, holdings, merchants, items) grows with later milestones.
5       ``documents.sha256`` is UNIQUE so re-ingesting the same file is a no-op.
6       ``offers.source`` records provenance ('text' = exact text layer, 'ocr' = approximate).
7       """
```

```

    from __future__ import annotations

    import os
    import sqlite3
    from collections.abc import Iterator
    from contextlib import contextmanager
    from pathlib import Path

    from muneem.errors import EncryptedDatabaseError  # re-exported so
db.EncryptedDatabaseError works

    SCHEMA_VERSION = 4

    # Unencrypted SQLite files start with this 16-byte magic header; an encrypted
(SQLCipher)
    # file does not, so we can tell them apart without a key.
    _SQLITE_MAGIC = b"SQLite format 3\x00"

    _SCHEMA = """
    CREATE TABLE IF NOT EXISTS documents (
        id            INTEGER PRIMARY KEY,
        source        TEXT NOT NULL,
        external_id   TEXT,
        sender        TEXT,
        subject       TEXT,
        received_date TEXT,
        doc_type      TEXT,
        sha256        TEXT NOT NULL UNIQUE,
        path          TEXT,
        status        TEXT NOT NULL DEFAULT 'parsed',
        created_at    TEXT NOT NULL DEFAULT (datetime('now'))
    );

    CREATE TABLE IF NOT EXISTS offers (
        id          INTEGER PRIMARY KEY,
        document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
        merchant    TEXT,
        promo_code  TEXT,
        description TEXT,
        expiry_raw  TEXT,
        expiry_type TEXT,
        source      TEXT NOT NULL DEFAULT 'text',
        redeemed    INTEGER NOT NULL DEFAULT 0,
        source_text TEXT
    );

    CREATE INDEX IF NOT EXISTS idx_offers_document ON offers(document_id);

    -- We use separate tables for each bank (e.g. axis_transactions, hdfc_transactions)
    -- instead of a unified table to preserve raw schema fidelity of the source PDFs.
    -- Normalization into a unified view layer is planned for a later phase
    -- once all bank schemas are stabilized.
    CREATE TABLE IF NOT EXISTS axis_transactions (
        id INTEGER PRIMARY KEY,
        document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
        account_number TEXT,
        txn_date TEXT,
        value_date TEXT,
        particulars TEXT,
        debit REAL,
        credit REAL,
        balance REAL
    );

```

```
    CREATE TABLE IF NOT EXISTS axis_cc_transactions (
        id INTEGER PRIMARY KEY,
        document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
        account_number TEXT,
        txn_date TEXT,
        particulars TEXT,
        merchant_category TEXT,
        amount REAL,
        type TEXT
    );

    CREATE TABLE IF NOT EXISTS au_transactions (
        id INTEGER PRIMARY KEY,
        document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
        account_number TEXT,
        txn_date TEXT,
        value_date TEXT,
        particulars TEXT,
        debit REAL,
        credit REAL,
        balance REAL
    );

    CREATE TABLE IF NOT EXISTS hdfc_transactions (
        id INTEGER PRIMARY KEY,
        document_id INTEGER NOT NULL REFERENCES documents(id) ON DELETE CASCADE,
        account_number TEXT,
        txn_date TEXT,
        value_date TEXT,
        particulars TEXT,
        ref TEXT,
        withdrawal REAL,
        deposit REAL,
        closing_balance REAL
    );
    """


    def _raw_key_pragma(key: str) -> str:
        """The SQLCipher raw-key PRAGMA: use the 256-bit ``key`` (64 hex chars) directly, no
KDF."""
        return f"PRAGMA key = \"x'{key}'\""


    def is_plaintext_sqlite(db_path: Path | str) -> bool:
        """True iff the file exists and begins with the *unencrypted* SQLite magic header.

        Lets us refuse to silently treat a plaintext ledger as encrypted (and vice-versa)
        without needing the key.
        """
        try:
            with open(db_path, "rb") as fh:
                return fh.read(16) == _SQLITE_MAGIC
        except OSError:
            return False


    def _harden_permissions(path: str) -> None:
        """Best-effort: restrict the DB file to its owner (0600). No-op on failure or
``:memory:``."""
        if path == ":memory:":
            return
        try:
            os.chmod(path, 0o600)
        except OSError:
```

```
            pass


    def _open_encrypted(path: str, key: str) -> sqlite3.Connection:
        """Open an encrypted connection via SQLCipher, verifying the key before returning.

        A keyed open NEVER silently downgrades to plaintext: if the binding is missing, or
the
        key is wrong (or the file is not an encrypted muneem ledger), this raises.
        """
        try:
            import sqlcipher3
        except ImportError as exc:  # pragma: no cover - exercised only without the binding
            raise EncryptedDatabaseError(
                "database encryption requested but the 'sqlcipher3' package is not
installed"
            ) from exc

        conn = sqlcipher3.connect(path)
        conn.execute(_raw_key_pragma(key))  # MUST be the first statement on the connection
        try:
            conn.execute("SELECT count(*) FROM sqlite_master")  # forces key verification
        except sqlcipher3.DatabaseError as exc:
            conn.close()
            raise EncryptedDatabaseError(
                f"could not open encrypted database {Path(path).name}: wrong key, or not an
"
                "encrypted muneem ledger"
            ) from exc
        conn.row_factory = sqlcipher3.Row
        return conn


    @contextmanager
    def connect(db_path: Path | str, *, key: str | None = None) ->
Iterator[sqlite3.Connection]:
        """Open a connection (commit on success, always close). Parent dirs created.

        With ``key`` (a 64-hex-char SQLCipher key) the database is **encrypted at rest** via
        SQLCipher. Without it, a plaintext stdlib :mod:`sqlite3` connection is used ? the
path for
        ``:memory:`` and tests. A keyed open never silently falls back to plaintext (see
        :func:`_open_encrypted`); the production ledger is opened via
:func:`connect_encrypted`.
        """
        path = str(db_path)
        if path != ":memory:":
            Path(path).parent.mkdir(parents=True, exist_ok=True)
        if key is not None:
            conn = _open_encrypted(path, key)
            _harden_permissions(path)
        else:
            conn = sqlite3.connect(path)
            conn.row_factory = sqlite3.Row
        conn.execute("PRAGMA foreign_keys = ON")
        try:
            yield conn
            conn.commit()
        finally:
            conn.close()


    @contextmanager
    def connect_encrypted(db_path: Path | str) -> Iterator[sqlite3.Connection]:
        """Open the real on-disk ledger, encrypted with the keychain-managed key.
```

```
193
194         The production path: resolves (or mints, on first use) the 256-bit key from the OS
195         keychain and opens with SQLCipher. If ``db_path`` is a pre-existing *plaintext*
SQLite
196         file, refuses with a clear pointer to ``muneem db encrypt`` rather than a cryptic
197         "file is not a database" error.
198         """
199         from muneem.db_key import get_or_create_key
200
201         path = str(db_path)
202         if is_plaintext_sqlite(path):
203             raise EncryptedDatabaseError(
204                 f"{Path(path).name} is an unencrypted SQLite database; run 'muneem db
encrypt' "
205                 "to migrate it to an encrypted ledger"
206             )
207         with connect(path, key=get_or_create_key()) as conn:
208             yield conn
209
210
211     def encrypt_database(db_path: Path | str, key: str) -> int:
212         """Migrate a plaintext SQLite ledger to an encrypted one in place, via
``sqlcipher_export``.
213
214         The original is preserved as ``<name>.plaintext.bak`` for the caller to verify and
then
215         securely delete (it still holds the cleartext data). Returns the number of user
tables
216         copied. Raises :class:`EncryptedDatabaseError` if ``db_path`` is not a plaintext
SQLite file.
217         """
218         import sqlcipher3
219
220         path = Path(db_path)
221         if not is_plaintext_sqlite(path):
222             raise EncryptedDatabaseError(
223                 f"{path.name} is not a plaintext SQLite database (already encrypted or
missing)"
224             )
225
226         enc_tmp = path.with_name(path.name + ".enc.tmp")
227         enc_tmp.unlink(missing_ok=True)
228         try:
229             src = sqlcipher3.connect(str(path))  # open the plaintext source (no key)
230             try:
231                 src_tables = src.execute(
232                     "SELECT count(*) FROM sqlite_master WHERE type = 'table'"
233                 ).fetchone()[0]
234                 target = str(enc_tmp).replace("\\", "/")  # SQL string literal: forward
slashes
235                 src.execute(f"ATTACH DATABASE '{target}' AS encrypted KEY \"x'{key}'\"")
236                 src.execute("SELECT sqlcipher_export('encrypted')")
237                 src.execute("DETACH DATABASE encrypted")
238             finally:
239                 src.close()
240
241             # Verify the encrypted copy BEFORE the destructive swap: it must open with the
key, pass
242             # an integrity check, and carry every table across. (The plaintext .bak is kept
either way,
243             # so a failed check is recoverable ? but we refuse to swap in a bad file.)
244             with connect(str(enc_tmp), key=key) as check:
245                 integrity = check.execute("PRAGMA integrity_check").fetchone()[0]
246                 ntables = check.execute(
247                     "SELECT count(*) FROM sqlite_master WHERE type = 'table'"
```

```
                ).fetchone()[0]
            if integrity != "ok" or ntables != src_tables:
                raise EncryptedDatabaseError(
                    f"encryption migration verification failed for {path.name} "
                    f"(integrity={integrity!r}, tables {ntables}/{src_tables}); original
left untouched"
                )

            backup = path.with_name(path.name + ".plaintext.bak")
            backup.unlink(missing_ok=True)
            path.rename(backup)
            enc_tmp.rename(path)  # consumed on success
            _harden_permissions(str(path))
        finally:
            # Never leave a partial temp behind: no-op on success (already renamed), cleans
up on error.
            enc_tmp.unlink(missing_ok=True)
        return int(ntables)


    def rekey_database(db_path: Path | str, current_key: str, new_key: str) -> None:
        """Re-encrypt the ledger under ``new_key`` in place (SQLCipher ``PRAGMA rekey``).

        Opens with ``current_key`` first (which verifies it), then rekeys. Raises
        :class:`EncryptedDatabaseError` if ``current_key`` is wrong or the file is not an
encrypted
        muneem ledger. The caller is responsible for updating the keychain entry to
``new_key`` only
        after this returns successfully.
        """
        conn = _open_encrypted(str(db_path), current_key)  # validates current_key
        try:
            conn.execute(f"PRAGMA rekey = \"x'{new_key}'\"")
            conn.commit()
        finally:
            conn.close()
        _harden_permissions(str(db_path))


    def backup_database(db_path: Path | str, dest: Path | str, key: str) -> None:
        """Write an **encrypted** online-backup copy of the ledger to ``dest`` (same key).

        Uses the SQLite Online Backup API for a consistent snapshot ? safe even with WAL
active, so no
        naive file copy and no manual checkpoint needed. The destination is keyed with the
same key, so
        it stays encrypted at rest. Raises :class:`EncryptedDatabaseError` if ``key`` is
wrong.
        """
        import sqlcipher3

        dest_path = Path(dest)
        dest_path.parent.mkdir(parents=True, exist_ok=True)
        dest_path.unlink(missing_ok=True)
        source = _open_encrypted(str(db_path), key)  # validates key
        try:
            target = sqlcipher3.connect(str(dest_path))
            try:
                target.execute(_raw_key_pragma(key))  # key the destination so the backup is
encrypted
                source.backup(target)  # consistent snapshot of the live DB
            finally:
                target.close()
        finally:
            source.close()
```

```
305         _harden_permissions(str(dest_path))
306
307
308     def init_db(conn: sqlite3.Connection) -> None:
309         """Bring the schema up to date via the migration runner (creates tables on a fresh
DB).
310
311         The current ``_SCHEMA`` is the frozen *baseline* (version ``SCHEMA_VERSION``);
future changes
312         are forward migrations in :mod:`muneem.migrations`, recorded in
``schema_migrations``.
313         """
314         from muneem import migrations
315
316         migrations.run_migrations(conn, baseline_sql=_SCHEMA,
baseline_version=SCHEMA_VERSION)
317
318
319     def get_document_by_sha(conn: sqlite3.Connection, sha256: str) -> sqlite3.Row | None:
320         return conn.execute("SELECT * FROM documents WHERE sha256 = ?",
(sha256,)).fetchone()
321
322
323     def _insert_row(conn: sqlite3.Connection, table: str, fields: dict) -> int:
324         """Internal helper for the simple INSERT pattern used by documents, offers, and bank
txns."""
325         cols = ", ".join(fields)
326         placeholders = ", ".join("?" for _ in fields)
327         cur = conn.execute(
328             f"INSERT INTO {table} ({cols}) VALUES ({placeholders})",
329             tuple(fields.values()),
330         )
331         rowid = cur.lastrowid  # always set after a successful INSERT; guard for the type-
checker
332         if rowid is None:  # pragma: no cover - defensive; INSERT always sets lastrowid
333             raise RuntimeError(f"INSERT into {table} did not return a row id")
334         return int(rowid)
335
336
337     def insert_document(conn: sqlite3.Connection, **fields: object) -> int:
338         """Insert into documents table. Delegates to the shared helper for DRY."""
339         return _insert_row(conn, "documents", dict(fields))
340
341
342     def record_provenance(
343         conn: sqlite3.Connection,
344         *,
345         document_id: int,
346         source_type: str,
347         parser: str | None = None,
348         parser_version: str | None = None,
349         mapping_layer: str | None = None,
350         confidence: str | None = None,
351     ) -> None:
352         """Upsert the provenance row for a parse (one per document ? see DESIGN §8.7).
353
354         First-class but *not* per transaction row: every txn table FKs ``documents(id)``, so
a single
355         provenance record per document is reachable by join. Idempotent on ``document_id``
(a re-parse
356         overwrites rather than duplicating).
357         """
358         conn.execute(
359             "INSERT INTO provenance "
360             "(document_id, source_type, parser, parser_version, mapping_layer, confidence) "
```

```
            "VALUES (?, ?, ?, ?, ?, ?) "
            "ON CONFLICT(document_id) DO UPDATE SET "
            "source_type=excluded.source_type, parser=excluded.parser, "
            "parser_version=excluded.parser_version, mapping_layer=excluded.mapping_layer, "
            "confidence=excluded.confidence",
            (document_id, source_type, parser, parser_version, mapping_layer, confidence),
        )


    def record_parse(
        conn: sqlite3.Connection,
        document_id: int,
        *,
        confidence: str,
        parser: str,
        source_type: str = "pdf_statement",
        parser_version: str | None = None,
        mapping_layer: str = "deterministic",
    ) -> None:
        """Record a parse outcome: set ``documents.status`` and upsert provenance (one per
parse)."""
        conn.execute("UPDATE documents SET status = ? WHERE id = ?", (confidence,
document_id))
        record_provenance(
            conn,
            document_id=document_id,
            source_type=source_type,
            parser=parser,
            parser_version=parser_version,
            mapping_layer=mapping_layer,
            confidence=confidence,
        )


    def get_provenance(conn: sqlite3.Connection, document_id: int) -> sqlite3.Row | None:
        return conn.execute("SELECT * FROM provenance WHERE document_id = ?",
(document_id,)).fetchone()


    def insert_offer(conn: sqlite3.Connection, **fields: object) -> int:
        return _insert_row(conn, "offers", fields)


    def list_offers(conn: sqlite3.Connection) -> list[sqlite3.Row]:
        return conn.execute(
            """
            SELECT o.id, o.merchant, o.promo_code, o.expiry_raw, o.description, o.source,
                   d.external_id, d.path
            FROM offers o
            JOIN documents d ON d.id = o.document_id
            ORDER BY o.id
            """
        ).fetchall()


    def insert_axis_transaction(conn: sqlite3.Connection, **fields: object) -> int:
        return _insert_row(conn, "axis_transactions", fields)


    def insert_axis_cc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
        return _insert_row(conn, "axis_cc_transactions", fields)


    def list_axis_cc_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
```

```
422         return conn.execute(
423             "SELECT * FROM axis_cc_transactions ORDER BY id DESC LIMIT ?", (limit,)
424         ).fetchall()
425
426
427     def list_axis_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
428         return conn.execute(
429             "SELECT * FROM axis_transactions ORDER BY id DESC LIMIT ?", (limit,)
430         ).fetchall()
431
432
433     def insert_hdfc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
434         return _insert_row(conn, "hdfc_transactions", fields)
435
436
437     def list_hdfc_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
438         return conn.execute(
439             "SELECT * FROM hdfc_transactions ORDER BY id DESC LIMIT ?", (limit,)
440         ).fetchall()
441
442
443     def insert_au_transaction(conn: sqlite3.Connection, **fields: object) -> int:
444         return _insert_row(conn, "au_transactions", fields)
445
446
447     def list_all_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
448         """Cross-bank rows from the derived serving view, with each parse's confidence
joined in.
449
450         Read-only over ``v_transactions`` (migration v6) ? the flat per-issuer tables remain
the
451         source of truth (DESIGN §8.7).
452         """
453         return conn.execute(
454             """
455             SELECT v.*, p.confidence
456             FROM v_transactions v
457             LEFT JOIN provenance p ON p.document_id = v.document_id
458             ORDER BY v.document_id DESC
459             LIMIT ?
460             """,
461             (limit,),
462         ).fetchall()
463
464
465     def list_au_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
466         return conn.execute(
467             "SELECT * FROM au_transactions ORDER BY id DESC LIMIT ?", (limit,)
468         ).fetchall()
469
```

40. user (2026-06-10T17:01:16.273Z)

```
Task #1 created successfully: A1: rotate-key atomicity (pending-key protocol + broad rollback +
tests + docs)
```

41. user (2026-06-10T17:01:17.791Z)

```
Task #2 created successfully: A2: db backup destination guard
```

42. user (2026-06-10T17:01:20.039Z)

```
Task #3 created successfully: A3: key-mint guard (keychain error != absent; never mint over
existing ledger)
```

43. user (2026-06-10T17:01:21.820Z)

```
Task #4 created successfully: B1: savings oracle always checks bookends
```

44. user (2026-06-10T17:01:23.401Z)

```
Task #5 created successfully: B2: Cr/Dr suffix sign semantics in parse_amount
```

45. user (2026-06-10T17:01:54.355Z)

```
Task #21 created successfully: F1?F3: sister repo data-loss fixes (backfill retry; incremental-
sync trio; small fixes)
```

46. user (2026-06-10T17:01:25.279Z)

```
Task #6 created successfully: B3: HDFC quarantine parity + status-aware txns list
```

47. user (2026-06-10T17:01:27.163Z)

```
Task #7 created successfully: B4: tighten reconcile tolerance (one-paisa corruption must fail)
```

48. user (2026-06-10T17:01:28.616Z)

```
Task #8 created successfully: B5: anchor parser routing (no bare-substring hijack)
```

49. user (2026-06-10T17:01:30.460Z)

```
Task #9 created successfully: B6: status-aware dedup (re-ingest after parser fix)
```

50. user (2026-06-10T17:01:32.461Z)

```
Task #10 created successfully: B7: Axis CC verification floor
```

51. user (2026-06-10T17:01:33.757Z)

```
Task #11 created successfully: B8: date normalization at engine boundary
```

52. user (2026-06-10T17:01:36.606Z)

```
Task #12 created successfully: C1: hygiene ? delete dead snapshot fixture + real-CC JSONs; block
client_secret patterns
```

53. user (2026-06-10T17:01:38.001Z)

```
Task #13 created successfully: C2: dependency pinning (constraints lockfile + CI)
```

54. user (2026-06-10T17:01:39.874Z)

```
Task #14 created successfully: C3: migrations ? per-migration transaction + checksum re-verify +
partial-failure test
```

55. user (2026-06-10T17:01:42.700Z)

```
Task #15 created successfully: C4: small-fixes bundle (key hex validation, ATTACH quoting,
redaction patterns, root-logger filter, unlock mkdir, recovery KDF guard)
```

56. user (2026-06-10T17:01:44.640Z)

```
Task #16 created successfully: D1: broaden synthetic corpus (Axis savings/CC fixtures, multi-
page, wrapped narration, dropped-row, encrypted PDF through CLI)
```

57. user (2026-06-10T17:01:46.052Z)

```
Task #17 created successfully: D2: OCR CI cron + pytest-id redaction for local tests
```

58. user (2026-06-10T17:01:48.962Z)

```
Task #18 created successfully: E1: docs refresh (parser-architecture, L3 back-record, synthetic
wording, README)
```

59. user (2026-06-10T17:01:50.297Z)

```
Task #19 created successfully: E2: docs/recovery.md runbook
```

60. user (2026-06-10T17:01:52.177Z)

```
Task #20 created successfully: E3: sequencing consolidation (ROADMAP owns Phases A?H)
```

61. user (2026-06-10T17:02:01.960Z)

```
Updated task #1 status
```

62. user (2026-06-10T17:02:03.586Z)

```
389:    from muneem.db_key import peek_key
392:    key_state = "present in keychain" if peek_key() else "not yet minted"
412:    from muneem.db_key import get_or_create_key
427:        f"encrypted {Path(db_path).name}: {n} table(s) migrated. A plaintext backup remains
at "
435:    from muneem.db_key import peek_key
440:    key = peek_key()
447:@db_app.command("rotate-key")
448:def db_rotate_key(
451:    """Rotate the ledger's encryption key (SQLCipher rekey) and update the keychain."""
453:    from muneem import db_key
457:    fresh = db_key.new_key()
458:    # Keychain-first with rollback: if the file rekey fails, the keychain still matches the
file.
459:    db_key.set_key(fresh)
461:        dbmod.rekey_database(db_path, current, fresh)
463:        db_key.set_key(current)  # roll back; the file is still under the old key
466:    typer.echo(f"rotated the encryption key for {Path(db_path).name}.")
469:@db_app.command("backup")
470:def db_backup(
471:    dest: Path = typer.Argument(..., help="Destination path for the encrypted backup
copy."),
474:    """Write an encrypted online-backup copy of the ledger (consistent even with WAL
active)."""
480:        dbmod.backup_database(db_path, dest, key)
487:@db_app.command("setup-recovery")
488:def db_setup_recovery(
490:        None, help="Recovery file path (default: <data dir>/muneem-recovery.json)."
493:    """Create a passphrase-wrapped recovery file for the ledger's encryption key.
495:    Lets you restore the key (and thus the ledger + backups) if the OS keychain is ever
lost. The
500:    from muneem import recovery
501:    from muneem.db_key import peek_key
503:    key = peek_key()
507:    out_path = out or (load_config().data_dir / "muneem-recovery.json")
508:    passphrase = getpass.getpass("New recovery passphrase: ")
513:        recovery.write_recovery_file(out_path, key, passphrase)
514:    except recovery.RecoveryError as exc:
```

```
518:        f"wrote recovery file to {out_path}. Store it OFF this machine (password manager / "
519:        "separate drive) and remember the passphrase ? you need BOTH to recover. Without
them, a "
520:        "lost keychain means the ledger and its backups are unrecoverable."
524:@db_app.command("recover")
525:def db_recover(
529:    """Restore the ledger's encryption key from a recovery file + passphrase (into the
keychain)."""
533:    from muneem import db_key, recovery
537:        key = recovery.read_recovery_file(file, passphrase)
538:    except recovery.RecoveryError as exc:
542:    # If the ledger is present, verify the recovered key actually opens it before committing
it.
548:            typer.echo("recovered a key, but it does not open the current ledger.",
err=True)
550:    db_key.set_key(key)
551:    typer.echo("recovered the encryption key into the keychain.")
```

63. user (2026-06-10T17:02:10.318Z)

```
380             typer.echo(f"[{row['account_number']}] {row['txn_date']} : {part:<50} |
{amount:>10}")
381
382
383     @db_app.command("status")
384     def db_status(
385         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
386     ) -> None:
387         """Show the ledger's encryption state (no statement data is read or printed)."""
388         from muneem import db as dbmod
389         from muneem.db_key import peek_key
390
391         db_path = db or load_config().db_path
392         key_state = "present in keychain" if peek_key() else "not yet minted"
393         if not Path(db_path).exists():
394             typer.echo(f"ledger : {db_path} (does not exist yet)")
395             typer.echo(f"key    : {key_state}")
396             return
397         if dbmod.is_plaintext_sqlite(db_path):
398             state = "PLAINTEXT ? run 'muneem db encrypt' to harden it"
399         else:
400             state = "encrypted (SQLCipher)"
401         typer.echo(f"ledger : {db_path}")
402         typer.echo(f"state  : {state}")
403         typer.echo(f"key    : {key_state}")
404
405
406     @db_app.command("encrypt")
407     def db_encrypt(
408         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
409     ) -> None:
410         """Migrate an existing *plaintext* ledger to an encrypted one (SQLCipher), in
place."""
411         from muneem import db as dbmod
412         from muneem.db_key import get_or_create_key
413
414         db_path = db or load_config().db_path
415         if not Path(db_path).exists():
416             typer.echo(
417                 f"nothing to encrypt: {db_path} does not exist (a new ledger is born
encrypted)."
418             )
419             return
```

```
        if not dbmod.is_plaintext_sqlite(db_path):
            typer.echo(
                f"{Path(db_path).name} is already encrypted (or not a SQLite file); nothing
to do."
            )
            return
        n = dbmod.encrypt_database(db_path, get_or_create_key())
        typer.echo(
            f"encrypted {Path(db_path).name}: {n} table(s) migrated. A plaintext backup
remains at "
            f"{Path(db_path).name}.plaintext.bak ? verify, then securely delete it."
        )


    def _require_encrypted_ledger(db_path: Path) -> str:
        """Return the ledger's keychain key, or exit with a clear message if it can't be
used."""
        from muneem import db as dbmod
        from muneem.db_key import peek_key

        if not Path(db_path).exists() or dbmod.is_plaintext_sqlite(db_path):
            typer.echo("no encrypted ledger here (run 'muneem db encrypt' first).",
err=True)
            raise typer.Exit(1)
        key = peek_key()
        if not key:
            typer.echo("no encryption key in the keychain for this ledger.", err=True)
            raise typer.Exit(1)
        return key


    @db_app.command("rotate-key")
    def db_rotate_key(
        db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
    ) -> None:
        """Rotate the ledger's encryption key (SQLCipher rekey) and update the keychain."""
        from muneem import db as dbmod
        from muneem import db_key

        db_path = db or load_config().db_path
        current = _require_encrypted_ledger(db_path)
        fresh = db_key.new_key()
        # Keychain-first with rollback: if the file rekey fails, the keychain still matches
the file.
        db_key.set_key(fresh)
        try:
            dbmod.rekey_database(db_path, current, fresh)
        except dbmod.EncryptedDatabaseError as exc:
            db_key.set_key(current)  # roll back; the file is still under the old key
            typer.echo(str(exc), err=True)
            raise typer.Exit(1) from exc
        typer.echo(f"rotated the encryption key for {Path(db_path).name}.")


    @db_app.command("backup")
    def db_backup(
        dest: Path = typer.Argument(..., help="Destination path for the encrypted backup
copy."),
        db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
    ) -> None:
        """Write an encrypted online-backup copy of the ledger (consistent even with WAL
active)."""
        from muneem import db as dbmod
```

```
476
477         db_path = db or load_config().db_path
478         key = _require_encrypted_ledger(db_path)
479         try:
480             dbmod.backup_database(db_path, dest, key)
481         except dbmod.EncryptedDatabaseError as exc:
482             typer.echo(str(exc), err=True)
483             raise typer.Exit(1) from exc
484         typer.echo(f"backed up {Path(db_path).name} to {dest} (encrypted).")
485
486
487     @db_app.command("setup-recovery")
488     def db_setup_recovery(
489         out: Path | None = typer.Option(
490             None, help="Recovery file path (default: <data dir>/muneem-recovery.json)."
491         ),
492     ) -> None:
493         """Create a passphrase-wrapped recovery file for the ledger's encryption key.
494
495         Lets you restore the key (and thus the ledger + backups) if the OS keychain is ever
lost. The
496         file holds only the *wrapped* key ? useless without the passphrase.
497         """
498         import getpass
499
500         from muneem import recovery
501         from muneem.db_key import peek_key
502
503         key = peek_key()
504         if not key:
505             typer.echo("no encryption key in the keychain yet (create the ledger first).",
err=True)
506             raise typer.Exit(1)
507         out_path = out or (load_config().data_dir / "muneem-recovery.json")
508         passphrase = getpass.getpass("New recovery passphrase: ")
509         if passphrase != getpass.getpass("Confirm passphrase: "):
510             typer.echo("passphrases did not match.", err=True)
511             raise typer.Exit(1)
512         try:
513             recovery.write_recovery_file(out_path, key, passphrase)
514         except recovery.RecoveryError as exc:
515             typer.echo(str(exc), err=True)
516             raise typer.Exit(1) from exc
517         typer.echo(
518             f"wrote recovery file to {out_path}. Store it OFF this machine (password manager
/ "
519             "separate drive) and remember the passphrase ? you need BOTH to recover. Without
them, a "
520             "lost keychain means the ledger and its backups are unrecoverable."
521         )
522
523
524     @db_app.command("recover")
525     def db_recover(
526         file: Path = typer.Argument(..., exists=True, dir_okay=False, help="Recovery file
path."),
527         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
528     ) -> None:
529         """Restore the ledger's encryption key from a recovery file + passphrase (into the
keychain)."""
530         import getpass
531
532         from muneem import db as dbmod
533         from muneem import db_key, recovery
```

```
534
535         passphrase = getpass.getpass("Recovery passphrase: ")
536         try:
537             key = recovery.read_recovery_file(file, passphrase)
538         except recovery.RecoveryError as exc:
539             typer.echo(str(exc), err=True)
540             raise typer.Exit(1) from exc
541         db_path = db or load_config().db_path
542         # If the ledger is present, verify the recovered key actually opens it before
committing it.
543         if Path(db_path).exists() and not dbmod.is_plaintext_sqlite(db_path):
544             try:
545                 with dbmod.connect(db_path, key=key) as conn:
546                     conn.execute("SELECT 1")
547             except dbmod.EncryptedDatabaseError as exc:
548                 typer.echo("recovered a key, but it does not open the current ledger.",
err=True)
549                 raise typer.Exit(1) from exc
550         db_key.set_key(key)
551         typer.echo("recovered the encryption key into the keychain.")
552
553
554     if __name__ == "__main__":
555         app()
556
```

64. assistant (2026-06-10T17:03:00.496Z)

Now reading the existing rotation tests and the doc that needs the inverted-claim fix:

65. user (2026-06-10T17:03:00.971Z)

```
1       """Crypto operations on the encrypted ledger: key rotation, encrypted backup, migration
temp
2       cleanup, correctness at size, and the plaintext-refusal guidance ? plus their CLI
commands.
3
4       Offline, fabricated data only; the autouse conftest fixture keeps the real keychain
untouched.
5       """
6
7       import sqlite3
8
9       import pytest
10
11      pytest.importorskip("sqlcipher3")
12
13      from typer.testing import CliRunner  # noqa: E402
14
15      from muneem import db as dbmod  # noqa: E402
16      from muneem import db_key  # noqa: E402
17      from muneem.cli import app  # noqa: E402
18
19      runner = CliRunner()
20
21      KEY_A = "00112233445566778899aabbccddeeff00112233445566778899aabbccddeeff"
22      KEY_B = "ffeeddccbbaa99887766554433221100ffeeddccbbaa99887766554433221100"
23
24
25      def _seed_encrypted(path, key, n=3):
26          with dbmod.connect(path, key=key) as conn:
27              conn.execute("CREATE TABLE t (id INTEGER PRIMARY KEY, amount REAL)")
28              conn.executemany("INSERT INTO t (amount) VALUES (?)", [(float(i),) for i in
range(n)])
29
```

```

    # --- rekey / rotation
-----------------------------------------------------------------------


    def test_rekey_changes_the_key_and_keeps_data(tmp_path):
        p = tmp_path / "led.db"
        _seed_encrypted(p, KEY_A, n=5)
        dbmod.rekey_database(p, KEY_A, KEY_B)
        with pytest.raises(dbmod.EncryptedDatabaseError):  # old key no longer opens it
            with dbmod.connect(p, key=KEY_A):
                pass
        with dbmod.connect(p, key=KEY_B) as conn:  # new key works, data intact
            assert conn.execute("SELECT count(*) FROM t").fetchone()[0] == 5


    def test_rekey_with_wrong_current_key_raises(tmp_path):
        p = tmp_path / "led.db"
        _seed_encrypted(p, KEY_A)
        with pytest.raises(dbmod.EncryptedDatabaseError):
            dbmod.rekey_database(p, KEY_B, KEY_A)


    # --- encrypted backup
-----------------------------------------------------------------------


    def test_backup_is_encrypted_and_intact(tmp_path):
        p = tmp_path / "led.db"
        _seed_encrypted(p, KEY_A, n=4)
        dest = tmp_path / "backup" / "led.bak.db"
        dbmod.backup_database(p, dest, KEY_A)
        assert dest.exists() and not dbmod.is_plaintext_sqlite(dest)  # backup stays
encrypted
        with dbmod.connect(dest, key=KEY_A) as conn:
            assert conn.execute("SELECT count(*) FROM t").fetchone()[0] == 4
        with pytest.raises(dbmod.EncryptedDatabaseError):  # wrong key can't read the backup
            with dbmod.connect(dest, key=KEY_B):
                pass


    def test_backup_with_wrong_key_raises(tmp_path):
        p = tmp_path / "led.db"
        _seed_encrypted(p, KEY_A)
        with pytest.raises(dbmod.EncryptedDatabaseError):
            dbmod.backup_database(p, tmp_path / "b.db", KEY_B)


    # --- migration cleanup + correctness at size + plaintext refusal
------------------------------


    def test_encrypt_database_cleans_temp_on_failure(tmp_path, monkeypatch):
        p = tmp_path / "led.db"
        con = sqlite3.connect(p)
        con.execute("CREATE TABLE t (id INTEGER PRIMARY KEY)")
        con.commit()
        con.close()

        def boom(*_a, **_k):  # fail the post-export verification open
            raise RuntimeError("boom during verify")

        monkeypatch.setattr(dbmod, "_open_encrypted", boom)
        with pytest.raises(RuntimeError):
            dbmod.encrypt_database(p, KEY_A)
```

```
91          assert not (tmp_path / "led.db.enc.tmp").exists()  # partial temp removed
92          assert dbmod.is_plaintext_sqlite(p)  # original left untouched
93
94
95      def test_encrypted_ledger_handles_a_larger_table(tmp_path):
96          p = tmp_path / "led.db"
97          with dbmod.connect(p, key=KEY_A) as conn:
98              conn.execute("CREATE TABLE t (id INTEGER PRIMARY KEY, amount REAL)")
99              conn.executemany("INSERT INTO t (amount) VALUES (?)", [(float(i),) for i in
range(2000)])
100         with dbmod.connect(p, key=KEY_A) as conn:
101             total = conn.execute("SELECT SUM(amount) FROM t").fetchone()[0]
102             cnt = conn.execute("SELECT count(*) FROM t WHERE amount >= 1000").fetchone()[0]
103         assert cnt == 1000 and total == float(sum(range(2000)))
104
105
106     def test_connect_encrypted_plaintext_message_points_to_db_encrypt(tmp_path):
107         p = tmp_path / "led.db"
108         con = sqlite3.connect(p)
109         con.execute("CREATE TABLE t (id INTEGER)")
110         con.commit()
111         con.close()
112         with pytest.raises(dbmod.EncryptedDatabaseError) as ei:
113             with dbmod.connect_encrypted(p):
114                 pass
115         assert "muneem db encrypt" in str(ei.value)  # actionable guidance, not a cryptic
error
116
117
118     # --- CLI: db rotate-key / db backup
------------------------------------------------------------
119
120
121     def _new_encrypted_ledger(path):
122         with dbmod.connect_encrypted(path) as conn:
123             dbmod.init_db(conn)
124             dbmod.insert_document(conn, source="file", sha256="s", status="parsed")
125
126
127     def test_db_rotate_key_cli(tmp_path):
128         led = tmp_path / "ledger.db"
129         _new_encrypted_ledger(led)
130         old = db_key.peek_key()
131         res = runner.invoke(app, ["db", "rotate-key", "--db", str(led)])
132         assert res.exit_code == 0, res.output
133         new = db_key.peek_key()
134         assert new and new != old
135         with dbmod.connect(led, key=new) as conn:  # opens under the new key
136             assert dbmod.get_document_by_sha(conn, "s") is not None
137         with pytest.raises(dbmod.EncryptedDatabaseError):  # old key no longer works
138             with dbmod.connect(led, key=old):
139                 pass
140
141
142     def test_db_backup_cli(tmp_path):
143         led = tmp_path / "ledger.db"
144         _new_encrypted_ledger(led)
145         dest = tmp_path / "led.backup.db"
146         res = runner.invoke(app, ["db", "backup", str(dest), "--db", str(led)])
147         assert res.exit_code == 0, res.output
148         assert dest.exists() and not dbmod.is_plaintext_sqlite(dest)
149         with dbmod.connect(dest, key=db_key.peek_key()) as conn:
150             assert dbmod.get_document_by_sha(conn, "s") is not None
151
152
```

```
153     def test_db_rotate_key_refuses_when_no_encrypted_ledger(tmp_path):
154         res = runner.invoke(app, ["db", "rotate-key", "--db", str(tmp_path / "nope.db")])
155         assert res.exit_code == 1 and "db encrypt" in res.output
156
```

66. user (2026-06-10T17:03:02.442Z)

```
33-- **Never** written to the repo, config, CLI args, or logs. `db.py` centralizes the `PRAGMA
key`
34-  so the raw-hex form is used everywhere (mixing raw-key and passphrase forms silently looks
like
35-  corruption).
36-
37:**Key rotation** is implemented: `muneem db rotate-key` mints a fresh 256-bit key, re-
encrypts the
38:file (`PRAGMA rekey`), and updates the keychain ? keychain-first with rollback, so a failed
file
39:rekey leaves the keychain matching the file. Rotate on suspected compromise; no scheduled
rotation
40-needed for a single-user local key.
41-
42-### Disaster recovery ? the key-loss problem (solved)
43-
```

67. user (2026-06-10T17:04:07.881Z)

```
Switched to a new branch 'fix/rotate-key-atomicity'
```

68. user (2026-06-10T17:04:07.897Z)

```
1       """Central, redaction-aware logging.
2
3       Use ``get_logger(__name__)`` instead of the stdlib ``logging.getLogger`` so every muneem
logger
4       carries a :class:`RedactingFilter`. The filter scrubs money-/account-like substrings
from each
5       record as a backstop ? code should still pass already-sanitised context (see
6       :mod:`muneem.redaction`); this is defense-in-depth so an accidental sensitive f-string
cannot leak.
7
8       muneem stays library-style: we attach the filter but do **not** configure handlers or
levels (the
9       application / CLI owns that), so importing muneem never hijacks the root logger.
10      """
11
12      from __future__ import annotations
13
14      import logging
15
16      from muneem.redaction import redact_text
17
18
19      class RedactingFilter(logging.Filter):
20          """A logging filter that masks sensitive substrings in the formatted message."""
21
22          def filter(self, record: logging.LogRecord) -> bool:
23              try:
24                  message = record.getMessage()
25              except Exception:  # never drop a record just because formatting failed
26                  return True
27              redacted = redact_text(message)
28              if redacted != message:
29                  record.msg = redacted
30                  record.args = ()
```

```
31              return True
32
33
34      _filtered: set[str] = set()
35
36
37      def get_logger(name: str) -> logging.Logger:
38          """Return a logger that redacts sensitive substrings from its records
(idempotent)."""
39          logger = logging.getLogger(name)
40          if name not in _filtered:
41              logger.addFilter(RedactingFilter())
42              _filtered.add(name)
43          return logger
44
```

69. user (2026-06-10T17:04:26.495Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\db_key.py has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

70. assistant (2026-06-10T17:04:30.623Z)

Now `db.py` ? the `key_opens_database` helper and the self-healing `connect_encrypted`:

71. user (2026-06-10T17:04:41.745Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\db.py has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

72. user (2026-06-10T17:04:52.523Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\db.py has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

73. user (2026-06-10T17:05:05.369Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\cli.py has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

74. assistant (2026-06-10T17:05:12.114Z)

Now the `db status` pending-key surfacing and the doc fix:

75. user (2026-06-10T17:05:18.524Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\cli.py has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

76. user (2026-06-10T17:05:26.315Z)

```
<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>
```

77. user (2026-06-10T17:05:31.631Z)

```
25
26      ## Key management
27
28      - A random **256-bit** key is minted on first use (`secrets.token_bytes(32)`) and stored
in the
29        **OS keychain** via `muneem.secrets` (`muneem`/`db-key`) ? Windows Credential Manager
(DPAPI),
30        macOS Keychain, or Secret Service on Linux. Same mechanism as OAuth tokens.
```

```
31      - It is handed to SQLCipher in **raw-key form** (`PRAGMA key = "x'<64 hex>'"`), which
uses the key
32        directly and **skips the passphrase KDF** ? correct because the key is already full-
entropy.
33      - **Never** written to the repo, config, CLI args, or logs. `db.py` centralizes the
`PRAGMA key`
34        so the raw-hex form is used everywhere (mixing raw-key and passphrase forms silently
looks like
35        corruption).
36
37      **Key rotation** is implemented: `muneem db rotate-key` mints a fresh 256-bit key, re-
encrypts the
38      file (`PRAGMA rekey`), and updates the keychain ? keychain-first with rollback, so a
failed file
39      rekey leaves the keychain matching the file. Rotate on suspected compromise; no
scheduled rotation
40      needed for a single-user local key.
41
42      ### Disaster recovery ? the key-loss problem (solved)
43
44      Because the key lives only in the keychain and `db backup` encrypts backups with that
*same* key, a
45      **lost keychain** (OS reinstall, new machine, profile corruption) would otherwise
destroy the ledger
46      *and every backup*. So a **passphrase-wrapped recovery file** is implemented
(`muneem.recovery`):
47
48      - **`muneem db setup-recovery`** ? wraps the current DB key under a user passphrase
49        (**Argon2id** KDF ? **AES-256-GCM**) into `muneem-recovery.json`. The file holds only
the *wrapped*
50        key (salt/nonce/ciphertext + KDF params) ? never the key, never the passphrase.
**Store it
51        off-machine** (password manager / separate drive). The pre-commit guard blocks
`*recovery*.json`
52        from the repo.
53      - **`muneem db recover <file>`** ? prompts for the passphrase, unwraps the key (a wrong
passphrase
54        fails the GCM tag ? clean error, no oracle), verifies it opens the ledger, and
restores it to the
55        keychain. You need **both** the file and the passphrase.
56
57      **Deferred (opt-in, ROADMAP D.2 Phase 2):** a full passphrase-*lock* mode (the working
key itself
58      derived from a passphrase, never stored) for stronger-than-keychain protection on a
shared machine ?
59      distinct from the recovery file above, which keeps the frictionless keychain key as
primary.
60
61      ## Threat model ? stated honestly
62
63      Keychain + SQLCipher **defends against**: a stolen or discarded **disk/file**, the DB
ending up in
64      a **cloud-synced backup / USB / email**, and **other OS users**. It does **not** defend
against
```

78. user (2026-06-10T17:05:43.296Z)

```
The file C:\Users\avidu\Projects\muneem\docs\encryption-at-rest.md has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

79. assistant (2026-06-10T17:05:58.860Z)

Now the failure-path tests the review flagged as missing:

80. user (2026-06-10T17:06:19.238Z)

```
The file C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

81. user (2026-06-10T17:06:29.409Z)

```
.....................................                                    [100%]
39 passed in 1.44s
```

82. assistant (2026-06-10T17:06:36.953Z)

Test subset green. Full gate now (ruff + mypy + full suite with the coverage gate):

83. user (2026-06-10T17:06:40.621Z)

```
E501 Line too long (112 > 100)
   --> tests\test_db_crypto_ops.py:204:101
    |
202 |     _new_encrypted_ledger(led)
203 |     old = db_key.peek_key()
204 |     monkeypatch.setattr(dbmod, "rekey_database", lambda *_a, **_k: (_ for _ in
()).throw(OSError("disk error")))
    |
^^^^^^^^^^^^
205 |     monkeypatch.setattr(dbmod, "key_opens_database", lambda *_a, **_k: False)
206 |     res = runner.invoke(app, ["db", "rotate-key", "--db", str(led)])
    |

Found 1 error.
Would reformat: tests\test_db_crypto_ops.py
1 file would be reformatted, 78 files already formatted
Success: no issues found in 33 source files
```

84. user (2026-06-10T17:06:49.200Z)

```
The file C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

85. user (2026-06-10T17:06:59.704Z)

```
Exit code 1
All checks passed!
1 file would be reformatted, 78 files already formatted
```

86. user (2026-06-10T17:07:08.113Z)

```
1 file reformatted
79 files already formatted
```

87. user (2026-06-10T17:07:24.346Z)

```
src\muneem\statement_unlock.py        42      4    90%
src\muneem\unlock.py                  56      5    91%
-------------------------------------------------------
TOTAL                               2018    267    87%
Required test coverage of 80% reached. Total coverage: 86.77%
401 passed, 21 deselected in 5.51s
```

88. assistant (2026-06-10T17:07:30.709Z)

Full gate green ? 401 passed, coverage 86.77%, ruff/mypy clean. Committing and opening the PR:

89. user (2026-06-10T17:07:47.252Z)

```
[fix/rotate-key-atomicity 1d44410] fix(db): atomic key rotation ? pending-key protocol + self-
healing open
 5 files changed, 216 insertions(+), 12 deletions(-)
muneem pre-push: ruff check ...
All checks passed!
muneem pre-push: ruff format --check ...
79 files already formatted
muneem pre-push: mypy ...
Success: no issues found in 33 source files
muneem pre-push: pytest (offline suite) ...
.................................................................... [ 17%]
.................................................................... [ 35%]
.................................................................... [ 53%]
.................................................................... [ 71%]
.................................................................... [ 89%]
.......................................                             [100%]
401 passed, 21 deselected in 3.84s
muneem pre-push: all checks passed.
remote:
remote: Create a pull request for 'fix/rotate-key-atomicity' on GitHub by visiting:
remote:      https://github.com/avidullu/muneem/pull/new/fix/rotate-key-atomicity
remote:
branch 'fix/rotate-key-atomicity' set up to track 'origin/fix/rotate-key-atomicity'.
To https://github.com/avidullu/muneem.git
 * [new branch]      fix/rotate-key-atomicity -> fix/rotate-key-atomicity
```

90. user (2026-06-10T17:08:04.581Z)

```
https://github.com/avidullu/muneem/pull/50
```

91. user (2026-06-10T17:08:19.387Z)

```
From https://github.com/avidullu/muneem
 * branch            main       -> FETCH_HEAD
   0e5602f..f13d62c  main       -> origin/main
Updating 0e5602f..f13d62c
Fast-forward
 docs/encryption-at-rest.md  |  16 +++++--
 src/muneem/cli.py           |  52 ++++++++++++++++++---
 src/muneem/db.py            |  37 +++++++++++++-
 src/muneem/db_key.py        |  23 +++++++-
 tests/test_db_crypto_ops.py | 100 ++++++++++++++++++++++++++++++++++++
 5 files changed, 216 insertions(+), 12 deletions(-)
Your branch is up to date with 'origin/main'.
Already on 'main'
Already up to date.
```

92. user (2026-06-10T17:08:53.423Z)

```
Updated task #1 status
```

93. user (2026-06-10T17:08:54.302Z)

```
Updated task #2 status
```

94. user (2026-06-10T17:08:55.653Z)

```
Switched to a new branch 'fix/backup-destination-guard'
```

95. user (2026-06-10T17:09:11.093Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\db.py has been updated successfully. (file
```

```
state is current in your context ? no need to Read it back)
```

96. user (2026-06-10T17:09:24.655Z)

```
The file C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

97. user (2026-06-10T17:09:34.229Z)

```
.........................                                                [100%]
27 passed in 1.20s
All checks passed!
1 file reformatted, 78 files left unchanged
Success: no issues found in 33 source files
```

98. user (2026-06-10T17:09:48.882Z)

```
TOTAL                               2030    266    87%
Required test coverage of 80% reached. Total coverage: 86.90%
404 passed, 21 deselected in 5.57s
```

99. user (2026-06-10T17:10:24.363Z)

```
[fix/backup-destination-guard 6a1e326] fix(db): backup can no longer destroy the ledger or a
previous backup
 2 files changed, 59 insertions(+), 4 deletions(-)
muneem pre-push: ruff check ...
All checks passed!
muneem pre-push: ruff format --check ...
79 files already formatted
muneem pre-push: mypy ...
Success: no issues found in 33 source files
muneem pre-push: pytest (offline suite) ...
.......................................................................... [ 17%]
.......................................................................... [ 35%]
.......................................................................... [ 53%]
.......................................................................... [ 71%]
.......................................................................... [ 89%]
.........................................                                [100%]
404 passed, 21 deselected in 4.06s
muneem pre-push: all checks passed.
remote:
remote: Create a pull request for 'fix/backup-destination-guard' on GitHub by visiting:
remote:      https://github.com/avidullu/muneem/pull/new/fix/backup-destination-guard
remote:
branch 'fix/backup-destination-guard' set up to track 'origin/fix/backup-destination-guard'.
To https://github.com/avidullu/muneem.git
 * [new branch]      fix/backup-destination-guard -> fix/backup-destination-guard
https://github.com/avidullu/muneem/pull/51
From https://github.com/avidullu/muneem
 * branch            main       -> FETCH_HEAD
   f13d62c..be0158b  main       -> origin/main
Updating f13d62c..be0158b
Fast-forward
 src/muneem/db.py            | 27 ++++++++++++++++++++++----
 tests/test_db_crypto_ops.py | 36 ++++++++++++++++++++++++++++++++++++
 2 files changed, 59 insertions(+), 4 deletions(-)
Your branch is up to date with 'origin/main'.
Already on 'main'
Already up to date.
```

100. user (2026-06-10T17:11:03.674Z)

```
Updated task #2 status
```

101. user (2026-06-10T17:11:04.043Z)

```
Updated task #3 status
```

102. user (2026-06-10T17:11:04.756Z)

```
1       import json
2       import os
3       from pathlib import Path
4
5       import pytest
6
7
8       @pytest.fixture(autouse=True)
9       def _isolate_keychain(monkeypatch):
10          """Never touch the real OS keychain in tests.
11
12          Replaces :mod:`muneem.secrets` with an in-memory store so the suite cannot pollute
the
13          developer's keychain (e.g. minting a DB key) and runs cleanly in headless CI where
no
14          keyring backend exists. Tests that need specific keychain behaviour re-patch on top.
15          """
16          store: dict[str, str] = {}
17
18          def _set(name, value):
19              store[name] = value
20
21          monkeypatch.setattr("muneem.secrets.get_secret", store.get)
22          monkeypatch.setattr("muneem.secrets.set_secret", _set)
23          monkeypatch.setattr("muneem.secrets.delete_secret", lambda name: store.pop(name,
None))
24          return store
25
26
27      @pytest.fixture
28      def snapshot():
29          def _assert_snapshot(name: str, data: list | dict):
30              snapshot_dir = Path("testdata/snapshots")
31              snapshot_dir.mkdir(parents=True, exist_ok=True)
32              snapshot_file = snapshot_dir / f"{name}.json"
33
34              if os.environ.get("UPDATE_SNAPSHOTS") == "1":
35                  with snapshot_file.open("w", encoding="utf-8") as f:
36                      json.dump(data, f, indent=2, ensure_ascii=False)
37                  return True
38
39              if not snapshot_file.exists():
40                  pytest.fail(f"Snapshot not found at {snapshot_file}. Run with
UPDATE_SNAPSHOTS=1.")
41
42              with snapshot_file.open("r", encoding="utf-8") as f:
43                  expected = json.load(f)
44
45              assert data == expected, f"Data does not match snapshot {name}"
46              return True
47
48          return _assert_snapshot
49
```

103. user (2026-06-10T17:11:26.165Z)

```
src\muneem\cli.py:459:    from muneem.errors import KeychainError
src\muneem\cli.py:493:    except KeychainError as exc:
src\muneem\db_key.py:30:    existing = _keychain.get_secret(_DB_KEY_NAME)
```

```
src\muneem\db_key.py:40:    return _keychain.get_secret(_DB_KEY_NAME)
src\muneem\db_key.py:57:    return _keychain.get_secret(_DB_KEY_PENDING_NAME)
src\muneem\errors.py:37:class KeychainError(MuneemError):
src\muneem\secrets.py:8:- **read** (:func:`get_secret`) is *graceful* ? a missing or unavailable
backend returns ``None``
src\muneem\secrets.py:10:- **write** (:func:`set_secret`) is *loud* ? a failure raises
:class:`~muneem.errors.KeychainError`,
src\muneem\secrets.py:20:from muneem.errors import KeychainError
src\muneem\secrets.py:25:def get_secret(name: str) -> str | None:
src\muneem\secrets.py:34:    """Store (or replace) a secret in the OS keychain. Raises
KeychainError on failure."""
src\muneem\secrets.py:39:        raise KeychainError(f"could not store secret {name!r} in the OS
keychain") from exc
src\muneem\statement_unlock.py:23:from muneem.errors import KeychainError
src\muneem\statement_unlock.py:34:    raw = secrets.get_secret(_CACHE_KEY)
src\muneem\statement_unlock.py:55:    except KeychainError:
src\muneem\ingestion\gmail.py:28:        raw = secrets.get_secret(_TOKEN_KEY)
src\muneem\ingestion\outlook.py:39:        raw = secrets.get_secret(_TOKEN_KEY)
src\muneem\ingestion\imap.py:29:        password =
secrets.get_secret(f"{_PASS_KEY}_{self.username}")
```

104. user (2026-06-10T17:11:54.637Z)

```
20      from pathlib import Path
21
22      from muneem import secrets
23      from muneem.errors import KeychainError
24      from muneem.password_rules import candidate_passwords
25      from muneem.unlock import is_encrypted, unlock
26
27      # Keychain entry holding a JSON list of known-good statement passwords (the learned
candidates).
28      _CACHE_KEY = "statement_unlock_candidates"
29      # Bound the cached list so the keychain entry can't grow without limit; most-recently-
used kept.
30      _MAX_CACHED = 20
31
32
33      def _cached_candidates() -> list[str]:
34          raw = secrets.get_secret(_CACHE_KEY)
35          if not raw:
36              return []
37          try:
38              return [p for p in json.loads(raw) if isinstance(p, str) and p]
39          except (ValueError, TypeError):
40              return []
41
42
43      def _remember(password: str) -> None:
44          """Cache a working password in the keychain (MRU order, deduped, capped). Never
logged.
45
46          Best-effort: a keychain write failure is swallowed (the password already unlocked
the file this
47          run); we never crash an ingest over a caching hiccup.
48          """
49          if not password or not password.strip():
50              return
51          current = [p for p in _cached_candidates() if p != password]
52          updated = [password, *current][:_MAX_CACHED]  # most-recently-used first, bounded
53          try:
54              secrets.set_secret(_CACHE_KEY, json.dumps(updated))
55          except KeychainError:
56              pass
57
```

```
58
59      def resolve_password(pdf_path: Path | str, *, interactive: bool = True) -> str | None:
60          """Return a working password for ``pdf_path``.
61
62          ``""`` means the PDF is not encrypted (no password needed). ``None`` means it is
encrypted and
63          no candidate (or prompt) unlocked it. A prompt-resolved password is cached for next
time.
64          """
```

105. user (2026-06-10T17:11:55.572Z)

```
18
19          def authorize(self) -> None:
20              from google.auth.transport.requests import Request
21              from google.oauth2.credentials import Credentials
22              from google_auth_oauthlib.flow import InstalledAppFlow
23              from googleapiclient.discovery import build
24
25              from muneem import secrets
26
27              creds = None
28              raw = secrets.get_secret(_TOKEN_KEY)
29              if raw:
30                  creds = Credentials.from_authorized_user_info(json.loads(raw), SCOPES)
31              if creds and creds.valid:
32                  pass
33              elif creds and creds.expired and creds.refresh_token:
34                  creds.refresh(Request())
35              else:
36                  flow = InstalledAppFlow.from_client_secrets_file(str(self.credentials_path),
SCOPES)
37                  creds = flow.run_local_server(port=0, open_browser=self.open_browser)
38              secrets.set_secret(_TOKEN_KEY, creds.to_json())
39              self.service = build("gmail", "v1", credentials=creds, cache_discovery=False)
40
41          def _build_query(self, criteria: SearchCriteria) -> str:
42              parts = []
43              if criteria.sender:
44                  parts.append(f"from:{criteria.sender}")
45              if criteria.has_attachment:
46                  parts.append("has:attachment")
47              if criteria.newer_than_days:
```

106. user (2026-06-10T17:11:55.877Z)

```
1       """The OS-keychain wrappers: graceful read, attributed (value-free) write, swallowed
delete.
2
3       The autouse conftest fixture swaps ``muneem.secrets`` for an in-memory store, so we
capture the
4       *real* wrapper functions at import time and drive them by patching the underlying
``keyring`` lib.
5       """
6
7       import keyring
8       import pytest
9       from keyring.errors import KeyringError
10
11      import muneem.secrets as secretsmod
12      from muneem.errors import KeychainError
13
14      # Captured before the autouse keychain fixture rebinds the module attributes per-test.
15      _real_get = secretsmod.get_secret
16      _real_set = secretsmod.set_secret
```

```
    _real_delete = secretsmod.delete_secret


    def test_get_secret_returns_none_when_keyring_unavailable(monkeypatch):
        def boom(service, name):
            raise KeyringError("no backend")

        monkeypatch.setattr(keyring, "get_password", boom)
        assert _real_get("anything") is None  # graceful: absence, not a crash


    def test_get_secret_returns_the_value(monkeypatch):
        monkeypatch.setattr(keyring, "get_password", lambda s, n: "v" if n == "k" else None)
        assert _real_get("k") == "v"
        assert _real_get("missing") is None


    def test_set_secret_raises_keychain_error_with_name_but_not_value(monkeypatch):
        def boom(service, name, value):
            raise KeyringError("locked")

        monkeypatch.setattr(keyring, "set_password", boom)
        with pytest.raises(KeychainError) as ei:
            _real_set("db-key", "super-secret-value")
        msg = str(ei.value)
        assert "db-key" in msg  # the name is safe to surface
        assert "super-secret-value" not in msg  # the value must NEVER appear


    def test_delete_secret_swallows_keyring_errors(monkeypatch):
        def boom(service, name):
            raise KeyringError("nope")

        monkeypatch.setattr(keyring, "delete_password", boom)
        _real_delete("k")  # must not raise

```

107. user (2026-06-10T17:12:11.204Z)

```


    class IMAPProvider(EmailProvider):
        def __init__(self, host: str, username: str):
            self.host = host
            self.username = username
            self.mail = None

        def authorize(self) -> None:
            password = secrets.get_secret(f"{_PASS_KEY}_{self.username}")
            if not password:
                import getpass

                password = getpass.getpass(f"Enter App Password for {self.username} at
{self.host}: ")
                secrets.set_secret(f"{_PASS_KEY}_{self.username}", password)

            self.mail = imaplib.IMAP4_SSL(self.host)
            self.mail.login(self.username, password)
            self.mail.select("inbox", readonly=True)

```

108. assistant (2026-06-10T17:12:08.238Z)

All context in hand. Implementing A3 across the six touchpoints:

109. user (2026-06-10T17:12:10.483Z)

```
Switched to a new branch 'fix/key-mint-guard'
```

110. user (2026-06-10T17:12:10.655Z)

```
30              except ImportError:
31                  msg = "msal is required for OutlookProvider. Run pip install msal"
32                  raise RuntimeError(msg) from None
33
34              from muneem import secrets
35
36              app = PublicClientApplication(self.client_id, authority=self.authority)
37
38              # Try to find a cached token (simplified)
39              raw = secrets.get_secret(_TOKEN_KEY)
40              if raw:
41                  token_data = json.loads(raw)
42                  self.access_token = token_data.get("access_token")
43                  # Ideally, handle refresh logic here via msal TokenCache
44                  return
45
46              # Interactive login
47              flow = app.initiate_device_flow(scopes=SCOPES)
48              if "user_code" not in flow:
49                  raise ValueError("Failed to create device flow")
```

111. user (2026-06-10T17:12:24.570Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\secrets.py has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

112. user (2026-06-10T17:12:27.922Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\secrets.py has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

113. assistant (2026-06-10T17:12:34.233Z)

Now the graceful callers and the no-mint-over-ledger guard:

114. user (2026-06-10T17:12:37.040Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\statement_unlock.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

115. user (2026-06-10T17:12:39.395Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\ingestion\gmail.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

116. user (2026-06-10T17:12:44.855Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\ingestion\outlook.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

117. user (2026-06-10T17:12:44.885Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\ingestion\imap.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

118. user (2026-06-10T17:12:56.603Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\db_key.py has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

119. user (2026-06-10T17:13:02.018Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\db.py has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

120. user (2026-06-10T17:13:14.397Z)

```
The file C:\Users\avidu\Projects\muneem\tests\test_secrets.py has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

121. user (2026-06-10T17:13:17.675Z)

```
The file C:\Users\avidu\Projects\muneem\tests\test_secrets.py has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

122. user (2026-06-10T17:13:23.562Z)

```
The file C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

123. user (2026-06-10T17:13:35.009Z)

```
Exit code 1
tests\test_db_crypto_ops.py:313: AssertionError
========================== short test summary info ==========================
FAILED
tests/test_db_crypto_ops.py::test_get_or_create_key_backend_failure_raises_instead_of_minting
1 failed, 405 passed, 21 deselected in 4.57s
```

124. user (2026-06-10T17:13:57.111Z)

```
The file C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

125. user (2026-06-10T17:14:12.936Z)

```
TOTAL                              2049    280    86%
Required test coverage of 80% reached. Total coverage: 86.33%
406 passed, 21 deselected in 6.18s
All checks passed!
79 files left unchanged
Success: no issues found in 33 source files
```

126. user (2026-06-10T17:14:49.708Z)

```
[fix/key-mint-guard 38f3e62] fix(secrets): keychain failure is never mistaken for key absence;
no minting over an existing ledger
 9 files changed, 94 insertions(+), 14 deletions(-)
muneem pre-push: ruff check ...
All checks passed!
muneem pre-push: ruff format --check ...
79 files already formatted
muneem pre-push: mypy ...
Success: no issues found in 33 source files
muneem pre-push: pytest (offline suite) ...
.......................................................................... [ 17%]
.......................................................................... [ 35%]
.......................................................................... [ 53%]
.......................................................................... [ 70%]
.......................................................................... [ 88%]
```

```
........................................                                 [100%]
406 passed, 21 deselected in 4.24s
muneem pre-push: all checks passed.
remote:
remote: Create a pull request for 'fix/key-mint-guard' on GitHub by visiting:
remote:      https://github.com/avidullu/muneem/pull/new/fix/key-mint-guard
remote:
branch 'fix/key-mint-guard' set up to track 'origin/fix/key-mint-guard'.
To https://github.com/avidullu/muneem.git
 * [new branch]      fix/key-mint-guard -> fix/key-mint-guard
https://github.com/avidullu/muneem/pull/52
From https://github.com/avidullu/muneem
 * branch            main       -> FETCH_HEAD
   be0158b..1ea952f  main       -> origin/main
Updating be0158b..1ea952f
Fast-forward
 src/muneem/db.py                | 14 ++++++++++++-
 src/muneem/db_key.py            |  8 +++++++-
 src/muneem/ingestion/gmail.py   |  6 +++++-
 src/muneem/ingestion/imap.py    |  7 +++++++
 src/muneem/ingestion/outlook.py |  6 +++++-
 src/muneem/secrets.py           | 18 ++++++++++++++++++-----
 src/muneem/statement_unlock.py  |  5 ++++-
 tests/test_db_crypto_ops.py     | 33 +++++++++++++++++++++++++++++++++
 tests/test_secrets.py           | 11 ++++++++++++++---
 9 files changed, 94 insertions(+), 14 deletions(-)
Your branch is up to date with 'origin/main'.
Already on 'main'
Already up to date.
```

127. user (2026-06-10T17:14:57.424Z)

```
Updated task #3 status
```

128. user (2026-06-10T17:14:58.095Z)

```
Updated task #4 status
```

129. user (2026-06-10T17:14:58.352Z)

```
1       """The **verifier** ? reconciliation as a layout-independent truth oracle.
2
3       Reconciliation proves a parse is **complete and correct without inspecting any sensitive
value**:
4       if a transaction is missed or misread, the arithmetic stops adding up. That is what lets
us trust a
5       parse of a statement we never eyeball, and (later) what makes a probabilistic extractor
safe on
6       money ? *extraction proposes, reconciliation disposes* (see
7       [`docs/parser-architecture.md`](../../../docs/parser-architecture.md) §2, §5).
8
9       This module grew from a single running-balance walk into the verifier the architecture
calls for:
10
11      * **per-row walk** ? ``reconcile_running_balance``: ``balance[i] == balance[i-1] ? debit
+ credit``.
12      * **total reconcile** ? ``reconcile_total``: ``opening + ?credit ? ?debit == closing``
(works even
13        with no per-row balance, e.g. Axis savings).
14      * **bookend** ? ``verify``: parsed Dr/Cr counts + opening/closing vs the bank's printed
SUMMARY.
15      * **selection** ? ``select_reconciling``: among candidate concept mappings, pick the one
arithmetic
16        endorses (the seam the L2 mapping search plugs into).
```

```
    * **confidence** ? ``Confidence`` + ``BookendReport.confidence``: a recorded verdict,
not an
      assumption.

    Reports carry counts / booleans / a verdict only ? **never an amount**.
    """

    from __future__ import annotations

    import re
    from collections.abc import Sequence
    from dataclasses import dataclass
    from enum import StrEnum

    from .concepts import ConceptRow, StatementConcepts

    _CR_DR_SUFFIX = re.compile(r"(?i)\s*(cr|dr)\s*$")


    def parse_amount(value: object) -> float | None:
        """Parse a statement money cell to a float.

        Returns ``None`` only for a genuinely absent cell (None / empty / ``"-"`` /
        non-numeric); ``"0.00"`` parses to ``0.0``. This deliberately distinguishes
        *absent* from *zero* ? unlike ``base.safe_amount``, which maps ``"0.00"`` to
        ``None`` and would corrupt a real zero balance and the reconciliation math.
        """
        if value is None:
            return None
        text = _CR_DR_SUFFIX.sub("", str(value).strip()).replace(",", "").strip()
        if text in ("", "-"):
            return None
        try:
            return float(text)
        except ValueError:
            return None


    @dataclass
    class ReconcileReport:
        """Result of a running-balance walk. Counts only, never an amount."""

        n: int  # rows considered
        compared: int  # consecutive (balance-bearing) pairs actually checked
        breaks: int
        first_break: int | None = None

        @property
        def ok(self) -> bool:
            return self.compared > 0 and self.breaks == 0


    def reconcile_running_balance(
        entries: Sequence[tuple[float | None, float | None, float | None]],
        tol: float = 0.01,
    ) -> ReconcileReport:
        """Verify ``balance[i] == balance[i-1] - debit[i] + credit[i]`` across rows.

        ``entries`` is an ordered list of ``(debit, credit, balance)``; ``debit`` and
        ``credit`` may be ``None`` (treated as 0.0). A row whose ``balance`` is ``None``
        is skipped (can't anchor on it). Because the walk spans the whole statement, a
        clean result also rules out any row missed *between* two balances ? which is the
        failure mode silent ``extract_tables()`` row-drops would otherwise hide.
        """
        breaks = 0
```

```
        first_break: int | None = None
        compared = 0
        prev: float | None = None
        for i, (debit, credit, balance) in enumerate(entries):
            if balance is None:
                continue
            if prev is not None:
                expected = prev - (debit or 0.0) + (credit or 0.0)
                compared += 1
                if abs(expected - balance) > tol:
                    breaks += 1
                    if first_break is None:
                        first_break = i
            prev = balance
        return ReconcileReport(
            n=len(entries), compared=compared, breaks=breaks, first_break=first_break
        )


    def _row_entries(
        rows: Sequence[ConceptRow],
    ) -> list[tuple[float | None, float | None, float | None]]:
        """Project concept rows onto the ``(debit, credit, balance)`` tuples the walk
consumes."""
        return [(r.debit, r.credit, r.balance) for r in rows]


    def reconcile_total(
        rows: Sequence[ConceptRow], opening: float, closing: float, tol: float = 0.01
    ) -> bool:
        """Verify the layout-independent identity ``opening + ?credit ? ?debit == closing``.

        This is the oracle for statements with **no reliable per-row balance** (e.g. Axis
savings):
        it checks the whole statement against its own opening/closing bookends without
needing each
        row's running balance. ``opening`` and ``closing`` come from the bank's
SUMMARY/header
        (``StatementConcepts``); the caller must have them.
        """
        delta = sum(r.credit or 0.0 for r in rows) - sum(r.debit or 0.0 for r in rows)
        return abs((opening + delta) - closing) <= tol


    class Confidence(StrEnum):
        """Recorded trust verdict for a parsed statement (mirrors ``documents.status``
values).

        ``RECONCILED`` ? the parse passed every available check, so it is trusted.
``UNRECONCILED`` ?
        a check failed (or none could run); the parse is suspect and is quarantined rather
than trusted.
        """

        RECONCILED = "reconciled"
        UNRECONCILED = "unreconciled"


    @dataclass
    class BookendReport:
        """A parse cross-checked against the bank's own SUMMARY. Booleans + counts only.

        ``None`` for a check means the figure wasn't available to verify (not a failure).
``ok``
        requires the per-row balance walk *plus* every available check to pass.
```

```
        """
        n: int
        reconciles: bool
        debit_count_ok: bool | None = None
        credit_count_ok: bool | None = None
        opening_ok: bool | None = None
        closing_ok: bool | None = None

        @property
        def ok(self) -> bool:
            checks = [self.debit_count_ok, self.credit_count_ok, self.opening_ok,
self.closing_ok]
            return self.n > 0 and self.reconciles and all(c is True for c in checks)

        @property
        def confidence(self) -> Confidence:
            return Confidence.RECONCILED if self.ok else Confidence.UNRECONCILED


    def verify(
        rows: Sequence[ConceptRow], concepts: StatementConcepts, tol: float = 0.01
    ) -> BookendReport:
        """Bookend a parse against the bank's printed SUMMARY: reconciliation + the printed
figures.

        The per-row walk catches any mid-statement error/omission; the Dr/Cr counts catch
any
        extra/missing row; the opening & closing balances catch a missing first/last row.
Each printed
        figure is only checked when present (``None`` ? not checked). The opening balance is
checked by
        implication (``balance[0] ? credit[0] + debit[0]`` should equal the printed
opening).
        """
        reconciles = reconcile_running_balance(_row_entries(rows), tol).ok
        debit_count_ok = credit_count_ok = opening_ok = closing_ok = None
        if concepts.debit_count is not None:
            debit_count_ok = sum(1 for r in rows if (r.debit or 0.0) > 0) ==
concepts.debit_count
        if concepts.credit_count is not None:
            credit_count_ok = sum(1 for r in rows if (r.credit or 0.0) > 0) ==
concepts.credit_count
        if rows and rows[-1].balance is not None and concepts.closing_balance is not None:
            closing_ok = abs(rows[-1].balance - concepts.closing_balance) <= tol
        if rows and rows[0].balance is not None and concepts.opening_balance is not None:
            implied_opening = rows[0].balance - (rows[0].credit or 0.0) + (rows[0].debit or
0.0)
            opening_ok = abs(implied_opening - concepts.opening_balance) <= tol
        return BookendReport(
            n=len(rows),
            reconciles=reconciles,
            debit_count_ok=debit_count_ok,
            credit_count_ok=credit_count_ok,
            opening_ok=opening_ok,
            closing_ok=closing_ok,
        )


    def select_reconciling(candidates: Sequence[Sequence[ConceptRow]], tol: float = 0.01) ->
int | None:
        """Among candidate concept-row mappings, return the index of the first that
reconciles.

        This is the *selection* half of "extraction proposes, reconciliation disposes": when
```

```
the
192         extraction ladder can't be sure which column?concept assignment is right, it
proposes several
193         and lets the per-row walk pick the correct one. Returns ``None`` if none reconcile.
194
195         The candidate *generation* (the L2 mapping search) is Step 2 of the architecture
plan; this is
196         the verifier-side seam it plugs into. Selection here uses the per-row walk, so
candidates need a
197         running balance ? balance-less statements select via :func:`reconcile_total` at the
call site.
198         """
199         for i, rows in enumerate(candidates):
200             if reconcile_running_balance(_row_entries(rows), tol).ok:
201                 return i
202         return None
203
204
205     __all__ = [
206         "parse_amount",
207         "ReconcileReport",
208         "reconcile_running_balance",
209         "reconcile_total",
210         "Confidence",
211         "BookendReport",
212         "verify",
213         "select_reconciling",
214     ]
215
```

130. user (2026-06-10T17:14:58.925Z)

```
1       """Shared engine-driven savings-statement parsing (Axis, AU Bank, and future issuers).
2
3       Every issuer whose statement is a transaction table follows the same recipe, so it lives
here once
4       and each bank supplies only a :class:`~muneem.parsers.engine.Profile`:
5
6       1. **Extraction proposes** candidate cell-matrices ? one per ``extract_tables()`` table,
plus a
7          coordinate-clustered whole-statement matrix (for layouts ``extract_tables`` mis-
segments).
8       2. The generic **engine** maps each candidate's columns onto the canonical concepts.
9       3. **Reconciliation disposes:** a parse is trusted only if its per-row balance walks, or
? when a
10         vintage has no per-row balance ? the totals reconcile to the printed
``opening``/``closing``.
11
12      A statement splits its ledger across several extracted tables (page breaks), so we map
each table
13      on its own, keep the rows the oracle endorses, and re-check the *combined* set (which
catches a
14      cross-table balance break, i.e. a missing transaction). The result is **oracle-gated**:
a layout we
15      can't yet read returns ``verified=False`` and the caller persists nothing ? never
mislabeled rows.
16
17      No statement content is logged; only counts/booleans travel out.
18      """
19
20      from __future__ import annotations
21
22      import re
23      from dataclasses import dataclass, field
24      from pathlib import Path
```

```
25
26      import pdfplumber
27
28      from .clustering import cluster_to_matrix
29      from .concepts import ConceptRow, StatementConcepts
30      from .engine import Profile, map_table
31      from .validation import parse_amount, reconcile_running_balance, reconcile_total
32
33      _MONEY_TOKEN = re.compile(r"^-?[\d,]+\.\d{2}(?:cr|dr)?$", re.IGNORECASE)
34      _OPENING_RE = re.compile(r"opening\s+balance[^\d\-]*(-?[\d,]+\.\d{2})", re.IGNORECASE)
35      _CLOSING_RE = re.compile(r"closing\s+balance[^\d\-]*(-?[\d,]+\.\d{2})", re.IGNORECASE)
36
37
38      @dataclass
39      class SavingsParse:
40          """Outcome of a savings parse: concept rows + whether reconciliation endorsed
them."""
41
42          account_number: str
43          rows: list[ConceptRow] = field(default_factory=list)
44          verified: bool = False
45
46
47      def parse_statement_summary(text: str) -> StatementConcepts:
48          """Read opening/closing balance from statement text (for the total-reconcile
oracle).
49
50          Best-effort regex; ``None`` when not found ? the oracle then relies on the per-row
walk.
51          """
52          opening = _OPENING_RE.search(text)
53          closing = _CLOSING_RE.search(text)
54          return StatementConcepts(
55              opening_balance=parse_amount(opening.group(1)) if opening else None,
56              closing_balance=parse_amount(closing.group(1)) if closing else None,
57          )
58
59
60      def txn_line_predicate(profile: Profile):
61          """A line is transaction-like if it carries one of the profile's dates *and* a money
token.
62
63          Used to restrict coordinate clustering to the transaction band (drops headers /
summaries /
64          address blocks that would otherwise wreck column inference).
65          """
66          patterns = profile.date_patterns
67
68          def keep(texts: list[str]) -> bool:
69              has_date = any(any(p.match(t) for p in patterns) for t in texts)
70              return has_date and any(_MONEY_TOKEN.match(t) for t in texts)
71
72          return keep
73
74
75      def reconcile_oracle(opening: float | None, closing: float | None):
76          """Trust a parse if the per-row balance walks, else if the totals reconcile to the
bookends."""
77
78          def check(rows: list[ConceptRow]) -> bool:
79              if reconcile_running_balance([(r.debit, r.credit, r.balance) for r in rows]).ok:
80                  return True
81              if rows and opening is not None and closing is not None:
82                  return reconcile_total(rows, opening, closing)
83              return False
```

```
84
85          return check
86
87
88      def _reconstruct_by_balance_delta(
89          matrix: list[list[str]], profile: Profile, opening: float | None, tol: float = 0.01
90      ) -> list[ConceptRow]:
91          """Recover debit/credit from the running balance when columns won't split cleanly.
92
93          Needs only three things per row ? a date, the transaction amount, and the running
balance ? so
94          it tolerates messy column inference (a sliver column, an interleaved value-date
column, or a
95          single merged withdrawal/deposit column). Direction is the sign of the balance
change; the
96          caller's reconciliation then **cross-checks** that the amount equals |?balance| on
every row, so
97          a misread or a dropped row still fails (it is *not* circular). Needs the printed
opening balance
98          to fix the first row's direction; returns ``[]`` if it can't form a clean series.
99          """
100         patterns = profile.date_patterns or (re.compile(r"^\d{2}[-/]\d{2}[-/]\d{4}$"),)
101         rows = [[(str(c).strip() if c else "") for c in r] for r in (matrix or []) if r]
102         if not rows or opening is None:
103             return []
104         ncols = max(len(r) for r in rows)
105
106         def is_date(s: str) -> bool:
107             return any(p.match(s) for p in patterns)
108
109         def money(s: str) -> float | None:
110             return parse_amount(s) if _MONEY_TOKEN.match(s) else None
111
112         # date column = the leftmost of the (first few) columns with the most date cells
113         date_counts: dict[int, int] = {}
114         for r in rows:
115             for c in range(min(ncols, 3)):
116                 if c < len(r) and is_date(r[c]):
117                     date_counts[c] = date_counts.get(c, 0) + 1
118         if not date_counts:
119             return []
120         top = max(date_counts.values())
121         date_col = min(c for c, n in date_counts.items() if n == top)
122
123         txn_rows = [r for r in rows if date_col < len(r) and is_date(r[date_col])]
124         if len(txn_rows) < 2:
125             return []
126
127         # A money column's money cells must outnumber its date cells ? so a sparsely-filled
deposit
128         # column still counts, but an interleaved value-date column (mostly dates) does not.
The
129         # running balance is the rightmost money column; the rest are the flow
(withdrawal/deposit).
130         money_cols = []
131         for c in range(ncols):
132             m = sum(1 for r in txn_rows if c < len(r) and money(r[c]) is not None)
133             d = sum(1 for r in txn_rows if c < len(r) and is_date(r[c]))
134             if m >= 1 and m > d:
135                 money_cols.append(c)
136         if len(money_cols) < 2:
137             return []
138         balance_col = max(money_cols)
139         flow_cols = [c for c in money_cols if c != balance_col]
140
```

```
        text_cols = [c for c in range(ncols) if c != date_col and c not in money_cols]
        narr_col = (
            max(text_cols, key=lambda c: sum(len(r[c]) for r in txn_rows if c < len(r)))
            if text_cols
            else None
        )

        out: list[ConceptRow] = []
        prev = opening
        for r in txn_rows:
            bal = money(r[balance_col]) if balance_col < len(r) else None
            if bal is None:
                return []  # a running balance on every row is required for the delta
            amt = next((money(r[c]) for c in flow_cols if c < len(r) and money(r[c]) is not
None), None)
            debit = credit = None
            if amt is not None:
                if bal < prev - tol:
                    debit = amt
                elif bal > prev + tol:
                    credit = amt
                else:
                    debit = amt  # zero net with an amount present ? reconciliation will
flag it
            out.append(
                ConceptRow(
                    txn_date=r[date_col],
                    narration=(r[narr_col] if narr_col is not None and narr_col < len(r)
else ""),
                    debit=debit,
                    credit=credit,
                    balance=bal,
                )
            )
            prev = bal
        return out


    def parse_savings(pdf_path: Path, profile: Profile, password: str | None = None) ->
SavingsParse:
        """Parse a savings statement into concept rows + a reconciliation verdict, per the
profile.

        Per-table mapping first (each extracted table mapped on its own; oracle-endorsed
rows combined
        and re-checked), then a coordinate-clustering fallback. Returns ``verified=False``
if neither
        reconciles, so the caller can refuse to persist.
        """
        with pdfplumber.open(pdf_path, password=password) as pdf:
            words_by_page = [
                page.extract_words(use_text_flow=False, keep_blank_chars=False) for page in
pdf.pages
            ]
            tables = [t for page in pdf.pages for t in (page.extract_tables() or [])]
            head = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(3,
len(pdf.pages))))

        account_number = profile.first_account(head) or "UNKNOWN"
        summary = parse_statement_summary(head)
        oracle = reconcile_oracle(summary.opening_balance, summary.closing_balance)

        confirmed: list[ConceptRow] = []
        for table in tables:
            res = map_table(table, profile, oracle=oracle)
```

```
            if res and res.verified:
                confirmed.extend(res.rows)
        if confirmed and oracle(confirmed):
            return SavingsParse(account_number, confirmed, True)

        matrix = cluster_to_matrix(words_by_page, keep_line=txn_line_predicate(profile))
        clustered = map_table(matrix, profile, oracle=oracle)
        if clustered and clustered.verified:
            return SavingsParse(account_number, clustered.rows, True)

        # 3) balance-delta reconstruction ? recovers vintages whose columns don't split into
a clean
        #    debit/credit (merged amount column, interleaved value-date, sliver columns):
derive the
        #    direction from the running balance and let the oracle cross-check amount ==
|?balance|.
        delta = _reconstruct_by_balance_delta(matrix, profile, summary.opening_balance)
        if delta and oracle(delta):
            return SavingsParse(account_number, delta, True)

        fallback = confirmed or (clustered.rows if clustered else [])
        return SavingsParse(account_number, fallback, False)


    __all__ = [
        "SavingsParse",
        "parse_savings",
        "parse_statement_summary",
        "txn_line_predicate",
        "reconcile_oracle",
    ]

```

131. user (2026-06-10T17:15:53.136Z)

```
      """The generic concept-mapping engine ? columns in, concepts out, arithmetic decides.

      This is the *generic-first* half of the concept-core architecture (see
      [`docs/parser-architecture.md`](../../../docs/parser-architecture.md) §4, §6).
      Bank-agnostic code that takes a **cell-matrix** (the shape `pdfplumber.extract_tables()`
returns,
      or what a coordinate-clustering extractor emits) and maps its columns onto the canonical
concepts
      via a cheap-first ladder, with a profile supplying priors:

      * **L0 ? header lexicon.** Match header-row labels against the profile's concept?label
aliases.
      * **profile-order.** If there is no header, fall back to the profile's known column
order (a prior).
      * **L1/L2 ? content + search.** Classify columns by content (dates, money), then
enumerate the
        plausible amount-column ? {debit, credit, balance} assignments and **keep the one the
        reconciliation oracle endorses** ? the self-identifying balance column, found by
*which mapping
        makes the math work* rather than by its label.

      The engine is **family-agnostic**: it is parameterized by a :class:`FamilySpec` (the
target concept
      names, which are date/amount/balance, and a typed-row builder) and an injected `oracle`
      (`rows -> bool`). Only the *bank* family ships today; an instrument family
(equity/MF/ETF/bond ? see
      the project scope) would reuse this control flow with its own concepts + oracle. Nothing
here logs a
      value; provenance is the winning layer + column map.
      """
```

```

    from __future__ import annotations

    import itertools
    import re
    from collections.abc import Callable, Sequence
    from dataclasses import dataclass, field

    from .concepts import ConceptRow
    from .validation import parse_amount

    Cell = object  # a table cell: str | None (we coerce defensively)
    Row = Sequence[Cell]

    # Generic Indian-statement date shapes; a profile can pin its own to avoid false
positives.
    _GENERIC_DATE_PATTERNS: tuple[re.Pattern, ...] = (
        re.compile(r"^\d{2}[/-]\d{2}[/-]\d{4}$"),  # dd/mm/yyyy or dd-mm-yyyy
        re.compile(r"^\d{2}[- ][A-Za-z]{3}[- ]\d{2,4}$"),  # dd-Mon-yyyy / dd Mon yy
    )
    # "Money" requires a decimal point so bare integers (e.g. a long reference number) are
NOT mistaken
    # for an amount column during content classification.
    _MONEY_RE = re.compile(r"(?i)^-?[\d,]+\.\d{1,2}(?:\s*(?:cr|dr))?$")
    _BALANCE_MARKERS = ("opening balance", "closing balance", "balance b/f", "balance c/f")

    # The generic bank lexicon. Profiles extend/override per issuer; keys are concept names,
values are
    # lower-cased header labels matched exactly (not by substring) during L0.
    _BANK_ALIASES: dict[str, tuple[str, ...]] = {
        "txn_date": ("date", "txn date", "tran date", "transaction date", "posting date"),
        "value_date": ("value date", "val date", "value dt"),
        "narration": (
            "narration",
            "particulars",
            "description",
            "transaction details",
            "details",
            "remarks",
            "transaction",
            "transaction remarks",
        ),
        "reference": (
            "ref",
            "reference",
            "ref no",
            "reference no",
            "cheque no",
            "chq no",
            "instrument no",
        ),
        "debit": ("debit", "withdrawal", "withdrawals", "dr", "paid out", "debit amount"),
        "credit": ("credit", "deposit", "deposits", "cr", "paid in", "credit amount"),
        "balance": (
            "balance",
            "closing balance",
            "running balance",
            "closing bal",
            "available balance",
        ),
    }


    def _text(cell: Cell) -> str:
        return "" if cell is None else str(cell).strip()
```

```
84
85
86      def _norm(label: str) -> str:
87          return re.sub(r"\s+", " ", label.strip().lower())
88
89
90      def _matches_date(text: str, patterns: Sequence[re.Pattern]) -> bool:
91          return any(p.match(text) for p in patterns)
92
93
94      def _looks_like_money(text: str) -> bool:
95          return bool(_MONEY_RE.match(text))
96
97
98      @dataclass
99      class FamilySpec:
100         """A concept *family*: the target vocabulary + how to build typed rows from a column
mapping.
101
102         The engine's L0/L1/L2 logic is written against this spec rather than against bank
concepts, so a
103         future instrument family (with its own concepts + reconciliation oracle) reuses the
same engine.
104         ``build`` turns per-row ``{concept: raw_cell}`` dicts into typed rows the oracle
understands.
105         """
106
107         name: str
108         concepts: tuple[str, ...]
109         date_concepts: tuple[str, ...]  # in order: the first is the primary (row-gating)
date
110         amount_concepts: tuple[str, ...]  # money columns; the non-balance ones are the
"flows"
111         balance_concept: str | None  # the running-balance concept, if the family has one
112         text_concepts: tuple[str, ...]  # free-text columns; the first gets the widest text
column
113         default_aliases: dict[str, tuple[str, ...]]
114         build: Callable[[list[dict[str, str]]], list]
115
116
117     def _build_bank_rows(mapped: list[dict[str, str]]) -> list[ConceptRow]:
118         """Turn mapped cells into :class:`ConceptRow`s (amounts parsed; absent stays
``None``)."""
119         rows: list[ConceptRow] = []
120         for m in mapped:
121             rows.append(
122                 ConceptRow(
123                     txn_date=_text(m.get("txn_date")),
124                     narration=_text(m.get("narration")),
125                     debit=parse_amount(m.get("debit")),
126                     credit=parse_amount(m.get("credit")),
127                     balance=parse_amount(m.get("balance")),
128                     value_date=_text(m.get("value_date")) or None,
129                     reference=_text(m.get("reference")) or None,
130                 )
131             )
132         return rows
133
134
135     BANK_FAMILY = FamilySpec(
136         name="bank",
137         concepts=("txn_date", "value_date", "narration", "reference", "debit", "credit",
"balance"),
138         date_concepts=("txn_date", "value_date"),
139         amount_concepts=("debit", "credit", "balance"),
```

```
        balance_concept="balance",
        text_concepts=("narration", "reference"),
        default_aliases=_BANK_ALIASES,
        build=_build_bank_rows,
    )


    @dataclass
    class MappingResult:
        """The outcome of mapping one table: typed rows + how we got there (provenance)."""

        rows: list  # typed rows from family.build (e.g. list[ConceptRow])
        mapped: list[
            dict[str, str]
        ]  # per-row {concept: raw cell}, incl. any profile-declared extra columns
        column_map: dict[str, int]  # the winning concept -> column-index assignment
        layer: str  # "L0" / "profile-order" / "L2" ? which rung produced it
        verified: bool  # did the oracle endorse it? (False if no oracle was supplied)


    @dataclass
    class Profile:
        """Declarative per-issuer knowledge ? *data, not code* (architecture §6).

        Generic detectors run first; the profile supplies priors: extra concept?label
aliases, the
        accepted date shape, a positional fallback when there's no header, table anchors,
account-number
        patterns, quirks, and (placeholder, wiring parked) the password rule.
        """

        name: str
        aliases: dict[str, tuple[str, ...]] = field(default_factory=dict)
        date_patterns: tuple[re.Pattern, ...] = ()
        column_order: tuple[str, ...] | None = None
        table_anchors: tuple[str, ...] = ()
        account_patterns: tuple[re.Pattern, ...] = ()
        quirks: frozenset[str] = frozenset()
        password_rule: str | None = None  # data only; sourcing is parked (DESIGN §8.2)

        def first_account(self, text: str) -> str | None:
            for rx in self.account_patterns:
                m = rx.search(text)
                if m:
                    return m.group(1).strip()
            return None


    def _alias_index(profile: Profile, family: FamilySpec) -> dict[str, str]:
        """Build ``normalized-label -> concept`` (profile entries override family
defaults)."""
        index: dict[str, str] = {}
        for concept, labels in family.default_aliases.items():
            for label in labels:
                index[_norm(label)] = concept
        for concept, labels in profile.aliases.items():
            for label in labels:
                index[_norm(label)] = concept
        return index


    def _date_patterns(profile: Profile) -> tuple[re.Pattern, ...]:
        return profile.date_patterns or _GENERIC_DATE_PATTERNS


```

```
    def _detect_header(
        rows: list[list[Cell]], alias_index: dict[str, str]
    ) -> tuple[int | None, dict[str, int]]:
        """Find the header row (most exact alias hits, >=2) and its concept->column map."""
        best_idx: int | None = None
        best_map: dict[str, int] = {}
        for i, row in enumerate(rows):
            cmap: dict[str, int] = {}
            for col, cell in enumerate(row):
                concept = alias_index.get(_norm(_text(cell)))
                if concept and concept not in cmap:
                    cmap[concept] = col
            if len(cmap) > len(best_map):
                best_map, best_idx = cmap, i
        if len(best_map) < 2:
            return None, {}
        return best_idx, best_map


    def _is_balance_marker(row: Row) -> bool:
        return any(any(mk in _norm(_text(c)) for mk in _BALANCE_MARKERS) for c in row)


    def _is_transaction_row(row: Row, date_col: int | None, patterns: Sequence[re.Pattern])
-> bool:
        if date_col is not None:
            cell = row[date_col] if date_col < len(row) else None
            return _matches_date(_text(cell), patterns)
        return any(_matches_date(_text(c), patterns) for c in row)


    def _classify_columns(
        data: list[list[Cell]], ncols: int, patterns: Sequence[re.Pattern]
    ) -> dict[int, str]:
        """Tag each column date / amount / text / empty by majority content."""
        kinds: dict[int, str] = {}
        for c in range(ncols):
            cells = [_text(r[c]) for r in data if c < len(r) and _text(r[c])]
            if not cells:
                kinds[c] = "empty"
                continue
            thresh = max(1, int(len(cells) * 0.6))
            if sum(1 for x in cells if _matches_date(x, patterns)) >= thresh:
                kinds[c] = "date"
            elif sum(1 for x in cells if _looks_like_money(x)) >= thresh:
                kinds[c] = "amount"
            else:
                kinds[c] = "text"
        return kinds


    def _base_map(
        col_kinds: dict[int, str], data: list[list[Cell]], family: FamilySpec
    ) -> dict[str, int]:
        """The unambiguous part of a content (L1) mapping: date columns + the primary text
column.

        Family-agnostic: date columns fill ``family.date_concepts`` in order; the widest
text column
        goes to the family's first text concept (if any). Amount columns are left to the
search.
        """
        base: dict[str, int] = {}
        date_cols = [c for c, k in sorted(col_kinds.items()) if k == "date"]
        text_cols = [c for c, k in sorted(col_kinds.items()) if k == "text"]
```

```
        for concept, col in zip(family.date_concepts, date_cols, strict=False):
            base[concept] = col
        if text_cols and family.text_concepts:
            base[family.text_concepts[0]] = max(
                text_cols, key=lambda c: sum(len(_text(r[c])) for r in data if c < len(r))
            )
        return base


    def _search_candidates(
        base: dict[str, int], col_kinds: dict[int, str], family: FamilySpec
    ) -> list[tuple[str, dict[str, int]]]:
        """Enumerate plausible amount-column ? concept assignments (the L2 search space).

        Family-agnostic: the "flows" are the family's amount concepts minus its balance
concept (for
        banks: debit, credit). We try each amount column as the balance and assign the rest
to the
        flows, then also the no-per-row-balance case (total-reconcile families, e.g. Axis
Oct).
        """
        amount_cols = [c for c, k in sorted(col_kinds.items()) if k == "amount"]
        flows = [c for c in family.amount_concepts if c != family.balance_concept]
        bal = family.balance_concept
        cands: list[dict[str, int]] = []
        if bal and len(amount_cols) >= len(flows) + 1:
            for bcol in amount_cols:
                rest = [c for c in amount_cols if c != bcol]
                for assign in itertools.permutations(rest, len(flows)):
                    cands.append({**base, **dict(zip(flows, assign, strict=True)), bal:
bcol})
        if flows and len(amount_cols) >= len(flows):
            for assign in itertools.permutations(amount_cols, len(flows)):
                cands.append({**base, **dict(zip(flows, assign, strict=True))})
        return [("L2", c) for c in cands[:24]]


    def _apply_map(data: list[list[Cell]], col_map: dict[str, int]) -> list[dict[str, str]]:
        out: list[dict[str, str]] = []
        for row in data:
            out.append({c: (_text(row[i]) if i < len(row) else "") for c, i in
col_map.items()})
        return out


    def map_table(
        table: Sequence[Row] | None,  # Sequence (covariant) so list[list[str | None]]
callers fit
        profile: Profile,
        family: FamilySpec = BANK_FAMILY,
        *,
        oracle: Callable[[list], bool] | None = None,
    ) -> MappingResult | None:
        """Map one extracted table onto ``family``'s concepts; ``oracle`` selects among
candidates.

        Resolution order: header lexicon (L0) ? profile column order ? content search (L2).
With an
        ``oracle`` the first candidate it endorses wins (``verified=True``); without one,
the first
        structurally-complete candidate is returned unverified (the caller reconciles
separately).
        Returns ``None`` if no transaction rows are found, or the best unverified candidate
if an oracle
        was supplied but nothing reconciled.
```

```
317         """
318         if not table:
319             return None
320         rows_in = [list(r) for r in table if r is not None]
321         if not rows_in:
322             return None
323         alias_index = _alias_index(profile, family)
324         patterns = _date_patterns(profile)
325 
326         header_idx, col_map_hdr = _detect_header(rows_in, alias_index)
327         start = header_idx + 1 if header_idx is not None else 0
328         date_col_hint = col_map_hdr.get("txn_date")
329         if date_col_hint is None and profile.column_order and "txn_date" in
profile.column_order:
330             date_col_hint = profile.column_order.index("txn_date")
331 
332         data = [
333             r
334             for r in rows_in[start:]
335             if not _is_balance_marker(r) and _is_transaction_row(r, date_col_hint, patterns)
336         ]
337         if not data:
338             return None
339         ncols = max(len(r) for r in data)
340         col_kinds = _classify_columns(data, ncols, patterns)
341         base = _base_map(col_kinds, data, family)
342         flows = [c for c in family.amount_concepts if c != family.balance_concept]
343 
344         candidates: list[tuple[str, dict[str, int]]] = []
345         if flows and all(c in col_map_hdr for c in flows):  # L0: header pinned the flow
concepts
346             candidates.append(("L0", {**base, **col_map_hdr}))
347         elif col_map_hdr:  # header present but didn't pin the flows: keep its hints, search
the rest
348             candidates.append(("L0", {**base, **col_map_hdr}))
349         if profile.column_order:  # positional prior (no/partial header)
350             candidates.append(("profile-order", {c: i for i, c in
enumerate(profile.column_order)}))
351         candidates += _search_candidates(base, col_kinds, family)
352 
353         primary_date = family.date_concepts[0] if family.date_concepts else None
354         best: MappingResult | None = None
355         for layer, cmap in candidates:
356             if primary_date and primary_date not in cmap:
357                 continue
358             if not (set(family.amount_concepts) & cmap.keys()):
359                 continue
360             mapped = _apply_map(data, cmap)
361             typed = family.build(mapped)
362             if oracle is None:
363                 return MappingResult(typed, mapped, cmap, layer, verified=False)
364             if oracle(typed):
365                 return MappingResult(typed, mapped, cmap, layer, verified=True)
366             if best is None:
367                 best = MappingResult(typed, mapped, cmap, layer, verified=False)
368         return best
369 
370 
371     __all__ = ["Profile", "FamilySpec", "BANK_FAMILY", "MappingResult", "map_table"]
372 
```

132. user (2026-06-10T17:16:25.758Z)

```
tests\parsers\test_validation.py:8:    reconcile_total,
tests\parsers\test_validation.py:58:def test_reconcile_total_clean_and_broken():
```

```
tests\parsers\test_validation.py:60:    assert reconcile_total(rows, opening=10000.0,
closing=9999.5)
tests\parsers\test_validation.py:61:    assert not reconcile_total(rows, opening=10000.0,
closing=9990.0)
tests\parsers\test_validation.py:64:def test_reconcile_total_needs_no_per_row_balance():
tests\parsers\test_validation.py:70:    assert reconcile_total(rows, opening=1000.0,
closing=800.0)  # 1000 ? 300 + 100
tests\parsers\test_validation.py:71:    assert not reconcile_total(rows, opening=1000.0,
closing=999.0)
src\muneem\parsers\au.py:22:from .savings import SavingsParse, parse_savings
src\muneem\parsers\au.py:28:    return parse_savings(pdf_path, AUBANK_PROFILE, password)
src\muneem\parsers\axis.py:29:from .savings import SavingsParse, parse_savings,
parse_statement_summary, txn_line_predicate
src\muneem\parsers\axis.py:54:    """Parse an Axis savings statement via the shared engine-
driven path (savings.parse_savings)."""
src\muneem\parsers\axis.py:55:    return parse_savings(pdf_path, AXIS_PROFILE, password)
src\muneem\parsers\savings.py:31:from .validation import parse_amount,
reconcile_running_balance, reconcile_total
src\muneem\parsers\savings.py:75:def reconcile_oracle(opening: float | None, closing: float |
None):
src\muneem\parsers\savings.py:82:            return reconcile_total(rows, opening, closing)
src\muneem\parsers\savings.py:176:def parse_savings(pdf_path: Path, profile: Profile, password:
str | None = None) -> SavingsParse:
src\muneem\parsers\savings.py:192:    oracle = reconcile_oracle(summary.opening_balance,
summary.closing_balance)
src\muneem\parsers\savings.py:220:    "parse_savings",
src\muneem\parsers\savings.py:223:    "reconcile_oracle",
src\muneem\parsers\validation.py:12:* **total reconcile** ? ``reconcile_total``: ``opening +
?credit ? ?debit == closing`` (works even
src\muneem\parsers\validation.py:107:def reconcile_total(
src\muneem\parsers\validation.py:197:    running balance ? balance-less statements select via
:func:`reconcile_total` at the call site.
src\muneem\parsers\validation.py:209:    "reconcile_total",
tests\test_verifier_properties.py:14:from muneem.parsers.validation import
reconcile_running_balance, reconcile_total
tests\test_verifier_properties.py:42:    assert reconcile_total(rows, opening=opening,
closing=closing)
tests\test_verifier_properties.py:67:    assert reconcile_total(rows, opening=opening,
closing=closing)
tests\test_verifier_properties.py:68:    assert not reconcile_total(rows, opening=opening,
closing=round(closing + 1.0, 2))
```

133. user (2026-06-10T17:16:42.748Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\savings.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

134. assistant (2026-06-10T17:16:31.020Z)

```
Clean ? `reconcile_oracle` is used only inside `parse_savings`. Implementing B1:
```

135. user (2026-06-10T17:16:33.174Z)

```
Switched to a new branch 'fix/savings-bookend-anchoring'
```

136. user (2026-06-10T17:16:56.353Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\savings.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

137. user (2026-06-10T17:17:04.257Z)

```
1       """Tests for the shared savings path's balance-delta reconstruction
(muneem.parsers.savings).
```

```
2
3       `_reconstruct_by_balance_delta` recovers debit/credit from the running-balance delta
when columns
4       don't split cleanly (a merged amount column, an interleaved value-date column, sparse
deposits). It
5       is exercised here on synthetic cell-matrices ? no PDF, no secrets ? including the
*dropped-row* case
6       (a balance jump matching no single amount) which must NOT reconcile (the real Axis May
2025 case).
7       """
8
9       from muneem.parsers import savings
10      from muneem.parsers.profiles import AXIS_PROFILE
11      from muneem.parsers.validation import reconcile_running_balance
12
13
14      def _matrix():
15          """June-style: date | narration | withdrawal(sparse) | deposit(sparse) | value-date
| balance.
16          Opening 10,000 ? 9,000 (?1,000) ? 14,000 (+5,000) ? 12,000 (?2,000)."""
17          return [
18              ["02-08-2025", "UPI-ABC", "1,000.00", "", "01-08-2025", "9,000.00"],
19              ["05-08-2025", "SALARY", "", "5,000.00", "04-08-2025", "14,000.00"],
20              ["10-08-2025", "ATM-CASH", "2,000.00", "", "09-08-2025", "12,000.00"],
21          ]
22
23
24      def _reconciles(rows):
25          return reconcile_running_balance([(r.debit, r.credit, r.balance) for r in rows]).ok
26
27
28      def test_reconstructs_direction_from_balance_delta_and_reconciles():
29          rows = savings._reconstruct_by_balance_delta(_matrix(), AXIS_PROFILE,
opening=10000.0)
30          assert len(rows) == 3
31          assert _reconciles(rows)
32          # direction derived from the sign of ?balance; the value-date column is ignored as a
flow
33          assert rows[0].debit == 1000.0 and rows[0].credit is None
34          assert rows[1].credit == 5000.0 and rows[1].debit is None
35          assert rows[2].debit == 2000.0 and rows[2].credit is None
36          assert [r.balance for r in rows] == [9000.0, 14000.0, 12000.0]
37
38
39      def test_a_dropped_transaction_does_not_reconcile():
40          # Drop a transaction (make the last balance jump): |?balance| no longer equals the
row's amount,
41          # so the reconstruction must NOT reconcile ? the oracle quarantines an incomplete
ledger (May).
42          m = _matrix()
43          m[2] = ["10-08-2025", "ATM-CASH", "2,000.00", "", "09-08-2025", "1,000.00"]
44          rows = savings._reconstruct_by_balance_delta(m, AXIS_PROFILE, opening=10000.0)
45          assert not _reconciles(rows)
46
47
48      def test_requires_opening_balance_to_fix_first_direction():
49          assert savings._reconstruct_by_balance_delta(_matrix(), AXIS_PROFILE, opening=None)
== []
50
51
52      def test_ignores_a_leading_non_transaction_noise_row():
53          # A header/summary line (no leading date) must be dropped, not parsed as a
transaction.
54          m = [["Opening Balance", "", "", "", "", "10,000.00"], *_matrix()]
55          rows = savings._reconstruct_by_balance_delta(m, AXIS_PROFILE, opening=10000.0)
```

```
56          assert len(rows) == 3 and _reconciles(rows)
57
```

138. user (2026-06-10T17:17:21.399Z)

```
The file C:\Users\avidu\Projects\muneem\tests\parsers\test_savings.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

139. user (2026-06-10T17:17:46.399Z)

```
.......................................................................... [ 87%]
.....................................................                    [100%]
412 passed, 21 deselected in 4.98s
=== local real-statement tier ===
========================== short test summary info ==========================
SKIPPED [7] tests\parsers\test_hdfc.py:227: needs MUNEEM_TEST_HDFC_PW + real statements (never
in CI)
SKIPPED [4] tests\parsers\test_local_au.py:31: needs the AU password (password_rules.yaml or
MUNEEM_TEST_AU_PW)
SKIPPED [7] tests\parsers\test_local_axis.py:29: needs the Axis password (password_rules.yaml or
MUNEEM_TEST_AXIS_PW)
SKIPPED [1] tests\test_unlock.py:63: needs MUNEEM_TEST_HDFC_PW + real statements (never in CI)
19 skipped, 414 deselected in 0.74s
```

140. user (2026-06-10T17:18:06.309Z)

```
All checks passed!
79 files left unchanged
Success: no issues found in 33 source files
[fix/savings-bookend-anchoring 80e64ff] fix(parsers): savings trust oracle anchors a passing
walk to the printed bookends
 2 files changed, 102 insertions(+), 7 deletions(-)
```

141. user (2026-06-10T17:18:43.498Z)

```
To https://github.com/avidullu/muneem.git
 * [new branch]      fix/savings-bookend-anchoring -> fix/savings-bookend-anchoring
https://github.com/avidullu/muneem/pull/53
From https://github.com/avidullu/muneem
 * branch            main       -> FETCH_HEAD
   1ea952f..a1c9210  main       -> origin/main
Updating 1ea952f..a1c9210
Fast-forward
 src/muneem/parsers/savings.py | 57 ++++++++++++++++++++++++++++++++++------
 tests/parsers/test_savings.py | 52 ++++++++++++++++++++++++++++++++++++
 2 files changed, 102 insertions(+), 7 deletions(-)
Your branch is up to date with 'origin/main'.
Already on 'main'
Already up to date.
```

142. user (2026-06-10T17:18:58.654Z)

```
Updated task #4 status
```

143. user (2026-06-10T17:18:59.149Z)

```
Updated task #5 status
```

144. user (2026-06-10T17:19:00.148Z)

```
C:\Users\avidu\Projects\muneem\src\muneem\parsers\axis.py-27-from .base import StatementParser
C:\Users\avidu\Projects\muneem\src\muneem\parsers\axis.py-28-from .profiles import AXIS_PROFILE
C:\Users\avidu\Projects\muneem\src\muneem\parsers\axis.py-29-from .savings import SavingsParse,
parse_savings, parse_statement_summary, txn_line_predicate
```

```
src\muneem\parsers\axis.py:30:from .validation import Confidence, parse_amount
C:\Users\avidu\Projects\muneem\src\muneem\parsers\axis.py-31-
src\muneem\parsers\axis.py-32-_CC_DATE_RE = re.compile(r"\d{2}\d{2}\d{4}")  # Axis credit card:
dd/mm/yyyy
C:\Users\avidu\Projects\muneem\src\muneem\parsers\axis.py-33-_CARD_RES =
[re.compile(r"(\d{4}XXXX\d{4})"), re.compile(r"(\d{6}\*+\d{4})")]
--
C:\Users\avidu\Projects\muneem\src\muneem\parsers\axis.py-136-                continue
C:\Users\avidu\Projects\muneem\src\muneem\parsers\axis.py-137-            amount_str =
str(row[8]).strip() if row[8] else ""
C:\Users\avidu\Projects\muneem\src\muneem\parsers\axis.py-138-            is_cr = "Cr" in
amount_str
src\muneem\parsers\axis.py:139:            amount = parse_amount(re.sub(r"[^\d.]", "",
amount_str)) or 0.0
C:\Users\avidu\Projects\muneem\src\muneem\parsers\axis.py-140-            rows.append(
C:\Users\avidu\Projects\muneem\src\muneem\parsers\axis.py-141-                AxisCCTxn(
C:\Users\avidu\Projects\muneem\src\muneem\parsers\axis.py-142-
txn_date=date_str,
--
C:\Users\avidu\Projects\muneem\src\muneem\parsers\base.py-11-    ?? This maps ``"0.00"`` to
``None`` (treating a printed zero as *absent*). That is fine for a
C:\Users\avidu\Projects\muneem\src\muneem\parsers\base.py-12-    credit-card-style amount column
but **wrong for balance-bearing rows**, where a real ``0.00``
C:\Users\avidu\Projects\muneem\src\muneem\parsers\base.py-13-    balance must survive the
reconciliation walk. For anything feeding reconciliation use
src\muneem\parsers\base.py:14:    ``muneem.parsers.validation.parse_amount`` instead (it keeps
``0.0`` distinct from ``None``).
C:\Users\avidu\Projects\muneem\src\muneem\parsers\base.py-15-    """
src\muneem\parsers\base.py-16-    if not s or s.strip() in ("-", "", "0.00"):
C:\Users\avidu\Projects\muneem\src\muneem\parsers\base.py-17-        return None
--
C:\Users\avidu\Projects\muneem\src\muneem\parsers\engine.py-28-from dataclasses import
dataclass, field
C:\Users\avidu\Projects\muneem\src\muneem\parsers\engine.py-29-
C:\Users\avidu\Projects\muneem\src\muneem\parsers\engine.py-30-from .concepts import ConceptRow
src\muneem\parsers\engine.py:31:from .validation import parse_amount
C:\Users\avidu\Projects\muneem\src\muneem\parsers\engine.py-32-
src\muneem\parsers\engine.py-33-Cell = object  # a table cell: str | None (we coerce
defensively)
C:\Users\avidu\Projects\muneem\src\muneem\parsers\engine.py-34-Row = Sequence[Cell]
--
C:\Users\avidu\Projects\muneem\src\muneem\parsers\engine.py-122-            ConceptRow(
C:\Users\avidu\Projects\muneem\src\muneem\parsers\engine.py-123-
txn_date=_text(m.get("txn_date")),
C:\Users\avidu\Projects\muneem\src\muneem\parsers\engine.py-124-
narration=_text(m.get("narration")),
src\muneem\parsers\engine.py:125:                debit=parse_amount(m.get("debit")),
src\muneem\parsers\engine.py:126:                credit=parse_amount(m.get("credit")),
src\muneem\parsers\engine.py:127:                balance=parse_amount(m.get("balance")),
C:\Users\avidu\Projects\muneem\src\muneem\parsers\engine.py-128-
value_date=_text(m.get("value_date")) or None,
C:\Users\avidu\Projects\muneem\src\muneem\parsers\engine.py-129-
reference=_text(m.get("reference")) or None,
C:\Users\avidu\Projects\muneem\src\muneem\parsers\engine.py-130-            )
--
C:\Users\avidu\Projects\muneem\src\muneem\parsers\savings.py-28-from .clustering import
cluster_to_matrix
C:\Users\avidu\Projects\muneem\src\muneem\parsers\savings.py-29-from .concepts import
ConceptRow, StatementConcepts
C:\Users\avidu\Projects\muneem\src\muneem\parsers\savings.py-30-from .engine import Profile,
map_table
src\muneem\parsers\savings.py:31:from .validation import parse_amount,
reconcile_running_balance, reconcile_total
C:\Users\avidu\Projects\muneem\src\muneem\parsers\savings.py-32-
src\muneem\parsers\savings.py-33-_MONEY_TOKEN = re.compile(r"^-?[\d,]+\d{2}(?:cr|dr)?$",
```

```
re.IGNORECASE)
C:\Users\avidu\Projects\muneem\src\muneem\parsers\savings.py-34-_OPENING_RE =
re.compile(r"opening\s+balance[^\d\-]*(-?[\d,]+\.\d{2})", re.IGNORECASE)
--
C:\Users\avidu\Projects\muneem\src\muneem\parsers\savings.py-52-    opening =
_OPENING_RE.search(text)
C:\Users\avidu\Projects\muneem\src\muneem\parsers\savings.py-53-    closing =
_CLOSING_RE.search(text)
C:\Users\avidu\Projects\muneem\src\muneem\parsers\savings.py-54-    return StatementConcepts(
src\muneem\parsers\savings.py:55:        opening_balance=parse_amount(opening.group(1)) if
opening else None,
src\muneem\parsers\savings.py:56:        closing_balance=parse_amount(closing.group(1)) if
closing else None,
C:\Users\avidu\Projects\muneem\src\muneem\parsers\savings.py-57-    )
C:\Users\avidu\Projects\muneem\src\muneem\parsers\savings.py-58-
C:\Users\avidu\Projects\muneem\src\muneem\parsers\savings.py-59-
--
C:\Users\avidu\Projects\muneem\src\muneem\parsers\savings.py-147-        return any(p.match(s)
for p in patterns)
C:\Users\avidu\Projects\muneem\src\muneem\parsers\savings.py-148-
src\muneem\parsers\savings.py-149-    def money(s: str) -> float | None:
src\muneem\parsers\savings.py:150:        return parse_amount(s) if _MONEY_TOKEN.match(s) else
None
C:\Users\avidu\Projects\muneem\src\muneem\parsers\savings.py-151-
C:\Users\avidu\Projects\muneem\src\muneem\parsers\savings.py-152-    # date column = the
leftmost of the (first few) columns with the most date cells
src\muneem\parsers\savings.py-153-    date_counts: dict[int, int] = {}
--
C:\Users\avidu\Projects\muneem\src\muneem\parsers\validation.py-32-_CR_DR_SUFFIX =
re.compile(r"(?i)\s*(cr|dr)\s*$")
C:\Users\avidu\Projects\muneem\src\muneem\parsers\validation.py-33-
C:\Users\avidu\Projects\muneem\src\muneem\parsers\validation.py-34-
src\muneem\parsers\validation.py:35:def parse_amount(value: object) -> float | None:
C:\Users\avidu\Projects\muneem\src\muneem\parsers\validation.py-36-    """Parse a statement
money cell to a float.
C:\Users\avidu\Projects\muneem\src\muneem\parsers\validation.py-37-
C:\Users\avidu\Projects\muneem\src\muneem\parsers\validation.py-38-    Returns ``None`` only for
a genuinely absent cell (None / empty / ``"-"`` /
--
C:\Users\avidu\Projects\muneem\src\muneem\parsers\validation.py-203-
C:\Users\avidu\Projects\muneem\src\muneem\parsers\validation.py-204-
C:\Users\avidu\Projects\muneem\src\muneem\parsers\validation.py-205-__all__ = [
src\muneem\parsers\validation.py:206:    "parse_amount",
C:\Users\avidu\Projects\muneem\src\muneem\parsers\validation.py-207-    "ReconcileReport",
C:\Users\avidu\Projects\muneem\src\muneem\parsers\validation.py-208-
"reconcile_running_balance",
C:\Users\avidu\Projects\muneem\src\muneem\parsers\validation.py-209-    "reconcile_total",
```

145. user (2026-06-10T17:19:55.770Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\validation.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

146. user (2026-06-10T17:19:45.554Z)

```
Switched to a new branch 'fix/crdr-sign-semantics'
```

147. assistant (2026-06-10T17:20:01.326Z)

Now the three balance call sites ? engine, summary regexes, delta reconstruction:

148. user (2026-06-10T17:20:03.653Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\engine.py has been updated
```

successfully. (file state is current in your context ? no need to Read it back)

149. user (2026-06-10T17:20:09.305Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\savings.py has been updated successfully. (file state is current in your context ? no need to Read it back)

150. user (2026-06-10T17:20:12.177Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\savings.py has been updated successfully. (file state is current in your context ? no need to Read it back)

151. user (2026-06-10T17:20:14.508Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\savings.py has been updated successfully. (file state is current in your context ? no need to Read it back)

152. user (2026-06-10T17:20:16.571Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\savings.py has been updated successfully. (file state is current in your context ? no need to Read it back)

153. assistant (2026-06-10T17:20:21.383Z)

Now the tests ? including the empirical inverted-overdraft demonstration:

154. user (2026-06-10T17:20:22.071Z)

```
1       """Tests for the verifier (muneem.parsers.validation): the per-row walk, the layout-
independent
2       total reconcile, the concept-level bookend, candidate selection, and the confidence
verdict."""
3
4       from muneem.parsers.concepts import ConceptRow, StatementConcepts
5       from muneem.parsers.validation import (
6           Confidence,
7           reconcile_running_balance,
8           reconcile_total,
9           select_reconciling,
10          verify,
11      )
12
13
14      def _concept_rows():
15          """3 rows that walk cleanly: 10000 -500-> 9500 +2000-> 11500 -1500.5-> 9999.5."""
16          return [
17              ConceptRow(txn_date="05/02/2026", narration="a", debit=500.0, credit=0.0,
balance=9500.0),
18              ConceptRow(txn_date="08/02/2026", narration="b", debit=0.0, credit=2000.0,
balance=11500.0),
19              ConceptRow(txn_date="10/02/2026", narration="c", debit=1500.5, credit=0.0,
balance=9999.5),
20          ]
21
22
23      def test_reconciles_clean():
24          entries = [(500.0, 0.0, 9500.0), (0.0, 2000.0, 11500.0), (1500.5, 0.0, 9999.5)]
25          r = reconcile_running_balance(entries)
26          assert r.n == 3 and r.compared == 2 and r.breaks == 0 and r.ok
27
28
29      def test_detects_break():
30          entries = [(500.0, 0.0, 9500.0), (0.0, 2000.0, 11500.0), (1500.5, 0.0, 9999.49)]
31          r = reconcile_running_balance(entries)
```

```
32          assert r.breaks == 1 and r.first_break == 2 and not r.ok
33
34
35      def test_none_debit_or_credit_treated_as_zero():
36          # Bank rows: exactly one of (debit, credit) is filled; the other is None.
37          entries = [(None, None, 1000.0), (250.0, None, 750.0), (None, 1250.0, 2000.0)]
38          r = reconcile_running_balance(entries)
39          assert r.compared == 2 and r.breaks == 0 and r.ok
40
41
42      def test_a_dropped_balance_surfaces_as_a_gap():
43          # If a row's balance goes missing (e.g. a silent extract_tables drop), the walk
44          # across the gap no longer adds up ? exactly the failure we want surfaced.
45          entries = [(None, None, 1000.0), (250.0, None, None), (None, None, 750.0)]
46          r = reconcile_running_balance(entries)
47          assert r.breaks == 1 and r.first_break == 2 and not r.ok
48
49
50      def test_empty_and_single_are_not_ok():
51          assert not reconcile_running_balance([]).ok
52          assert not reconcile_running_balance([(None, None, 100.0)]).ok  # nothing to compare
53
54
55      # --- total reconcile: opening + ?credit ? ?debit == closing (layout-independent)
-------------
```

155. user (2026-06-10T17:20:38.692Z)

```
The file C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

156. user (2026-06-10T17:20:47.773Z)

```
The file C:\Users\avidu\Projects\muneem\tests\parsers\test_savings.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

157. user (2026-06-10T17:21:01.953Z)

```
.................................................................... [ 86%]
.....................................................                [100%]
417 passed, 21 deselected in 4.36s
[*] 1 fixable with the `--fix` option.
79 files left unchanged
Success: no issues found in 33 source files
```

158. user (2026-06-10T17:21:16.057Z)

```
Found 1 error (1 fixed, 0 remaining).
All checks passed!
72 passed, 18 deselected in 0.99s
```

159. assistant (2026-06-10T17:21:24.089Z)

```
Gate green (417 passed). Shipping B2:
```

160. user (2026-06-10T17:21:56.049Z)

```
[fix/crdr-sign-semantics 43a9c8e] fix(parsers): Cr/Dr on a balance is a SIGN, not a tag to strip
 5 files changed, 89 insertions(+), 11 deletions(-)
 * [new branch]      fix/crdr-sign-semantics -> fix/crdr-sign-semantics
https://github.com/avidullu/muneem/pull/54
From https://github.com/avidullu/muneem
 * branch            main       -> FETCH_HEAD
   a1c9210..276c073  main       -> origin/main
```

```
Updating a1c9210..276c073
Fast-forward
 src/muneem/parsers/engine.py     |  3 ++-
 src/muneem/parsers/savings.py    | 26 ++++++++++++++++++-------
 src/muneem/parsers/validation.py | 20 ++++++++++++++++---
 tests/parsers/test_savings.py    | 13 +++++++++++++
 tests/parsers/test_validation.py | 38 ++++++++++++++++++++++++++++++++++++++
 5 files changed, 89 insertions(+), 11 deletions(-)
Your branch is up to date with 'origin/main'.
Already on 'main'
Already up to date.
```

161. user (2026-06-10T17:22:02.722Z)

```
Updated task #5 status
```

162. user (2026-06-10T17:22:03.217Z)

```
Updated task #6 status
```

163. user (2026-06-10T17:22:04.298Z)

```
380                     "narration",
381                     "ref",
382                     "withdrawal",
383                     "deposit",
384                     "closing_balance",
385                     "page",
386                 ]
387             )
388             for t in statement.transactions:
389                 writer.writerow(
390                     [
391                         t.date,
392                         t.value_date or "",
393                         t.narration,
394                         t.ref or "",
395                         f"{t.withdrawal:.2f}",
396                         f"{t.deposit:.2f}",
397                         f"{t.closing_balance:.2f}",
398                         t.page,
399                     ]
400                 )
401         return out
402
403
404     @register_parser
405     class HDFCParser(StatementParser):
406         """Registered parser for HDFC savings / transaction-account statements."""
407
408         def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
409             low = (text or "").lower()
410             is_hdfc = "hdfc bank" in low or "hdfcbank" in low or "hdfcbank" in (sender or
"").lower()
411             is_statement = "savings account details" in low or (
412                 "narration" in low and "closing balance" in low
413             )
414             return is_hdfc and is_statement
415
416         def parse_and_insert(
417             self,
418             pdf_path: Path,
419             conn,
```

```
            doc_id: int,
            password: str | None = None,
        ) -> int:
            """Parse, insert rows into ``hdfc_transactions``, and record parse quality.

            Sets ``documents.status`` to 'reconciled' / 'unreconciled' based on the
            balance walk (and the bank's SUMMARY when present) so a downstream consumer
            can tell a trustworthy parse from a suspect one. Returns the row count.
            """
            try:
                stmt = parse_statement(pdf_path, password=password or "")
            except ParseError:
                raise
            except Exception as exc:
                raise ParseError(f"failed to parse HDFC statement
{redact_filename(pdf_path)}") from exc
            for t in stmt.transactions:
                dbmod.insert_hdfc_transaction(
                    conn,
                    document_id=doc_id,
                    account_number=stmt.account_number,
                    txn_date=t.date,
                    value_date=t.value_date,
                    particulars=t.narration,
                    ref=t.ref,
                    withdrawal=t.withdrawal,
                    deposit=t.deposit,
                    closing_balance=t.closing_balance,
                )
            summary = parse_summary(pdf_path, password=password or "")
            ok = (
                validate_against_summary(stmt, summary).ok
                if summary is not None
                else reconcile(stmt).ok
            )
            verdict = Confidence.RECONCILED if ok else Confidence.UNRECONCILED
            dbmod.record_parse(
                conn, doc_id, confidence=verdict.value, parser="HDFCParser",
parser_version=__version__
            )
            return len(stmt.transactions)

```

164. user (2026-06-10T17:22:13.154Z)

```
                return m.group(1).strip()
        return None


    # --- Savings / transaction account (concept core)
-----------------------------------------
    # Back-compat names; the savings logic now lives in muneem.parsers.savings (shared with
AU Bank).
    AxisSavings = SavingsParse
    parse_axis_summary = parse_statement_summary
    _is_txn_line = txn_line_predicate(AXIS_PROFILE)


    def parse_axis_savings(pdf_path: Path, password: str | None = None) -> SavingsParse:
        """Parse an Axis savings statement via the shared engine-driven path
(savings.parse_savings)."""
        return parse_savings(pdf_path, AXIS_PROFILE, password)

```

```
57
58      @register_parser
59      class AxisParser(StatementParser):
60          def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
61              if sender and "axisbank.com" in sender.lower():
62                  return True
63              t = text or ""
64              return "Axis Bank" in t and ("Statement of Account" in t or "Account Statement"
in t)
65
66          def parse_and_insert(
67              self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
68          ) -> int:
69              """Persist a savings parse **only if it reconciles**; otherwise store nothing.
70
71              Sets ``documents.status`` to ``reconciled`` / ``unreconciled`` accordingly, so a
layout the
72              parser can't yet read surfaces loudly instead of saving mislabeled rows.
73              """
74              try:
75                  parsed = parse_axis_savings(pdf_path, password)
76              except ParseError:
77                  raise
78              except Exception as exc:
79                  # Filename only ? never statement text; original cause kept via `from`.
80                  name = redact_filename(pdf_path)
81                  raise ParseError(f"failed to parse Axis statement {name}") from exc
82
83              if not parsed.verified:
84                  dbmod.record_parse(
85                      conn,
86                      doc_id,
87                      confidence=Confidence.UNRECONCILED.value,
88                      parser="AxisParser",
89                      parser_version=__version__,
90                  )
91                  return 0
92
93              for r in parsed.rows:
94                  dbmod.insert_axis_transaction(
95                      conn,
96                      document_id=doc_id,
97                      account_number=parsed.account_number,
98                      txn_date=r.txn_date,
99                      value_date=r.value_date or "",
100                     particulars=r.narration,
101                     debit=r.debit,
102                     credit=r.credit,
103                     balance=r.balance,
104                 )
105             dbmod.record_parse(
106                 conn,
107                 doc_id,
108                 confidence=Confidence.RECONCILED.value,
109                 parser="AxisParser",
110                 parser_version=__version__,
111             )
112             return len(parsed.rows)
113
114
115     # --- Credit card (no running balance; count only)
-------------------------------------------
116
```

```
117
118     @dataclass
119     class AxisCCTxn:
120         txn_date: str
121         particulars: str
122         merchant_category: str
123         amount: float
124         type: str  # "Cr" or "Dr"
125
126
127     def _rows_from_cc_tables(tables: list) -> list[AxisCCTxn]:
128         """Build credit-card transactions from extracted tables. Pure: no PDF, no DB."""
129         rows: list[AxisCCTxn] = []
```

165. user (2026-06-10T17:22:13.768Z)

```
285                 status="parsed",
286             )
287             for code, src in found:
288                 dbmod.insert_offer(
289                     conn,
290                     document_id=doc_id,
291                     promo_code=code,
292                     expiry_raw=expiry_raw,
293                     expiry_type=expiry_type,
294                     source=src,
295                     source_text=ex.text[:2000],
296                 )
297
298         summary = ", ".join(f"{c} [{s}]" for c, s in found) if found else "none"
299         typer.echo(f"ingested {pdf.name} (doc {doc_id}); {len(found)} offer(s): {summary}")
300
301
302     @offers_app.command("list")
303     def offers_list(
304         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
305     ) -> None:
306         """List extracted offers."""
307         from muneem import db as dbmod
308
309         db_path = db or load_config().db_path
310         if not Path(db_path).exists():
311             typer.echo("no offers yet (database not created).")
312             return
313
314         with _ledger(db_path) as conn:
315             dbmod.init_db(conn)
316             rows = dbmod.list_offers(conn)
317
318         if not rows:
319             typer.echo("no offers found.")
320             return
321         for row in rows:
322             who = row["merchant"] or row["external_id"] or "?"
323             when = f" ? expires {row['expiry_raw']}" if row["expiry_raw"] else ""
324             tag = " [ocr ? verify]" if row["source"] == "ocr" else ""
325             typer.echo(f"[{row['id']}] {who}: {row['promo_code']}{when}{tag}")
326
327
328     @txns_app.command("list")
329     def txns_list(
330         bank: str = typer.Argument(
331             ..., help="Which bank to list (e.g. hdfc, axis) ? or 'all' for the cross-bank
view."
```

```
        ),
        db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
        limit: int = typer.Option(50, help="Max number of transactions to show."),
    ) -> None:
        """List parsed transactions for one bank, or across all banks via the serving
view."""
        from muneem import db as dbmod

        db_path = db or load_config().db_path
        if not Path(db_path).exists():
            typer.echo("database not created yet.")
            return

        bank = bank.lower()
        if bank == "all":
            with _ledger(db_path) as conn:
                dbmod.init_db(conn)
                rows = dbmod.list_all_transactions(conn, limit=limit)
            if not rows:
                typer.echo("no transactions found.")
                return
            for row in rows:
                amount = f"+{row['credit']}" if row["credit"] else f"-{row['debit']}"
                part = (row["particulars"] or "")[:40]
                verdict = row["confidence"] or "?"
                typer.echo(
                    f"[{row['bank']}:{row['account_number']}] {row['txn_date']} : "
                    f"{part:<40} | {amount:>10} [{verdict}]"
                )
            return

        if bank not in BANK_DISPATCH:
            typer.echo(f"Unknown bank: {bank}", err=True)
            return

        list_fn, neg_key, pos_key = BANK_DISPATCH[bank]
        with _ledger(db_path) as conn:
            dbmod.init_db(conn)
            rows = list_fn(conn, limit=limit)

        if not rows:
            typer.echo(f"no {bank} transactions found.")
            return
        for row in rows:
            if bank == "axis_cc":
                amount = f"+{row[pos_key]}" if row["type"] == "Cr" else f"-{row[neg_key]}"
            else:
                amount = f"+{row[pos_key]}" if row[pos_key] else f"-{row[neg_key]}"
            part = (row["particulars"] or "")[:50]
            typer.echo(f"[{row['account_number']}] {row['txn_date']} : {part:<50} |
{amount:>10}")


    @db_app.command("status")
    def db_status(
```

166. user (2026-06-10T17:22:28.796Z)

```
44-        with dbmod.connect_encrypted(db_path) as conn:
45-            yield conn
46-    except dbmod.EncryptedDatabaseError as exc:
47-        typer.echo(str(exc), err=True)
48-        raise typer.Exit(1) from exc
49-
```

```
50-
51-# Dispatch table for supported banks (extensible as new parsers land).
52:BANK_DISPATCH = {
53-    "axis": (dbmod.list_axis_transactions, "debit", "credit"),
54-    "axis_cc": (dbmod.list_axis_cc_transactions, "amount", "type"),
55-    "au": (dbmod.list_au_transactions, "debit", "credit"),
56-    "hdfc": (dbmod.list_hdfc_transactions, "withdrawal", "deposit"),
57-}
58-
59-
60-def _version_callback(value: bool) -> None:
--
354-            part = (row["particulars"] or "")[:40]
355-            verdict = row["confidence"] or "?"
356-            typer.echo(
357-                f"[{row['bank']}:{row['account_number']}] {row['txn_date']} : "
358-                f"{part:<40} | {amount:>10} [{verdict}]"
359-            )
360-        return
361-
362:    if bank not in BANK_DISPATCH:
363-        typer.echo(f"Unknown bank: {bank}", err=True)
364-        return
365-
366:    list_fn, neg_key, pos_key = BANK_DISPATCH[bank]
367-    with _ledger(db_path) as conn:
368-        dbmod.init_db(conn)
369-        rows = list_fn(conn, limit=limit)
370-
371-    if not rows:
372-        typer.echo(f"no {bank} transactions found.")
373-        return
374-    for row in rows:
```

167. user (2026-06-10T17:22:44.317Z)

```
1-"""Tests for the HDFC statement parser (muneem.parsers.hdfc).
2-
3-Column logic is exercised on *synthetic* word lists that mimic the real statement
4:geometry ? no real data, fast, offline. ``parse_and_insert`` is tested by
5-monkeypatching the parse functions (not by mocking pdfplumber), and a
6-``local_statements`` test runs the real parser on the user's PDFs (skipped in CI).
7-"""
8-
9-import os
10-from pathlib import Path
--
170-    # sender alone, with no statement markers, must NOT match (e.g. a promo from HDFC)
171-    assert not parser.can_handle("anything", sender="alerts@hdfcbank.net")
172-    assert not parser.can_handle("HDFC Bank promo: use code SAVE50")
173-    assert not parser.can_handle("Axis Bank Statement of Account Narration Closing Balance")
174-
175-
176:def test_parse_and_insert_records_reconciled(tmp_path, monkeypatch):
177-    monkeypatch.setattr(hdfc, "parse_statement", lambda *a, **k: _sample_statement())
178-    monkeypatch.setattr(
179-        hdfc,
180-        "parse_summary",
181-        lambda *a, **k: hdfc.StatementSummary(
182-            opening_balance=10000.0, closing_balance=9999.50, debit_count=2, credit_count=1
183-        ),
184-    )
185-    with dbmod.connect(tmp_path \ "t.db") as conn:
186-        dbmod.init_db(conn)
187-        doc_id = dbmod.insert_document(conn, source="file", sha256="h1", status="parsed")
```

```
188:        n = hdfc.HDFCParser().parse_and_insert(Path("x.pdf"), conn, doc_id)
189-        assert n == 3
190-        rows = dbmod.list_hdfc_transactions(conn, limit=100)
191-        assert len(rows) == 3
192-        assert rows[0]["account_number"] == "50100000000000"
193-        assert {r["particulars"] for r in rows}  # narration mapped to particulars
194-        assert dbmod.get_document_by_sha(conn, "h1")["status"] == "reconciled"
195-
196-
197:def test_parse_and_insert_records_unreconciled(tmp_path, monkeypatch):
198-    broken = _parse_sample()
199-    broken[2].closing_balance = 1.0  # break the balance walk
200-    monkeypatch.setattr(
201-        hdfc,
202-        "parse_statement",
203-        lambda *a, **k: hdfc.Statement(path=Path("x"), transactions=broken),
204-    )
205-    monkeypatch.setattr(hdfc, "parse_summary", lambda *a, **k: None)  # fall back to
reconcile()
206-    with dbmod.connect(tmp_path \ "t.db") as conn:
207-        dbmod.init_db(conn)
208-        doc_id = dbmod.insert_document(conn, source="file", sha256="h2", status="parsed")
209:        hdfc.HDFCParser().parse_and_insert(Path("x.pdf"), conn, doc_id)
210:        assert dbmod.get_document_by_sha(conn, "h2")["status"] == "unreconciled"
211-
212-
213-@pytest.mark.local_statements
214-@pytest.mark.parametrize(
215-    "pdf_path",
216-    sorted(Path("testdata/hdfc-statements").glob("*.pdf")),
--
238-    )
239-
240-    # End-to-end through the registered parser + DB, incl. the status it records.
241-    with dbmod.connect(tmp_path \ "t.db") as conn:
242-        dbmod.init_db(conn)
243-        doc_id = dbmod.insert_document(conn, source="file", sha256=pdf_path.name,
status="parsed")
244:        n = hdfc.HDFCParser().parse_and_insert(pdf_path, conn, doc_id, password=pw)
245-        assert n == len(stmt.transactions)
246-        assert dbmod.get_document_by_sha(conn, pdf_path.name)["status"] == "reconciled"
```

168. user (2026-06-10T17:23:05.922Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\hdfc.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

169. assistant (2026-06-10T17:22:53.427Z)

```
Implementing B3:
```

170. user (2026-06-10T17:22:55.084Z)

```
Switched to a new branch 'fix/hdfc-quarantine-parity'
```

171. assistant (2026-06-10T17:23:10.221Z)

```
Now the status-aware listings in db.py and the CLI display fixes:
```

172. user (2026-06-10T17:23:16.322Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\db.py has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

173. user (2026-06-10T17:23:19.614Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\db.py has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

174. user (2026-06-10T17:23:22.443Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\db.py has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

175. user (2026-06-10T17:23:26.203Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\cli.py has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

176. user (2026-06-10T17:23:30.561Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\cli.py has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

177. user (2026-06-10T17:23:46.976Z)

```
<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>
```

178. user (2026-06-10T17:23:52.531Z)

```
176     def test_parse_and_insert_records_reconciled(tmp_path, monkeypatch):
177         monkeypatch.setattr(hdfc, "parse_statement", lambda *a, **k: _sample_statement())
178         monkeypatch.setattr(
179             hdfc,
180             "parse_summary",
181             lambda *a, **k: hdfc.StatementSummary(
182                 opening_balance=10000.0, closing_balance=9999.50, debit_count=2,
credit_count=1
183             ),
184         )
185         with dbmod.connect(tmp_path / "t.db") as conn:
186             dbmod.init_db(conn)
187             doc_id = dbmod.insert_document(conn, source="file", sha256="h1",
status="parsed")
188             n = hdfc.HDFCParser().parse_and_insert(Path("x.pdf"), conn, doc_id)
189             assert n == 3
190             rows = dbmod.list_hdfc_transactions(conn, limit=100)
191             assert len(rows) == 3
192             assert rows[0]["account_number"] == "50100000000000"
193             assert {r["particulars"] for r in rows}  # narration mapped to particulars
194             assert dbmod.get_document_by_sha(conn, "h1")["status"] == "reconciled"
195
196
197     def test_parse_and_insert_records_unreconciled(tmp_path, monkeypatch):
198         broken = _parse_sample()
199         broken[2].closing_balance = 1.0  # break the balance walk
200         monkeypatch.setattr(
201             hdfc,
202             "parse_statement",
203             lambda *a, **k: hdfc.Statement(path=Path("x"), transactions=broken),
204         )
205         monkeypatch.setattr(hdfc, "parse_summary", lambda *a, **k: None)  # fall back to
reconcile()
206         with dbmod.connect(tmp_path / "t.db") as conn:
207             dbmod.init_db(conn)
208             doc_id = dbmod.insert_document(conn, source="file", sha256="h2",
status="parsed")
209             hdfc.HDFCParser().parse_and_insert(Path("x.pdf"), conn, doc_id)
```

```
210             assert dbmod.get_document_by_sha(conn, "h2")["status"] == "unreconciled"
211
212
213     @pytest.mark.local_statements
214     @pytest.mark.parametrize(
215         "pdf_path",
```

179. user (2026-06-10T17:24:05.545Z)

```
The file C:\Users\avidu\Projects\muneem\tests\parsers\test_hdfc.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

180. user (2026-06-10T17:24:12.088Z)

```
No matches found
```

181. user (2026-06-10T17:24:17.737Z)

```
Found 2 files
tests\parsers\test_hdfc.py
tests\test_serving_view.py
```

182. user (2026-06-10T17:24:22.246Z)

```
1       """The derived serving view (migration v6, DESIGN §8.7): one normalized, cross-bank
query surface
2       over the flat per-issuer source-of-truth tables ? column mapping, Dr/Cr derivation,
provenance
3       join, and the `txns list all` CLI. Fabricated data only.
4       """
5
6       from muneem import db as dbmod
7
8
9       def _seed(conn):
10          """One row in each per-issuer table, each under its own document (+ provenance for
one)."""
11          d_axis = dbmod.insert_document(conn, source="file", sha256="s-axis",
status="parsed")
12          d_au = dbmod.insert_document(conn, source="file", sha256="s-au", status="parsed")
13          d_hdfc = dbmod.insert_document(conn, source="file", sha256="s-hdfc",
status="parsed")
14          d_cc = dbmod.insert_document(conn, source="file", sha256="s-cc", status="parsed")
15          dbmod.insert_axis_transaction(
16              conn,
17              document_id=d_axis,
18              account_number="AX1",
19              txn_date="01/01/2026",
20              value_date="01/01/2026",
21              particulars="axis row",
22              debit=100.0,
23              credit=None,
24              balance=900.0,
25          )
26          dbmod.insert_au_transaction(
27              conn,
28              document_id=d_au,
29              account_number="AU1",
30              txn_date="02/01/2026",
31              value_date="02/01/2026",
32              particulars="au row",
33              debit=None,
34              credit=50.0,
35              balance=950.0,
```

```
        )
        dbmod.insert_hdfc_transaction(
            conn,
            document_id=d_hdfc,
            account_number="HD1",
            txn_date="03/01/2026",
            value_date="03/01/2026",
            particulars="hdfc row",
            ref="r1",
            withdrawal=25.0,
            deposit=None,
            closing_balance=925.0,
        )
        dbmod.insert_axis_cc_transaction(
            conn,
            document_id=d_cc,
            account_number="CC1",
            txn_date="04/01/2026",
            particulars="cc row",
            merchant_category="food",
            amount=75.0,
            type="Dr",
        )
        dbmod.record_parse(conn, d_hdfc, confidence="reconciled", parser="HDFCParser")
        return d_hdfc


    def test_view_unions_all_issuers_with_normalized_columns():
        with dbmod.connect(":memory:") as conn:
            dbmod.init_db(conn)
            _seed(conn)
            rows = {r["bank"]: r for r in conn.execute("SELECT * FROM v_transactions")}
            assert set(rows) == {"axis", "au", "hdfc", "axis_cc"}
            # HDFC's withdrawal/deposit/closing_balance land in debit/credit/balance.
            assert rows["hdfc"]["debit"] == 25.0 and rows["hdfc"]["credit"] is None
            assert rows["hdfc"]["balance"] == 925.0
            # Credit card derives debit/credit from the Dr/Cr type; no balance, no
value_date.
            assert rows["axis_cc"]["debit"] == 75.0 and rows["axis_cc"]["credit"] is None
            assert rows["axis_cc"]["balance"] is None and rows["axis_cc"]["value_date"] is
None
            # Plain savings rows pass through unchanged.
            assert rows["axis"]["debit"] == 100.0 and rows["au"]["credit"] == 50.0


    def test_view_joins_provenance_confidence():
        with dbmod.connect(":memory:") as conn:
            dbmod.init_db(conn)
            _seed(conn)
            rows = dbmod.list_all_transactions(conn)
            assert len(rows) == 4
            by_bank = {r["bank"]: r for r in rows}
            assert by_bank["hdfc"]["confidence"] == "reconciled"  # record_parse joined in
            assert by_bank["axis"]["confidence"] is None  # no provenance recorded for this
doc


    def test_view_is_read_only():
        import sqlite3

        import pytest

        with dbmod.connect(":memory:") as conn:
            dbmod.init_db(conn)
            with pytest.raises(sqlite3.OperationalError):  # views cannot be written to
```

```
98                 conn.execute("INSERT INTO v_transactions (bank) VALUES ('x')")
99
100
101     def test_cli_txns_list_all(tmp_path, monkeypatch):
102         import pytest
103
104         pytest.importorskip("sqlcipher3")
105         from typer.testing import CliRunner
106
107         from muneem.cli import app
108
109         led = tmp_path / "ledger.db"
110         with dbmod.connect_encrypted(led) as conn:
111             dbmod.init_db(conn)
112             _seed(conn)
113         result = CliRunner().invoke(app, ["txns", "list", "all", "--db", str(led)])
114         assert result.exit_code == 0, result.output
115         for tag in ("[axis:AX1]", "[au:AU1]", "[hdfc:HD1]", "[axis_cc:CC1]"):
116             assert tag in result.output
117         assert "[reconciled]" in result.output  # confidence surfaced for the hdfc row
118         assert "-25.0" in result.output  # hdfc withdrawal normalized to a debit
119
```

183. user (2026-06-10T17:24:42.429Z)

```
The file C:\Users\avidu\Projects\muneem\tests\test_serving_view.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

184. user (2026-06-10T17:24:53.995Z)

```
.................................................................... [ 68%]
.................................................................... [ 85%]
.....................................................               [100%]
419 passed, 21 deselected in 4.49s
```

185. user (2026-06-10T17:25:21.225Z)

```
All checks passed!
1 file reformatted, 78 files left unchanged
Success: no issues found in 33 source files
419 passed, 21 deselected in 4.26s
[fix/hdfc-quarantine-parity 10602f2] fix(parsers): HDFC quarantines unreconciled parses; txns
list surfaces parse status
 5 files changed, 115 insertions(+), 20 deletions(-)
 * [new branch]      fix/hdfc-quarantine-parity -> fix/hdfc-quarantine-parity
```

186. user (2026-06-10T17:25:45.031Z)

```
https://github.com/avidullu/muneem/pull/55
From https://github.com/avidullu/muneem
 * branch            main       -> FETCH_HEAD
   276c073..6804ab3  main       -> origin/main
Updating 276c073..6804ab3
Fast-forward
 src/muneem/cli.py          | 10 +++++---
 src/muneem/db.py           | 18 ++++++++++---
 src/muneem/parsers/hdfc.py | 36 ++++++++++++++++++++++++++++++++--------
 tests/parsers/test_hdfc.py |  8 ++++--
 tests/test_serving_view.py | 63 ++++++++++++++++++++++++++++++++++++++++++++++
 5 files changed, 115 insertions(+), 20 deletions(-)
Your branch is up to date with 'origin/main'.
Already on 'main'
Already up to date.
```

187. user (2026-06-10T17:26:01.566Z)

```
Updated task #6 status
```

188. user (2026-06-10T17:26:02.035Z)

```
Updated task #7 status
```

189. user (2026-06-10T17:26:03.802Z)

```
src\muneem\parsers\validation.py:84:    tol: float = 0.01,
src\muneem\parsers\validation.py:104:           if abs(expected - balance) > tol:
src\muneem\parsers\validation.py:122:    rows: Sequence[ConceptRow], opening: float, closing:
float, tol: float = 0.01
src\muneem\parsers\validation.py:132:    return abs((opening + delta) - closing) <= tol
src\muneem\parsers\validation.py:172:    rows: Sequence[ConceptRow], concepts:
StatementConcepts, tol: float = 0.01
src\muneem\parsers\validation.py:188:        closing_ok = abs(rows[-1].balance -
concepts.closing_balance) <= tol
src\muneem\parsers\validation.py:191:        opening_ok = abs(implied_opening -
concepts.opening_balance) <= tol
src\muneem\parsers\validation.py:202:def select_reconciling(candidates:
Sequence[Sequence[ConceptRow]], tol: float = 0.01) -> int | None:
src\muneem\parsers\clustering.py:35:        if lines and anchor_top is not None and abs(w["top"]
- anchor_top) <= tol:
src\muneem\parsers\hdfc.py:286:def reconcile(statement: Statement, tol: float = 0.01) ->
ReconcileReport:
src\muneem\parsers\hdfc.py:354:    statement: Statement, summary: StatementSummary, tol: float =
0.01
src\muneem\parsers\savings.py:83:def reconcile_oracle(opening: float | None, closing: float |
None, tol: float = 0.01):
src\muneem\parsers\savings.py:105:                if abs(implied - opening) > tol:
src\muneem\parsers\savings.py:108:                if abs(last.balance - closing) > tol:
src\muneem\parsers\savings.py:137:    matrix: list[list[str]], profile: Profile, opening: float
| None, tol: float = 0.01
```

190. user (2026-06-10T17:26:23.259Z)

```
Switched to a new branch 'fix/reconcile-tolerance'
```

191. user (2026-06-10T17:26:23.277Z)

```
1       """Property-style fuzzing of the reconciliation verifier ? deterministic (seeded), no
new dep.
2
3       The verifier is muneem's truth oracle: an off-by-one in the balance walk or the total
identity
4       would silently let mislabeled rows through. These tests generate many random
*consistent* ledgers
5       and assert the oracles accept them, then perturb a single value past the tolerance and
assert the
6       oracles catch it. A one-line slip in the reconcile arithmetic fails here.
7       """
8
9       import random
10
11      import pytest
12
13      from muneem.parsers.concepts import ConceptRow
14      from muneem.parsers.validation import reconcile_running_balance, reconcile_total
15
16
17      def _consistent_ledger(rng: random.Random, n: int, opening: float):
18          """Build a random clean ledger of ``n`` rows; return (rows, closing_balance)."""
```

```
        rows: list[ConceptRow] = []
        bal = opening
        for _ in range(n):
            amt = round(rng.uniform(0.01, 5000.0), 2)
            if rng.random() < 0.5:
                bal = round(bal - amt, 2)
                rows.append(ConceptRow(txn_date="01/01/2026", narration="x", debit=amt,
balance=bal))
            else:
                bal = round(bal + amt, 2)
                rows.append(ConceptRow(txn_date="01/01/2026", narration="x", credit=amt,
balance=bal))
        return rows, bal


    def _entries(rows):
        return [(r.debit, r.credit, r.balance) for r in rows]


    @pytest.mark.parametrize("seed", range(60))
    def test_consistent_ledger_always_reconciles(seed):
        rng = random.Random(seed)
        opening = round(rng.uniform(0, 100_000), 2)
        rows, closing = _consistent_ledger(rng, rng.randint(2, 40), opening)
        assert reconcile_running_balance(_entries(rows)).ok
        assert reconcile_total(rows, opening=opening, closing=closing)


    @pytest.mark.parametrize("seed", range(60))
    def test_single_balance_perturbation_is_caught_by_the_walk(seed):
        rng = random.Random(10_000 + seed)
        opening = round(rng.uniform(0, 100_000), 2)
        rows, closing = _consistent_ledger(rng, rng.randint(2, 40), opening)
        i = rng.randrange(len(rows))
        r = rows[i]
        rows[i] = ConceptRow(  # corrupt one balance by more than the tolerance
            txn_date=r.txn_date,
            narration=r.narration,
            debit=r.debit,
            credit=r.credit,
            balance=round((r.balance or 0.0) + 1.0, 2),
        )
        assert not reconcile_running_balance(_entries(rows)).ok


    @pytest.mark.parametrize("seed", range(60))
    def test_total_oracle_catches_a_wrong_closing_bookend(seed):
        rng = random.Random(20_000 + seed)
        opening = round(rng.uniform(0, 100_000), 2)
        rows, closing = _consistent_ledger(rng, rng.randint(2, 40), opening)
        assert reconcile_total(rows, opening=opening, closing=closing)
        assert not reconcile_total(rows, opening=opening, closing=round(closing + 1.0, 2))

```

192. assistant (2026-06-10T17:26:42.889Z)

Property tests perturb by 1.0, so they're compatible. Implementing the half-quantum tolerance:

193. user (2026-06-10T17:26:47.214Z)

The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\validation.py has been updated successfully. (file state is current in your context ? no need to Read it back)

194. user (2026-06-10T17:26:49.569Z)

```
 src/muneem/parsers/validation.py | 15 ++++++++++----
 1 file changed, 11 insertions(+), 4 deletions(-)
```

195. user (2026-06-10T17:27:00.042Z)

```
 src/muneem/parsers/hdfc.py       |  4 ++--
 src/muneem/parsers/savings.py    | 11 ++++++++---
 src/muneem/parsers/validation.py | 15 ++++++++++----
 3 files changed, 21 insertions(+), 9 deletions(-)
44: from .validation import (
286: def reconcile(statement: Statement, tol: float = BALANCE_TOLERANCE) -> ReconcileReport:
354: statement: Statement, summary: StatementSummary, tol: float = BALANCE_TOLERANCE
```

196. user (2026-06-10T17:27:06.527Z)

```
40      from . import register_parser
41      from .base import StatementParser
42      from .concepts import ConceptRow, StatementConcepts
43      from .profiles import HDFC_PROFILE
44      from .validation import (
45          BookendReport,
46          Confidence,
47          ReconcileReport,
48          reconcile_running_balance,
49          verify,
50      )
51
52      # --- Column geometry (PDF points; A4-portrait HDFC detailed statement) ---------
53      # Left-aligned columns key on x0; right-aligned amount columns key on their right
```

197. user (2026-06-10T17:27:14.393Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\hdfc.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

198. user (2026-06-10T17:27:15.807Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\validation.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

199. user (2026-06-10T17:27:25.432Z)

```
The file C:\Users\avidu\Projects\muneem\tests\test_verifier_properties.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

200. user (2026-06-10T17:27:39.871Z)

```
FAILED tests/test_verifier_properties.py::test_exact_one_paisa_corruption_is_always_caught[37]
FAILED tests/test_verifier_properties.py::test_exact_one_paisa_corruption_is_always_caught[49]
4 failed, 476 passed, 21 deselected in 4.71s
Found 1 error.
1 file reformatted, 78 files left unchanged
Success: no issues found in 33 source files
```

201. user (2026-06-10T17:28:00.086Z)

```
The file C:\Users\avidu\Projects\muneem\tests\test_verifier_properties.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

202. user (2026-06-10T17:28:15.262Z)

```
.............................................                            [100%]
480 passed, 21 deselected in 4.24s
```

```
All checks passed!
79 files already formatted
```

203. user (2026-06-10T17:28:49.560Z)

```
[fix/reconcile-tolerance f1a9e9b] fix(parsers): reconcile tolerance at half the currency quantum
 4 files changed, 57 insertions(+), 9 deletions(-)
 * [new branch]      fix/reconcile-tolerance -> fix/reconcile-tolerance
https://github.com/avidullu/muneem/pull/56
From https://github.com/avidullu/muneem
 * branch            main       -> FETCH_HEAD
   6804ab3..4554e12  main       -> origin/main
Updating 6804ab3..4554e12
Fast-forward
 src/muneem/parsers/hdfc.py        |  5 +++--
 src/muneem/parsers/savings.py     | 11 ++++++++---
 src/muneem/parsers/validation.py  | 18 ++++++++++++++----
 tests/test_verifier_properties.py | 32 ++++++++++++++++++++++++++++++++
 4 files changed, 57 insertions(+), 9 deletions(-)
Your branch is up to date with 'origin/main'.
Already on 'main'
Already up to date.
```

204. user (2026-06-10T17:28:57.130Z)

```
Updated task #7 status
```

205. user (2026-06-10T17:28:57.605Z)

```
Updated task #8 status
```

206. user (2026-06-10T17:28:58.549Z)

```
1       from __future__ import annotations
2
3       from .base import StatementParser, safe_amount
4
5       _REGISTRY: list[type[StatementParser]] = []
6
7
8       def register_parser(parser_cls: type[StatementParser]) -> type[StatementParser]:
9           """Decorator to register a parser."""
10          _REGISTRY.append(parser_cls)
11          return parser_cls
12
13
14      def get_parser(
15          text: str, sender: str | None = None, subject: str | None = None
16      ) -> StatementParser | None:
17          """Return an instantiated parser that can handle this document, or None."""
18          for parser_cls in _REGISTRY:
19              parser = parser_cls()
20              if parser.can_handle(text, sender, subject):
21                  return parser
22          return None
23
24
25      def _autoload_parsers() -> None:
26          """Import every parser submodule so its ``@register_parser`` runs.
27
28          Registration is an import *side-effect*, so the registry is only populated for
29          modules that have actually been imported. Without this, the production path
30          (``from muneem.parsers import get_parser``, as cli.py uses) would leave
31          ``_REGISTRY`` EMPTY ? only the tests imported the submodules. Auto-discovery
```

```
        fixes that and means new parser modules register themselves with no edit here.
        """
        import importlib
        import pkgutil

        for info in pkgutil.iter_modules(__path__):
            if info.name != "base" and not info.name.startswith("_"):
                importlib.import_module(f"{__name__}.{info.name}")


    _autoload_parsers()

    __all__ = ["StatementParser", "register_parser", "get_parser", "safe_amount"]

```

207. user (2026-06-10T17:28:58.729Z)

```
    """AU Small Finance Bank savings / transaction-account statement parser.

    Runs on the **shared engine-driven path** (:mod:`muneem.parsers.savings`): per-table +
    coordinate-clustered candidates ? engine mapping ? reconciliation oracle, **oracle-gated** so a
    vintage it can't read stores nothing rather than mislabeled rows. The AU specifics are entirely in
    ``AUBANK_PROFILE`` ? this parser is mostly the DB wiring. No statement content is logged here.
    """

    from __future__ import annotations

    import sqlite3
    from pathlib import Path

    from muneem import __version__
    from muneem import db as dbmod
    from muneem.errors import ParseError
    from muneem.redaction import redact_filename

    from . import register_parser
    from .base import StatementParser
    from .profiles import AUBANK_PROFILE
    from .savings import SavingsParse, parse_savings
    from .validation import Confidence


    def parse_au_savings(pdf_path: Path, password: str | None = None) -> SavingsParse:
        """Parse an AU Bank savings statement via the shared engine-driven path."""
        return parse_savings(pdf_path, AUBANK_PROFILE, password)


    @register_parser
    class AUParser(StatementParser):
        def can_handle(self, text: str, sender: str | None = None, subject: str | None = None) -> bool:
            if sender and "aubank" in sender.lower():
                return True
            low = (text or "").lower()
            return "au small finance bank" in low or "aubank" in low

        def parse_and_insert(
            self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str | None = None
        ) -> int:
            """Persist a savings parse **only if it reconciles**; otherwise store nothing."""
```

```
            try:
                parsed = parse_au_savings(pdf_path, password)
            except ParseError:
                raise
            except Exception as exc:
                raise ParseError(f"failed to parse AU statement
{redact_filename(pdf_path)}") from exc

            if not parsed.verified:
                dbmod.record_parse(
                    conn,
                    doc_id,
                    confidence=Confidence.UNRECONCILED.value,
                    parser="AUParser",
                    parser_version=__version__,
                )
                return 0

            for r in parsed.rows:
                dbmod.insert_au_transaction(
                    conn,
                    document_id=doc_id,
                    account_number=parsed.account_number,
                    txn_date=r.txn_date,
                    value_date=r.value_date or "",
                    particulars=r.narration,
                    debit=r.debit,
                    credit=r.credit,
                    balance=r.balance,
                )
            dbmod.record_parse(
                conn,
                doc_id,
                confidence=Confidence.RECONCILED.value,
                parser="AUParser",
                parser_version=__version__,
            )
            return len(parsed.rows)

```

208. user (2026-06-10T17:29:50.665Z)

```
    """Regression test for the parser registry (the PR #9-11 production-path bug).

    `from muneem.parsers import get_parser` must return a working selector ? i.e. the
    registry must be populated by package import alone, without anything importing the
    axis/hdfc/... submodules first. We check in a FRESH interpreter (subprocess) so a
    prior test importing a submodule can't mask a regression.
    """

    from __future__ import annotations

    import subprocess
    import sys


    def test_registry_is_populated_by_package_import_alone():
        code = (
            "from muneem.parsers import get_parser, _REGISTRY\n"
            "assert len(_REGISTRY) >= 2, f'registry empty: {_REGISTRY}'\n"
            "assert get_parser('Axis Bank Statement of Account') is not None\n"
            "assert get_parser('HDFC Bank savings account details') is not None\n"
            "assert get_parser('a wholly unrelated document') is None\n"
            "print('REGISTRY_OK', len(_REGISTRY))\n"
        )
```

```
24          result = subprocess.run([sys.executable, "-c", code], capture_output=True,
text=True)
25          assert result.returncode == 0, f"fresh-interpreter import failed:\n{result.stderr}"
26          assert "REGISTRY_OK" in result.stdout
27
28
29      def test_get_parser_selects_by_sender():
30          from muneem.parsers import get_parser
31
32          assert type(get_parser("", sender="alerts@axisbank.com")).__name__ == "AxisParser"
33
```

209. user (2026-06-10T17:29:52.119Z)

```
12:the bands the HDFC clusterer / engine expect.
```

210. user (2026-06-10T17:30:18.964Z)

```
4:``extract`` (pdfminer) ? ``get_parser`` ? ``parse_and_insert`` ? assert a **reconciled**
parse. This
14:from muneem.parsers import get_parser
59:def _au_savings_pdf(path: Path) -> Path:
60:    """Savings layout (the shared savings.py engine path used by Axis/AU): a clean
64:        at(50, 60, "AU Small Finance Bank"),
93:    parser = get_parser(ex.text)
111:    name, n, status = _run_pipeline(_au_savings_pdf(tmp_path / "au.pdf"))
112:    assert name == "AUParser"
```

211. user (2026-06-10T17:30:20.221Z)

```
15-from muneem.parsers.savings import SavingsParse
16-
17-
18:def test_can_handle():
19:    assert AUParser().can_handle("...customercare@aubank.in...")
20:    assert AUParser().can_handle("AU Small Finance Bank Statement")
21:    assert AUParser().can_handle("anything", sender="statements@aubank.in")
22:    assert not AUParser().can_handle("HDFC Bank savings account details")
23:    assert not AUParser().can_handle("Axis Bank Statement of Account")
24-
25-
26-def test_parse_au_savings_per_row_reconcile():
```

212. user (2026-06-10T17:30:45.326Z)

```
Switched to a new branch 'fix/routing-anchored-evidence'
```

213. user (2026-06-10T17:30:47.944Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\au.py has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

214. user (2026-06-10T17:30:52.781Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\hdfc.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

215. user (2026-06-10T17:31:07.167Z)

```
The file C:\Users\avidu\Projects\muneem\tests\parsers\test_registry.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

216. user (2026-06-10T17:31:21.467Z)

```
.............................................................. [ 89%]
.....................................................          [100%]
482 passed, 21 deselected in 5.55s
```

217. user (2026-06-10T17:32:00.535Z)

```
All checks passed!
1 file reformatted, 78 files left unchanged
Success: no issues found in 33 source files
[fix/routing-anchored-evidence ea5dc90] fix(parsers): routing evidence must be letterhead-
shaped, not narration tokens
 3 files changed, 43 insertions(+), 2 deletions(-)
 * [new branch]      fix/routing-anchored-evidence -> fix/routing-anchored-evidence
https://github.com/avidullu/muneem/pull/57
From https://github.com/avidullu/muneem
 * branch            main       -> FETCH_HEAD
   4554e12..9040f91  main       -> origin/main
Updating 4554e12..9040f91
Fast-forward
 src/muneem/parsers/au.py       |  7 ++++++-
 src/muneem/parsers/hdfc.py     |  8 +++++++-
 tests/parsers/test_registry.py | 30 ++++++++++++++++++++++++++++++
 3 files changed, 43 insertions(+), 2 deletions(-)
Your branch is up to date with 'origin/main'.
Already on 'main'
Already up to date.
```

218. user (2026-06-10T17:32:06.933Z)

```
Updated task #8 status
```

219. user (2026-06-10T17:32:07.422Z)

```
Updated task #9 status
```

220. user (2026-06-10T17:32:08.294Z)

```
90
91
92      def _ingest_bank_statement(pdf: Path, db_path: Path, doc_type: str, *, interactive:
bool) -> None:
93          """Unlock (if encrypted) ? extract ? select parser ? store reconciled transactions.
94
95          The hybrid password unlock (cached/configured candidates ? prompt) runs *before*
extraction,
96          since the text needed to pick a parser is itself encrypted. The resolved password is
forwarded
97          to extraction and the parser, used in-memory only ? never logged.
98          """
99          from muneem import db as dbmod
100         from muneem.extract import extract
101         from muneem.parsers import get_parser
102         from muneem.statement_unlock import resolve_password
103         from muneem.unlock import is_encrypted
104
105         password: str | None = ""
106         if is_encrypted(pdf):
107             password = resolve_password(pdf, interactive=interactive)
108             if password is None:
109                 hint = " (run without --no-prompt to enter it)" if not interactive else ""
110                 typer.echo(f"Could not unlock {pdf.name}: no working password{hint}.",
err=True)
111                 raise typer.Exit(1)
112
```

```
        ex = extract(pdf, password=password or "")  # resolved above; `or ""` keeps the type
str
        parser = get_parser(ex.text)
        if parser is None:
            typer.echo(f"No parser recognizes {pdf.name} as a supported bank statement.",
err=True)
            raise typer.Exit(1)

        with _ledger(db_path) as conn:
            dbmod.init_db(conn)
            existing = dbmod.get_document_by_sha(conn, ex.sha256)
            if existing is not None:
                typer.echo(f"already ingested (doc {existing['id']}): {pdf.name}")
                return
            doc_id = dbmod.insert_document(
                conn,
                source="file",
                external_id=pdf.name,
                doc_type=doc_type,
                sha256=ex.sha256,
                path=str(pdf.resolve()),
                status="parsed",
            )
            count = parser.parse_and_insert(pdf, conn, doc_id, password=password)
            doc = dbmod.get_document_by_sha(conn, ex.sha256)
            status = doc["status"] if doc else "parsed"

        typer.echo(
            f"ingested {pdf.name} (doc {doc_id}); parsed {count} transaction(s) "
            f"via {parser.__class__.__name__} [{status}]"
        )


    def _eval_statement(pdf: Path, *, interactive: bool) -> tuple[str, int, str | None]:
        """Run one statement through the full parse pipeline into an in-memory DB.

        Returns ``(outcome, row_count, parser_name)`` where ``outcome`` is the document
status
        (``reconciled`` / ``unreconciled``) or a failure tag (``locked`` / ``no-parser`` /
``error``).
        Nothing is persisted and no statement value is surfaced ? only the verdict and
counts.
        """
        from muneem import db as dbmod
        from muneem.extract import extract
        from muneem.parsers import get_parser
        from muneem.statement_unlock import resolve_password
        from muneem.unlock import is_encrypted

        password = ""
        if is_encrypted(pdf):
            password = resolve_password(pdf, interactive=interactive) or ""
            if not password:
                return ("locked", 0, None)
        try:
            ex = extract(pdf, password=password)
            parser = get_parser(ex.text)
        except Exception as exc:  # eval must not crash on one bad file; record the redacted
reason
            _log.warning(
                "eval: extract/route failed for %s (%s)", redact_filename(pdf),
type(exc).__name__
            )
            return ("error", 0, None)
        if parser is None:
```

```
171             return ("no-parser", 0, None)
172         name = parser.__class__.__name__
173         try:
174             with dbmod.connect(":memory:") as conn:
175                 dbmod.init_db(conn)
176                 doc_id = dbmod.insert_document(
177                     conn, source="file", external_id=pdf.name, sha256=ex.sha256,
status="parsed"
178                 )
179                 count = parser.parse_and_insert(pdf, conn, doc_id, password=password)
180                 doc = dbmod.get_document_by_sha(conn, ex.sha256)
181                 status = doc["status"] if doc else "parsed"
182         except Exception as exc:
183             _log.warning(
184                 "eval: parse/store failed for %s (%s)", redact_filename(pdf),
type(exc).__name__
185             )
186             return ("error", 0, name)
187         return (status, count, name)
188
189
190     @app.command("eval")
191     def eval_statements(
192         folder: Path = typer.Argument(
193             ...,
194             exists=True,
195             file_okay=False,
196             readable=True,
197             help="A LOCAL folder of statement PDFs to evaluate (e.g. your seed or golden
set).",
198         ),
199         prompt: bool = typer.Option(
```

221. user (2026-06-10T17:32:30.442Z)

```
tests\test_cli.py:24:def test_ingest_bank_statement_unlocks_and_passes_password(tmp_path,
monkeypatch):
tests\test_cli.py:49:        app, ["ingest", str(pdf), "--doc-type", "bank_statement", "--db",
str(tmp_path / "t.db")]
tests\test_cli.py:84:def test_ingest_bank_statement_unlock_failure_exits_nonzero(tmp_path,
monkeypatch):
tests\test_cli.py:96:            "--doc-type",
```

222. user (2026-06-10T17:32:36.393Z)

```
1       from pathlib import Path
2
3       from typer.testing import CliRunner
4
5       from muneem.cli import app
6       from muneem.extract import Extracted
7
8       runner = CliRunner()
9
10
11      def test_version_flag():
12          result = runner.invoke(app, ["--version"])
13          assert result.exit_code == 0
14          assert "muneem" in result.stdout
15
16
17      def test_info_lists_paths():
18          result = runner.invoke(app, ["info"])
19          assert result.exit_code == 0
20          assert "config dir" in result.stdout
```

```
        assert "staging" in result.stdout


    def test_ingest_bank_statement_unlocks_and_passes_password(tmp_path, monkeypatch):
        """The bank path resolves the password and forwards it to extract + the parser."""
        pdf = tmp_path / "stmt.pdf"
        pdf.write_bytes(b"%PDF-1.4 fake")
        captured = {}

        monkeypatch.setattr("muneem.unlock.is_encrypted", lambda p: True)
        monkeypatch.setattr(
            "muneem.statement_unlock.resolve_password", lambda p, interactive=True: "PW123"
        )

        def fake_extract(p, password=""):
            captured["extract_pw"] = password
            return Extracted(path=Path(p), sha256="sha-x", text="Axis Bank Account
Statement")

        monkeypatch.setattr("muneem.extract.extract", fake_extract)

        class FakeParser:
            def parse_and_insert(self, pdf_path, conn, doc_id, password=None):
                captured["parser_pw"] = password
                return 3

        monkeypatch.setattr("muneem.parsers.get_parser", lambda text: FakeParser())

        result = runner.invoke(
            app, ["ingest", str(pdf), "--doc-type", "bank_statement", "--db", str(tmp_path /
"t.db")]
        )
        assert result.exit_code == 0, result.output
        assert captured["extract_pw"] == "PW123" and captured["parser_pw"] == "PW123"
        assert "ingested" in result.output and "FakeParser" in result.output


    def test_db_status_on_missing_ledger(tmp_path):
        result = runner.invoke(app, ["db", "status", "--db", str(tmp_path / "nope.db")])
        assert result.exit_code == 0, result.output
        assert "does not exist yet" in result.output
        assert "not yet minted" in result.output


    def test_db_encrypt_migrates_and_status_reflects_it(tmp_path):
        import sqlite3

        p = tmp_path / "ledger.db"
        con = sqlite3.connect(p)
        con.execute("CREATE TABLE t (id INTEGER PRIMARY KEY, amount REAL)")
        con.execute("INSERT INTO t (amount) VALUES (10.0)")
        con.commit()
        con.close()

        before = runner.invoke(app, ["db", "status", "--db", str(p)])
        assert before.exit_code == 0 and "PLAINTEXT" in before.output

        enc = runner.invoke(app, ["db", "encrypt", "--db", str(p)])
        assert enc.exit_code == 0, enc.output
        assert "migrated" in enc.output and "plaintext.bak" in enc.output

        after = runner.invoke(app, ["db", "status", "--db", str(p)])
        assert after.exit_code == 0 and "encrypted" in after.output


```

```
84      def test_ingest_bank_statement_unlock_failure_exits_nonzero(tmp_path, monkeypatch):
85          pdf = tmp_path / "stmt.pdf"
86          pdf.write_bytes(b"%PDF-1.4 fake")
87          monkeypatch.setattr("muneem.unlock.is_encrypted", lambda p: True)
88          monkeypatch.setattr(
89              "muneem.statement_unlock.resolve_password", lambda p, interactive=True: None
90          )
91          result = runner.invoke(
92              app,
93              [
94                  "ingest",
95                  str(pdf),
96                  "--doc-type",
97                  "bank_statement",
98                  "--db",
99                  str(tmp_path / "t.db"),
100                 "--no-prompt",
101             ],
102         )
103         assert result.exit_code == 1
104         assert "Could not unlock" in result.output
105
```

223. user (2026-06-10T17:33:02.947Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\cli.py has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

224. assistant (2026-06-10T17:32:49.049Z)

```
Implementing B6:
```

225. user (2026-06-10T17:32:51.393Z)

```
Switched to a new branch 'fix/status-aware-dedup'
```

226. user (2026-06-10T17:32:55.706Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\db.py has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

227. user (2026-06-10T17:33:19.451Z)

```
The file C:\Users\avidu\Projects\muneem\tests\test_cli.py has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

228. user (2026-06-10T17:33:32.697Z)

```
................................................                     [100%]
483 passed, 21 deselected in 4.44s
All checks passed!
79 files left unchanged
Success: no issues found in 33 source files
```

229. user (2026-06-10T17:34:08.226Z)

```
[fix/status-aware-dedup d81e3fd] fix(cli): sha256 dedup is status-aware -- a quarantined parse
gets a second chance
 3 files changed, 103 insertions(+), 10 deletions(-)
 * [new branch]      fix/status-aware-dedup -> fix/status-aware-dedup
https://github.com/avidullu/muneem/pull/58
From https://github.com/avidullu/muneem
 * branch            main       -> FETCH_HEAD
   9040f91..73d77ff  main       -> origin/main
```

```
Updating 9040f91..73d77ff
Fast-forward
 src/muneem/cli.py | 30 +++++++++++++++++---------
 src/muneem/db.py  | 17 ++++++++++++++
 tests/test_cli.py | 66 ++++++++++++++++++++++++++++++++++++++++++++++++++++++
 3 files changed, 103 insertions(+), 10 deletions(-)
Your branch is up to date with 'origin/main'.
Already on 'main'
Already up to date.
```

230. user (2026-06-10T17:34:17.377Z)

```
Updated task #9 status
```

231. user (2026-06-10T17:34:17.853Z)

```
Updated task #10 status
```

232. user (2026-06-10T17:34:18.448Z)

```
115     # --- Credit card (no running balance; count only)
-------------------------------------------
116
117
118     @dataclass
119     class AxisCCTxn:
120         txn_date: str
121         particulars: str
122         merchant_category: str
123         amount: float
124         type: str  # "Cr" or "Dr"
125
126
127     def _rows_from_cc_tables(tables: list) -> list[AxisCCTxn]:
128         """Build credit-card transactions from extracted tables. Pure: no PDF, no DB."""
129         rows: list[AxisCCTxn] = []
130         for table in tables or []:
131             for row in table or []:
132                 if not row or len(row) < 9:
133                     continue
134                 date_str = str(row[0]).strip() if row[0] else ""
135                 if not _CC_DATE_RE.match(date_str):
136                     continue
137                 amount_str = str(row[8]).strip() if row[8] else ""
138                 is_cr = "Cr" in amount_str
139                 amount = parse_amount(re.sub(r"[^\d.]", "", amount_str)) or 0.0
140                 rows.append(
141                     AxisCCTxn(
142                         txn_date=date_str,
143                         particulars=str(row[2]).strip() if row[2] else "",
144                         merchant_category=str(row[7]).strip() if row[7] else "",
145                         amount=amount,
146                         type="Cr" if is_cr else "Dr",
147                     )
148                 )
149         return rows
150
151
152     def parse_axis_cc(pdf_path: Path, password: str | None = None) -> tuple[str,
list[AxisCCTxn]]:
153         """Open an Axis credit-card statement and return (card_number, transactions)."""
154         card_number = "UNKNOWN"
155         rows: list[AxisCCTxn] = []
156         with pdfplumber.open(pdf_path, password=password) as pdf:
```

```
157             head = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(3,
len(pdf.pages))))
158             card_number = _first_match(head, _CARD_RES) or "UNKNOWN"
159             for page in pdf.pages:
160                 rows.extend(_rows_from_cc_tables(page.extract_tables()))
161         return card_number, rows
162
163
164     @register_parser
165     class AxisCreditCardParser(StatementParser):
166         def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
167             return "Axis Bank" in text and "Credit Card Statement" in text
168
169         def parse_and_insert(
170             self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
171         ) -> int:
172             try:
173                 card_number, rows = parse_axis_cc(pdf_path, password)
174             except ParseError:
175                 raise
176             except Exception as exc:
177                 raise ParseError(
178                     f"failed to parse Axis CC statement {redact_filename(pdf_path)}"
179                 ) from exc
180             for t in rows:
181                 dbmod.insert_axis_cc_transaction(
182                     conn,
183                     document_id=doc_id,
184                     account_number=card_number,
185                     txn_date=t.txn_date,
186                     particulars=t.particulars,
187                     merchant_category=t.merchant_category,
188                     amount=t.amount,
189                     type=t.type,
190                 )
191             # No running balance to reconcile; documents.status stays as the caller set it.
We still
192             # record provenance (with no confidence verdict).
193             dbmod.record_provenance(
194                 conn,
195                 document_id=doc_id,
196                 source_type="pdf_statement",
197                 parser="AxisCreditCardParser",
198                 parser_version=__version__,
199                 mapping_layer="deterministic",
200             )
201             return len(rows)
202
```

233. user (2026-06-10T17:34:52.457Z)

```
15-from muneem.parsers.concepts import ConceptRow
16-
17-
18-def test_can_handle():
19-    assert AxisParser().can_handle("x", sender="alerts@axisbank.com")
20:    assert AxisParser().can_handle("Axis Bank Statement of Account")
21:    assert AxisParser().can_handle("Axis Bank Account Statement")  # the real wording on
statements
22-    assert not AxisParser().can_handle("HDFC Bank")
23-    assert AxisCreditCardParser().can_handle("Axis Bank Credit Card Statement")
24:    assert not AxisCreditCardParser().can_handle("Axis Bank Statement of Account")
25-
```

```
26-
27-def test_parse_axis_summary_reads_opening_closing():
28-    s = parse_axis_summary("Opening Balance 1,000.00\n...rows...\nClosing Balance 2,500.50")
29-    assert s.opening_balance == 1000.0 and s.closing_balance == 2500.5
--
83-    assert not r.verified
84-
85-
86-def test_parse_and_insert_stores_when_verified(tmp_path, monkeypatch):
87-    rows = [ConceptRow(txn_date="01-02-2026", narration="UPI", debit=500.0, balance=9500.0)]
88:    monkeypatch.setattr(axis, "parse_axis_savings", lambda *a, **k: AxisSavings("ACC1", rows,
True))
89-    with dbmod.connect(tmp_path \ "t.db") as conn:
90-        dbmod.init_db(conn)
91-        doc_id = dbmod.insert_document(conn, source="file", sha256="ax1", status="parsed")
92-        n = AxisParser().parse_and_insert(Path("x.pdf"), conn, doc_id)
93-        assert n == 1
94-        stored = dbmod.list_axis_transactions(conn)
95-        assert (
96-            len(stored) == 1
97:            and stored[0]["account_number"] == "ACC1"
98-            and stored[0]["debit"] == 500.0
99-        )
100-        assert dbmod.get_document_by_sha(conn, "ax1")["status"] == "reconciled"
101-
102-
103-def test_parse_and_insert_refuses_when_unverified(tmp_path, monkeypatch):
104-    rows = [ConceptRow(txn_date="01-02-2026", narration="UPI", debit=500.0, balance=1.0)]
105-    monkeypatch.setattr(
106:        axis, "parse_axis_savings", lambda *a, **k: AxisSavings("ACC1", rows, False)
107-    )
108-    with dbmod.connect(tmp_path \ "t.db") as conn:
109-        dbmod.init_db(conn)
110-        doc_id = dbmod.insert_document(conn, source="file", sha256="ax2", status="parsed")
111-        n = AxisParser().parse_and_insert(Path("x.pdf"), conn, doc_id)
112-        assert n == 0
113-        assert dbmod.list_axis_transactions(conn) == []
114-        assert dbmod.get_document_by_sha(conn, "ax2")["status"] == "unreconciled"
115-
116-
117:def test_rows_from_cc_tables():
118-    tables = [
119-        [
120-            ["Date", "c1", "Particulars", "c3", "c4", "c5", "c6", "Category", "Amount"],
121-            ["05/02/2026", "x", "Swiggy", "", "", "", "", "Food", "250.00 Dr"],
122-            ["06/02/2026", "x", "Refund", "", "", "", "", "Food", "100.00 Cr"],
123-            ["bad", "too", "short"],  # < 9 cols -> skipped
124-        ]
125-    ]
126:    rows = axis._rows_from_cc_tables(tables)
127-    assert len(rows) == 2
128-    assert rows[0].amount == 250.0 and rows[0].type == "Dr" and rows[0].merchant_category ==
"Food"
129-    assert rows[1].amount == 100.0 and rows[1].type == "Cr"
```

234. user (2026-06-10T17:35:22.260Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\axis.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

235. assistant (2026-06-10T17:35:07.607Z)

```
Implementing B7:
```

236. user (2026-06-10T17:35:09.369Z)

```
Switched to a new branch 'fix/axis-cc-verification-floor'
```

237. user (2026-06-10T17:35:13.842Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\validation.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

238. user (2026-06-10T17:35:34.522Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\axis.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

239. user (2026-06-10T17:35:44.823Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\axis.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

240. user (2026-06-10T17:36:03.288Z)

```
<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>
```

241. user (2026-06-10T17:36:09.311Z)

```
1       """Tests for the Axis parsers (savings via the concept core + credit card).
2
3       Savings runs end-to-end on synthetic data through a pdfplumber mock (extract_tables /
extract_text /
4       extract_words), so the per-table ? engine ? reconcile path is exercised without a real
PDF;
5       ``parse_and_insert``'s store-iff-reconciled gate is checked by monkeypatching that
function.
6       Real encrypted statements run under the ``local_statements`` marker
(test_local_axis.py).
7       """
8
9       from pathlib import Path
10      from unittest.mock import MagicMock, patch
11
12      import muneem.db as dbmod
13      from muneem.parsers import axis
14      from muneem.parsers.axis import AxisCreditCardParser, AxisParser, AxisSavings,
parse_axis_summary
15      from muneem.parsers.concepts import ConceptRow
16
17
18      def test_can_handle():
19          assert AxisParser().can_handle("x", sender="alerts@axisbank.com")
20          assert AxisParser().can_handle("Axis Bank Statement of Account")
21          assert AxisParser().can_handle("Axis Bank Account Statement")  # the real wording on
statements
22          assert not AxisParser().can_handle("HDFC Bank")
23          assert AxisCreditCardParser().can_handle("Axis Bank Credit Card Statement")
24          assert not AxisCreditCardParser().can_handle("Axis Bank Statement of Account")
25
26
27      def test_parse_axis_summary_reads_opening_closing():
28          s = parse_axis_summary("Opening Balance 1,000.00\n...rows...\nClosing Balance
2,500.50")
29          assert s.opening_balance == 1000.0 and s.closing_balance == 2500.5
30
```

242. user (2026-06-10T17:36:28.417Z)

```
The file C:\Users\avidu\Projects\muneem\tests\parsers\test_axis.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

243. user (2026-06-10T17:36:42.097Z)

```
========================== short test summary info ==========================
FAILED tests/test_provenance.py::test_credit_card_parser_records_provenance_without_confidence
1 failed, 485 passed, 21 deselected in 4.66s
All checks passed!
79 files left unchanged
Success: no issues found in 33 source files
```

244. user (2026-06-10T17:36:48.582Z)

```
85:def test_credit_card_parser_records_provenance_without_confidence(monkeypatch):
86-    monkeypatch.setattr(axis, "parse_axis_cc", lambda *a, **k: ("CARD1234", []))
87-    with dbmod.connect(":memory:") as conn:
88-        dbmod.init_db(conn)
89-        doc = _doc(conn)
90-        axis.AxisCreditCardParser().parse_and_insert(Path("cc.pdf"), conn, doc,
password=None)
91-        prov = dbmod.get_provenance(conn, doc)
92-        assert prov is not None
93-        assert prov["parser"] == "AxisCreditCardParser"
94-        assert prov["confidence"] is None  # CC has no running balance / reconcile verdict
```

245. user (2026-06-10T17:37:00.650Z)

```
<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>
```

246. user (2026-06-10T17:37:08.198Z)

```
80              n = au.AUParser().parse_and_insert(Path("x.pdf"), conn, doc, password=None)
81              assert n == 0
82              assert dbmod.get_provenance(conn, doc)["confidence"] == "unreconciled"
83
84
85      def test_credit_card_parser_records_provenance_without_confidence(monkeypatch):
86          monkeypatch.setattr(axis, "parse_axis_cc", lambda *a, **k: ("CARD1234", []))
87          with dbmod.connect(":memory:") as conn:
88              dbmod.init_db(conn)
89              doc = _doc(conn)
90              axis.AxisCreditCardParser().parse_and_insert(Path("cc.pdf"), conn, doc,
password=None)
91              prov = dbmod.get_provenance(conn, doc)
92              assert prov is not None
93              assert prov["parser"] == "AxisCreditCardParser"
94              assert prov["confidence"] is None  # CC has no running balance / reconcile
verdict
95
```

247. user (2026-06-10T17:37:18.249Z)

```
The file C:\Users\avidu\Projects\muneem\tests\test_provenance.py has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

248. user (2026-06-10T17:37:44.350Z)

```
..................................................                   [100%]
486 passed, 21 deselected in 4.50s
[fix/axis-cc-verification-floor f5e6a4e] fix(parsers): Axis CC floor -- explicit "unverified"
verdict, no silent 0.0 amounts
 4 files changed, 96 insertions(+), 21 deletions(-)
 * [new branch]      fix/axis-cc-verification-floor -> fix/axis-cc-verification-floor
```

249. user (2026-06-10T17:38:05.844Z)

```
https://github.com/avidullu/muneem/pull/59
From https://github.com/avidullu/muneem
 * branch            main       -> FETCH_HEAD
   73d77ff..3ab48ff  main       -> origin/main
Updating 73d77ff..3ab48ff
Fast-forward
 src/muneem/parsers/axis.py       | 57 +++++++++++++++++++++++++++++-----------
 src/muneem/parsers/validation.py |  7 ++++-
 tests/parsers/test_axis.py       | 45 +++++++++++++++++++++++++++++++--
 tests/test_provenance.py         |  8 +++---
 4 files changed, 96 insertions(+), 21 deletions(-)
Your branch is up to date with 'origin/main'.
Already on 'main'
Already up to date.
```

250. user (2026-06-10T17:38:28.128Z)

```
Updated task #10 status
```

251. user (2026-06-10T17:38:28.600Z)

```
Updated task #11 status
```

252. user (2026-06-10T17:38:29.605Z)

```
1       """The canonical *concept schema* ? one bank-agnostic vocabulary every extractor
targets.
2
3       This is the **contract** at the heart of the concept-core parser architecture (see
4       [`docs/parser-architecture.md`](../../../docs/parser-architecture.md) §3): a parser's
job is to
5       *map this bank's columns/fields onto these concepts, then let the verifier check them.*
Because the
6       concepts are layout-independent, a format change becomes "re-map onto the same
concepts," not
7       "rewrite the parser."
8
9       These are pure data ? no PDF, no DB, no logic ? so they import nothing and are safe to
use anywhere.
10      The verifier (``muneem.parsers.validation``) consumes them; bank extractors produce
them.
11      """
12
13      from __future__ import annotations
14
15      from dataclasses import dataclass
16
17
18      @dataclass
19      class ConceptRow:
20          """One normalized transaction, in the bank-agnostic vocabulary.
21
22          ``debit`` is money out (withdrawal / spend / Dr); ``credit`` is money in (deposit /
Cr).
23          Exactly one is typically non-``None`` on a real row, but both may be ``None`` for a
24          non-amount marker row. ``balance`` is the running account balance *after* this row,
or
25          ``None`` when the statement carries no per-row balance.
26
27          **Credit cards** (no running balance) map onto the same row: put the charge in
``debit`` or
28          the payment/refund in ``credit`` according to the printed Dr/Cr direction, and leave
```

```
29          ``balance=None``. That keeps one row type for the whole ladder; total reconciliation
(not the
30          per-row walk) is what validates a balance-less statement.
31          """
32
33          txn_date: str
34          narration: str
35          debit: float | None = None
36          credit: float | None = None
37          balance: float | None = None
38          value_date: str | None = None
39          reference: str | None = None
40
41
42      @dataclass
43      class StatementConcepts:
44          """Statement-level concepts ? the figures a bank prints in its own SUMMARY / header.
45
46          Every field is optional: ``None`` means "this statement didn't expose it," which the
verifier
47          treats as *not checkable* (never as a failure). ``debit_amount`` / ``credit_amount``
are the
48          bank's own printed totals; they are recorded here for completeness but are not yet
asserted by
49          :func:`muneem.parsers.validation.verify` (the current bookend checks counts +
opening/closing,
50          matching the established HDFC behavior).
51          """
52
53          opening_balance: float | None = None
54          closing_balance: float | None = None
55          debit_count: int | None = None
56          credit_count: int | None = None
57          debit_amount: float | None = None
58          credit_amount: float | None = None
59          account_number: str | None = None
60          period_start: str | None = None
61          period_end: str | None = None
62          currency: str | None = None
63
64
65      __all__ = ["ConceptRow", "StatementConcepts"]
66
```

253. user (2026-06-10T17:38:30.776Z)

```
30:    date_patterns=(re.compile(r"^\d{2}[-/]\d{2}[-/]\d{4}$"),),  # dd-mm-yyyy or dd/mm/yyyy
56:    date_patterns=(
74:    date_patterns=(re.compile(r"^\d{2}/\d{2}/\d{4}$"),),  # dd/mm/yyyy
```

254. user (2026-06-10T17:39:14.223Z)

```
File created successfully at: C:\Users\avidu\Projects\muneem\src\muneem\parsers\dates.py (file
state is current in your context ? no need to Read it back)
```

255. user (2026-06-10T17:39:06.670Z)

```
Switched to a new branch 'feat/date-normalization'
```

256. assistant (2026-06-10T17:39:14.208Z)

```
"""Statement date canonicalization ? day-first Indian formats to ISO-8601.

Dates carry no arithmetic the reconciliation oracle can check, so a misread date would
```

```
persist silently forever. Normalizing at the engine/persistence boundary guarantees the
*stored* form is a real, validated calendar date in one format (``yyyy-mm-dd``) regardless
of the issuer's print style (``dd/mm/yyyy``, ``dd-mm-yyyy``, ``dd-Mon-yy``, ?).

Day-first always: these are Indian statements; ``01/02/2026`` is the 1st of February. A
token that parses under no known format is returned **verbatim** ? never invented, never
dropped ? so an unrecognized vintage degrades to today's raw-string behavior instead of
losing data.
"""

from __future__ import annotations

from datetime import datetime
```

Day-first formats observed across Indian bank/CC statements. Order matters only for performance; the shapes are mutually exclusive. %b month names are the English abbreviations the banks print (strptime's C-locale set).

```
_FORMATS = (
    "%d/%m/%Y",
    "%d-%m-%Y",
    "%d/%m/%y",
    "%d-%m-%y",
    "%d-%b-%Y",
    "%d %b %Y",
    "%d-%b-%y",
    "%d %b %y",
    "%d%m%Y",
)


def normalize_statement_date(raw: str | None) -> str | None:
    """Return ISO-8601 (``yyyy-mm-dd``) when ``raw`` parses as a day-first statement date.

    ``None``/empty and unrecognized tokens pass through unchanged. Impossible dates
    (``32/01/2026``) do not parse and therefore also pass through verbatim ? visible in the
    ledger as the anomaly they are, rather than silently coerced.
    """
    if not raw:
        return raw
    text = raw.strip()
    for fmt in _FORMATS:
        try:
            return datetime.strptime(text, fmt).date().isoformat()
        except ValueError:
            continue
    return raw


__all__ = ["normalize_statement_date"]
```

257. user (2026-06-10T17:39:23.338Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\engine.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

258. user (2026-06-10T17:39:30.408Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\engine.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

259. user (2026-06-10T17:39:32.694Z)

<tool_use_error>File has been modified since read, either by the user or by a linter. Read it
again before attempting to write it.</tool_use_error>

260. user (2026-06-10T17:39:38.675Z)

```
26      import pdfplumber
27
28      from .clustering import cluster_to_matrix
29      from .concepts import ConceptRow, StatementConcepts
30      from .engine import Profile, map_table
31      from .validation import (
32          BALANCE_TOLERANCE,
33          parse_amount,
34          reconcile_running_balance,
35          reconcile_total,
36      )
37
38      _MONEY_TOKEN = re.compile(r"^-?[\d,]+\.\d{2}(?:cr|dr)?$", re.IGNORECASE)
39      # Capture an optional trailing Cr/Dr marker: "Opening Balance: 5,000.00 Dr" is NEGATIVE
40      # (overdrawn) and must anchor the oracle with its true sign.
41      _OPENING_RE = re.compile(
42          r"opening\s+balance[^\d\-]*(-?[\d,]+\.\d{2})(\s*(?:cr|dr))?", re.IGNORECASE
43      )
44      _CLOSING_RE = re.compile(
45          r"closing\s+balance[^\d\-]*(-?[\d,]+\.\d{2})(\s*(?:cr|dr))?", re.IGNORECASE
46      )
47
```

261. user (2026-06-10T17:39:45.912Z)

```
218-            else:
219-                debit = amt  # zero net with an amount present ? reconciliation will flag it
220-        out.append(
221-            ConceptRow(
222:                txn_date=r[date_col],
223-                narration=(r[narr_col] if narr_col is not None and narr_col < len(r) else
""),
224-                debit=debit,
225-                credit=credit,
226-                balance=bal,
```

262. user (2026-06-10T17:39:52.593Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\savings.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

263. user (2026-06-10T17:39:55.311Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\savings.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

264. assistant (2026-06-10T17:40:01.832Z)

Now the HDFC and CC persistence sites:

265. user (2026-06-10T17:40:05.646Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\hdfc.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

266. user (2026-06-10T17:40:07.979Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\hdfc.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

267. user (2026-06-10T17:40:10.695Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\axis.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

268. user (2026-06-10T17:40:18.574Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\parsers\axis.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

269. assistant (2026-06-10T17:40:31.114Z)

```
"""Statement date canonicalization (muneem.parsers.dates): day-first Indian formats ? ISO-8601;
anything unrecognized passes through verbatim (never invented, never dropped)."""

from muneem.parsers.dates import normalize_statement_date


def test_common_indian_formats_normalize_to_iso():
    assert normalize_statement_date("01/02/2026") == "2026-02-01"  # day-first, NOT Jan 2
    assert normalize_statement_date("01-02-2026") == "2026-02-01"
    assert normalize_statement_date("01/02/26") == "2026-02-01"
    assert normalize_statement_date("05-Feb-2026") == "2026-02-05"
    assert normalize_statement_date("05 Feb 2026") == "2026-02-05"
    assert normalize_statement_date("05-Feb-26") == "2026-02-05"
    assert normalize_statement_date("01022026") == "2026-02-01"
    assert normalize_statement_date(" 01/02/2026 ") == "2026-02-01"  # padding tolerated


def test_iso_input_and_unknown_tokens_pass_through():
    assert normalize_statement_date("2026-02-01") == "2026-02-01"  # idempotent on ISO? no ?
    # ISO is not in the day-first format list; it passes through VERBATIM, which is the same
    # string. Unknown shapes likewise:
    assert normalize_statement_date("Feb 1") == "Feb 1"
    assert normalize_statement_date("UPI-REF-12345678") == "UPI-REF-12345678"


def test_impossible_dates_are_not_invented():
    # 32nd of January / 13th month: visible in the ledger as the anomaly they are.
    assert normalize_statement_date("32/01/2026") == "32/01/2026"
    assert normalize_statement_date("01/13/2026") == "01/13/2026"  # would be month 13 day-first


def test_none_and_empty_pass_through():
    assert normalize_statement_date(None) is None
    assert normalize_statement_date("") == ""


def test_engine_builds_rows_with_normalized_dates():
    from muneem.parsers.engine import _build_bank_rows

    rows = _build_bank_rows(
        [
            {
                "txn_date": "05/02/2026",
                "value_date": "04-02-2026",
                "narration": "x",
                "debit": "100.00",
                "balance": "900.00",
            }
        ]
    )
    assert rows[0].txn_date == "2026-02-05"
    assert rows[0].value_date == "2026-02-04"
```

270. user (2026-06-10T17:40:31.125Z)

```
File created successfully at: C:\Users\avidu\Projects\muneem\tests\parsers\test_dates.py (file
state is current in your context ? no need to Read it back)
```

271. user (2026-06-10T17:40:42.556Z)

```
Exit code 1
tests\parsers\test_engine.py:49: AssertionError
========================== short test summary info ==========================
FAILED tests/parsers/test_engine.py::test_l0_header_lexicon_maps_generic_labels
FAILED tests/parsers/test_engine.py::test_l1_detects_value_date_as_second_date_column
2 failed, 489 passed, 21 deselected in 4.71s
```

272. user (2026-06-10T17:40:47.699Z)

```
30              ["03/01/2026", "Salary", "", "1250.00", "2000.00"],
31          ]
32          r = map_table(table, PROFILE, BANK_FAMILY)
33          assert r is not None and r.layer == "L0"
34          assert [row.txn_date for row in r.rows] == ["02/01/2026", "03/01/2026"]
35          assert r.rows[0].debit == 250.0 and r.rows[0].balance == 750.0
36          assert r.rows[1].credit == 1250.0 and r.rows[1].balance == 2000.0
37          assert r.column_map["balance"] == 4
38
39
40      def test_l1_detects_value_date_as_second_date_column():
41          table = [
42              ["Tran Date", "Value Date", "Particulars", "Debit", "Credit", "Balance"],
43              ["02/01/2026", "01/01/2026", "UPI", "250.00", "", "750.00"],
44              ["03/01/2026", "02/01/2026", "Salary", "", "1250.00", "2000.00"],
45          ]
46          r = map_table(table, PROFILE, BANK_FAMILY)
47          assert r is not None
48          assert r.column_map["txn_date"] == 0 and r.column_map["value_date"] == 1
49          assert r.rows[0].value_date == "01/01/2026"
50
51
52      # --- profile-order: positional prior when there is no header
---------------------------------
53
54
55      def test_profile_order_fallback_when_no_header():
56          p = Profile(
57              name="t",
58              date_patterns=DMY,
59              column_order=("txn_date", "narration", "debit", "credit", "balance"),
60          )
61          table = [
62              ["02/01/2026", "UPI", "250.00", "", "750.00"],
63              ["03/01/2026", "Salary", "", "1250.00", "2000.00"],
64          ]
65          r = map_table(table, p, BANK_FAMILY)
66          assert r is not None and r.layer == "profile-order"
67          assert r.rows[0].debit == 250.0 and r.rows[1].credit == 1250.0
68
69
70      # --- L2: the self-identifying balance column, found by the oracle, not by a label
-------------
71
72
73      def test_l2_search_finds_self_identifying_balance_via_oracle():
74          # Unusual order, no header, no column_order: balance sits BEFORE the flows. Only the
assignment
75          # whose running balance walks cleanly should survive ? the oracle picks it.
```

```
76          table = [
77              ["02/01/2026", "UPI", "750.00", "250.00", ""],  # date, narr, BALANCE, DEBIT,
CREDIT
78              ["03/01/2026", "Salary", "2000.00", "", "1250.00"],
79          ]
```

273. assistant (2026-06-10T17:40:55.259Z)

The two failures pin the old raw dates ? updating them to the new intended ISO behavior:

274. user (2026-06-10T17:40:58.682Z)

```
The file C:\Users\avidu\Projects\muneem\tests\parsers\test_engine.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

275. user (2026-06-10T17:41:00.493Z)

```
The file C:\Users\avidu\Projects\muneem\tests\parsers\test_engine.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

276. user (2026-06-10T17:41:13.497Z)

```
.........................................................              [100%]
491 passed, 21 deselected in 4.61s
Found 1 error (1 fixed, 0 remaining).
81 files left unchanged
Success: no issues found in 34 source files
```

277. user (2026-06-10T17:41:49.909Z)

```
[feat/date-normalization d97b6f7] feat(parsers): canonicalize statement dates to ISO-8601 at the
build boundary
 7 files changed, 123 insertions(+), 9 deletions(-)
 create mode 100644 src/muneem/parsers/dates.py
 create mode 100644 tests/parsers/test_dates.py
 * [new branch]      feat/date-normalization -> feat/date-normalization
https://github.com/avidullu/muneem/pull/60
From https://github.com/avidullu/muneem
 * branch            main       -> FETCH_HEAD
   3ab48ff..11ebb82  main       -> origin/main
Updating 3ab48ff..11ebb82
Fast-forward
 src/muneem/parsers/axis.py    |  3 ++-
 src/muneem/parsers/dates.py   | 52 +++++++++++++++++++++++++++++++++++++++++++
 src/muneem/parsers/engine.py  | 12 +++++++---
 src/muneem/parsers/hdfc.py    |  5 +++--
 src/muneem/parsers/savings.py |  3 ++-
 tests/parsers/test_dates.py   | 52 +++++++++++++++++++++++++++++++++++++++++++
 tests/parsers/test_engine.py  |  5 +++--
 7 files changed, 123 insertions(+), 9 deletions(-)
 create mode 100644 src/muneem/parsers/dates.py
 create mode 100644 tests/parsers/test_dates.py
Your branch is up to date with 'origin/main'.
Already on 'main'
Already up to date.
```

278. user (2026-06-10T17:42:00.971Z)

```
Updated task #11 status
```

279. user (2026-06-10T17:42:01.332Z)

```
1       ---
2       name: session-handoff
```

```
3       description: "? Start here ? living handoff for muneem (and its sister repo gmail-
attachments-to-gdrive); current state, next steps, ramp-up kit, key decisions."
4       metadata:
5         node_type: memory
6         type: project
7         originSessionId: 02b9e9c9-3806-45ca-9b24-2ecc35a81efd
8       ---
9
10      # Session handoff ? muneem (+ gmail-attachments-to-gdrive)
11
12      > Last refreshed: 2026-06-10, after merging the antigravity-feedback action items (#48,
#49).
13
14      ## 1. You are here
15      - muneem `main` @ **0e5602f** ? local == origin, full gate green (394 passed, cov 86.4%,
mypy/ruff clean). Parser precision on real data: HDFC 6/6, AU 4/4, Axis 6/7. Encryption +
recovery + migrations + provenance shipped. Sister repo `gmail-attachments-to-gdrive` master @
a764030 (auditability invariant, PR #14).
16      - **Just merged (2026-06-10):**
17        - **#48** ? antigravity-feedback response artefact (`docs/feedback-antigravity-
response.md`: schema critique rejected, AA-pivot refuted, sister `processedIds` claim refuted by
its own code) + **Apps Script deprecation trigger** (retire sister archiver when `fetch`/B.4
ships; ROADMAP Track B + feedbackJune2026 P7) + monetization parked (feedbackJune2026 §8).
18        - **#49** ? **schema v6**: migration `serving_view` ? `v_transactions` (normalized
UNION over the flat per-issuer tables, provenance-joined) + `muneem txns list all` + schema-lock
5?6 + CURRENT_STATE refresh (v6; "unify storage" step superseded ? instrument family on the flat
pattern; D.3-before-L3). Closes the deep-review items "ship the dropped serving view + txns all"
and the CURRENT_STATE half of the doc-debt.
19      - ~~GitHub auth blocker~~ **RESOLVED:** the fine-grained PAT (shared with rally-
annotator) needed `avidullu/muneem` under Repository access **and** Contents/Pull-requests =
Read & write under Permissions. If pushes 403 again, check that token first.
20      - Unreviewed remote branch from another session: `origin/docs/proposal-opendataloader-
pdf-eval` (not merged; not mine to judge here).
21      - A 35-agent deep review of BOTH repos (code, security, architecture, roadmap, PMF)
completed 2026-06-10. **All findings in [[deep-review-2026-06]]** ? 9 confirmed critical/high
code bugs, the cross-repo Gmail-ownership contradiction, roadmap gaps, and a researched grocery-
vs-finance-vs-tax-pack wedge assessment delivered to Avi. **Awaiting Avi's direction** on (a)
which fixes to start, (b) the cross-repo seam decision, (c) the first-vertical/wedge choice.
22
23      ## 2. Next steps / open threads
24      1. Avi to react to the review: pick wedge (recommendation: finance stays first
personally; grocery as 2nd/front-door corpus; tax-season pack as the public wedge candidate).
25      2. If fixes approved, P0 order: rotate-key atomicity ? `db backup` destination guard ?
savings-oracle bookend check ? Cr/Dr sign semantics ? HDFC unreconciled persistence ? gmail
backfill retry. One PR per item (see [[pr-workflow-cadence]]).
26      3. Cross-repo ADR needed: who reads Gmail (recommended middle path: sister archives to
Drive in-account; muneem ingests the locally-synced Drive folder; invariant scoped to the public
product). Record in both repos.
27      4. Doc debt: consolidate sequencing into one owner (ROADMAP vs feedbackJune2026 Phases
A?H); refresh parser-architecture (stale precision numbers, superseded §9). ~~Refresh
CURRENT_STATE (schema)~~ and ~~ship the dropped Phase-B serving view + `txns all`~~ ? **done,
merged in #49.**
28      5. Hygiene quick wins: delete dead `snapshot` fixture + the two real-CC-row JSONs in
testdata/snapshots/; block `client_secret*.json` everywhere; add lockfile.
29
30      ## 3. Ramp-up kit
31      - [[deep-review-2026-06]] ? the full review findings (memory dir).
32      - muneem: `CLAUDE.md` ? `DESIGN.md` ? `docs/feedbackJune2026.md` (the governing phase
plan) ? `docs/CURRENT_STATE.md` (stale re: schema v5 ? see review).
33      - Sister: `docs/VISION.md` ? `docs/ARCHITECTURE.md` (ADR-008 invariant) ? `ROADMAP.md`.
34      - Memory index `MEMORY.md` for standing decisions (licenses, local-first, storage shape,
L3, AA-deferred).
35
36      ## 4. Key decisions (don't relitigate)
37      - Python-only; PDF-first/AA-deferred; flat per-issuer tables + separate provenance table
```

+ (promised, unbuilt) serving view; hand-rolled migrations (no Alembic); SQLCipher + keychain + passphrase recovery; D.3 eval before L3; licenses MIT/BSD/Apache only (PyMuPDF banned); no cloud LLMs on sensitive docs; sister repo: <100-user Testing mode, CASA deferred.
38   - Still open (genuinely, verified in code): log format; self-hosted runner; macOS stance; the cross-repo Gmail seam (NEW ? surfaced by the review); instrument-family artifact choice (CAS vs broker stmt vs tradebook).
39

280. assistant (2026-06-10T17:42:33.737Z)

```
---
name: session-handoff
description: "? Start here ? living handoff for muneem (and its sister repo gmail-attachments-to-gdrive); current state, next steps, ramp-up kit, key decisions."
metadata:
  node_type: memory
  type: project
  originSessionId: 02b9e9c9-3806-45ca-9b24-2ecc35a81efd
---
```

Session handoff ? muneem (+ gmail-attachments-to-gdrive)

> Last refreshed: 2026-06-10, mid "civilization plan" execution ? Tracks A+B done (#50?#60).

1. You are here

- muneem `main` @ **11ebb82** ? local == origin, full gate green (491 passed, cov ~86%, mypy/ruff clean). Sister repo `gmail-attachments-to-gdrive` master @ a764030.
- **Avi approved the post-review "civilization plan"** (fix everything checked in, tiered A?F, one PR each, squash-merge as verified). **Done ? Track A (ledger safety):** #50 rotate-key pending-key protocol + self-healing open · #51 backup destination guard + atomic temp-rename · #52 keychain-failure?absence + never-mint-over-existing-ledger. **Done ? Track B (oracle honesty):** #53 bookend-anchored trust oracle (+ `_selection_oracle` for fragments) · #54 Cr/Dr is a SIGN on balances (`parse_amount(signed=)`) · #55 HDFC quarantine parity + status-tagged `txns list` + axis_cc `+Cr` display fix · #56 `BALANCE_TOLERANCE=0.005` (one-paisa corruption always caught) · #57 letterhead-anchored routing (UPI-handle hijack killed) · #58 status-aware dedup (quarantined docs re-ingestable; `db.clear_document_rows`) · #59 Axis CC floor (`Confidence.UNVERIFIED`, strict amounts) · #60 ISO date normalization (`parsers/dates.py`).
- Earlier same day: #48 (antigravity-feedback response + Apps Script deprecation trigger: retire sister archiver when `fetch`/B.4 ships) and #49 (schema v6 serving view + `txns list all` + CURRENT_STATE refresh).
- **?? Avi asked for one manual verification:** run `pytest -m local_statements` (or `muneem eval` over the statement folders) once ? #53/#54/#57 tightened the oracle and routing; if a real Axis/AU vintage now quarantines or misroutes, fix `parse_statement_summary` regexes / routing evidence, never weaken the oracle. Passwords weren't resolvable in the work session, so this is unverified on real data.
- The deep review that seeded the plan: [[deep-review-2026-06]]. PAT note: pushes 403 ? check the fine-grained PAT (muneem under Repository access + Contents/PRs RW).
- Unreviewed remote branch from another session: `origin/docs/proposal-opendataloader-pdf-eval` (not merged; not mine to judge).

2. Next steps / open threads

1. **Remaining civilization-plan tracks (in order):** C1 hygiene (delete dead `snapshot` fixture + 2 real-CC-row JSONs in testdata/snapshots/; block `client_secret*.json` in .gitignore + pre-commit + hygiene test; fix stale FORBIDDEN_PREFIXES) ? C2 constraints lockfile + CI ? C3 migrations per-txn transaction + checksum re-verify + partial-failure test ? C4 small-fixes bundle (64-hex key validation, ATTACH quoting, redaction patterns PAN/spaced-cards/Aadhaar + root-logger filter, unlock mkdir, recovery KDF guard) ? D1 synthetic corpus breadth (Axis savings/CC fixtures, multi-page, dropped-row-must-quarantine at PDF level, encrypted PDF through CLI) ? D2 OCR CI cron + pytest-id redaction ? E1 docs refresh (parser-architecture stale numbers/§9; back-record L3 in DESIGN §8.3; synthetic-fixture wording; README) ? E2 docs/recovery.md runbook ? E3 ROADMAP owns Phases A?H (rename collisions) ? F1?F3 sister repo data-loss fixes (lower priority per #48 deprecation trigger, but archiver still runs daily).
2. Avi reacting to the review wedge question (recommendation: finance stays first personally;

grocery as 2nd/front-door corpus; tax-season pack as the public wedge candidate).
3. Cross-repo ADR: who reads Gmail (review recommended middle path: sister archives to Drive in-account; muneem ingests locally-synced Drive folder; invariant scoped to public product). #48's deprecation trigger leans toward muneem-keeps-ingest ? needs the explicit ADR in both repos.

3. Ramp-up kit

- [[deep-review-2026-06]] ? full review findings (memory dir).
- muneem: `CLAUDE.md` ? `DESIGN.md` ? `docs/feedbackJune2026.md` (governing phase plan) ? `docs/CURRENT_STATE.md` (refreshed in #49).
- Sister: `docs/VISION.md` ? `docs/ARCHITECTURE.md` (ADR-008 invariant) ? `ROADMAP.md`.
- Memory index `MEMORY.md` for standing decisions.

4. Key decisions (don't relitigate)

- Python-only; PDF-first/AA-deferred; flat per-issuer tables + separate provenance table + serving view (v6); hand-rolled migrations (no Alembic); SQLCipher + keychain + passphrase recovery; D.3 eval before L3; licenses MIT/BSD/Apache only (PyMuPDF banned); no cloud LLMs on sensitive docs; sister repo: <100-user Testing mode, CASA deferred; Apps Script archiver deprecates when `muneem fetch` ships (#48).
- New status vocabulary (this session): documents.status ? reconciled / unreconciled (quarantined, stores nothing) / **unverified** (family has no oracle by design, e.g. CC ? persisted but tagged); listings tag anything ? reconciled.
- Still open (verified in code): log format; self-hosted runner; macOS stance; cross-repo Gmail seam ADR; instrument-family artifact choice.

281. user (2026-06-10T17:42:33.754Z)

```
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\memory\session-handoff.md has been updated successfully. (file state is current in your context ? no need to Read it back)
```

282. assistant (2026-06-10T17:42:43.730Z)

Handoff refreshed. Now C1 ? hygiene. Reading the three guard layers:

283. user (2026-06-10T17:42:44.204Z)

```
Updated task #12 status
```

284. user (2026-06-10T17:42:44.700Z)

```
1       #!/usr/bin/env python3
2       """muneem sensitive-file guard (invoked by the pre-commit hook).
3
4       Refuses to *stage* files that policy says must never be committed: real
5       statements (any PDF, or anything under ``testdata/`` outside the allowed
6       corpora), OAuth secrets, password rules, local databases, key material, and
7       ``.env`` files. This is the PRE-commit backstop to ``.gitignore`` ? it stops the
8       ``git add -f`` / ``git add -A`` mistake before it becomes a commit (the very
9       thing that caused an earlier statement leak). The post-commit complement is
10      ``tests/test_repo_hygiene.py``.
11
12      Exit 0 when nothing sensitive is staged, 1 otherwise. A deliberate, reviewed
13      exception can be committed with ``git commit --no-verify``.
14      """
15
16      from __future__ import annotations
17
18      import fnmatch
19      import os
20      import subprocess
21      import sys
22
23      # Only these subtrees of testdata/ are non-sensitive and may be committed.
```

```
    ALLOWED_PREFIXES = ("testdata/promo-emails/", "testdata/synthetic/")
    SECRET_NAMES = {"credentials.json", "token.json", "token.pickle", "passwords.txt"}


    def is_sensitive(path: str) -> str | None:
        """Return a human-readable reason if ``path`` must not be committed, else None."""
        p = path.replace("\\", "/").strip()
        if not p or p.startswith(ALLOWED_PREFIXES):
            return None
        name = p.rsplit("/", 1)[-1]
        low = p.lower()
        nlow = name.lower()
        if p.startswith("testdata/"):
            return "under testdata/ ? sensitive by default (only promo-emails/ and
synthetic/ allowed)"
        if low.endswith(".pdf"):
            return "PDF outside the allowed corpora (real statements must stay local)"
        if low.endswith((".db", ".sqlite", ".sqlite3")):
            return "local database (may contain parsed statement data)"
        if low.endswith(".plaintext.bak"):
            return "plaintext database backup (cleartext from 'muneem db encrypt'
migration)"
        if fnmatch.fnmatch(nlow, "*recovery*.json"):
            return "DB-key recovery file (passphrase-wrapped key ? keep it off the repo)"
        if low.endswith((".pem", ".key")):
            return "key material"
        if nlow in SECRET_NAMES:
            return "OAuth / credential secret"
        if nlow.startswith("oauth_token"):
            return "OAuth token"
        # Block the rules *data* file (password_rules.yaml/.yml/.json/?) but not the loader
module
        # (src/muneem/password_rules.py is code, not a secret).
        if fnmatch.fnmatch(nlow, "password_rules.*") and not nlow.endswith(".py"):
            return "password rules"
        if (nlow == ".env" or nlow.startswith(".env.")) and nlow != ".env.example":
            return ".env file (may contain secrets)"
        return None


    def _staged_paths() -> list[str]:
        # Tests inject a newline-separated list via PRECOMMIT_TEST_FILES (no git needed).
        override = os.environ.get("PRECOMMIT_TEST_FILES")
        if override is not None:
            return [ln for ln in override.replace("\r\n", "\n").split("\n") if ln.strip()]
        # -z keeps paths with spaces/unicode intact (the real statements have spaces).
        out = subprocess.run(
            ["git", "diff", "--cached", "--name-only", "-z", "--diff-filter=ACM"],
            capture_output=True,
            text=True,
            check=True,
        ).stdout
        return [p for p in out.split("\0") if p.strip()]


    def main() -> int:
        offenders = [(p, reason) for p in _staged_paths() if (reason := is_sensitive(p))]
        if not offenders:
            return 0
        print("muneem pre-commit: refusing to commit sensitive file(s):", file=sys.stderr)
        for p, reason in offenders:
            print(f"  x {p}\n      {reason}", file=sys.stderr)
        print(
            "\nThese must never reach git (CLAUDE.md security rules; see .gitignore).\n"
            "If this is a deliberate, reviewed exception: git commit --no-verify",
```

```
            file=sys.stderr,
        )
        return 1


    if __name__ == "__main__":
        raise SystemExit(main())

```

285. user (2026-06-10T17:42:45.048Z)

```
    """Security guard: no sensitive paths may be tracked in git.

    Encodes DESIGN.md section 5 ("Nothing sensitive ever in git") and docs/TESTING.md
    section 9 as a failing-build assertion. If this test fails, something that must
    stay local has been committed.
    """

    from __future__ import annotations

    import subprocess
    from pathlib import Path

    REPO_ROOT = Path(__file__).resolve().parents[1]

    FORBIDDEN_PREFIXES = ("testdata/statements/", "testdata/private/")
    FORBIDDEN_SUFFIXES = (".db", ".sqlite", ".sqlite3", ".env", ".pem", ".key")
    FORBIDDEN_NAMES = (
        "token.json",
        "token.pickle",
        "credentials.json",
        "password_rules.yaml",
        "password_rules.yml",
        "password_rules.json",
        "passwords.txt",
    )


    def _tracked_files() -> list[str]:
        out = subprocess.run(
            ["git", "ls-files"],
            cwd=REPO_ROOT,
            capture_output=True,
            text=True,
            check=True,
        )
        return [line.strip() for line in out.stdout.splitlines() if line.strip()]


    def test_no_sensitive_paths_tracked():
        offenders = []
        for path in _tracked_files():
            low = path.lower()
            if (
                low.startswith(FORBIDDEN_PREFIXES)
                or low.endswith(FORBIDDEN_SUFFIXES)
                or Path(path).name.lower() in FORBIDDEN_NAMES
            ):
                offenders.append(path)
        assert not offenders, f"sensitive paths must never be tracked: {offenders}"


    def test_only_corpus_pdfs_are_tracked():
        """Statements are PDFs ? the only PDFs git may track are the non-sensitive promo
corpus.
```

```
54
55          This is the guard that would have caught real HDFC statements being committed to
56          testdata/hdfc-statements/.
57          """
58          offenders = [
59              path
60              for path in _tracked_files()
61              if path.lower().endswith(".pdf") and not path.startswith("testdata/promo-
emails/")
62          ]
63          assert not offenders, f"only testdata/promo-emails/ PDFs may be tracked; found:
{offenders}"
64
65
66      def test_no_noncorpus_testdata_is_tracked():
67          """Mirror of the pre-commit guard: under testdata/, only the non-sensitive
68          promo-emails/ and synthetic/ corpora may be tracked ? nothing else (real
69          statements, zips, credentials, snapshots all stay local)."""
70          allowed = ("testdata/promo-emails/", "testdata/synthetic/")
71          offenders = [
72              path
73              for path in _tracked_files()
74              if path.startswith("testdata/") and not path.startswith(allowed)
75          ]
76          assert not offenders, (
77              f"only promo-emails/ and synthetic/ may be tracked under testdata/; found:
{offenders}"
78          )
79
```

286. user (2026-06-10T17:43:00.956Z)

```
Switched to a new branch 'chore/hygiene-footguns'
conftest.py:28: def snapshot():
conftest.py:29: def _assert_snapshot(name: str, data: list | dict):
conftest.py:30: snapshot_dir = Path("testdata/snapshots")
conftest.py:31: snapshot_dir.mkdir(parents=True, exist_ok=True)
conftest.py:32: snapshot_file = snapshot_dir / f"{name}.json"
conftest.py:34: if os.environ.get("UPDATE_SNAPSHOTS") == "1":
conftest.py:35: with snapshot_file.open("w", encoding="utf-8") as f:
conftest.py:39: if not snapshot_file.exists():
conftest.py:40: pytest.fail(f"Snapshot not found at {snapshot_file}. Run with
UPDATE_SNAPSHOTS=1.")
conftest.py:42: with snapshot_file.open("r", encoding="utf-8") as f:
conftest.py:45: assert data == expected, f"Data does not match snapshot {name}"
conftest.py:48: return _assert_snapshot
test_repo_hygiene.py:69: statements, zips, credentials, snapshots all stay local)."""
```

287. user (2026-06-10T17:43:00.973Z)

```
1       # --- Sensitive data ? NEVER commit ---
2       # Real financial documents stay local only. Everything under testdata/ is sensitive by
3       # default ? ONLY the non-sensitive promo corpus and committed synthetic fixtures are
tracked.
4       # Real statements in ANY subdir (e.g. testdata/hdfc-statements/) must never be
committed.
5       testdata/*
6       !testdata/promo-emails/
7       !testdata/synthetic/
8       *.db
9       *.sqlite
10      *.sqlite3
11      muneem.db*
12
13      # Secrets and credentials
```

```
.env
.env.*
!.env.example
*.pem
*.key
credentials.json
token.json
token.pickle
oauth_token*
password_rules.yaml
password_rules.yml
password_rules.json
passwords.txt

# Logs (may contain extracted statement text ? sanitize before committing any log)
*.log
logs/

# --- Python ---
__pycache__/
*.py[cod]
*$py.class
*.so
.Python
build/
develop-eggs/
dist/
downloads/
eggs/
.eggs/
lib/
lib64/
parts/
sdist/
var/
wheels/
*.egg-info/
.installed.cfg
*.egg
MANIFEST
.venv/
venv/
env/
ENV/
.pytest_cache/
.mypy_cache/
.ruff_cache/
.coverage
htmlcov/
.tox/

# --- Go ---
*.exe
*.exe~
*.dll
*.dylib
*.test
*.out
vendor/

# --- Node (if a UI later lands) ---
node_modules/
npm-debug.log*
yarn-debug.log*
yarn-error.log*
```

```
79      .next/
80      .nuxt/
81      dist/
82
83      # --- IDE / OS ---
84      .vscode/
85      .idea/
86      *.swp
87      *.swo
88      *~
89      .DS_Store
90      Thumbs.db
91      desktop.ini
92
93      # --- Jupyter (in case notebooks are used for exploration) ---
94      .ipynb_checkpoints/
95
96      # --- Scratch / local-only ---
97      scratch/
98      local/
99      tmp/
100     ruff-bin/
101     ruff.zip
102
103
```

288. assistant (2026-06-10T17:43:11.646Z)

```
All confirmed. Making the C1 edits:
```

289. user (2026-06-10T17:43:22.565Z)

```
The file C:\Users\avidu\Projects\muneem\tests\conftest.py has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

290. user (2026-06-10T17:43:25.862Z)

```
The file C:\Users\avidu\Projects\muneem\.githooks\precommit_check.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

291. user (2026-06-10T17:43:37.630Z)

```
The file C:\Users\avidu\Projects\muneem\tests\test_repo_hygiene.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

292. user (2026-06-10T17:43:40.233Z)

```
The file C:\Users\avidu\Projects\muneem\.gitignore has been updated successfully. (file state is
current in your context ? no need to Read it back)
```

293. user (2026-06-10T17:43:47.819Z)

```
3:Tests the pure rule function + the exit-code path (via PRECOMMIT_TEST_FILES), so
54:def test_sensitive_paths_are_blocked(path):
73:def test_allowed_paths_pass(path):
77:def test_main_blocks_when_sensitive_staged(monkeypatch):
78:    monkeypatch.setenv("PRECOMMIT_TEST_FILES", "src/muneem/hdfc.py\ntestdata/hdfc-
statements/x.pdf")
82:def test_main_passes_when_clean(monkeypatch):
83:    monkeypatch.setenv("PRECOMMIT_TEST_FILES", "src/muneem/hdfc.py\nREADME.md")
```

294. user (2026-06-10T17:43:55.172Z)

```
28
```

```
29      @pytest.mark.parametrize(
30          "path",
31          [
32              "testdata/hdfc-statements/Avi_Dullu_28022026.pdf",
33              "testdata/axis-statements/AXIS BANK  Statement for October 2025.pdf",  # spaces
34              "testdata/aubank-statements/logs_71559053525.zip",  # non-PDF under testdata/
35              "testdata/credentials/client_secret.json",
36              "some/dir/Statement.PDF",  # PDF anywhere, any case
37              "muneem.db",
38              "data/ledger.sqlite3",
39              "data/muneem.db.plaintext.bak",  # cleartext backup from the encrypt migration
40              "muneem-recovery.json",  # passphrase-wrapped DB-key recovery file
41              "backups/my-recovery.json",
42              "credentials.json",
43              "src/token.json",
44              "oauth_token_cache",
45              "password_rules.yaml",
46              "password_rules.yml",
47              "password_rules.json",
48              ".env",
49              ".env.local",
50              "secrets/server.pem",
51              "id_rsa.key",
52          ],
53      )
54      def test_sensitive_paths_are_blocked(path):
55          assert pc.is_sensitive(path) is not None, f"should block: {path}"
56
57
58      @pytest.mark.parametrize(
59          "path",
60          [
61              "testdata/promo-emails/swiggy.pdf",  # the allowed corpus
62              "testdata/synthetic/fixture.pdf",
63              "src/muneem/hdfc.py",
64              "src/muneem/password_rules.py",  # the loader module is code, not the secret
rules file
65              "tests/test_hdfc.py",
66              "README.md",
67              ".env.example",
68              "docs/DESIGN.md",
69              "pyproject.toml",
70              "Makefile",
71          ],
72      )
73      def test_allowed_paths_pass(path):
74          assert pc.is_sensitive(path) is None, f"should allow: {path}"
75
76
77      def test_main_blocks_when_sensitive_staged(monkeypatch):
```

295. user (2026-06-10T17:44:05.643Z)

```
The file C:\Users\avidu\Projects\muneem\tests\test_precommit_hook.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

296. user (2026-06-10T17:44:07.112Z)

```
False
```

297. user (2026-06-10T17:44:22.077Z)

```
.........................................................          [100%]
494 passed, 21 deselected in 4.91s
```

```
All checks passed!
81 files left unchanged
Success: no issues found in 34 source files
```

298. user (2026-06-10T17:44:58.452Z)

```
[chore/hygiene-footguns 9cf64e8] chore(security): remove the real-data snapshot footgun; block
client_secret patterns everywhere
 5 files changed, 43 insertions(+), 28 deletions(-)
 * [new branch]      chore/hygiene-footguns -> chore/hygiene-footguns
https://github.com/avidullu/muneem/pull/61
From https://github.com/avidullu/muneem
 * branch            main       -> FETCH_HEAD
   11ebb82..fd9abff  main       -> origin/main
Updating 11ebb82..fd9abff
Fast-forward
 .githooks/precommit_check.py |  7 +++++++
 .gitignore                   |  4 ++++
 tests/conftest.py            | 31 +++++++++++++++++--------------
 tests/test_precommit_hook.py |  5 +++++
 tests/test_repo_hygiene.py   | 24 ++++++++++++++++++++++--
 5 files changed, 43 insertions(+), 28 deletions(-)
Your branch is up to date with 'origin/main'.
Already on 'main'
Already up to date.
```

299. user (2026-06-10T17:45:17.104Z)

```
Updated task #12 status
```

300. user (2026-06-10T17:45:17.570Z)

```
Updated task #13 status
```

301. user (2026-06-10T17:45:17.811Z)

```
1       name: ci
2
3       on:
4         # All-branch pushes so feature branches are gated even before a PR exists
5         # (the regression fixtures in tests/test_regression_synth.py run here, secret-free).
6         push:
7         pull_request:
8         workflow_dispatch:
9
10      jobs:
11        test:
12          runs-on: ${{ matrix.os }}
13          strategy:
14            fail-fast: false
15            matrix:
16              os: [ubuntu-latest]
17              python-version: ["3.11", "3.12", "3.13"]
18              include:
19                # The dev box is Windows: test it on the primary Python (DPAPI secret backend,
Windows
20                # path handling, the sqlcipher3 win_amd64 wheel). Linux covers the 3.11/3.12
spread.
21                - os: windows-latest
22                  python-version: "3.13"
23          steps:
24            - uses: actions/checkout@v6
25            - uses: actions/setup-python@v6
26              with:
```

```
27                python-version: ${{ matrix.python-version }}
28                cache: pip
29                cache-dependency-path: pyproject.toml
30            - name: Install
31              run: |
32                python -m pip install --upgrade pip
33                python -m pip install -e '.[dev]'
34            - name: Lint
35              run: ruff check .
36            - name: Format check
37              run: ruff format --check .
38            - name: Type check
39              run: mypy
40            - name: Test
41              run: pytest --cov=muneem --cov-report=term-missing --cov-fail-under=80
42
43        ocr:
44          # OCR tier needs the Tesseract binary; runs on demand (Linux) to avoid per-PR
flakiness.
45          if: github.event_name == 'workflow_dispatch'
46          runs-on: ubuntu-latest
47          steps:
48            - uses: actions/checkout@v6
49            - uses: actions/setup-python@v6
50              with:
51                python-version: "3.13"
52                cache: pip
53                cache-dependency-path: pyproject.toml
54            - name: Install Tesseract
55              run: sudo apt-get update && sudo apt-get install -y tesseract-ocr
56            - name: Install
57              run: |
58                python -m pip install --upgrade pip
59                python -m pip install -e '.[dev]'
60            - name: OCR tests
61              run: pytest -m ocr
62
```

302. user (2026-06-10T17:45:27.697Z)

```
Switched to a new branch 'chore/constraints-lockfile'
annotated-doc==0.0.4
ast_serialize==0.5.0
certifi==2026.5.20
cffi==2.0.0
charset-normalizer==3.4.7
colorama==0.4.6
coverage==7.14.1
cryptography==48.0.0
google-api-core==2.30.3
google-api-python-client==2.197.0
google-auth-httplib2==0.4.0
google-auth-oauthlib==1.4.0
google-auth==2.53.0
googleapis-common-protos==1.75.0
httplib2==0.31.2
idna==3.17
iniconfig==2.3.0
jaraco.classes==3.4.0
jaraco.context==6.1.2
jaraco.functools==4.5.0
keyring==25.7.0
librt==0.11.0
markdown-it-py==4.2.0
mdurl==0.1.2
```

```
more-itertools==11.1.0
mypy_extensions==1.1.0
mypy==2.1.0
oauthlib==3.3.1
packaging==26.2
pathspec==1.1.1
pdfminer.six==20251230
pdfplumber==0.11.9
pillow==12.2.0
platformdirs==4.10.0
pluggy==1.6.0
proto-plus==1.28.0
protobuf==7.35.0
pyasn1_modules==0.4.2
pyasn1==0.6.3
pycparser==3.0
Pygments==2.20.0
pyparsing==3.3.2
pypdf==6.12.2
pypdfium2==5.8.0
pytesseract==0.3.13
pytest-cov==7.1.0
pytest==9.0.3
pywin32-ctypes==0.2.3
PyYAML==6.0.3
requests-oauthlib==2.0.0
requests==2.34.2
rich==15.0.0
ruff==0.15.15
shellingham==1.5.4
sqlcipher3==0.6.2
typer==0.26.4
types-PyYAML==6.0.12.20260518
typing_extensions==4.15.0
uritemplate==4.2.0
urllib3==2.7.0
```

303. user (2026-06-10T17:45:51.007Z)

```
[notice] A new release of pip is available: 26.1.1 -> 26.1.2
[notice] To update, run: C:\Users\avidu\Projects\muneem\.venv\Scripts\python.exe -m pip
install --upgrade pip
No broken requirements found.
```

304. user (2026-06-10T17:45:58.664Z)

```
The file C:\Users\avidu\Projects\muneem\.github\workflows\ci.yml has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

305. user (2026-06-10T17:46:01.844Z)

```
The file C:\Users\avidu\Projects\muneem\.github\workflows\ci.yml has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

306. user (2026-06-10T17:46:11.946Z)

```
1:# muneem ? Dependency policy & license allowlist
7:## Rules
22:## Current dependencies
51:## System tools (not Python packages)
57:## Current / recently added (permissive)
72:## Planned (future, permissive, no paid)
```

307. user (2026-06-10T17:46:20.696Z)

```
7       ## Rules
8
9       1. Only permissive, commercial-safe licenses: **MIT, BSD (2/3-clause), Apache-2.0, ISC,
HPND**.
10      2. **Banned:** GPL, LGPL, **AGPL**, and source-available / non-commercial licenses.
11         ? **PyMuPDF / `fitz` is AGPL-3.0 ? do not use it.** (Its own metadata reads
12         *"Dual Licensed - GNU AFFERO GPL 3.0 or Artifex Commercial"*; the non-AGPL option
13         is a *paid* license, which also violates rule 4.)
14      3. **MPL-2.0** (weak, file-level copyleft) only as a fallback when no permissive option
15         exists, and flagged here.
16      4. Prefer **local / self-hosted / open-source** runtimes over paid APIs. Paid APIs
17         (cloud LLMs, paid OCR/vision) are a **fallback only**, gated behind explicit
approval.
18         We are willing to **train our own** image/OCR models rather than take a paid
dependency.
19      5. Adding a dependency requires updating **both** this table **and** the allowlist in
20         `tests/test_dependency_licenses.py` ? otherwise the build fails.
21
22      ## Current dependencies
```

308. user (2026-06-10T17:46:31.035Z)

```
The file C:\Users\avidu\Projects\muneem\docs\DEPENDENCIES.md has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

309. user (2026-06-10T17:46:57.647Z)

```
............................................................           [100%]
494 passed, 21 deselected in 4.54s
warning: in the working copy of 'constraints.txt', CRLF will be replaced by LF the next time Git
touches it
[chore/constraints-lockfile 456865d] chore(deps): pin dependency versions via a pip constraints
file
 3 files changed, 84 insertions(+), 4 deletions(-)
 create mode 100644 constraints.txt
 * [new branch]      chore/constraints-lockfile -> chore/constraints-lockfile
https://github.com/avidullu/muneem/pull/62
```

310. user (2026-06-10T17:47:08.626Z)

```
Your branch is up to date with 'origin/main'.
Switched to branch 'main'
Switched to a new branch 'fix/migration-atomicity'
```

311. user (2026-06-10T17:47:08.640Z)

```
1       """Hand-rolled, versioned, checksummed schema migrations (feedbackJune2026 P6).
2
3       Deliberately **not Alembic** ? muneem stays raw-`sqlite3` / stdlib-light. Migrations are
an ordered
4       list; each is applied exactly once, *inside* the (possibly encrypted) connection, and
recorded in a
5       ``schema_migrations`` table with a checksum. The schema-lock test
(`tests/test_db_invariants.py`)
6       pins the resulting shape, so accidental drift and deliberate changes both surface in
review.
7
8       Model:
9       - **Baseline.** The current schema (`db._SCHEMA`, version `db.SCHEMA_VERSION`) is the
*frozen*
10        baseline ? never edit it for new changes; add a migration instead. A fresh DB gets the
baseline
11        applied; a legacy DB created before this framework (tables present, no
`schema_migrations`) is
```

```
12        detected and its baseline row back-filled (no DDL re-run).
13      - **Forward migrations.** Entries in `MIGRATIONS` with ``version > baseline`` run in
order. (None
14        yet ? the first lands with the unified-schema work, feedbackJune2026 P3.)
15
16      Known limitation (single-user, reviewed migrations): a crash *between* applying a
migration's DDL
17      and recording it could require manual cleanup; per-migration transactions can be added
when the
18      first non-trivial migration lands.
19      """
20
21      from __future__ import annotations
22
23      import hashlib
24      import sqlite3
25      from dataclasses import dataclass
26
27      from muneem.errors import MigrationError
28
29      _SCHEMA_MIGRATIONS = """
30      CREATE TABLE IF NOT EXISTS schema_migrations (
31          version    INTEGER PRIMARY KEY,
32          name       TEXT NOT NULL,
33          checksum   TEXT NOT NULL,
34          applied_at TEXT NOT NULL DEFAULT (datetime('now'))
35      );
36      """
37
38
39      @dataclass(frozen=True)
40      class Migration:
41          """One forward migration: a version, a name, and the DDL to apply (run via
executescript)."""
42
43          version: int
44          name: str
45          sql: str
46
47          @property
48          def checksum(self) -> str:
49              return hashlib.sha256(self.sql.encode("utf-8")).hexdigest()
50
51
52      # --- Forward migrations (version > baseline), applied in ascending order
----------------------
53
54      # v5: provenance/confidence as a FIRST-CLASS but SEPARATE table (DESIGN §8.7,
feedbackJune2026 P3).
55      # Keyed by document_id (1:1) so every txn row reaches its provenance by the existing FK
join ? no
56      # per-row redundancy. source_type readies it for non-PDF sources (aa/sms) we may add
later.
57      _M5_PROVENANCE = """
58      CREATE TABLE IF NOT EXISTS provenance (
59          id             INTEGER PRIMARY KEY,
60          document_id    INTEGER NOT NULL UNIQUE REFERENCES documents(id) ON DELETE CASCADE,
61          source_type    TEXT NOT NULL,
62          parser         TEXT,
63          parser_version TEXT,
64          mapping_layer  TEXT,
65          confidence     TEXT,
66          created_at     TEXT NOT NULL DEFAULT (datetime('now'))
67      );
68      """
```

```

    # v6: the SERVING LAYER's first piece (DESIGN §8.7) ? a derived, read-only view unioning the flat
    # per-issuer source-of-truth tables into one normalized query surface (bank, dates, narration,
    # debit/credit, balance, document_id). HDFC's withdrawal/deposit/closing_balance map onto
    # debit/credit/balance; credit-card rows derive debit/credit from the Dr/Cr type (no balance).
    # The flat tables stay the rollback-able truth; this view is recreatable at any time.
    _M6_SERVING_VIEW = """
    CREATE VIEW IF NOT EXISTS v_transactions AS
    SELECT 'axis' AS bank, document_id, account_number, txn_date, value_date, particulars,
           debit, credit, balance
    FROM axis_transactions
    UNION ALL
    SELECT 'au', document_id, account_number, txn_date, value_date, particulars,
           debit, credit, balance
    FROM au_transactions
    UNION ALL
    SELECT 'hdfc', document_id, account_number, txn_date, value_date, particulars,
           withdrawal, deposit, closing_balance
    FROM hdfc_transactions
    UNION ALL
    SELECT 'axis_cc', document_id, account_number, txn_date, NULL, particulars,
           CASE WHEN type = 'Dr' THEN amount END,
           CASE WHEN type = 'Cr' THEN amount END,
           NULL
    FROM axis_cc_transactions;
    """

    MIGRATIONS: list[Migration] = [
        Migration(version=5, name="provenance", sql=_M5_PROVENANCE),
        Migration(version=6, name="serving_view", sql=_M6_SERVING_VIEW),
    ]


    def _checksum(sql: str) -> str:
        return hashlib.sha256(sql.encode("utf-8")).hexdigest()


    def _user_tables(conn: sqlite3.Connection) -> list[str]:
        return [
            r[0]
            for r in conn.execute(
                "SELECT name FROM sqlite_master WHERE type='table' "
                "AND name NOT LIKE 'sqlite_%' AND name != 'schema_migrations'"
            )
        ]


    def _record(conn: sqlite3.Connection, version: int, name: str, checksum: str) -> None:
        conn.execute(
            "INSERT INTO schema_migrations (version, name, checksum) VALUES (?, ?, ?)",
            (version, name, checksum),
        )


    def run_migrations(
        conn: sqlite3.Connection,
        *,
        baseline_sql: str,
        baseline_version: int,
        migrations: list[Migration] | None = None,
    ) -> int:
```

```
130         """Bring ``conn``'s schema up to date. Returns the resulting ``user_version``.
131
132         Idempotent: re-running applies nothing already recorded. Raises
:class:`MigrationError` if the
133         frozen baseline's checksum no longer matches what was recorded (i.e. someone edited
the baseline
134         instead of adding a migration).
135         """
136         migrations = MIGRATIONS if migrations is None else migrations
137         conn.executescript(_SCHEMA_MIGRATIONS)  # bookkeeping table first (idempotent)
138         applied = {r[0] for r in conn.execute("SELECT version FROM schema_migrations")}
139         current = conn.execute("PRAGMA user_version").fetchone()[0]
140         baseline_checksum = _checksum(baseline_sql)
141
142         if baseline_version in applied:
143             recorded = conn.execute(
144                 "SELECT checksum FROM schema_migrations WHERE version = ?",
(baseline_version,)
145             ).fetchone()
146             if recorded and recorded[0] != baseline_checksum:
147                 raise MigrationError(
148                     "baseline schema changed without a migration ? add a migration instead
of editing "
149                     "the frozen baseline (feedbackJune2026 P6)"
150                 )
151         else:
152             # Fresh DB ? create the baseline; legacy DB (tables already present) ? just
record it.
153             if not _user_tables(conn):
154                 conn.executescript(baseline_sql)
155             _record(conn, baseline_version, "baseline", baseline_checksum)
156             if current < baseline_version:
157                 conn.execute(f"PRAGMA user_version = {baseline_version}")
158                 current = baseline_version
159
160         for migration in sorted(migrations, key=lambda m: m.version):
161             if migration.version <= baseline_version or migration.version in applied:
162                 continue
163             conn.executescript(migration.sql)
164             _record(conn, migration.version, migration.name, migration.checksum)
165             conn.execute(f"PRAGMA user_version = {migration.version}")
166             current = migration.version
167
168         return int(current)
169
```

312. user (2026-06-10T17:48:07.975Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\migrations.py has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

313. assistant (2026-06-10T17:47:41.477Z)

```
(The ci.yml/DEPENDENCIES.md notes reflect the branch switch ? the C2 changes live on the PR
branch.) Now hardening `migrations.py`:
```

314. user (2026-06-10T17:47:45.727Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\migrations.py has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

315. user (2026-06-10T17:48:17.435Z)

```
1       """The hand-rolled migration runner: baseline apply, idempotency, legacy back-fill,
```

```
forward
2       migrations, baseline-drift detection, and running inside an encrypted connection.
3       """
4
5       import pytest
6
7       from muneem import db as dbmod
8       from muneem import migrations
9       from muneem.errors import MigrationError
10
11      _BASE = dbmod._SCHEMA
12      _VER = dbmod.SCHEMA_VERSION  # 4
13
14
15      def _applied(conn):
16          return {r[0]: r[1] for r in conn.execute("SELECT version, name FROM
schema_migrations")}
17
18
19      def test_fresh_db_applies_baseline():
20          with dbmod.connect(":memory:") as conn:
21              # Isolate baseline behaviour from the real forward migrations (migrations=[]).
22              v = migrations.run_migrations(
23                  conn, baseline_sql=_BASE, baseline_version=_VER, migrations=[]
24              )
25              assert v == _VER
26              assert conn.execute("PRAGMA user_version").fetchone()[0] == _VER
27              assert _VER in _applied(conn) and _applied(conn)[_VER] == "baseline"
28              names = {r[0] for r in conn.execute("SELECT name FROM sqlite_master WHERE
type='table'")}
29              assert {"documents", "offers", "schema_migrations"} <= names
30
31
32      def test_default_path_applies_all_forward_migrations():
33          # init_db uses the real MIGRATIONS list ? baseline (4) + provenance (5) + serving
view (6).
34          with dbmod.connect(":memory:") as conn:
35              dbmod.init_db(conn)
36              assert conn.execute("PRAGMA user_version").fetchone()[0] == 6
37              applied = _applied(conn)
38              assert applied.get(5) == "provenance"
39              assert applied.get(6) == "serving_view"
40              names = {r[0] for r in conn.execute("SELECT name FROM sqlite_master WHERE
type='table'")}
41              assert "provenance" in names
42              views = {r[0] for r in conn.execute("SELECT name FROM sqlite_master WHERE
type='view'")}
43              assert "v_transactions" in views
44
45
46      def test_idempotent_rerun_changes_nothing():
47          with dbmod.connect(":memory:") as conn:
48              migrations.run_migrations(conn, baseline_sql=_BASE, baseline_version=_VER)
49              before = _applied(conn)
50              migrations.run_migrations(conn, baseline_sql=_BASE, baseline_version=_VER)
51              assert _applied(conn) == before
52
53
54      def test_legacy_db_is_backfilled_without_re_running_ddl():
55          with dbmod.connect(":memory:") as conn:
56              # Pre-framework init: schema present + user_version set, but no
schema_migrations table.
57              conn.executescript(_BASE)
58              conn.execute(f"PRAGMA user_version = {_VER}")
59              conn.execute("INSERT INTO documents (source, sha256) VALUES ('file', 'abc')")
```

```
60              migrations.run_migrations(conn, baseline_sql=_BASE, baseline_version=_VER)
61              assert _VER in _applied(conn)  # baseline recorded
62              assert conn.execute("SELECT count(*) FROM documents").fetchone()[0] == 1  # data
survived
63
64
65      def test_forward_migration_applies_once_and_bumps_version():
66          m = migrations.Migration(
67              version=_VER + 1, name="add_demo_col", sql="ALTER TABLE documents ADD COLUMN
demo TEXT"
68          )
69          with dbmod.connect(":memory:") as conn:
70              v = migrations.run_migrations(
71                  conn, baseline_sql=_BASE, baseline_version=_VER, migrations=[m]
72              )
73              assert v == _VER + 1
74              assert conn.execute("PRAGMA user_version").fetchone()[0] == _VER + 1
75              assert "demo" in {r[1] for r in conn.execute("PRAGMA table_info(documents)")}
76              # Re-run must NOT re-ALTER (that would raise "duplicate column").
77              v2 = migrations.run_migrations(
78                  conn, baseline_sql=_BASE, baseline_version=_VER, migrations=[m]
79              )
80              assert v2 == _VER + 1
81
82
83      def test_baseline_drift_is_detected():
84          with dbmod.connect(":memory:") as conn:
85              migrations.run_migrations(conn, baseline_sql=_BASE, baseline_version=_VER)
86              with pytest.raises(MigrationError):  # baseline edited without a migration
87                  migrations.run_migrations(
88                      conn, baseline_sql=_BASE + "\n-- edited\n", baseline_version=_VER
89                  )
90
91
92      def test_runs_inside_an_encrypted_connection(tmp_path):
93          pytest.importorskip("sqlcipher3")
94          key = "00112233445566778899aabbccddeeff00112233445566778899aabbccddeeff"
95          with dbmod.connect(tmp_path / "led.db", key=key) as conn:
96              dbmod.init_db(conn)  # the production path through the runner, encrypted
97              assert _VER in {r[0] for r in conn.execute("SELECT version FROM
schema_migrations")}
98
```

316. user (2026-06-10T17:48:36.459Z)

```
The file C:\Users\avidu\Projects\muneem\tests\test_migrations.py has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

317. user (2026-06-10T17:48:48.129Z)

```
.................................................................... [ 86%]
.........................................................            [100%]
497 passed, 21 deselected in 4.70s
```

318. user (2026-06-10T17:49:24.480Z)

```
All checks passed!
81 files left unchanged
Success: no issues found in 34 source files
[fix/migration-atomicity e9146c5] fix(db): atomic migrations + frozen-migration checksum re-
verification
 2 files changed, 146 insertions(+), 23 deletions(-)
 * [new branch]      fix/migration-atomicity -> fix/migration-atomicity
https://github.com/avidullu/muneem/pull/63
```

```
From https://github.com/avidullu/muneem
 * branch            main       -> FETCH_HEAD
   fd9abff..a7b073e  main       -> origin/main
Updating fd9abff..a7b073e
Fast-forward
 src/muneem/migrations.py | 107 ++++++++++++++++++++++++++++++++++++++++----------
 tests/test_migrations.py |  62 ++++++++++++++++++++++++++
 2 files changed, 146 insertions(+), 23 deletions(-)
Your branch is up to date with 'origin/main'.
Already on 'main'
Already up to date.
```

319. user (2026-06-10T17:49:33.474Z)

```
test (ubuntu-latest, 3.11)      pass    34s
https://github.com/avidullu/muneem/actions/runs/27294928817/job/80625004894
test (ubuntu-latest, 3.11)      pass    36s
https://github.com/avidullu/muneem/actions/runs/27294930684/job/80625008641
test (ubuntu-latest, 3.12)      pass    47s
https://github.com/avidullu/muneem/actions/runs/27294928817/job/80625004985
test (ubuntu-latest, 3.12)      pass    46s
https://github.com/avidullu/muneem/actions/runs/27294930684/job/80625008765
test (ubuntu-latest, 3.13)      pass    39s
https://github.com/avidullu/muneem/actions/runs/27294928817/job/80625004784
test (ubuntu-latest, 3.13)      pass    45s
https://github.com/avidullu/muneem/actions/runs/27294930684/job/80625008776
test (windows-latest, 3.13)     pass    1m56s
https://github.com/avidullu/muneem/actions/runs/27294928817/job/80625004825
test (windows-latest, 3.13)     pass    1m29s
https://github.com/avidullu/muneem/actions/runs/27294930684/job/80625008893
ocr     skipping        0
https://github.com/avidullu/muneem/actions/runs/27294928817/job/80625005305
ocr     skipping        0
https://github.com/avidullu/muneem/actions/runs/27294930684/job/80625009578
```

320. assistant (2026-06-10T17:49:40.790Z)

C2's full matrix is green against the pins. Merging:

321. user (2026-06-10T17:49:48.405Z)

```
Your branch is up to date with 'origin/main'.
Already on 'main'
 create mode 100644 constraints.txt
```

322. user (2026-06-10T17:49:54.720Z)

```
Updated task #13 status
```

323. user (2026-06-10T17:49:55.196Z)

```
Updated task #14 status
```

324. user (2026-06-10T17:49:55.670Z)

```
Updated task #15 status
```

325. user (2026-06-10T17:49:57.766Z)

```
a84690d chore(deps): pin dependency versions via a pip constraints file (#62)
a7b073e fix(db): atomic migrations + frozen-migration checksum re-verification (#63)
fd9abff chore(security): remove the real-data snapshot footgun; block client_secret patterns
everywhere (#61)
11ebb82 feat(parsers): canonicalize statement dates to ISO-8601 at the build boundary (#60)
```

```
Switched to a new branch 'fix/small-hardening-bundle'
```

326. assistant (2026-06-10T17:50:18.977Z)

```
C4 ? the small-hardening bundle. Reading the four target files:
```

327. user (2026-06-10T17:50:19.452Z)

```
1       """Redaction helpers ? keep sensitive data out of logs, exceptions, and stdout.
2
3       muneem aggregates the user's whole financial life, so a balance, account number, or
password that
4       slips into a log line or an exception message is a real incident. These helpers produce
*safe*
5       representations for diagnostics. They are deliberately conservative: it is better to
over-mask a
6       diagnostic string than to leak one.
7       """
8
9       from __future__ import annotations
10
11      import re
12      from pathlib import Path
13
14      # Long digit runs look like account / card / customer numbers.
15      _ACCOUNT_RE = re.compile(r"\d{9,}")
16      # Money-like: Indian or Western grouping with a decimal, or a plain N.NN amount.
17      _AMOUNT_RE = re.compile(r"\d{1,3}(?:,\d{2,3})+(?:\.\d{1,2})?|\d+\.\d{2}")
18      # Digit runs in a filename that may be PII (DOB, account suffix, customer id).
19      _FILENAME_DIGITS_RE = re.compile(r"\d{4,}")
20
21
22      def redact_filename(path: str | Path) -> str:
23          """Return a file's *name only* (never the full path), with long digit runs masked.
24
25          The full path can expose the user's home/account layout, and a bare basename can
still carry PII
26          (Indian statement names are often ``Name_DDMMYYYY.pdf``), so digit runs of 4+ are
masked. Keeps
27          enough of the name to be useful in a diagnostic.
28          """
29          name = Path(str(path)).name
30          return _FILENAME_DIGITS_RE.sub(lambda m: "#" * len(m.group()), name)
31
32
33      def redact_text(text: str) -> str:
34          """Mask account-number-like and money-like substrings in free text (e.g. before
logging).
35
36          A backstop for accidental leaks ? code should already pass sanitised context; this
catches what
37          slips through. Returns the input unchanged if there is nothing to mask.
38          """
39          if not text:
40              return text
41          masked = _AMOUNT_RE.sub("?amt?", text)
42          masked = _ACCOUNT_RE.sub("?acct?", masked)
43          return masked
44
45
46      def mask_secret(secret: str | None) -> str:
47          """A non-reversible placeholder for a password/key ? its length only, never the
value."""
48          if not secret:
49              return "?none?"
```

```
        return f"?secret:{len(secret)} chars?"

```

328. user (2026-06-10T17:50:19.932Z)

```
    """Decrypt password-protected statement PDFs (Stage 2).

    Uses pypdf (BSD-3-Clause) ? permissive, pure-Python. Implements the hybrid
    password strategy from DESIGN.md § 8.2: try a candidate dictionary first, then
    fall back to an interactive prompt. The working password is returned so the
    caller can cache it in the OS keychain (muneem.secrets); **it is never logged**.
    """

    from __future__ import annotations

    import shutil
    from collections.abc import Callable, Iterable
    from dataclasses import dataclass
    from pathlib import Path

    import pypdf


    @dataclass
    class UnlockResult:
        ok: bool
        method: str | None  # "plaintext" | "candidate" | "prompt"
        password: str | None  # working password ? do NOT log; None if plaintext/failed
        output_path: Path | None  # decrypted copy, if a destination was given


    def is_encrypted(path: Path | str) -> bool:
        return pypdf.PdfReader(str(path)).is_encrypted


    def password_works(path: Path | str, password: str) -> bool:
        reader = pypdf.PdfReader(str(path))
        if not reader.is_encrypted:
            return True
        try:
            return reader.decrypt(password) != pypdf.PasswordType.NOT_DECRYPTED
        except Exception:
            return False


    def write_decrypted(path: Path | str, password: str | None, dest: Path | str) -> Path:
        """Write a decrypted copy of the PDF to dest (password may be '' for plaintext)."""
        src = Path(path)
        dest = Path(dest)
        dest.parent.mkdir(parents=True, exist_ok=True)
        reader = pypdf.PdfReader(str(src))
        if reader.is_encrypted:
            reader.decrypt(password or "")
        writer = pypdf.PdfWriter()
        writer.append(reader)
        with open(dest, "wb") as fh:
            writer.write(fh)
        return dest


    def unlock(
        path: Path | str,
        candidates: Iterable[str] = (),
        prompt_fn: Callable[[], str | None] | None = None,
        max_prompts: int = 3,
```

```
        dest: Path | str | None = None,
    ) -> UnlockResult:
        """Hybrid unlock: plaintext? -> candidate dictionary -> interactive prompt.

        If dest is given and unlock succeeds, a decrypted copy is written there. The
        working password is returned in the result but must never be logged.
        """
        src = Path(path)
        if not is_encrypted(src):
            out = Path(shutil.copyfile(src, dest)) if dest else None
            return UnlockResult(True, "plaintext", None, out)

        tried: set[str] = set()
        for pw in candidates:
            if pw in tried:
                continue
            tried.add(pw)
            if password_works(src, pw):
                out = write_decrypted(src, pw, dest) if dest else None
                return UnlockResult(True, "candidate", pw, out)

        if prompt_fn is not None:
            for _ in range(max_prompts):
                entered = prompt_fn()
                if not entered:
                    break
                if password_works(src, entered):
                    out = write_decrypted(src, entered, dest) if dest else None
                    return UnlockResult(True, "prompt", entered, out)

        return UnlockResult(False, None, None, None)

```

329. user (2026-06-10T17:50:20.866Z)

```
    """Disaster recovery for the encrypted ledger ? a passphrase-wrapped recovery file.

    The DB key lives in the OS keychain (fast, full-entropy). If that keychain is ever lost
? OS
    reinstall, new machine, profile/Credential-Manager corruption ? the ledger **and every
same-key
    backup** become permanently unrecoverable. This module lets the user wrap that key under
a
    passphrase they choose, into a recovery file they back up **off-machine**; with the file
*and* the
    passphrase
    they can restore the key into a fresh keychain.

    Crypto: **Argon2id** (memory-hard KDF) derives a 256-bit wrapping key from the
passphrase + a random
    salt; **AES-256-GCM** (authenticated) wraps the DB key. The recovery file holds only
    ``{version, kdf params, salt, nonce, ciphertext}`` ? never the key, never the
passphrase. A wrong
    passphrase fails the GCM authentication tag ? a clean error, with no decryption oracle.
    """

    from __future__ import annotations

    import base64
    import json
    import os
    from pathlib import Path
    from typing import Any

    from cryptography.exceptions import InvalidTag
```

```
    from cryptography.hazmat.primitives.ciphers.aead import AESGCM
    from cryptography.hazmat.primitives.kdf.argon2 import Argon2id

    from muneem.errors import MuneemError

    RECOVERY_VERSION = 1
    _AAD = b"muneem-recovery-v1"  # binds the ciphertext to this format/version

    # Argon2id parameters (memory in KiB). Conservative-but-real; the file stores them so
they can be
    # raised over time without breaking older recovery files.
    _TIME_COST = 3
    _MEMORY_KIB = 64 * 1024  # 64 MiB
    _LANES = 4
    _SALT_BYTES = 16
    _NONCE_BYTES = 12
    _KEY_BYTES = 32
    _MIN_PASSPHRASE = 8


    class RecoveryError(MuneemError):
        """A recovery file could not be created, or could not be opened (wrong passphrase /
corrupt)."""


    def _derive(passphrase: str, salt: bytes, *, time_cost: int, memory_kib: int, lanes:
int) -> bytes:
        kdf = Argon2id(
            salt=salt,
            length=_KEY_BYTES,
            iterations=time_cost,
            lanes=lanes,
            memory_cost=memory_kib,
        )
        return kdf.derive(passphrase.encode("utf-8"))


    def wrap_key(db_key: str, passphrase: str) -> dict[str, Any]:
        """Return a JSON-serialisable recovery payload wrapping ``db_key`` under
``passphrase``."""
        if not passphrase or len(passphrase) < _MIN_PASSPHRASE:
            raise RecoveryError(f"recovery passphrase must be at least {_MIN_PASSPHRASE}
characters")
        salt = os.urandom(_SALT_BYTES)
        nonce = os.urandom(_NONCE_BYTES)
        key = _derive(passphrase, salt, time_cost=_TIME_COST, memory_kib=_MEMORY_KIB,
lanes=_LANES)
        ciphertext = AESGCM(key).encrypt(nonce, db_key.encode("utf-8"), _AAD)
        return {
            "version": RECOVERY_VERSION,
            "kdf": {
                "algo": "argon2id",
                "time_cost": _TIME_COST,
                "memory_kib": _MEMORY_KIB,
                "lanes": _LANES,
            },
            "salt": base64.b64encode(salt).decode(),
            "nonce": base64.b64encode(nonce).decode(),
            "ciphertext": base64.b64encode(ciphertext).decode(),
        }


    def unwrap_key(payload: dict[str, Any], passphrase: str) -> str:
        """Recover the DB key from a recovery ``payload`` + ``passphrase``. Raises
RecoveryError."""
```

```
83          try:
84              salt = base64.b64decode(payload["salt"])
85              nonce = base64.b64decode(payload["nonce"])
86              ciphertext = base64.b64decode(payload["ciphertext"])
87              kdf = payload.get("kdf", {})
88          except (KeyError, ValueError, TypeError) as exc:
89              raise RecoveryError("recovery file is malformed") from exc
90          key = _derive(
91              passphrase,
92              salt,
93              time_cost=int(kdf.get("time_cost", _TIME_COST)),
94              memory_kib=int(kdf.get("memory_kib", _MEMORY_KIB)),
95              lanes=int(kdf.get("lanes", _LANES)),
96          )
97          try:
98              plain = AESGCM(key).decrypt(nonce, ciphertext, _AAD)
99          except InvalidTag as exc:
100             raise RecoveryError("wrong passphrase or corrupt recovery file") from exc
101         return plain.decode("utf-8")
102
103
104     def write_recovery_file(path: Path | str, db_key: str, passphrase: str) -> Path:
105         """Wrap ``db_key`` under ``passphrase`` and write the recovery file (owner-only
perms)."""
106         payload = wrap_key(db_key, passphrase)
107         out = Path(path)
108         out.parent.mkdir(parents=True, exist_ok=True)
109         out.write_text(json.dumps(payload, indent=2), encoding="utf-8")
110         try:
111             os.chmod(out, 0o600)
112         except OSError:  # pragma: no cover - best-effort on platforms without POSIX perms
113             pass
114         return out
115
116
117     def read_recovery_file(path: Path | str, passphrase: str) -> str:
118         """Read a recovery file and return the unwrapped DB key. Raises RecoveryError on any
failure."""
119         p = Path(path)
120         try:
121             payload = json.loads(p.read_text(encoding="utf-8"))
122         except (OSError, ValueError) as exc:
123             raise RecoveryError(f"could not read recovery file {p.name}") from exc
124         return unwrap_key(payload, passphrase)
125
```

330. user (2026-06-10T17:50:21.699Z)

```
No matches found
```

331. user (2026-06-10T17:50:57.243Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\db.py has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

332. user (2026-06-10T17:51:03.650Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\db.py has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

333. user (2026-06-10T17:51:06.045Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\db.py has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

334. user (2026-06-10T17:51:08.877Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\db.py has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

335. user (2026-06-10T17:51:11.581Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\db.py has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

336. user (2026-06-10T17:51:23.919Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\redaction.py has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

337. user (2026-06-10T17:51:26.274Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\redaction.py has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

338. user (2026-06-10T17:51:29.138Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\unlock.py has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

339. user (2026-06-10T17:51:34.452Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\recovery.py has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

340. assistant (2026-06-10T17:51:41.322Z)

Now the tests for the bundle, including the logger-discipline guard:

341. user (2026-06-10T17:51:49.769Z)

```
The file C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

342. user (2026-06-10T17:51:54.939Z)

```
<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>
```

343. user (2026-06-10T17:52:03.084Z)

```
1       """Redaction helpers keep sensitive data out of logs / exceptions / stdout."""
2
3       from pathlib import Path
4
5       from muneem.redaction import mask_secret, redact_filename, redact_text
6
7
8       def test_redact_filename_drops_path_and_masks_pii_digits():
9           out = redact_filename(Path("/home/avi/statements/Avi_Dullu_28022026.pdf"))
10          assert "/" not in out and "\\" not in out  # no directory components
11          assert "statements" not in out and "home" not in out
12          assert "28022026" not in out  # DOB-like digit run masked
13          assert out.startswith("Avi_Dullu_") and out.endswith(".pdf")  # still recognisable
14          assert "#" in out
15
16
17      def test_redact_text_masks_account_and_amount():
18          assert "?acct?" in redact_text("balance for 123456789012 today")
```

```
19          assert "?amt?" in redact_text("debit 1,23,456.78 INR")  # Indian grouping
20          assert "?amt?" in redact_text("amount 1234.56")
21          assert "123456789012" not in redact_text("acct 123456789012")
22
23
24      def test_redact_text_leaves_clean_text_untouched():
25          assert redact_text("opened 6 files for 2 banks") == "opened 6 files for 2 banks"
26          assert redact_text("") == ""
27
28
29      def test_mask_secret_reveals_length_only():
30          assert mask_secret("hunter2") == "?secret:7 chars?"
31          assert mask_secret("") == "?none?"
32          assert mask_secret(None) == "?none?"
33
```

344. user (2026-06-10T17:52:15.633Z)

```
The file C:\Users\avidu\Projects\muneem\tests\test_redaction.py has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

345. user (2026-06-10T17:52:26.088Z)

```
17: def test_wrap_unwrap_roundtrip():
18: payload = recovery.wrap_key(_DBKEY, _PASS)
19: assert recovery.unwrap_key(payload, _PASS) == _DBKEY
22: def test_wrong_passphrase_is_rejected():
23: payload = recovery.wrap_key(_DBKEY, _PASS)
25: recovery.unwrap_key(payload, "a different passphrase")
28: def test_tampered_ciphertext_is_rejected():
29: payload = recovery.wrap_key(_DBKEY, _PASS)
30: payload["ciphertext"] = payload["ciphertext"][:-4] + "AAAA"  # flip the tail
32: recovery.unwrap_key(payload, _PASS)
35: def test_short_passphrase_refused():
37: recovery.wrap_key(_DBKEY, "short")
10: def _write_pdf(path: Path, password: str | None = None) -> Path:
11: writer = pypdf.PdfWriter()
12: writer.add_blank_page(width=200, height=200)
30: def test_is_encrypted(encrypted_pdf, plain_pdf):
35: def test_unlock_via_candidate(encrypted_pdf, tmp_path):
42: def test_unlock_via_prompt(encrypted_pdf):
47: def test_unlock_fails_and_leaks_no_password(encrypted_pdf):
52: def test_plaintext_is_copied(plain_pdf, tmp_path):
58: def test_real_hdfc_statements_decrypt():
```

346. user (2026-06-10T17:52:41.182Z)

```
Exit code 1
========================== short test summary info ==========================
FAILED tests/test_unlock.py::test_plaintext_copy_creates_parent_dirs - TypeEr...
1 failed, 508 passed, 21 deselected in 4.52s
```

347. user (2026-06-10T17:52:50.884Z)

```
Exit code 1
E       TypeError: 'module' object is not callable
tests\test_unlock.py:72: TypeError
```

348. user (2026-06-10T17:52:58.742Z)

```
import os
from pathlib import Path
import pypdf
import pytest
```

```
from muneem import unlock
```

349. user (2026-06-10T17:53:14.052Z)

```
.....                                                                    [100%]
509 passed, 21 deselected in 4.45s
Found 1 error.
3 files reformatted, 78 files left unchanged
Success: no issues found in 34 source files
```

350. user (2026-06-10T17:53:23.012Z)

```
All checks passed!
```

351. user (2026-06-10T17:54:07.086Z)

```
.....                                                                    [100%]
509 passed, 21 deselected in 4.34s
[fix/small-hardening-bundle 2349ba1] fix(security): small-hardening bundle across the
crypto/redaction seams
 8 files changed, 134 insertions(+), 15 deletions(-)
 * [new branch]      fix/small-hardening-bundle -> fix/small-hardening-bundle
https://github.com/avidullu/muneem/pull/64
From https://github.com/avidullu/muneem
 * branch            main       -> FETCH_HEAD
   a84690d..fffbdc7  main       -> origin/main
Updating a84690d..fffbdc7
Fast-forward
 src/muneem/db.py            | 36 +++++++++++++++++++++++++++------
 src/muneem/recovery.py      | 13 ++++++------
 src/muneem/redaction.py     |  9 ++++++++-
 src/muneem/unlock.py        |  5 ++++-
 tests/test_db_crypto_ops.py | 40 ++++++++++++++++++++++++++++++++++++++
 tests/test_recovery.py      |  7 +++++++
 tests/test_redaction.py     | 32 ++++++++++++++++++++++++++++++
 tests/test_unlock.py        |  7 +++++++
 8 files changed, 134 insertions(+), 15 deletions(-)
Your branch is up to date with 'origin/main'.
Already on 'main'
Already up to date.
```

352. user (2026-06-10T17:54:20.270Z)

```
Updated task #15 status
```

353. user (2026-06-10T17:54:20.745Z)

```
Updated task #17 status
```

354. user (2026-06-10T17:54:21.941Z)

```
tests\test_db_crypto_ops.py:34:@pytest.mark.parametrize(
tests\test_precommit_hook.py:29:@pytest.mark.parametrize(
tests\test_precommit_hook.py:63:@pytest.mark.parametrize(
tests\test_promo_regression.py:26:@pytest.mark.parametrize("name", DETERMINISTIC)
tests\parsers\test_hdfc.py:131:@pytest.mark.parametrize(
tests\parsers\test_hdfc.py:218:@pytest.mark.parametrize(
tests\parsers\test_hdfc.py:221:    ids=lambda p: p.name,
tests\parsers\test_local_axis.py:25:@pytest.mark.parametrize("pdf_path", AXIS_PDFS, ids=lambda
p: p.name)
tests\parsers\test_parse_errors.py:32:@pytest.mark.parametrize("patch_target, parser_cls",
CASES)
tests\parsers\test_local_au.py:23:@pytest.mark.parametrize("pdf_path", AU_PDFS, ids=lambda p:
p.name)
```

```
tests\test_verifier_properties.py:36:@pytest.mark.parametrize("seed", range(60))
tests\test_verifier_properties.py:45:@pytest.mark.parametrize("seed", range(60))
tests\test_verifier_properties.py:62:@pytest.mark.parametrize("seed", range(60))
tests\test_verifier_properties.py:71:@pytest.mark.parametrize("seed", range(60))
```

355. user (2026-06-10T17:54:34.518Z)

```
Switched to a new branch 'chore/ocr-cron-and-test-ids'
test_local_axis.py:25: @pytest.mark.parametrize("pdf_path", AXIS_PDFS, ids=lambda p:
redact_filename(p),)
test_local_au.py:23: @pytest.mark.parametrize("pdf_path", AU_PDFS, ids=lambda p:
redact_filename(p),)
test_hdfc.py:221: ids=lambda p: redact_filename(p),
```

356. user (2026-06-10T17:54:44.203Z)

```
test_local_axis.py:20
test_local_au.py:18
test_hdfc.py:16
```

357. user (2026-06-10T17:54:53.368Z)

```
<tool_use_error>File has been modified since read, either by the user or by a linter. Read it
again before attempting to write it.</tool_use_error>
```

358. user (2026-06-10T17:54:55.499Z)

```
<tool_use_error>File has been modified since read, either by the user or by a linter. Read it
again before attempting to write it.</tool_use_error>
```

359. user (2026-06-10T17:55:01.400Z)

```
1       name: ci
2
3       on:
4         # All-branch pushes so feature branches are gated even before a PR exists
5         # (the regression fixtures in tests/test_regression_synth.py run here, secret-free).
6         push:
7         pull_request:
8         workflow_dispatch:
9
10      jobs:
11        test:
12          runs-on: ${{ matrix.os }}
13          strategy:
14            fail-fast: false
15            matrix:
```

360. user (2026-06-10T17:55:11.136Z)

```
The file C:\Users\avidu\Projects\muneem\.github\workflows\ci.yml has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

361. user (2026-06-10T17:55:16.601Z)

```
47              run: pytest --cov=muneem --cov-report=term-missing --cov-fail-under=80
48
49        ocr:
50          # OCR tier needs the Tesseract binary; runs on demand (Linux) to avoid per-PR
flakiness.
51          if: github.event_name == 'workflow_dispatch'
52          runs-on: ubuntu-latest
53          steps:
54            - uses: actions/checkout@v6
```

```
55            - uses: actions/setup-python@v6
56              with:
57                python-version: "3.13"
58                cache: pip
```

362. user (2026-06-10T17:55:24.358Z)

```
The file C:\Users\avidu\Projects\muneem\.github\workflows\ci.yml has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

363. user (2026-06-10T17:55:41.113Z)

```
.....                                                                    [100%]
509 passed, 21 deselected in 4.18s
tests/parsers/test_hdfc.py::test_real_hdfc_statement_validates[Avi_Dullu_########_#########.pdf0
]
tests/parsers/test_hdfc.py::test_real_hdfc_statement_validates[Avi_Dullu_########_#########.pdf1
]
tests/parsers/test_hdfc.py::test_real_hdfc_statement_validates[Avi_Dullu_########_#########.pdf2
]
tests/parsers/test_hdfc.py::test_real_hdfc_statement_validates[Avi_Dullu_########_#########.pdf3
]
tests/parsers/test_hdfc.py::test_real_hdfc_statement_validates[Avi_Dullu_########_#########.pdf4
]
tests/parsers/test_hdfc.py::test_real_hdfc_statement_validates[Avi_Dullu_########_#########.pdf5
]
All checks passed!
2 files reformatted, 79 files left unchanged
```

364. user (2026-06-10T17:56:13.343Z)

```
[chore/ocr-cron-and-test-ids e6ab2fc] chore(ci): weekly OCR-tier schedule; redact real filenames
from pytest ids
 4 files changed, 21 insertions(+), 5 deletions(-)
 * [new branch]      chore/ocr-cron-and-test-ids -> chore/ocr-cron-and-test-ids
https://github.com/avidullu/muneem/pull/65
From https://github.com/avidullu/muneem
 * branch            main       -> FETCH_HEAD
   fffbdc7..e7aca66  main       -> origin/main
Updating fffbdc7..e7aca66
Fast-forward
 .github/workflows/ci.yml         | 9 +++++++--
 tests/parsers/test_hdfc.py       | 3 ++-
 tests/parsers/test_local_au.py   | 7 ++++++-
 tests/parsers/test_local_axis.py | 7 ++++++-
 4 files changed, 21 insertions(+), 5 deletions(-)
Your branch is up to date with 'origin/main'.
Already on 'main'
Already up to date.
```

365. assistant (2026-06-10T17:56:27.265Z)

```
D2 merged as [#65](https://github.com/avidullu/muneem/pull/65). E2 next ? the prescribed
recovery runbook:
```

366. user (2026-06-10T17:56:27.743Z)

```
Updated task #17 status
```

367. user (2026-06-10T17:56:28.243Z)

```
Updated task #19 status
```

368. user (2026-06-10T17:56:29.759Z)

```
Switched to a new branch 'docs/recovery-runbook'
```

369. user (2026-06-10T17:57:03.070Z)

```
File created successfully at: C:\Users\avidu\Projects\muneem\docs\recovery.md (file state is
current in your context ? no need to Read it back)
```

370. assistant (2026-06-10T17:57:03.045Z)

Recovery runbook ? keychain loss, new machine, backups, migrations

> The runbook feedbackJune2026 §2 and P6 prescribed. Every command here exists today; nothing is
> aspirational. Background: [encryption-at-rest.md](./encryption-at-rest.md). Threat recap: the
> 256-bit ledger key lives **only** in the OS keychain, and `db backup` encrypts backups with
the
> **same** key ? so a lost keychain without a recovery file means the ledger *and every backup*
> are permanently unreadable. The recovery file is what breaks that single point of failure.

0. One-time setup (do this before anything else matters)

```text
muneem db setup-recovery            # prompts for a recovery passphrase (min 8 chars; use more)
```

Writes `<data dir>/muneem-recovery.json` (see `muneem info` for the data dir) ? a
passphrase-wrapped copy of the DB key (Argon2id ? AES-256-GCM). It is useless without the
passphrase, and the passphrase is useless without the file.

- **Store the file OFF this machine**: password manager attachment, a separate drive, printed.
- **Remember the passphrase** (password manager). You need **both** to recover.
- The repo guards (`.gitignore`, pre-commit hook) block `*recovery*.json` from ever being
  committed ? keep it out of the repo directory anyway.

Re-run `setup-recovery` **after every key rotation** ? a rotation mints a new key, so the old
recovery file wraps a key that no longer opens the ledger. (`db rotate-key` reminds you.)

1. "The keychain is gone" (OS reinstall, profile corruption, Credential Manager wiped)

Symptoms: `muneem db status` says `key: not yet minted` while the ledger file exists; opening
fails with *"?its key is not in the OS keychain. Restore it with 'muneem db recover'?"* (muneem
deliberately refuses to mint a fresh key over an existing ledger).

```text
muneem db recover <path-to-muneem-recovery.json>   # prompts for the recovery passphrase
muneem db status                                   # key: present in keychain
muneem txns list all                               # data readable again
```

`db recover` verifies the unwrapped key actually opens the current ledger **before** writing it
to the keychain ? a stale recovery file (pre-rotation) fails cleanly instead of clobbering
anything.

2. Moving to a new machine

1. Install muneem; run `muneem info` to find the data dir.
2. Copy the ledger file (and any backups) from the old machine into place.
3. `muneem db recover <recovery-file>` (as in §1).
4. Verify: `muneem db status` ? encrypted + key present; `muneem txns list all` shows rows.
5. Create a fresh backup on the new machine: `muneem db backup <dest>`.

If the old machine still works, an alternative to the recovery file is rotating *into* the move:
back up, copy, then `setup-recovery` again on the new machine.

3. Verifying a backup (do this when you take one, not when you need one)

Backups are encrypted with the **current** ledger key, so they open through the same CLI:

```text
muneem db backup D:\backups\muneem-2026-06.db     # consistent online snapshot, refuses the live
                                                  # ledger path, atomic replace of a prior
backup
muneem db status --db D:\backups\muneem-2026-06.db    # state: encrypted (SQLCipher)
muneem txns list all --db D:\backups\muneem-2026-06.db --limit 5   # rows actually readable
```

A backup taken **before** a key rotation no longer opens with the current key ? after rotating, re-run `db backup` (the CLI reminds you) and re-verify.

4. Interrupted key rotation

`db rotate-key` stages the new key under a *pending* keychain entry before touching the file, so an interruption is recoverable at every point:

- `muneem db status` shows `pending: a staged rotation key is present?` after an interrupted run.
- The **next ledger open** (any command) reconciles it automatically: if the file was already rekeyed, the pending key is promoted; if not, the stale staging entry is discarded.
- If a rotation failed and the message said the ledger opens with **neither** key: do not touch the keychain entries (`db-key`, `db-key-pending` ? both were deliberately kept); restore the ledger file from a backup, then `db status` to confirm which key it opens under.

5. After a schema migration ("migrate then re-verify", P6)

Migrations run atomically inside the encrypted connection on first open after an upgrade ? the ledger is always fully before or fully after a migration. After upgrading muneem:

1. `muneem db backup <dest>` **before** the first post-upgrade command, if you want a pre-migration
   snapshot to roll back to (the flat per-issuer tables are the rollback-able source of truth).
2. Run any command (e.g. `muneem db status`) ? migrations apply; a checksum mismatch or mid-script
   failure aborts loudly and rolls back rather than half-migrating.
3. Re-verify the data surface: `muneem txns list all --limit 10` and, per issuer,
   `muneem txns list hdfc` etc. ? listings tag anything not cleanly `reconciled`.
4. A deeper re-verification command (`muneem verify`: re-run reconciliation over *stored* rows) is planned (feedbackJune2026 P9) but does not exist yet ? until it lands, re-ingesting a recent statement (`muneem ingest ? --doc-type bank_statement`) and seeing it reconcile is the practical end-to-end check.

The one rule

A recovery file + passphrase stored off-machine turns "keychain lost" from a catastrophe into a five-minute fix. Without them there is no fix. Set it up now (§0), refresh it after every rotation, and verify backups when you take them.

371. user (2026-06-10T17:57:09.625Z)

52:### Disaster recovery ? the key-loss problem (solved)

372. user (2026-06-10T17:57:16.160Z)

The file C:\Users\avidu\Projects\muneem\docs\encryption-at-rest.md has been updated successfully. (file state is current in your context ? no need to Read it back)

373. user (2026-06-10T17:57:49.714Z)

```
38 passed in 0.33s
[docs/recovery-runbook cfca001] docs: recovery runbook (keychain lost / new machine / verify
backups / interrupted rotation / migrate-then-reverify)
 2 files changed, 103 insertions(+)
 create mode 100644 docs/recovery.md
 * [new branch]      docs/recovery-runbook -> docs/recovery-runbook
https://github.com/avidullu/muneem/pull/66
From https://github.com/avidullu/muneem
 * branch            main       -> FETCH_HEAD
   e7aca66..87273c5  main       -> origin/main
Updating e7aca66..87273c5
Fast-forward
 docs/encryption-at-rest.md |   3 ++
 docs/recovery.md           | 100 ++++++++++++++++++++++++++++++++++++++++++++++
 2 files changed, 103 insertions(+)
 create mode 100644 docs/recovery.md
Your branch is up to date with 'origin/main'.
Already on 'main'
Already up to date.
```

374. user (2026-06-10T17:58:37.231Z)

```
Updated task #19 status
```

375. user (2026-06-10T17:59:15.424Z)

```
Async agent launched successfully.
agentId: a0a70ddb1d7197b83 (internal ID - do not mention to user. Use SendMessage with to:
'a0a70ddb1d7197b83' to continue this agent.)
The agent is working in the background. You will be notified automatically when it completes.
Do not duplicate this agent's work ? avoid working with the same files or topics it is using.
Work on non-overlapping tasks, or briefly tell the user what you launched and end your response.
output_file: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-muneem\d1586dfe-
adae-4d40-a85d-ed0bb23738b8\tasks\a0a70ddb1d7197b83.output
Do NOT Read or tail this file via the shell tool ? it is the full subagent JSONL transcript and
reading it will overflow your context. If the user asks for progress, say the agent is still
running; you'll get a completion notification.
```

376. user (2026-06-10T17:59:40.687Z)

```
Async agent launched successfully.
agentId: afa9c802e3657ad51 (internal ID - do not mention to user. Use SendMessage with to:
'afa9c802e3657ad51' to continue this agent.)
The agent is working in the background. You will be notified automatically when it completes.
Do not duplicate this agent's work ? avoid working with the same files or topics it is using.
Work on non-overlapping tasks, or briefly tell the user what you launched and end your response.
output_file: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-muneem\d1586dfe-
adae-4d40-a85d-ed0bb23738b8\tasks\afa9c802e3657ad51.output
Do NOT Read or tail this file via the shell tool ? it is the full subagent JSONL transcript and
reading it will overflow your context. If the user asks for progress, say the agent is still
running; you'll get a completion notification.
```

377. user (2026-06-10T17:59:59.911Z)

```
Switched to a new branch 'docs/refresh-stale-claims'
```

378. user (2026-06-10T18:00:00.198Z)

```
10:> 4/4, Axis 2/4 on real statements). The §11 open questions remain.
158:   oracle (per-row walk, else total reconcile), oracle-gated. Real-data precision (local):
**AU 4/4**
159:   reconcile + persist; **Axis 2/4** (Aug'24, Oct'25) ? May/Jun'25 are vintages
`extract_tables`
177:  HDFC 4/4 and AU 4/4 qualify today. Harvest them as the eval set; no cloud, no new risk.
```

379. user (2026-06-10T18:00:08.021Z)

```
1       # muneem ? Parser Architecture (concept-core, verification-first)
2
3       > **Status: DRAFT for discussion.** Direction chosen 2026-06: *"concept core + profiles-
as-data."*
4       > The deterministic core below (L0?L2, schema, profiles, verifier, traps) is settled
enough to
5       > build against. The local-VLM research is **done**: the **L3 vision fallback** is
**DeepSeek-OCR-3B
6       > (MIT)**, with **Qwen2.5-VL-7B (Apache-2.0)** as a heavier fallback, reconcile-gated
(§4, §7) ? and on
7       > the dev RTX 2070S (8 GB, Turing) it runs 4-bit + SDPA. The L3 *build* is deferred.
8       > **Steps 1?3 of the plan (§10) have landed** ? the verifier + concept schema, the
generic
9       > concept-mapping engine + profiles, and the shared savings path (Axis + AU Bank run
through it: AU
10      > 4/4, Axis 2/4 on real statements). The §11 open questions remain.
11      > Companion to [`DESIGN.md`](./DESIGN.md) (what/why) and [`ROADMAP.md`](./ROADMAP.md)
(sequencing).
12
13      ---
14
15      ## 1. Why we're rethinking this
16
17      The current parsers are brittle for two reasons:
18
19      1. **They encode *syntax*, not *semantics*.** HDFC hard-codes x-positions; Axis hard-
codes column
20         indices. They know *where* a value sits, not *what* it is ? so any format nudge maps
the wrong
21         column and produces wrong data.
22      2. **They parse open-loop.** They emit whatever they extracted with no check, so a wrong
parse looks
23         like a right one. (The Axis parser confidently produced 15 mislabeled rows ? see
ROADMAP Known
24         Issues.)
25
26      Goals for the new architecture: **format-resilient** (small layout/label changes don't
break us),
27      **extensible** (a new issuer or a changed format is mostly data, not code), **semantic**
(we *know*
28      the critical placeholders ? balance, spend, deposit, ?), and **failsafe** (drift is
loud, never
29      silent).
30
```

380. user (2026-06-10T18:00:08.421Z)

```
150        ? parameterized by a `FamilySpec` (concept names + typed-row builder) and an injected
oracle, so
151        an instrument family (equity/MF/ETF/bond) reuses the same control flow with its own
concepts +
152        oracle. Issuer knowledge is `profiles.py` ? **data, not code**. The engine ships
validated on
153        synthetic fixtures; HDFC consumes its profile (account pattern) while keeping its
bespoke
154        coordinate-clustering extraction (the documented `extract_tables` exception). The
first real
155        issuers driven through the engine are **Axis + AU Bank** (step 3).
156     3. **? Axis + AU Bank rewrite onto the core ? landed.** Both run the shared engine-
driven savings
157        path (`savings.py`): per-table + coordinate-clustered candidates ? engine mapping ?
reconciliation
158        oracle (per-row walk, else total reconcile), oracle-gated. Real-data precision
```

(local): **AU 4/4**
159   reconcile + persist; **Axis 2/4** (Aug'24, Oct'25) ? May/Jun'25 are vintages
`extract_tables`
160   mangles, so they store nothing (the safety invariant holds). HDFC stays bespoke
clustering.
161   Recovering the remaining Axis vintages (better clustering) is follow-up work.
162   4. **L3 local-vision fallback** (research done; build deferred): **DeepSeek-OCR-3B
(MIT)**, fired
163   *only* when the deterministic ladder fails to reconcile (e.g. the Axis May/Jun
vintages); output
164   accepted only if it reconciles, self-consistency re-sample on a miss, then human
review. Dev RTX
165   2070S ? 4-bit + SDPA; Qwen2.5-VL-7B (Apache) is the heavier / fine-tune option on a
bigger GPU.
166   See §4, §7, and the `l3-vision-and-eval` memory.
167   5. **Unify storage** (§9) + begin **Tier-2 transaction semantics**.
168
169   Each step is a green, reversible PR; the verifier guarantees we never regress into
storing garbage.
170
171   ## 10a. Golden eval sets (high-confidence labels)
172
173   Ground truth is built three ways, cheapest-risk first ? confidence comes from
*reconciliation +
174   consensus*, never from trusting any single extractor:
175
176   - **Local-golden (free, now):** any parse that **reconciles + bookends** is already a
gold label ?
177   HDFC 4/4 and AU 4/4 qualify today. Harvest them as the eval set; no cloud, no new
risk.
178   - **Synthetic-golden (free, CI-able):** statements authored in the real issuer layouts
with
179   fabricated PII/amounts ? safe to commit (`testdata/synthetic/`) and to run any model
against.
180   - **Cloud-golden (eval-only, approval-gated, off by default):** for a layout no local
method can
181   label, a *zero-retention* cloud doc-AI may extract a **minimal** real subset as **one
voter**,
182   cross-checked by reconciliation ? never in the runtime path, never without explicit
per-category
183   approval (DESIGN §5).
184
185   This dovetails with the **Regression CI** plan (ROADMAP § "Planned ? Regression CI for
statement
186   parsing"): synthetic-golden ? the always-on cloud-CI layer; local-golden ? the self-
hosted
187   real-statement layer.
188
189   ## 11. Open questions (for discussion)
190
191   - **Storage unification timing** ? keep per-bank tables until the concept core is
proven, then migrate?
192   - **Generic-vs-profile balance** ? how much to lean on profiles as priors vs. pure
content inference.
193   *Provisional (Step 2): generic detectors run first; the profile supplies aliases /
column-order /
194   anchors / account patterns as priors. Revisit once more issuers land.*
195   - **L3 scope** ? *model resolved (DeepSeek-OCR-3B; §4).* Still open: rare fallback vs
routine
196   cross-check on every statement (lean **rare / reconcile-fail-only**, given the 2070S
budget).
197   - **Tier-2** ? when do transaction classification + merchant canonicalization become in-
scope?
198   - **Confidence policy** ? what does the CLI/ledger do with a `quarantined` statement
(skip, show with

```
199       a warning, queue for the VLM)?
200
```

381. user (2026-06-10T18:00:15.355Z)

```
130:## 9. Data-model evolution
131-
132-Today: per-bank raw tables (`hdfc_transactions`, `axis_transactions`, ?) ? fine for
fidelity, but the
133-concept schema points at a **unified `transactions` table** keyed on the canonical concepts,
plus
134-`account`/`document` context and **`provenance` + `confidence`** columns. Migration is
additive (the
135-per-bank tables can become views or be backfilled); sequence it after the core lands so we
don't churn
136-storage twice.
137-
138-## 10. Implementation plan (incremental, merge-as-we-go)
139-
140-1. **? Core extraction ? landed.** `validation.py` is now the verifier: the per-row walk,
**total
141-   reconcile** (`opening + ?credit ? ?debit == closing`), the concept-level **bookend**
(`verify`),
142-   **candidate selection** (`select_reconciling`), and a recorded **confidence** verdict
143-   (`Confidence`). The concept schema lives in `concepts.py` (`ConceptRow`,
`StatementConcepts`).
144-   **HDFC is refactored onto it as the reference implementation** ? it maps its rows onto
the
```

382. assistant (2026-06-10T18:00:20.080Z)

Now applying the E1 edits to parser-architecture.md:

383. user (2026-06-10T18:00:24.873Z)

```
The file C:\Users\avidu\Projects\muneem\docs\parser-architecture.md has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

384. user (2026-06-10T18:00:30.973Z)

```
The file C:\Users\avidu\Projects\muneem\docs\parser-architecture.md has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

385. user (2026-06-10T18:00:37.557Z)

```
The file C:\Users\avidu\Projects\muneem\docs\parser-architecture.md has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

386. user (2026-06-10T18:00:40.380Z)

```
The file C:\Users\avidu\Projects\muneem\docs\parser-architecture.md has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

387. user (2026-06-10T18:00:44.991Z)

```
The file C:\Users\avidu\Projects\muneem\docs\parser-architecture.md has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

388. user (2026-06-10T18:00:52.203Z)

```
Updated task #18 status
```

389. user (2026-06-10T18:00:53.153Z)

Updated task #16 owner, status

390. user (2026-06-10T18:00:53.624Z)

Updated task #21 owner, status

391. user (2026-06-10T18:00:55.665Z)

```
293-
294-Carried from DESIGN.md § 8, not yet resolved ? surface and ask when a milestone forces them:
295-
296:- **§ 8.3 ? LLM model for sensitive categories.** Coupled to your hardware (GPU? RAM?) and
latency tolerance. Forced at the start of the generalize phase (local-LLM fallback). Ollama +
Llama/Qwen/Mistral is the likely shape, but unconfirmed.
297-- **§ 8.5 ? Merchant canonicalization.** Deferred until real receipt data exposes the mess.
Forced when `items`/`merchants` land.
298-- ~~**Encryption-at-rest timing** (D.2) ? SQLCipher now vs. later.~~ **Resolved 2026-05:**
SQLCipher adopted now (`sqlcipher3` + keychain key), forced by storing real numbers; research-
backed. See [`docs/encryption-at-rest.md`](./encryption-at-rest.md).
299-[Omitted long context line]
```

392. user (2026-06-10T18:00:56.682Z)

```
15-- Bank/card performance comparison
16-- Catalog of purchased items with their bills (warranty, returns, age)
17-- Forgotten deals and coupons sitting unread in your inbox
18-
19:## Status
20-
21-**Early implementation.** The design and plan are settled ? see [`DESIGN.md`](.\DESIGN.md)
(architecture + threat model), [`docs\ROADMAP.md`](.\docs\ROADMAP.md) (milestones), and
[`docs\TESTING.md`](.\docs\TESTING.md) (test strategy). The Python project skeleton, CLI, and
offline CI are in place; build proceeds by the roadmap's milestones. Prior art: [`docs/prior-
art-email-mining.md`](./docs/prior-art-email-mining.md).
22-
23-**Ingesting bank statements** (HDFC \ Axis \ AU Bank, with automatic password unlock):
[`docs/ingesting-statements.md`](./docs/ingesting-statements.md).
24-
25-## Getting started
26-
27:Runs on Linux, macOS, and Windows. Requires **Python ? 3.11** and git.
28-
29-### Linux / macOS
30-
31-```bash
```

393. user (2026-06-10T18:01:06.508Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

394. user (2026-06-10T18:01:13.624Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

395. user (2026-06-10T18:01:15.356Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

396. user (2026-06-10T18:01:22.456Z)

```
290     ---
291
292     ## 9. Still-open decisions (do NOT pick silently)
293
```

```
294     Carried from DESIGN.md § 8, not yet resolved ? surface and ask when a milestone forces
them:
295
296     - **§ 8.3 ? LLM model for sensitive categories.** Coupled to your hardware (GPU? RAM?)
and latency tolerance. Forced at the start of the generalize phase (local-LLM fallback). Ollama
+ Llama/Qwen/Mistral is the likely shape, but unconfirmed.
297     - **§ 8.5 ? Merchant canonicalization.** Deferred until real receipt data exposes the
mess. Forced when `items`/`merchants` land.
298     - ~~**Encryption-at-rest timing** (D.2) ? SQLCipher now vs. later.~~ **Resolved
2026-05:** SQLCipher adopted now (`sqlcipher3` + keychain key), forced by storing real numbers;
research-backed. See [`docs/encryption-at-rest.md`](./encryption-at-rest.md).
299     - ~~**Ingestion source ? Open Banking (Account Aggregator) vs PDF parsing.**~~
**Resolved 2026-06:** **PDF-first; AA deferred** as an opt-in complement only (regulatory FIU
gate + paid cloud-decrypting TSP make it non-viable and a net privacy regression for a single-
user local tool; re-adopt triggers tracked). Research-backed ? see [`docs/research-open-banking-
india.md`](./research-open-banking-india.md) and DESIGN § 8.6. Surfaced a new local-only
candidate: **SMS/email transaction-alert parsing** (freshness complement to PDF, zero third
party).
300     - ~~**Storage shape.**~~ **Resolved 2026-06:** **flat per-issuer source-of-truth tables
+ a derived serving layer; provenance/confidence in a separate `provenance` table** (DESIGN §
8.7, feedbackJune2026 P3). First forward migration adds `provenance`.
301     - **Promo-track LLM** (A.3) ? only reached if deterministic extraction underperforms;
cloud is policy-permitted for promo but still needs a quick okay on which vendor.
```

397. user (2026-06-10T18:01:22.932Z)

```
15      - Bank/card performance comparison
16      - Catalog of purchased items with their bills (warranty, returns, age)
17      - Forgotten deals and coupons sitting unread in your inbox
18
19      ## Status
20
21      **Early implementation.** The design and plan are settled ? see
[`DESIGN.md`](./DESIGN.md) (architecture + threat model), [`docs/ROADMAP.md`](./docs/ROADMAP.md)
(milestones), and [`docs/TESTING.md`](./docs/TESTING.md) (test strategy). The Python project
skeleton, CLI, and offline CI are in place; build proceeds by the roadmap's milestones. Prior
art: [`docs/prior-art-email-mining.md`](./docs/prior-art-email-mining.md).
22
23      **Ingesting bank statements** (HDFC / Axis / AU Bank, with automatic password unlock):
[`docs/ingesting-statements.md`](./docs/ingesting-statements.md).
24
25      ## Getting started
26
27      Runs on Linux, macOS, and Windows. Requires **Python ? 3.11** and git.
28
29      ### Linux / macOS
30
```

398. user (2026-06-10T18:01:34.869Z)

```
The file C:\Users\avidu\Projects\muneem\docs\ROADMAP.md has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

399. user (2026-06-10T18:01:41.031Z)

```
The file C:\Users\avidu\Projects\muneem\README.md has been updated successfully. (file state is
current in your context ? no need to Read it back)
```

400. user (2026-06-10T18:01:42.750Z)

```
The file C:\Users\avidu\Projects\muneem\README.md has been updated successfully. (file state is
current in your context ? no need to Read it back)
```

401. assistant (2026-06-10T18:01:53.038Z)

Now DESIGN §8.3, the synthetic-fixture wording in three places, and the cli docstring:

402. user (2026-06-10T18:02:00.169Z)

The file C:\Users\avidu\Projects\muneem\DESIGN.md has been updated successfully. (file state is current in your context ? no need to Read it back)

403. user (2026-06-10T18:02:01.571Z)

5:> Created 2026-05-30. Grounded in a direct inspection of the committed promo corpus (`testdata/promo-emails/`), not in the abstract.
13:3. **Test on safe data by default.** The committed promo corpus is non-sensitive and drives CI. Sensitive paths (statements) are tested on **synthetic/redacted fixtures** in CI and on **local-only, gitignored** real data behind an opt-in marker ? never real statements in CI or the repo.
45:[Omitted long matching line]
64:A single committed file is the source of truth; tests read it, they don't hardcode expectations.
109:- **Image dedup**: MD5-of-PNG-bytes equality and SSIM ? 0.95 near-duplicate detection (port + test Email-Mining's logic on synthetic image pairs).
142:1. **Synthetic fixtures, committed.** A small generator emits fake **HDFC-layout** statement PDFs with faker-generated names/amounts/dates and a *known* derived password. These exercise the parser and the unlocker in CI with zero real data. Stored under `testdata/synthetic/` (committed; clearly fake).
145:**Unlocker tests (hybrid strategy, C.1):** encrypt a synthetic fixture with a password derived by a known rule, then assert the resolver tries **dictionary ? rule ? prompt** in order, caches the winner, and ? critically ? that **no password material appears in logs or exceptions** (log-capture assertion).
147:**Parser tests (C.2):** synthetic HDFC statement ? assert normalized `transactions` (date, amount, direction, balance) match the generator's known ledger.
166:- **Log redaction.** Feed synthetic account numbers / balances / a known password through the logging path; assert they are masked in captured output.
183:`expected.yaml` committed with every verified row; **T1 recall + precision = 100%** in default CI; **T3 negatives extract nothing**; **T2 passes locally with OCR**; idempotency + the § 9 security guards green. That is the bar for calling the promo "forgotten deals" feature trustworthy.

404. user (2026-06-10T18:02:07.298Z)

```
138     ## 7. The sensitive path (statements) ? coverage without real data in CI
139
140     The corpus is promo-only by design. Statement parsing (ROADMAP Milestone C) is tested two ways:
141
142     1. **Synthetic fixtures, committed.** A small generator emits fake **HDFC-layout** statement PDFs with faker-generated names/amounts/dates and a *known* derived password. These exercise the parser and the unlocker in CI with zero real data. Stored under `testdata/synthetic/` (committed; clearly fake).
143     2. **Real statements, local-only.** Behind `@pytest.mark.local_statements`, reading `testdata/statements/` (gitignored). Skipped in CI; run by the user locally.
144
145     **Unlocker tests (hybrid strategy, C.1):** encrypt a synthetic fixture with a password derived by a known rule, then assert the resolver tries **dictionary ? rule ? prompt** in order, caches the winner, and ? critically ? that **no password material appears in logs or exceptions** (log-capture assertion).
146
147     **Parser tests (C.2):** synthetic HDFC statement ? assert normalized `transactions` (date, amount, direction, balance) match the generator's known ledger.
148
149     ---
150
151     ## 8. CI shape (GitHub Actions)
152
```

405. user (2026-06-10T18:02:09.238Z)

```
140-The corpus is promo-only by design. Statement parsing (ROADMAP Milestone C) is tested two
ways:
141-
142-1. **Synthetic fixtures, committed.** A small generator emits fake **HDFC-layout** statement
PDFs with faker-generated names/amounts/dates and a *known* derived password. These exercise the
parser and the unlocker in CI with zero real data. Stored under `testdata/synthetic/`
(committed; clearly fake).
143:2. **Real statements, local-only.** Behind `@pytest.mark.local_statements`, reading
`testdata/statements/` (gitignored). Skipped in CI; run by the user locally.
144-
145-**Unlocker tests (hybrid strategy, C.1):** encrypt a synthetic fixture with a password
derived by a known rule, then assert the resolver tries **dictionary ? rule ? prompt** in order,
caches the winner, and ? critically ? that **no password material appears in logs or
exceptions** (log-capture assertion).
146-
--
161-
162-These encode the DESIGN § 5 standing rules as failing-build checks:
163-
164:- **No sensitive paths tracked.** A test runs `git ls-files` and fails if anything under
`testdata/statements/`, `testdata/private/`, `*.db`, `*.sqlite*`, `token*`, `.env`, or
`password_rules.*` is tracked.
165-- **Cloud-LLM category gate.** Calling the cloud-LLM client with a category in {bank, cc,
broker, mf, insurance} raises/refuses; only {promo, ?explicitly-approved} are allowed. A test
asserts the refusal.
166-- **Log redaction.** Feed synthetic account numbers / balances / a known password through
the logging path; assert they are masked in captured output.
167-- **Token storage.** Token persistence goes through the `keyring` abstraction (mocked in
tests); a test asserts nothing is written to a plaintext file on disk.
--
180-
181-## 11. Definition of done (testing) for Track A
182-
183:`expected.yaml` committed with every verified row; **T1 recall + precision = 100%** in
default CI; **T3 negatives extract nothing**; **T2 passes locally with OCR**; idempotency + the
§ 9 security guards green. That is the bar for calling the promo "forgotten deals" feature
trustworthy.
```

406. user (2026-06-10T18:02:24.737Z)

```
The file C:\Users\avidu\Projects\muneem\docs\TESTING.md has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

407. user (2026-06-10T18:02:26.355Z)

```
The file C:\Users\avidu\Projects\muneem\docs\TESTING.md has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

408. user (2026-06-10T18:02:34.193Z)

```
7-Two principles do the heavy lifting:
8-- **Reconciliation is the oracle** ? a parse is trusted only if the running balance walks (or
the
9-  totals match the printed opening/closing). So a wrong parse is *rejected*, never stored.
10-- **Real statements never leave the machine, ever committed.** We validate on local data and
ship
11:  *synthetic* fixtures (real layout, fabricated values) for cloud CI.
12-
13-## The loop
14-
15-### 1. Seed (2?3 real statements) ? build/tune the profile
--
```

```
41-
42-### 3. Synthetic fixture ? gate the parser in cloud CI
43-Add a born-digital fixture in the new bank's **layout** with **fabricated** data, using the
stdlib
44-generator `tests/synthpdf.py`, and assert a full-pipeline `reconciled` (see
45:`tests/test_regression_synth.py`). Commit it to `testdata/synthetic/` ? it's safe (no real
data) and
46-runs on every push (Layer-1), so the parser is regression-gated without exposing statements.
47-
48-### 4. Land
49:A PR adds: the `Profile` (+ a thin parser module if required) + the synthetic fixture. The
golden
50-real statements are registered locally for Layer-2 ? **never committed** (`.gitignore` +
51-`.githooks/precommit_check.py` enforce this).
52-
53-## Checklist
54:- [ ] Real seeds in a gitignored folder; password in `password_rules.yaml` (config dir) or
keychain.
55:- [ ] `Profile` added (aliases / date patterns / account regex / quirks).
56:- [ ] `muneem eval <seed-folder>` ? seeds `reconciled`.
57:- [ ] `muneem eval <golden-folder>` ? golden set `reconciled` (your local high-confidence
set).
58:- [ ] Synthetic fixture in `testdata/synthetic/` + a `test_regression_synth`-style assertion
(Layer-1).
59:- [ ] PR = profile (+ parser module if needed) + synthetic fixture. No real statements
committed.
```

409. user (2026-06-10T18:02:46.476Z)

```
<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>
```

410. user (2026-06-10T18:02:48.854Z)

```
<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>
```

411. user (2026-06-10T18:02:50.056Z)

```
1-# --- Sensitive data ? NEVER commit ---
2-# Real financial documents stay local only. Everything under testdata/ is sensitive by
3:# default ? ONLY the non-sensitive promo corpus and committed synthetic fixtures are tracked.
4-# Real statements in ANY subdir (e.g. testdata/hdfc-statements/) must never be committed.
5-testdata/*
6-!testdata/promo-emails/
7:!testdata/synthetic/
8-*.db
9-*.sqlite
10-*.sqlite3
```

412. user (2026-06-10T18:02:57.618Z)

```
40      folder is the quick local gate.
41
42      ### 3. Synthetic fixture ? gate the parser in cloud CI
43      Add a born-digital fixture in the new bank's **layout** with **fabricated** data, using
the stdlib
44      generator `tests/synthpdf.py`, and assert a full-pipeline `reconciled` (see
45      `tests/test_regression_synth.py`). Commit it to `testdata/synthetic/` ? it's safe (no
real data) and
46      runs on every push (Layer-1), so the parser is regression-gated without exposing
statements.
47
48      ### 4. Land
49      A PR adds: the `Profile` (+ a thin parser module if required) + the synthetic fixture.
The golden
```

```
50      real statements are registered locally for Layer-2 ? **never committed** (`.gitignore` +
51      `.githooks/precommit_check.py` enforce this).
52
53      ## Checklist
54      - [ ] Real seeds in a gitignored folder; password in `password_rules.yaml` (config dir)
or keychain.
55      - [ ] `Profile` added (aliases / date patterns / account regex / quirks).
56      - [ ] `muneem eval <seed-folder>` ? seeds `reconciled`.
57      - [ ] `muneem eval <golden-folder>` ? golden set `reconciled` (your local high-
confidence set).
58      - [ ] Synthetic fixture in `testdata/synthetic/` + a `test_regression_synth`-style
assertion (Layer-1).
59      - [ ] PR = profile (+ parser module if needed) + synthetic fixture. No real statements
committed.
60
```

413. user (2026-06-10T18:03:08.416Z)

```
The file C:\Users\avidu\Projects\muneem\docs\onboarding-a-bank.md has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

414. user (2026-06-10T18:03:09.614Z)

```
The file C:\Users\avidu\Projects\muneem\docs\onboarding-a-bank.md has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

415. user (2026-06-10T18:03:13.203Z)

```
The file C:\Users\avidu\Projects\muneem\.gitignore has been updated successfully. (file state is
current in your context ? no need to Read it back)
```

416. user (2026-06-10T18:03:19.794Z)

```
2-
3-Track A ships the promo spine: ``ingest`` a PDF into the local store and
4-``offers list`` to read back what was found. ``--ocr`` adds local image OCR for
5-codes rasterized into banners (approximate; flagged for verification). Pipeline
6:subcommands (``fetch``, ``txns``) land with their milestones.
7-"""
8-
9-from __future__ import annotations
10-
```

417. user (2026-06-10T18:03:31.993Z)

```
The file C:\Users\avidu\Projects\muneem\src\muneem\cli.py has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

418. user (2026-06-10T18:03:39.281Z)

```
.....                                                                    [100%]
509 passed, 21 deselected in 4.20s
All checks passed!
81 files already formatted
Success: no issues found in 34 source files
```

419. user (2026-06-10T18:04:09.608Z)

```
warning: in the working copy of 'docs/TESTING.md', CRLF will be replaced by LF the next time Git
touches it
[docs/refresh-stale-claims ffce877] docs: refresh every stale claim the deep review caught
 8 files changed, 57 insertions(+), 31 deletions(-)
 * [new branch]      docs/refresh-stale-claims -> docs/refresh-stale-claims
https://github.com/avidullu/muneem/pull/67
```

```
From https://github.com/avidullu/muneem
 * branch            main       -> FETCH_HEAD
   87273c5..d94b1fa  main       -> origin/main
Updating 87273c5..d94b1fa
Fast-forward
 .gitignore                  |  5 +++--
 DESIGN.md                   |  8 ++++++++
 README.md                   |  4 ++--
 docs/ROADMAP.md             |  2 +-
 docs/TESTING.md             |  6 +++---
 docs/onboarding-a-bank.md   | 13 +++++++------
 docs/parser-architecture.md | 40 ++++++++++++++++++++++++++++++++-------------
 src/muneem/cli.py           | 10 ++++++----
 8 files changed, 57 insertions(+), 31 deletions(-)
Your branch is up to date with 'origin/main'.
Already on 'main'
Already up to date.
```

420. user (2026-06-10T18:04:29.390Z)

```
Updated task #18 status
```

421. user (2026-06-10T18:04:29.820Z)

```
Updated task #20 status
```

422. user (2026-06-10T18:04:30.834Z)

```
1       # muneem ? Project Plan & Roadmap
2
3       > Status: **active plan; mid-implementation.** Milestone 0 + Track A are done and the
parser concept-core has landed (see the Progress snapshot below;
[`CURRENT_STATE.md`](./CURRENT_STATE.md) is the live status). This is the "how and in what
order" companion to [`DESIGN.md`](../DESIGN.md) (the "what and why"); DESIGN.md owns the
architecture and threat model, this doc owns sequencing. Where they disagree, DESIGN.md wins.
4       >
5       > Created 2026-05-30.
6       >
7       > **Scope:** muneem is a _whole-finance_ bookkeeper. Bank/CC statements are the first
slice;
8       > **equity, mutual-fund, ETF, and bond statements are core scope** (not merely the § 6
"generalize"
9       > backlog), enabled by the family-agnostic concept core. Sequencing still leads with the
bank slice,
10      > but the data model and parser core are built to host an instrument family +
instrument-event
11      > classification (buy / sell / dividend / fees).
12      >
13      > **Progress snapshot (2026-06):** Milestone 0 (scaffolding / offline CI / config +
keyring) ? and
14      > **Track A** (promo spine: `ingest` ? SQLite ? `offers list`, green regression) ? are
done.
15      > **Track B:** Gmail OAuth2 + search + attachment download ?; the `muneem fetch` CLI
(B.4) is not
16      > built yet. **Parsers:** the concept-core rework (parser-architecture.md **steps 1?3**)
landed ?
17      > HDFC 6/6 real, AU Bank 4/4, Axis 6/7, ICICI removed. **Password unlock is now wired
into `ingest`**
18      > (C.1 ? hybrid candidates ? prompt, keychain-cached; see ingesting-statements.md).
**Not yet:** the
19      > unified `transactions`/`holdings` schema (C.3 / step 5). The
20      > checkbox lists below predate the concept-core pivot ? treat
[`CURRENT_STATE.md`](./CURRENT_STATE.md)
21      > as the live status.
```

?? Known issues

> **Axis savings ? rewrite landed (2026-06), partially complete.** The old fixed-column
> `extract_tables()` mapping (wrong columns, layouts that drift between vintages) was replaced by the
> concept-core path: coordinate clustering + per-table mapping ? engine ? reconciliation oracle
> (per-row walk, else total reconcile), oracle-gated. See [`parser-architecture.md`](./parser-architecture.md)
> step 3. **Real-data precision: 6/7** ? June 2025 was recovered via **balance-delta reconstruction**
> (derive debit/credit from the running-balance sign, then cross-check `amount == |?balance|`). **May
> 2025 still fails**: the clusterer drops one transaction line ? incomplete ledger, which the oracle
> correctly rejects (it stores **nothing** ? never mislabeled rows).
>
> **Remaining:** recover the dropped line (smarter line recovery in the clusterer) for May 2025 and
> future vintages.
> The credit-card parser (`AxisCreditCardParser`) is still unverified on real data.

? Open critical work items (tracked)

Live cross-cutting work, each detailed in its linked home:

1. **Regression CI for statement parsing** ? two-layer (synthetic cloud CI + self-hosted real-statement
   runner); decided, **build deferred**. ? § "Planned ? Regression CI for statement parsing" (below).
2. **Axis May 2025 recovery** ? recover the one transaction line the clusterer drops (incomplete
   ledger; oracle-gated, stores nothing). June 2025 recovered via balance-delta. ? "?? Known issues".
3. **L3 local-vision fallback** ? DeepSeek-OCR-3B (MIT), reconcile-gated; dev RTX 2070S ? 4-bit + SDPA.
   Research done, **build deferred**. ? `parser-architecture.md` §4/§7.
4. **Golden eval sets** ? local-golden + synthetic-golden; cloud-golden eval-only & approval-gated.
   ? `parser-architecture.md` §10a. **Seed?golden onboarding harness landed:** `muneem eval <folder>`
   (reconcile-rate report) + [`onboarding-a-bank.md`](./onboarding-a-bank.md).
5. **Unified `transactions`/`holdings` schema + instrument family** (equity/MF/ETF/bond + buy/sell/
   dividend/fee events) ? the whole-finance milestone. ? §10 step 5 / `scope-all-personal-finance` memory.

---

1. Decisions locked (2026-05-30)

These resolve open decisions from DESIGN.md § 8. They were made by the user explicitly; this section is the start of the decision log.

DESIGN.md ref	Decision	Choice
§ 8.1 Language	Implementation language for v1	**Python-only.** Accept the heavier deployment story in exchange for the best PDF/OCR/LLM ecosystem and fastest path on the hard parsing work. Revisit only if the ingestion daemon outgrows Python.
§ 8.4 First slice	What to build first	**Both tracks in parallel** ? the promo-

```
corpus parse?store?query spine *and* the Gmail ingestion plumbing, developed as two decoupled
tracks that converge at the first bank slice. |
68      | § 8.2 Passwords | Password sourcing strategy | **Hybrid: candidate dictionary ? per-
sender rule template ? interactive prompt.** Cache the winning method per sender. Secrets never
touch the repo or logs. |
69      | (practical) | First real bank parser target | **HDFC** ? born-digital statements,
name+DDMM-style PDF password (per CLAUDE.md), and the issuer with the longest consistent
statement history to test against. |
70
71      **Still parked** (do not pick silently ? see § 9 below): § 8.3 LLM model for *sensitive*
categories, and § 8.5 merchant canonicalization. _(Encryption-at-rest ? D.2 ? is now **decided &
implemented**: SQLCipher via `sqlcipher3` + keychain key. See [`docs/encryption-at-
rest.md`](./encryption-at-rest.md).)_
72
73      ---
74
75      ## 2. Guiding principles (inherited from DESIGN.md § 5 + CLAUDE.md)
76
77      1. **Local by default.** All raw statement data and the DB stay on the machine. OCR/LLM
work runs locally for any sensitive category. No cloud sync of staging or DB, ever.
78      2. **No cloud LLM on sensitive contents** (bank, CC, broker, MF, insurance) without
explicit per-category approval in-session. The **promo corpus is the one non-sensitive
playground** where cloud LLMs are policy-permitted ? so it's where we validate any LLM-
extraction pattern *before* it could ever see a statement.
79      3. **Test on safe, committed data wherever possible.** `testdata/promo-emails/` is the
regression corpus and runs in CI. Statement parsing gets CI coverage via **synthetic/redacted
fixtures** in the issuer's layout ? never real statements.
80      4. **Secrets discipline.** OAuth tokens ? OS keychain (Windows Credential Manager).
Nothing sensitive in git; don't weaken `.gitignore`. Logs are redacted (no account numbers,
balances, line items, or statement text).
```

423. user (2026-06-10T18:05:02.566Z)

```
The file C:\Users\avidu\Projects\muneem\docs\ROADMAP.md has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

424. user (2026-06-10T18:05:12.776Z)

```
<tool_use_error>File has been modified since read, either by the user or by a linter. Read it
again before attempting to write it.</tool_use_error>
```

425. user (2026-06-10T18:05:16.603Z)

```
The file C:\Users\avidu\Projects\muneem\docs\ROADMAP.md has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

426. user (2026-06-10T18:05:21.810Z)

```
176     - **Local web UI:** ? defer ? CLI stays authoritative; a web tier is new attack surface
+ scope for
177       no current need. Revisit only if a real UX gap appears.
178
179     ---
180
181     ## 4. Proposed sequencing (dependency-ordered)
182
183     > Rule of thumb: **recoverability + foundation first; then the one big schema decision;
then
184     > everything that depends on it.** Each phase is shippable and individually PR-able.
185
```

427. user (2026-06-10T18:05:30.205Z)

```
The file C:\Users\avidu\Projects\muneem\docs\feedbackJune2026.md has been updated successfully.
```

```
(file state is current in your context ? no need to Read it back)
```

428. user (2026-06-10T18:06:01.787Z)

```
Switched to a new branch 'docs/sequencing-single-owner'
3 passed in 0.29s
[docs/sequencing-single-owner 83f564d] docs: ROADMAP section 0 becomes the single sequencing
owner (Phases A-H)
 2 files changed, 31 insertions(+), 1 deletion(-)
 * [new branch]      docs/sequencing-single-owner -> docs/sequencing-single-owner
https://github.com/avidullu/muneem/pull/68
From https://github.com/avidullu/muneem
 * branch            main       -> FETCH_HEAD
   d94b1fa..e1d0b2b  main       -> origin/main
Updating d94b1fa..e1d0b2b
Fast-forward
 docs/ROADMAP.md          | 28 ++++++++++++++++++++++++++-
 docs/feedbackJune2026.md |  4 ++++
 2 files changed, 31 insertions(+), 1 deletion(-)
Your branch is up to date with 'origin/main'.
Already on 'main'
Already up to date.
```

429. user (2026-06-10T18:06:11.154Z)

```
Updated task #20 status
```

430. user (2026-06-10T18:06:42.786Z)

```
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\memory\session-
handoff.md has been updated successfully. (file state is current in your context ? no need to
Read it back)
```

431. user (2026-06-10T18:06:58.790Z)

```
Next wakeup scheduled for 00:07:00 (in 1801s). Nothing more to do this turn ? the harness re-
invokes you when the wakeup fires or a task-notification arrives.
```

432. user (2026-06-10T18:06:58.860Z)

```
Do sync the repo from remote first
Some files have been recently updated
I'm sure this is a redundant message but still would request you to update the next steps based
on your recommendations and the other docs in this repo so that I can take a fresh look.
```

433. user (2026-06-10T18:07:21.057Z)

```
Your branch is up to date with 'origin/main'.
 - [deleted]         (none)     -> origin/provenance-table
 - [deleted]         (none)     -> origin/secrets-password-hardening
 - [deleted]         (none)     -> origin/serving-view
Already up to date.
e1d0b2b docs: ROADMAP section 0 becomes the single sequencing owner (Phases A-H) (#68)
d94b1fa docs: refresh every stale claim the deep review caught (#67)
87273c5 docs: recovery runbook (keychain lost / new machine / verify backups / interrupted
rotation / migrate-then-reverify) (#66)
e7aca66 chore(ci): weekly OCR-tier schedule; redact real filenames from pytest ids (#65)
fffbdc7 fix(security): small-hardening bundle across the crypto/redaction seams (#64)
a84690d chore(deps): pin dependency versions via a pip constraints file (#62)
a7b073e fix(db): atomic migrations + frozen-migration checksum re-verification (#63)
fd9abff chore(security): remove the real-data snapshot footgun; block client_secret patterns
everywhere (#61)
=== open PRs ===
47      docs: Add proposal for evaluating opendataloader-pdf integration + quick comparison
```

```
harness     docs/proposal-opendataloader-pdf-eval   OPEN    2026-06-08T05:55:51Z
=== branches ===
  origin/HEAD -> origin/main
  origin/docs/proposal-opendataloader-pdf-eval
  origin/main
```

434. assistant (2026-06-10T18:07:31.285Z)

muneem main is current (e1d0b2b) and the D1 agent hasn't opened its PR yet. Checking local modifications and the sister repo:

435. user (2026-06-10T18:07:42.308Z)

```
=== sister repo ===
a764030 Codify the auditability invariant (Option A) + enforce no egress (#14)
90c85ba Merge pull request #13 from avidullu/docs/vision-and-architecture
4975254 docs: capture extraction-pivot vision and architecture
172b3b3 Validate setup input + traceable attachment-copy errors (#12)
964b58c Portable attachment names + robust content-type matching (#11)
=== sister PRs ===
=== sister branches ===
* master
  remotes/origin/HEAD -> origin/master
  remotes/origin/master
```

436. user (2026-06-10T18:08:06.394Z)

```
1       # Current State
2
3       > **Last Updated:** 2026-06 (concept-core engine + issuer refocus: HDFC / Axis / AU
Bank)
4       > Summarizes current capabilities, completed features, and near-term next steps for
`muneem`.
5       > See [`ROADMAP.md`](./ROADMAP.md) for milestone tracking and [`parser-
architecture.md`](./parser-architecture.md) for the parser design.
6
7       > **Scope reminder ? muneem is a _whole-finance_ bookkeeper, not a bank tool.** Bank &
credit-card
8       > statements are the first vertical slice; **equities, mutual funds, ETFs, and bonds are
core
9       > targets**, parsed into the same local ledger, with transaction/instrument-event
classification to
10      > follow. The parser core is deliberately **family-agnostic** (see §2), so adding an
instrument
11      > family is *new data + a reconciliation rule*, not a new pipeline.
12
13      ## 1. Core Ingestion Pipeline
14      The ingestion pipeline supports a generic `SearchCriteria` interface and multiple
providers (Track B plumbing):
15
16      * **Gmail** (`muneem.ingestion.gmail`): Full OAuth2 read-only + search + attachment
download implemented.
17      * **IMAP** (`muneem.ingestion.imap`): Basic auth + search implemented; `get_headers` /
attachment methods are NotImplemented (experimental).
18      * **Outlook** (`muneem.ingestion.outlook`): Device-flow auth + basic search implemented;
attachment/header/download NotImplemented (experimental).
19
20      Companion one-time auth tools live in `tools/`. No high-level `muneem fetch` CLI command
yet (see Roadmap B.4).
21
22      ## 2. Statement Parsers (concept-core architecture)
23      Deterministic parsers live under `muneem.parsers`, registered via decorator and selected
by `get_parser` on text heuristics + sender. Submodules are **auto-loaded on package import**
(`_autoload_parsers`), so the registry is populated in the production path (not just under
```

tests).

The parser core is **concept-first and verification-first** (see [`parser-architecture.md`](./parser-architecture.md)): an extractor maps an issuer's columns onto a bank-agnostic **concept schema** (`concepts.py`); a generic **engine** (`engine.py`) does the column?concept mapping (L0 header lexicon ? profile column-order ? L2 content search); and a shared **verifier** (`validation.py`) decides trust by *arithmetic* ? per-row balance walk + total reconcile + SUMMARY bookend + a recorded confidence verdict. The engine is **family-agnostic**: parameterized by a `FamilySpec` + an injected reconciliation oracle, so a future **instrument family** (equity/MF/ETF/bond holdings + buy/sell/dividend/fee events) reuses the same control flow with its own concepts + oracle. Issuer knowledge lives in `profiles.py` as **data, not code**. Bank **savings** parsers share one engine-driven path ? `savings.py` (per-table + coordinate-clustered candidates ? engine ? oracle), so adding a savings issuer is mostly a profile; HDFC keeps bespoke coordinate clustering (the documented `extract_tables` exception).

Active issuers ? we have real statements + passwords, so these get validated on real data:

* **HDFC Bank** (`hdfc.py`): Savings / transaction-account statements via **coordinate clustering** (deliberately *not* `extract_tables()` ? HDFC's table is semi-ruled). **Self-validating:** running-balance reconciliation + a cross-check against the bank's own SUMMARY (Dr/Cr counts, opening/closing balance); records `documents.status` = `reconciled`/`unreconciled`. Refactored onto the concept core as the **reference implementation**. ? **Verified on the user's 6 real statements** ? all reconcile *and* bookend, without inspecting any value.
* **Axis Bank** (`axis.py`): Savings + Credit Card. Savings runs the shared engine-driven path (`savings.py`) ? per-table + coordinate-clustered candidates ? engine mapping ? reconciliation oracle ? plus a **balance-delta reconstruction**: when columns don't split into a clean debit/credit (a merged amount column, an interleaved value-date, a sparse deposit column), it derives direction from the running-balance sign and the oracle cross-checks `amount == |?balance|`. Oracle-gated, so it never persists unreconciled rows. ? **6/7 real statements reconcile + persist**; the 1 miss (May 2025) has a transaction line the clusterer drops ? incomplete ledger ? the oracle quarantines it (stores **nothing**). Credit-card has no running balance (count only).
* **AU Bank** (`au.py`): engine-driven savings via the shared `savings.py` path ? same recipe, just `AUBANK_PROFILE`. ? **4/4 real statements reconcile + persist** (the cleanest issuer so far).

> _Dropped: **ICICI** ? no statements on hand to validate against, so it was removed to focus effort on the issuers we can actually verify (HDFC/Axis/AU). It can return as a profile when data exists._

All balance-bearing parsers share the verifier and split parsing from DB-insert, so row logic is unit-testable without a PDF/DB and a parse is validated before it's trusted. **Concept-core steps 1?3 have landed** (the verifier + concept schema; the generic mapping engine + profiles; the shared savings path with Axis + AU Bank on it). Parsers use `pdfplumber` (with optional password). **Password resolution is now wired into `muneem ingest --doc-type bank_statement`** ? the hybrid unlock (keychain-cached + `password_rules.yaml` candidates ? interactive prompt; `unlock.py` + `statement_unlock.py`) runs before extraction and forwards the password to the parser, in-memory only. See [ingesting-statements.md](./ingesting-statements.md).

See `password_rules.py` (prefers user config dir) and `cli.py:ingest`.

3. Database & CLI

* **Schema v6** ? documented in [`schema.md`](./schema.md): the frozen **v4 baseline** (`documents`, `offers`, flat per-issuer txn tables ? *by design*, DESIGN §8.7) + forward migrations **v5 `provenance`** (per-document source/parser/confidence, joined ? not per-row) and **v6 `v_transactions`** (the derived **serving view** unioning the per-issuer tables into one normalized, provenance-joined query surface). Dedup on sha256. **Versioned migrations** via the hand-rolled runner (`muneem/migrations.py`): checksummed `schema_migrations` records, baseline-drift rejected (feedbackJune2026 P6). The flat tables remain the rollback-able source of truth; the view is the first piece of the serving layer.
* **The on-disk ledger is encrypted at rest** (decision D.2) ? **SQLCipher** transparent AES-256 via `sqlcipher3`, opened through `db.connect_encrypted()`; the 256-bit key lives in the

OS keychain. SQL queries are unaffected. `:memory:` and the CI fixtures stay on plaintext stdlib `sqlite3`. See [encryption-at-rest.md](./encryption-at-rest.md).
42   * CLI: `ingest <pdf>` (promo, or `--doc-type bank_statement` ? **auto-unlocks encrypted PDFs**, see [ingesting-statements.md](./ingesting-statements.md)), `offers list`, `txns list <bank|all>` (`all` = the cross-bank **serving view**, provenance-joined ? see [schema.md](./schema.md)), `eval <folder>` (reconcile-rate report ? the seed?golden onboarding loop, see [onboarding-a-bank.md](./onboarding-a-bank.md)), and `db` encryption ops (`status`, `encrypt`, `rotate-key`, `backup`, `setup-recovery`, `recover` ? see [encryption-at-rest.md](./encryption-at-rest.md)).
43   * Config uses platformdirs for local dirs; secrets (OAuth tokens, cached statement passwords, **the DB encryption key**) via keyring.
44
45   ## 4. Testing & Hygiene
46   * Strong offline regression on `testdata/promo-emails/` + synthetic parser/engine fixtures (default CI).
47   * Security guards in `test_repo_hygiene.py` + license allowlist test.
48   * **DB invariant tripwires** (`test_db_invariants.py`, default CI): a **schema lock** (pins `SCHEMA_VERSION` + every table/column shape ? an accidental drift fails the merge; a deliberate change is "explicitly allowed" by updating the lock in the same PR) plus **encryption guards** (production ingest must write an *encrypted* ledger; a keyed open never silently downgrades to plaintext; foreign keys enforced + cascade; `documents.sha256` dedup).
49   * **Local git hooks** (`.githooks/`, enable via `make hooks`): `pre-commit` blocks staging sensitive files; `pre-push` runs ruff + the offline suite (incl. the invariant tripwires) so only green code reaches GitHub. The pre-push gate is the **local** stand-in for server-side branch protection, which GitHub gates behind Pro / public repos (muneem stays private). Both are bypassable with `--no-verify` (the conscious "explicitly allowed" escape).
50   * Local-only tests for real statements (gitignored data; passwords via env/keyring, never committed). The suite never touches the real OS keychain (autouse fixture in `tests/conftest.py`) ? headless-CI safe.
51
52   ## 5. Next Steps
53   Concept-core **steps 1?3 are done**, and **password unlock is now wired into `muneem ingest`** (C.1 ? hybrid candidates ? prompt, keychain-cached; see [ingesting-statements.md](./ingesting-statements.md)). Real-statement precision: **HDFC 6/6, AU 4/4, Axis 6/7** (the 1 Axis miss stores nothing ? oracle-gated). In rough order from here:
54
55   * **Instrument family on the flat per-issuer pattern** (the whole-finance milestone): per-issuer/instrument source-of-truth tables (broker/MF/CAS) + a holdings-style `FamilySpec` with its own reconciliation oracle, surfaced through the serving layer. *(Supersedes the old "unify storage into one `transactions` table" step ? the storage decision landed the other way: flat truth + derived serving layer, DESIGN §8.7.)*
56   * **Recover the May 2025 Axis vintage**: the clusterer drops one transaction line ? incomplete ledger; needs smarter line recovery. Isolated and safe (it currently stores nothing; June 2025 was recovered via balance-delta reconstruction).
57   * **D.3 evaluation writeup, then Step 4 ? L3 local-vision fallback**: characterize where the deterministic ladder actually breaks (D.3) before building the fallback (DeepSeek-OCR-3B, local) for layouts it can't reconcile.
58   * **Regression CI for statements** (decided 2026-06, *deferred ? not yet built*): synthetic full-pipeline fixtures in cloud CI + a self-hosted runner exercising the real statements (which stay local, off GitHub). Plan in [ROADMAP.md](./ROADMAP.md) (§ "Planned ? Regression CI for statement parsing").
59
60   Parked: DESIGN §8.3 (local LLM for sensitive categories), §8.5 (merchant canonicalization). _(Encryption-at-rest timing, D.2, is now resolved & implemented ? SQLCipher; see [encryption-at-rest.md](./encryption-at-rest.md).)_
61
62   The project stays **local-first**, secrets in the OS keychain, promo corpus safe for CI/LLM experiments.
63

437. assistant (2026-06-10T18:08:27.922Z)

Everything is current. Now updating CURRENT_STATE.md ? today's shipped state plus a decision-oriented Next Steps section ? so the repo docs alone give you the fresh look:

438. user (2026-06-10T18:08:29.610Z)

```
Switched to a new branch 'docs/current-state-next-steps'
```

439. user (2026-06-10T18:08:32.704Z)

```
The file C:\Users\avidu\Projects\muneem\docs\CURRENT_STATE.md has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

440. user (2026-06-10T18:08:47.779Z)

```
The file C:\Users\avidu\Projects\muneem\docs\CURRENT_STATE.md has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

441. assistant (2026-06-10T18:08:53.555Z)

Now §3/§4 touch-ups and the rewritten §5:

442. user (2026-06-10T18:09:10.485Z)

```
The file C:\Users\avidu\Projects\muneem\docs\CURRENT_STATE.md has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

443. user (2026-06-10T18:09:34.984Z)

```
The file C:\Users\avidu\Projects\muneem\docs\CURRENT_STATE.md has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

444. user (2026-06-10T18:10:07.243Z)

```
3 passed in 0.30s
[docs/current-state-next-steps 4adbde9] docs(current-state): post-hardening status + a decision-
oriented Next Steps
 1 file changed, 63 insertions(+), 17 deletions(-)
 * [new branch]      docs/current-state-next-steps -> docs/current-state-next-steps
https://github.com/avidullu/muneem/pull/69
From https://github.com/avidullu/muneem
 * branch            main       -> FETCH_HEAD
   e1d0b2b..b548fb6  main       -> origin/main
Updating e1d0b2b..b548fb6
Fast-forward
 docs/CURRENT_STATE.md | 80 ++++++++++++++++++++++++++++++++++++++++++++-----------
 1 file changed, 63 insertions(+), 17 deletions(-)
Your branch is up to date with 'origin/main'.
Already on 'main'
Already up to date.
```

445. assistant (2026-06-10T18:10:13.284Z)

Merged as [#69](https://github.com/avidullu/muneem/pull/69). One last handoff pointer update, then the summary:

446. user (2026-06-10T18:10:17.960Z)

```
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\memory\session-
handoff.md has been updated successfully. (file state is current in your context ? no need to
Read it back)
```

447. assistant (2026-06-10T18:10:33.953Z)

Done. Summary of what the sync found and what changed:

Sync result: both repos were already current ? muneem `main` had nothing new on the remote

beyond what we merged this session (now at `b548fb6` after #69), the sister repo is untouched at `a764030`, and three stale remote branches got pruned locally. The two background agents haven't pushed their PRs yet (the synthetic-corpus one and the sister-repo fixes are still in progress ? I'll report when they land).

The fresh-look update, merged as [#69](https://github.com/avidullu/muneem/pull/69): [CURRENT_STATE.md](docs/CURRENT_STATE.md) is now your entry point, brought current with all 19 hardening PRs (the hardened-verifier note, crash-safe key lifecycle, atomic migrations, `unverified` status vocabulary, pinned deps), and its **§5 Next Steps is rewritten as a decision queue** in three bands:

1. **Do now (mechanical):** the one-time `pytest -m local_statements` re-verification under the stricter oracle ? the single thing only you can run, since it needs your statement passwords; reviewing the in-flight agent PRs (sister-repo merges auto-deploy your live archiver ? flagged); Axis May-2025 recovery + CC real-data validation as the cheapest unverified work.
2. **Decide next:** each item names what it blocks plus the review's recommendation ? the **cross-repo Gmail-seam ADR** (blocks `muneem fetch`; #48's deprecation trigger vs the Drive-folder middle path), the **C?D phase order** (safety net before the instrument family), the **product wedge** (finance personally, grocery as the sister repo's corpus, tax-pack as the public wedge), and the three cheap standing decisions (log format, self-hosted runner, macOS).
3. **Then build:** per [ROADMAP §0](docs/ROADMAP.md), with the untriaged review-surfaced categories (cash entry, backfill/retention, quarantine UX, definition of done, SMS owner) pointed at but deliberately undecided.

Reading order for the fresh look: **CURRENT_STATE §5 ? ROADMAP §0 ? feedbackJune2026** (decisions context) ? everything else is reachable from those three. No decisions were made anywhere; they're queued with their context.

448. user (2026-06-10T18:11:30.840Z)

<task-notification>
<task-id>a0a70ddb1d7197b83</task-id>
<tool-use-id>toolu_01BqU5DvvtLhFKVwcqmHosnW</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-muneem\d1586dfe-adae-4d40-a85d-ed0bb23738b8\tasks\a0a70ddb1d7197b83.output</output-file>
<status>completed</status>
<summary>Agent "D1: broaden synthetic corpus" completed</summary>
<result>PR: **https://github.com/avidullu/muneem/pull/70** (branch `test/synthetic-corpus-breadth`, not merged)

Deliverables ? all 5 shipped, none blocked:

1. **Axis savings** ? born-digital fixture routed via title-case letterhead ("Axis Bank" + "Statement of Account") to `AxisParser`, through coordinate clustering + L2 engine search, reconciled with printed Opening/Closing Balance lines anchoring the walk. 3 rows persisted, `reconciled`.
2. **Axis CC** ? true PDF route through `extract_tables()` worked: added `Line`/`grid` primitives to `tests/synthpdf.py` that stroke `m`/`l`/`S` paths (pdfminer ? `LTLine` ? pdfplumber "lines" strategy). The 9-column ruled table extracts cleanly; 3 rows persist with status `unverified`. No fallback seam needed.
3. **Multi-page HDFC** ? 3+2 rows across 2 pages, header repeated, SUMMARY on page 2; walk + bookend span the page break. 5 rows, `reconciled`. (`synthpdf` gained `build_pdf_pages`/`Page`; `build_pdf` unchanged for existing callers.)
4. **Dropped-row quarantine** ? AU statement, internally walk-consistent but missing the first transaction vs the printed Opening Balance: 0 rows stored, `unreconciled`. Verified by mutation: disabling the opening-anchor in `savings.reconcile_oracle` made it wrongly persist 2 rows ? the test exercises exactly the new bookend anchoring (mutation reverted).
5. **Encrypted-PDF CLI** ? pypdf RC4-128 encryption on a synthetic AU statement, driven through real `muneem ingest --doc-type bank_statement --no-prompt` via `CliRunner`. `resolve_password` not patched; candidate seeded via `statement_unlock._remember` against the conftest in-memory keychain. Exit 0, `[reconciled]`, 3 rows read back from the encrypted ledger.

Test counts: 7 tests in `tests/test_regression_synth.py` (up from 2); full default suite 514 passed, 6 deselected; coverage 88.42% (gate 80). `ruff check`, `ruff format`, `mypy src` all clean. Verified the worktree src (not the editable install) was under test via the mutation

check.

Caveats: the CLI test pins `muneem.password_rules.CONFIG_DIR` to an empty tmp dir + `chdir(tmp_path)` for hermeticity (a real `password_rules.yaml` on the dev machine would otherwise add unlock candidates ? harmless, since the seeded cache is tried first, but nondeterministic). `_run_pipeline`'s return shape changed (now includes stored-row count), updating the two pre-existing tests in the same file. CC encryption uses pypdf's default RC4-128 ? decryptable by pdfminer/pdfplumber with no extra backend.</result>
<usage><subagent_tokens>154388</subagent_tokens><tool_uses>38</tool_uses><duration_ms>734691</duration_ms></usage>
<worktree><worktreePath>C:\Users\avidu\Projects\muneem\.claude\worktrees\agent-a0a70ddb1d7197b83</worktreePath><worktreeBranch>worktree-agent-a0a70ddb1d7197b83</worktreeBranch></worktree>
</task-notification>

449. assistant (2026-06-10T18:11:41.924Z)

The D1 agent finished ? all five fixtures shipped on [#70](https://github.com/avidullu/muneem/pull/70), including the mutation-verified bookend-anchoring test. Reviewing the diff before merging:

450. user (2026-06-10T18:11:45.672Z)

```
Exit code 1
unknown flag: --stat

Usage:  gh pr diff [<number> | <url> | <branch>] [flags]

Flags:
      --color string       Use color in diff output: {always|never|auto} (default "auto")
  -e, --exclude patterns   Exclude files matching glob patterns from the diff
      --name-only          Display only names of changed files
      --patch              Display diff in patch format
  -w, --web                Open the pull request diff in the browser

test (ubuntu-latest, 3.11)      pending 0
https://github.com/avidullu/muneem/actions/runs/27296377394/job/80630045175
test (ubuntu-latest, 3.12)      pending 0
https://github.com/avidullu/muneem/actions/runs/27296377394/job/80630045111
test (ubuntu-latest, 3.13)      pending 0
https://github.com/avidullu/muneem/actions/runs/27296377394/job/80630045173
test (windows-latest, 3.13)     pending 0
https://github.com/avidullu/muneem/actions/runs/27296377394/job/80630045104
ocr     skipping        0
https://github.com/avidullu/muneem/actions/runs/27296377394/job/80630045916
test (ubuntu-latest, 3.11)      pass    35s
https://github.com/avidullu/muneem/actions/runs/27296351013/job/80629950438
test (ubuntu-latest, 3.12)      pass    46s
https://github.com/avidullu/muneem/actions/runs/27296351013/job/80629950341
test (ubuntu-latest, 3.13)      pass    39s
https://github.com/avidullu/muneem/actions/runs/27296351013/job/80629950267
```

451. user (2026-06-10T18:11:56.180Z)

```
tests/synthpdf.py
tests/test_regression_synth.py
 * branch            test/synthetic-corpus-breadth -> FETCH_HEAD
 tests/synthpdf.py              | 118 ++++++++++++---
 tests/test_regression_synth.py | 340 +++++++++++++++++++++++++++++++++++------
 2 files changed, 392 insertions(+), 66 deletions(-)
```

452. assistant (2026-06-10T18:12:10.494Z)

Test-only (two files). Reading the full patch:

453. user (2026-06-10T18:12:12.679Z)

```
diff --git a/tests/synthpdf.py b/tests/synthpdf.py
index 63a0255..4dd4a3e 100644
--- a/tests/synthpdf.py
+++ b/tests/synthpdf.py
@@ -10,11 +10,18 @@ to PDF's bottom-up baseline. Absolute `top` need not be exact ? parsers key
on
 grouping, and row order ? but words on one row share a baseline (one `top`) and rows are
separated.
 Amount columns are **right-aligned** via real Helvetica metrics so their right edges (`x1`)
land in
 the bands the HDFC clusterer / engine expect.
+
+Beyond text, a page can carry stroked **ruling lines** (`m`/`l`/`S` path operators). pdfminer
turns
+a stroked two-point path into an ``LTLine``, which is exactly what pdfplumber's default
+``extract_tables()`` "lines" strategy keys on ? so a :func:`grid` makes a hand-rolled table
visible
+to the cell-matrix extraction path (the Axis credit-card layout). Multi-page statements are
built
+with :func:`build_pdf_pages`; :func:`build_pdf` stays the single-page convenience.
 """

 from __future__ import annotations

-from dataclasses import dataclass
+from collections.abc import Sequence
+from dataclasses import dataclass, field
 from pathlib import Path

 # Helvetica advance widths (per 1000 em). Exact for the chars that matter for right-aligned
amounts
@@ -92,34 +99,92 @@ def right(x1: float, top: float, text: str, size: float = 9.0) -> Placement:
     return Placement(x=x1 - helvetica_width(text, size), top=top, text=text, size=size)


-def _esc(text: str) -> str:
-    return text.replace("\\", r"\\").replace("(", r"\(").replace(")", r"\)")
+@dataclass
+class Line:
+    """One stroked straight segment, in the same top-down coordinates as :class:`Placement`.

+    pdfminer reports a stroked two-point path as an ``LTLine``, which feeds pdfplumber's
+    ``extract_tables()`` "lines" strategy ? draw a full :func:`grid` to make a table ruled.
+    """
+
+    x0: float
+    top0: float
+    x1: float
+    top1: float
+    width: float = 0.75

-def build_pdf(
-    placements: list[Placement], path: Path | str, *, width: float = 595.0, height: float =
842.0
-) -> Path:
-    """Write a single-page PDF with the given positioned text. Returns the path.

-    Hand-rolls a minimal PDF (catalog ? pages ? page ? Helvetica font ? content stream) with a
-    byte-accurate xref table so pdfminer/pdfplumber parse it.
+def grid(x_edges: Sequence[float], top_edges: Sequence[float]) -> list[Line]:
+    """The full ruling of a table: a vertical line per column edge, horizontal per row edge.
+
+    ``extract_tables()`' default strategy intersects these to find cells, so every boundary
```

```
+    must be drawn (a borderless or semi-ruled table is exactly what it fails on).
     """
+    top_lo, top_hi = min(top_edges), max(top_edges)
+    x_lo, x_hi = min(x_edges), max(x_edges)
+    lines = [Line(x, top_lo, x, top_hi) for x in x_edges]
+    lines += [Line(x_lo, t, x_hi, t) for t in top_edges]
+    return lines
+
+
+@dataclass
+class Page:
+    """One page's content: positioned text plus any ruling lines."""
+
+    placements: list[Placement]
+    lines: list[Line] = field(default_factory=list)
+
+
+def _esc(text: str) -> str:
+    return text.replace("\\", r"\\").replace("(", r"\(").replace(")", r"\)")
+
+
+def _content_stream(page: Page, height: float) -> bytes:
+    """Render one page's lines + text runs as PDF content-stream operators."""
     ops: list[str] = []
-    for p in placements:
+    for ln in page.lines:
+        # q/Q isolate the line width; top-down -> bottom-up like the text path below.
+        ops.append(
+            f"q {ln.width:.2f} w {ln.x0:.2f} {height - ln.top0:.2f} m "
+            f"{ln.x1:.2f} {height - ln.top1:.2f} l S Q"
+        )
+    for p in page.placements:
         y = height - p.top  # top-down -> PDF bottom-up baseline
         ops.append(f"BT /F1 {p.size:.2f} Tf 1 0 0 1 {p.x:.2f} {y:.2f} Tm ({_esc(p.text)}) Tj
ET")
-    content = ("\n".join(ops)).encode("latin-1")
+    return ("\n".join(ops)).encode("latin-1")
+

+def build_pdf_pages(
+    pages: Sequence[Page], path: Path | str, *, width: float = 595.0, height: float = 842.0
+) -> Path:
+    """Write a multi-page PDF with the given per-page content. Returns the path.
+
+    Hand-rolls a minimal PDF (catalog ? pages ? font ? page/content pairs) with a byte-accurate
+    xref table so pdfminer/pdfplumber parse it. Object layout: 1 catalog, 2 page tree, 3 the
shared
+    Helvetica font, then each page as a (page, content) object pair.
+    """
+    page_obj_ids = [4 + 2 * i for i in range(len(pages))]
+    kids = " ".join(f"{n} 0 R" for n in page_obj_ids)
     objects: list[bytes] = [
         b"<< /Type /Catalog /Pages 2 0 R >>",
-        b"<< /Type /Pages /Kids [3 0 R] /Count 1 >>",
-        (
-            f"<< /Type /Page /Parent 2 0 R /MediaBox [0 0 {width:.0f} {height:.0f}] "
-            f"/Resources << /Font << /F1 4 0 R >> >> /Contents 5 0 R >>"
-        ).encode("latin-1"),
+        f"<< /Type /Pages /Kids [{kids}] /Count {len(pages)} >>".encode("latin-1"),
         b"<< /Type /Font /Subtype /Type1 /BaseFont /Helvetica >>",
-        b"<< /Length " + str(len(content)).encode() + b" >>\nstream\n" + content +
b"\nendstream",
     ]
+    for i, page in enumerate(pages):
+        content = _content_stream(page, height)
```

```
+        objects.append(
+            (
+                f"<< /Type /Page /Parent 2 0 R /MediaBox [0 0 {width:.0f} {height:.0f}] "
+                f"/Resources << /Font << /F1 3 0 R >> >> /Contents {page_obj_ids[i] + 1} 0 R
>>"
+            ).encode("latin-1")
+        )
+        objects.append(
+            b"<< /Length "
+            + str(len(content)).encode()
+            + b" >>\nstream\n"
+            + content
+            + b"\nendstream"
+        )

     out = bytearray(b"%PDF-1.4\n")
     offsets: list[int] = []
@@ -143,4 +208,21 @@ def build_pdf(
     return out_path


-__all__ = ["Placement", "at", "right", "helvetica_width", "build_pdf"]
+def build_pdf(
+    placements: list[Placement], path: Path | str, *, width: float = 595.0, height: float =
842.0
+) -> Path:
+    """Write a single-page PDF with the given positioned text. Returns the path."""
+    return build_pdf_pages([Page(placements)], path, width=width, height=height)
+
+
+__all__ = [
+    "Placement",
+    "Line",
+    "Page",
+    "at",
+    "right",
+    "grid",
+    "helvetica_width",
+    "build_pdf",
+    "build_pdf_pages",
+]
```

454. user (2026-06-10T18:12:20.783Z)

```
diff --git a/tests/test_regression_synth.py b/tests/test_regression_synth.py
index 3771624..a7fefe8 100644
--- a/tests/test_regression_synth.py
+++ b/tests/test_regression_synth.py
@@ -1,42 +1,60 @@
 """Layer-1 regression CI ? full-pipeline fixtures on synthetic born-digital PDFs.

 Generates *real* PDFs (no committed statements, no secrets) and runs the production pipeline ?
-``extract`` (pdfminer) ? ``get_parser`` ? ``parse_and_insert`` ? assert a **reconciled** parse.
This
+``extract`` (pdfminer) ? ``get_parser`` ? ``parse_and_insert`` ? assert the recorded verdict.
This
 covers the extraction / coordinate-clustering / engine layers end-to-end, the gap the synthetic
 word-list & table unit tests (which bypass pdfplumber) don't reach. See ROADMAP § "Planned ?
 Regression CI for statement parsing" (Layer 1). Always-on, secret-free ? runs in cloud CI.
+
+Corpus breadth (deep-review D1): per-issuer layouts (HDFC, AU, Axis savings), the ruled Axis
+credit-card table through ``extract_tables()``, a 2-page statement with a repeated header, a
+bookend-violating statement that must quarantine, and a password-protected statement driven
+through the real CLI hybrid-unlock path.
 """
```

```
-from pathlib import Path
+import pypdf
+from typer.testing import CliRunner

 from muneem import db as dbmod
+from muneem import statement_unlock
+from muneem.cli import app
 from muneem.extract import extract
 from muneem.parsers import get_parser
-from synthpdf import at, build_pdf, right
+from muneem.unlock import is_encrypted
+from synthpdf import Page, Placement, at, build_pdf, build_pdf_pages, grid, right

+runner = CliRunner()

-def _hdfc_pdf(path: Path) -> Path:
-    """HDFC layout: leading date x0?52, narration x0?111, three right-aligned amount columns
whose
-    right edges land in the clusterer's bands (withdrawal <400, deposit <490, balance ?490),
plus a
-    SUMMARY block for the bookend. Three rows reconcile (opening 10,000 ? closing 10,000)."""
-    p = [
+
+# --- HDFC layout (bespoke coordinate clustering; SUMMARY bookend)
------------------------------
+
+
+def _hdfc_letterhead() -> list[Placement]:
+    return [
         at(50, 60, "HDFC BANK"),
         at(50, 72, "Savings Account Details"),
         at(50, 84, "Account Number : 50100012345678"),
-        # header ? must carry "Narration" + a "Closing" token so _table_body locates the table
-        at(72, 110, "Date"),
-        at(175, 110, "Narration"),
-        at(300, 110, "Withdrawals"),
-        at(390, 110, "Deposits"),
-        at(480, 110, "Closing"),
-        at(515, 110, "Balance"),
     ]
-    rows = [
-        ("05/02/2026", "UPI-SWIGGY-PAY", "500.00", "0.00", "9,500.00"),
-        ("08/02/2026", "SALARY-CREDIT", "0.00", "2,000.00", "11,500.00"),
-        ("10/02/2026", "ATM-WDL", "1,500.00", "0.00", "10,000.00"),
+
+
+def _hdfc_table_header(top: float = 110) -> list[Placement]:
+    """Column header ? must carry "Narration" + a "Closing" token so _table_body locates the
+    table; repeated on every page of a real HDFC statement."""
+    return [
+        at(72, top, "Date"),
+        at(175, top, "Narration"),
+        at(300, top, "Withdrawals"),
+        at(390, top, "Deposits"),
+        at(480, top, "Closing"),
+        at(515, top, "Balance"),
     ]
-    top = 130
+
+
+def _hdfc_rows(rows: list[tuple[str, str, str, str, str]], top: float = 130) ->
list[Placement]:
+    """Transaction rows in HDFC geometry: leading date x0?52, narration x0?111, three
+    right-aligned amount columns whose right edges land in the clusterer's bands
+    (withdrawal <400, deposit <490, balance ?490)."""
```

```
+    p: list[Placement] = []
     for d, narr, wd, dep, bal in rows:
         p += [
             at(52, top, d),
@@ -46,49 +64,195 @@ def _hdfc_pdf(path: Path) -> Path:
             right(561, top, bal),
         ]
         top += 15
-    p += [
-        at(50, 200, "SUMMARY"),
-        at(50, 212, "Opening Balance Debit Amount Credit Amount Closing Balance"),
-        at(50, 224, "10,000.00 2,000.00 2,000.00 10,000.00"),
-        at(50, 236, "Debit Count Credit Count"),
-        at(50, 248, "2 1"),
+    return p
+
+
+def _hdfc_summary(figures: str, counts: str, top: float = 200) -> list[Placement]:
+    return [
+        at(50, top, "SUMMARY"),
+        at(50, top + 12, "Opening Balance Debit Amount Credit Amount Closing Balance"),
+        at(50, top + 24, figures),
+        at(50, top + 36, "Debit Count Credit Count"),
+        at(50, top + 48, counts),
     ]
+
+
+def _hdfc_pdf(path):
+    """Single-page HDFC statement: three rows that walk (opening 10,000 ? closing 10,000) plus
+    the SUMMARY block for the bookend."""
+    rows = [
+        ("05/02/2026", "UPI-SWIGGY-PAY", "500.00", "0.00", "9,500.00"),
+        ("08/02/2026", "SALARY-CREDIT", "0.00", "2,000.00", "11,500.00"),
+        ("10/02/2026", "ATM-WDL", "1,500.00", "0.00", "10,000.00"),
     ]
+    p = (
+        _hdfc_letterhead()
+        + _hdfc_table_header()
+        + _hdfc_rows(rows)
+        + _hdfc_summary("10,000.00 2,000.00 2,000.00 10,000.00", "2 1")
+    )
     return build_pdf(p, path)


-def _au_savings_pdf(path: Path) -> Path:
-    """Savings layout (the shared savings.py engine path used by Axis/AU): a clean
-    date | narration | debit | credit | balance table in well-separated columns ? coordinate
-    clustering ? engine mapping ? per-row reconcile. Three rows that walk (opening 50,000)."""
-    p = [
-        at(50, 60, "AU Small Finance Bank"),
-        at(50, 72, "Account Number : 1234567890123"),
-        # header (clustering's date+money filter drops it; here only for realism / text
markers)
+def _hdfc_multipage_pdf(path):
+    """Two-page HDFC statement: the transaction band splits across pages with the column header
+    repeated (as the real statements do), SUMMARY on the last page. The walk and the SUMMARY
+    bookend both span the page break, so a clean verdict proves page stitching."""
+    page1_rows = [
+        ("05/02/2026", "UPI-SWIGGY-PAY", "500.00", "0.00", "9,500.00"),
+        ("08/02/2026", "SALARY-CREDIT", "0.00", "2,000.00", "11,500.00"),
+        ("10/02/2026", "ATM-WDL", "1,500.00", "0.00", "10,000.00"),
+    ]
+    page2_rows = [
+        ("12/02/2026", "UPI-BIGBASKET", "800.00", "0.00", "9,200.00"),
+        ("15/02/2026", "NEFT-REFUND", "0.00", "1,300.00", "10,500.00"),
```

```
+    ]
+    page1 = _hdfc_letterhead() + _hdfc_table_header() + _hdfc_rows(page1_rows)
+    page2 = (
+        _hdfc_table_header()
+        + _hdfc_rows(page2_rows)
+        + _hdfc_summary("10,000.00 2,800.00 3,300.00 10,500.00", "3 2")
+    )
+    return build_pdf_pages([Page(page1), Page(page2)], path)
+
+
+# --- Shared savings layout (the engine-driven savings.py path used by Axis/AU)
-----------------
+
+
+def _savings_pdf(
+    path,
+    letterhead: list[str],
+    rows: list[tuple[str, str, str, str, str]],
+    *,
+    opening: str | None = None,
+    closing: str | None = None,
+):
+    """Whitespace-aligned savings table: date | narration | debit | credit | balance in
+    well-separated columns ? coordinate clustering ? engine mapping ? per-row reconcile.
+
+    ``letterhead`` carries the routing markers and the account line; ``opening``/``closing``
+    print the summary lines the trust oracle anchors a passing walk to
(savings.reconcile_oracle).
+    """
+    p: list[Placement] = []
+    top = 60
+    for line in letterhead:
+        p.append(at(50, top, line))
+        top += 12
+    # header (clustering's date+money filter drops it; here only for realism / text markers)
+    p += [
         at(52, 100, "Date"),
         at(110, 100, "Transaction"),
         at(310, 100, "Withdrawal"),
         at(395, 100, "Deposit"),
         at(485, 100, "Balance"),
     ]
-    rows = [
-        ("02/08/2025", "UPI-ZOMATO", "1,200.00", "", "48,800.00"),
-        ("05/08/2025", "SALARY", "", "30,000.00", "78,800.00"),
-        ("10/08/2025", "ATM-CASH", "2,000.00", "", "76,800.00"),
-    ]
     top = 124
     for d, narr, debit, credit, bal in rows:
-        line = [at(52, top, d), at(110, top, narr), right(520, top, bal)]
+        line_p = [at(52, top, d), at(110, top, narr), right(520, top, bal)]
         if debit:
-            line.append(right(340, top, debit))
+            line_p.append(right(340, top, debit))
         if credit:
-            line.append(right(420, top, credit))
-        p += line
+            line_p.append(right(420, top, credit))
+        p += line_p
         top += 15
+    if opening is not None:
+        p.append(at(50, top + 20, f"Opening Balance : {opening}"))
+    if closing is not None:
+        p.append(at(50, top + 32, f"Closing Balance : {closing}"))
     return build_pdf(p, path)
```

```
-def _run_pipeline(pdf: Path) -> tuple[str, int, str]:
-    """Mirror `muneem ingest`: extract text ? pick parser ? parse+insert ? read back status."""
+_AU_LETTERHEAD = ["AU Small Finance Bank", "Account Number : 1234567890123"]
+# Three rows that walk from an implied opening of 50,000.00.
+_AU_ROWS = [
+    ("02/08/2025", "UPI-ZOMATO", "1,200.00", "", "48,800.00"),
+    ("05/08/2025", "SALARY", "", "30,000.00", "78,800.00"),
+    ("10/08/2025", "ATM-CASH", "2,000.00", "", "76,800.00"),
+]
+
+
+def _au_savings_pdf(path):
+    return _savings_pdf(path, _AU_LETTERHEAD, _AU_ROWS)
+
+
+def _axis_savings_pdf(path):
+    """Axis savings: AXIS_PROFILE dates (dd-mm-yyyy) + the letterhead markers AxisParser routes
+    on, with printed Opening/Closing Balance lines consistent with the rows ? exercising the
+    bookend-anchored trust oracle on the happy path."""
+    rows = [
```

455. user (2026-06-10T18:12:28.005Z)

```
+        ("03-09-2025", "UPI-FLIPKART", "5,000.00", "", "20,000.00"),
+        ("10-09-2025", "NEFT-SALARY", "", "40,000.00", "60,000.00"),
+        ("18-09-2025", "ATM-WDL", "10,000.00", "", "50,000.00"),
+    ]
+    return _savings_pdf(
+        path,
+        ["Axis Bank", "Statement of Account", "Account Number : 912345678901"],
+        rows,
+        opening="25,000.00",
+        closing="50,000.00",
+    )
+
+
+def _au_dropped_first_row_pdf(path):
+    """The Axis May-2025 failure class, reproduced at the PDF level: the printed rows are
+    internally walk-consistent (the chain 78,800 ? 76,800 holds) but the FIRST transaction of
+    the real set is missing, so the first row's implied opening (48,800) contradicts the
printed
+    Opening Balance (50,000) of the full set. Only the bookend anchoring can catch this."""
+    return _savings_pdf(
+        path, _AU_LETTERHEAD, _AU_ROWS[1:], opening="50,000.00", closing="76,800.00"
+    )
+
+
+# --- Axis credit card (ruled table through pdfplumber extract_tables)
--------------------------
+
+
+def _axis_cc_pdf(path):
+    """Axis credit-card statement: a fully *ruled* 9-column table (a synthpdf `grid` of stroked
+    lines), because the CC parser reads ``page.extract_tables()`` whose default "lines"
strategy
+    needs real ruling. Column contract per axis._rows_from_cc_tables: date col 0 (dd/mm/yyyy),
+    particulars col 2, merchant category col 7, amount col 8 with a Dr/Cr suffix."""
+    x_edges = [40.0, 100.0, 140.0, 280.0, 315.0, 350.0, 385.0, 420.0, 510.0, 565.0]
+    tops = [150.0, 170.0, 190.0, 210.0, 230.0]
+    p = [
+        at(50, 60, "Axis Bank"),
+        at(50, 72, "Credit Card Statement"),
+        at(50, 84, "Card Number : 4321XXXX9876"),
+    ]
+    header = (
```

```
+        "Date",
+        "Ref",
+        "Transaction Details",
+        "EMI",
+        "Intl",
+        "Cash",
+        "Pts",
+        "Merchant Category",
+        "Amount",
+    )
+    rows = [
+        ("05/01/2026", "", "AMAZON PAY INDIA", "", "", "", "", "ECOMMERCE", "2,499.00 Dr"),
+        ("11/01/2026", "", "SWIGGY BANGALORE", "", "", "", "", "RESTAURANT", "651.50 Dr"),
+        ("18/01/2026", "", "PAYMENT RECEIVED", "", "", "", "", "PAYMENT", "3,000.00 Cr"),
+    ]
+    for table_row, cells in enumerate((header, *rows)):
+        for col, text in enumerate(cells):
+            if text:
+                p.append(at(x_edges[col] + 3, tops[table_row] + 14, text, size=8))
+    return build_pdf_pages([Page(p, lines=grid(x_edges, tops))], path)
+
+
+# --- The pipeline under test
---------------------------------------------------------------------
+
+
+_TXN_TABLES = ("axis_transactions", "axis_cc_transactions", "au_transactions",
"hdfc_transactions")
+
+
+def _run_pipeline(pdf) -> tuple[str, int, str, int]:
+    """Mirror `muneem ingest`: extract text ? pick parser ? parse+insert ? read back the
verdict.
+
+    Returns ``(parser name, rows reported, document status, rows actually stored across every
+    per-issuer table)`` ? the stored count lets quarantine tests prove nothing landed.
+    """
     ex = extract(pdf)
     parser = get_parser(ex.text)
     assert parser is not None, "no parser routed this synthetic statement"
@@ -97,18 +261,98 @@ def _run_pipeline(pdf: Path) -> tuple[str, int, str]:
         doc_id = dbmod.insert_document(conn, source="file", sha256=ex.sha256, status="parsed")
         n = parser.parse_and_insert(pdf, conn, doc_id)
         status = dbmod.get_document_by_sha(conn, ex.sha256)["status"]
-    return type(parser).__name__, n, status
+        stored = sum(conn.execute(f"SELECT COUNT(*) FROM {t}").fetchone()[0] for t in
_TXN_TABLES)
+    return type(parser).__name__, n, status, stored


 def test_hdfc_layout_full_pipeline_reconciles(tmp_path):
-    name, n, status = _run_pipeline(_hdfc_pdf(tmp_path / "hdfc.pdf"))
+    name, n, status, stored = _run_pipeline(_hdfc_pdf(tmp_path / "hdfc.pdf"))
     assert name == "HDFCParser"
     assert n == 3, f"expected 3 transactions, got {n}"
     assert status == "reconciled", f"expected reconciled, got {status}"
+    assert stored == 3


 def test_savings_layout_full_pipeline_reconciles(tmp_path):
-    name, n, status = _run_pipeline(_au_savings_pdf(tmp_path / "au.pdf"))
+    name, n, status, stored = _run_pipeline(_au_savings_pdf(tmp_path / "au.pdf"))
     assert name == "AUParser"
     assert n == 3, f"expected 3 transactions, got {n}"
     assert status == "reconciled", f"expected reconciled, got {status}"
```

```
+    assert stored == 3
+
+
+def test_axis_savings_layout_full_pipeline_reconciles(tmp_path):
+    """Axis savings routes via the title-case letterhead evidence and reconciles through the
+    shared engine path, with the printed bookends anchoring the walk."""
+    name, n, status, stored = _run_pipeline(_axis_savings_pdf(tmp_path / "axis.pdf"))
+    assert name == "AxisParser"
+    assert n == 3, f"expected 3 transactions, got {n}"
+    assert status == "reconciled", f"expected reconciled, got {status}"
+    assert stored == 3
+
+
+def test_axis_cc_ruled_table_persists_unverified(tmp_path):
+    """A CC bill has no running balance, so no oracle can endorse it: rows persist but the
+    document is explicitly 'unverified' (Confidence.UNVERIFIED), never silently trusted."""
+    name, n, status, stored = _run_pipeline(_axis_cc_pdf(tmp_path / "axis-cc.pdf"))
+    assert name == "AxisCreditCardParser"
+    assert n == 3, f"expected 3 transactions, got {n}"
+    assert status == "unverified", f"expected unverified, got {status}"
+    assert stored == 3
+
+
+def test_multipage_hdfc_reconciles_across_page_break(tmp_path):
+    """Rows split across two pages (header repeated) must stitch into one combined set: the
+    balance walk and the last-page SUMMARY bookend both span the page break."""
+    name, n, status, stored = _run_pipeline(_hdfc_multipage_pdf(tmp_path / "hdfc-2p.pdf"))
+    assert name == "HDFCParser"
+    assert n == 5, f"expected 5 transactions, got {n}"
+    assert status == "reconciled", f"expected reconciled, got {status}"
+    assert stored == 5
+
+
+def test_dropped_first_row_quarantines_at_pdf_level(tmp_path):
+    """A statement missing its first transaction still walks internally ? only the printed
+    Opening Balance exposes the gap. The pipeline must store NOTHING and record 'unreconciled'
+    (the bookend anchoring in savings.reconcile_oracle)."""
+    name, n, status, stored = _run_pipeline(_au_dropped_first_row_pdf(tmp_path / "au-gap.pdf"))
+    assert name == "AUParser"
+    assert n == 0, f"expected 0 rows reported, got {n}"
+    assert status == "unreconciled", f"expected unreconciled, got {status}"
+    assert stored == 0, f"quarantine must persist nothing, found {stored} row(s)"
+
+
+def test_encrypted_statement_through_real_cli_hybrid_unlock(tmp_path, monkeypatch):
+    """A password-protected statement driven through the REAL `muneem ingest` CLI: pypdf
+    encryption on a synthetic AU statement, unlocked by the genuine hybrid path (the keychain
+    candidate cache, seeded the same way a successful prompt-unlock caches a password ?
+    resolve_password itself is NOT patched). Asserts exit 0 and reconciled rows in the ledger.
+    """
+    plain = _au_savings_pdf(tmp_path / "plain.pdf")
+    enc = tmp_path / "statement.pdf"
+    reader = pypdf.PdfReader(str(plain))
+    writer = pypdf.PdfWriter()
+    writer.append(reader)
+    writer.encrypt("TEST1234")  # synthetic fixture password ? not real material
+    with open(enc, "wb") as fh:
+        writer.write(fh)
+    assert is_encrypted(enc), "the fixture must actually exercise the unlock path"
+
+    # Hermetic: a real password_rules.yaml on the dev machine must not feed candidates.
+    monkeypatch.setattr("muneem.password_rules.CONFIG_DIR", tmp_path / "no-config")
+    monkeypatch.chdir(tmp_path)
+    # Seed the keychain candidate cache through the legitimate seam (conftest isolates the
+    # keychain to an in-memory store, so nothing touches the real OS keychain).
```

```
+    statement_unlock._remember("TEST1234")
+
+    db = tmp_path / "ledger.db"
+    result = runner.invoke(
+        app, ["ingest", str(enc), "--doc-type", "bank_statement", "--db", str(db), "--no-
prompt"]
+    )
+    assert result.exit_code == 0, result.output
+    assert "[reconciled]" in result.output and "3 transaction(s)" in result.output
+
+    with dbmod.connect_encrypted(db) as conn:
+        rows = dbmod.list_au_transactions(conn, limit=10)
+    assert len(rows) == 3
+    assert all(r["doc_status"] == "reconciled" for r in rows)
```

456. user (2026-06-10T18:12:40.582Z)

```
test (windows-latest, 3.13)     pending 0
https://github.com/avidullu/muneem/actions/runs/27296377394/job/80630045104
test (ubuntu-latest, 3.11)      pass    39s
https://github.com/avidullu/muneem/actions/runs/27296377394/job/80630045175
test (ubuntu-latest, 3.12)      pass    1m0s
https://github.com/avidullu/muneem/actions/runs/27296377394/job/80630045111
test (ubuntu-latest, 3.13)      pass    42s
https://github.com/avidullu/muneem/actions/runs/27296377394/job/80630045173
ocr     skipping        0
https://github.com/avidullu/muneem/actions/runs/27296377394/job/80630045916
test (ubuntu-latest, 3.11)      pass    35s
https://github.com/avidullu/muneem/actions/runs/27296351013/job/80629950438
```

457. user (2026-06-10T18:12:57.356Z)

```
fatal: 'test/synthetic-corpus-breadth' is already used by worktree at
'C:/Users/avidu/Projects/muneem/.claude/worktrees/agent-a0a70ddb1d7197b83'
..                                                                       [100%]
2 passed in 0.38s
.....                                                                    [100%]
509 passed, 21 deselected in 4.39s
Your branch is up to date with 'origin/main'.
```

458. assistant (2026-06-10T18:13:06.177Z)

```
The branch lives in the agent's worktree ? running the suite there directly with the worktree's
src under test:
```

459. user (2026-06-10T18:13:14.232Z)

```
..........                                                               [100%]
514 passed, 6 deselected in 4.58s
```

460. user (2026-06-10T18:13:23.341Z)

```
test (windows-latest, 3.13)     pass    1m51s
https://github.com/avidullu/muneem/actions/runs/27296377394/job/80630045104
test (windows-latest, 3.13)     pass    1m39s
https://github.com/avidullu/muneem/actions/runs/27296351013/job/80629950373
```

461. assistant (2026-06-10T18:13:30.007Z)

```
All green ? full CI matrix plus my local Windows run (514 passed). Merging:
```

462. user (2026-06-10T18:13:37.396Z)

```
failed to delete local branch test/synthetic-corpus-breadth: failed to run git: error:
```

```
cannot delete branch 'test/synthetic-corpus-breadth' used by worktree at
'C:/Users/avidu/Projects/muneem/.claude/worktrees/agent-a0a70ddb1d7197b83'

Your branch is up to date with 'origin/main'.
 tests/test_regression_synth.py | 340 ++++++++++++++++++++++++++++++++++++------
 2 files changed, 392 insertions(+), 66 deletions(-)
ee8a8c2 test: broaden the synthetic regression corpus (deep-review D1) (#70)
b548fb6 docs(current-state): post-hardening status + a decision-oriented Next Steps (#69)
```

463. assistant (2026-06-10T18:13:44.737Z)

Merged as `ee8a8c2`. Cleaning up the agent worktree that's pinning the local branch:

464. user (2026-06-10T18:13:53.630Z)

```
Deleted branch test/synthetic-corpus-breadth (was b01a71d).
C:/Users/avidu/Projects/muneem  ee8a8c2 [main]
514 passed, 21 deselected in 4.35s
```

465. user (2026-06-10T18:14:00.477Z)

Updated task #16 status

466. user (2026-06-10T18:14:06.181Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\memory\session-handoff.md has been updated successfully. (file state is current in your context ? no need to Read it back)

467. assistant (2026-06-10T18:14:21.800Z)

D1 is done ? [#70](https://github.com/avidullu/muneem/pull/70) reviewed and merged. The verdict on the agent's work after reading the full diff: genuinely good. Highlights worth knowing about:

- **All five fixtures shipped**, including the one I expected to be infeasible: a truly *ruled* 9-column Axis CC table, done by teaching `synthpdf.py` to stroke PDF line paths (`m`/`l`/`S` operators) so pdfplumber's `extract_tables()` sees real ruling ? the CC parser now has full-pipeline CI coverage for the first time.
- **The dropped-row quarantine test is the crown jewel**: a statement that walks perfectly but is missing its first transaction against the printed opening balance ? it stores nothing. The agent mutation-tested it (disabled the bookend anchoring from #53, watched the test wrongly persist rows, reverted), so the test provably guards exactly that fix.
- **The encrypted-PDF test drives the real CLI** end-to-end ? pypdf-encrypted statement through `muneem ingest`, the genuine hybrid unlock (candidate cache seeded through the legitimate seam, nothing patched), rows read back from the encrypted ledger.

Verified before merging: full CI matrix green + my own local Windows run of the branch (514 passed). The agent's worktree is cleaned up; `main` is at `ee8a8c2` with the suite green.

Remaining: only the sister-repo agent is still working (the two gmail-repo PRs, which stay unmerged for your review since merging deploys). The handoff and CURRENT_STATE §5 are current ? your fresh-look entry points are [CURRENT_STATE §5](docs/CURRENT_STATE.md) ? [ROADMAP §0](docs/ROADMAP.md), and the one thing only you can run remains the `pytest -m local_statements` confirmation.

468. user (2026-06-10T18:14:24.252Z)

I ran the command but got

```
"
pytest -m local_statements
============================================================ test session starts
============================================================
```

```
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\avidu\Projects\muneem
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.13.0, cov-7.1.0
collected 44 items / 35 errors / 44 deselected / 2 skipped / 0 selected

=========================================================================== ERRORS
===========================================================================
_____________________________________________________ ERROR collecting tests/ingestion/test_gmail.py
_____________________________________________________
ImportError while importing test module
'C:\Users\avidu\Projects\muneem\tests\ingestion\test_gmail.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\ingestion\test_gmail.py:5: in <module>
    from muneem.ingestion.base import SearchCriteria
E   ModuleNotFoundError: No module named 'muneem'
_____________________________________________ ERROR collecting tests/ingestion/test_providers.py
_____________________________________________
ImportError while importing test module
'C:\Users\avidu\Projects\muneem\tests\ingestion\test_providers.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\ingestion\test_providers.py:1: in <module>
    from muneem.ingestion.base import SearchCriteria
E   ModuleNotFoundError: No module named 'muneem'
_____________________________________________________ ERROR collecting tests/parsers/test_au.py
_____________________________________________________
ImportError while importing test module
'C:\Users\avidu\Projects\muneem\tests\parsers\test_au.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\parsers\test_au.py:11: in <module>
    import muneem.db as dbmod
E   ModuleNotFoundError: No module named 'muneem'
_____________________________________________________ ERROR collecting tests/parsers/test_axis.py
_____________________________________________________
ImportError while importing test module
'C:\Users\avidu\Projects\muneem\tests\parsers\test_axis.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\parsers\test_axis.py:12: in <module>
    import muneem.db as dbmod
E   ModuleNotFoundError: No module named 'muneem'
_____________________________________________ ERROR collecting tests/parsers/test_clustering.py
_____________________________________________
ImportError while importing test module
'C:\Users\avidu\Projects\muneem\tests\parsers\test_clustering.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
```

```
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\parsers\test_clustering.py:7: in <module>
    from muneem.parsers.clustering import cluster_to_matrix, group_lines, infer_columns
E   ModuleNotFoundError: No module named 'muneem'
_____________________________________________ ERROR collecting tests/parsers/test_dates.py
_____________________________________________
ImportError while importing test module
'C:\Users\avidu\Projects\muneem\tests\parsers\test_dates.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\parsers\test_dates.py:4: in <module>
    from muneem.parsers.dates import normalize_statement_date
E   ModuleNotFoundError: No module named 'muneem'
_____________________________________________ ERROR collecting tests/parsers/test_engine.py
_____________________________________________
ImportError while importing test module
'C:\Users\avidu\Projects\muneem\tests\parsers\test_engine.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\parsers\test_engine.py:11: in <module>
    from muneem.parsers.engine import BANK_FAMILY, FamilySpec, Profile, map_table
E   ModuleNotFoundError: No module named 'muneem'
_____________________________________________ ERROR collecting tests/parsers/test_hdfc.py
_____________________________________________
ImportError while importing test module
'C:\Users\avidu\Projects\muneem\tests\parsers\test_hdfc.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\parsers\test_hdfc.py:14: in <module>
    from muneem import db as dbmod
E   ModuleNotFoundError: No module named 'muneem'
_____________________________________________ ERROR collecting tests/parsers/test_local_au.py
_____________________________________________
ImportError while importing test module
'C:\Users\avidu\Projects\muneem\tests\parsers\test_local_au.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\parsers\test_local_au.py:15: in <module>
    from muneem import db as dbmod
E   ModuleNotFoundError: No module named 'muneem'
_____________________________________________ ERROR collecting tests/parsers/test_local_axis.py
_____________________________________________
ImportError while importing test module
'C:\Users\avidu\Projects\muneem\tests\parsers\test_local_axis.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\parsers\test_local_axis.py:17: in <module>
    from muneem import db as dbmod
E   ModuleNotFoundError: No module named 'muneem'
_____________________________________________ ERROR collecting tests/parsers/test_parse_errors.py
```

```
_____________________________________________
ImportError while importing test module
'C:\Users\avidu\Projects\muneem\tests\parsers\test_parse_errors.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\parsers\test_parse_errors.py:12: in <module>
    from muneem import db as dbmod
E   ModuleNotFoundError: No module named 'muneem'
_____________________________________________ ERROR collecting tests/parsers/test_savings.py
_____________________________________________
ImportError while importing test module
'C:\Users\avidu\Projects\muneem\tests\parsers\test_savings.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\parsers\test_savings.py:9: in <module>
    from muneem.parsers import savings
E   ModuleNotFoundError: No module named 'muneem'
_____________________________________________ ERROR collecting tests/parsers/test_validation.py
_____________________________________________
ImportError while importing test module
'C:\Users\avidu\Projects\muneem\tests\parsers\test_validation.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\parsers\test_validation.py:4: in <module>
    from muneem.parsers.concepts import ConceptRow, StatementConcepts
E   ModuleNotFoundError: No module named 'muneem'
_____________________________________________________ ERROR collecting tests/test_cli.py
_____________________________________________________
ImportError while importing test module 'C:\Users\avidu\Projects\muneem\tests\test_cli.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\test_cli.py:3: in <module>
    from typer.testing import CliRunner
E   ModuleNotFoundError: No module named 'typer'
_____________________________________________________ ERROR collecting tests/test_db.py
_____________________________________________________
ImportError while importing test module 'C:\Users\avidu\Projects\muneem\tests\test_db.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\test_db.py:1: in <module>
    from muneem import db as dbmod
E   ModuleNotFoundError: No module named 'muneem'
_____________________________________________ ERROR collecting tests/test_db_invariants.py
_____________________________________________
ImportError while importing test module
'C:\Users\avidu\Projects\muneem\tests\test_db_invariants.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
```

```
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\test_db_invariants.py:29: in <module>
    from muneem import db as dbmod
E   ModuleNotFoundError: No module named 'muneem'
_____________________________________________________ ERROR collecting tests/test_errors.py
_____________________________________________________
ImportError while importing test module 'C:\Users\avidu\Projects\muneem\tests\test_errors.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\test_errors.py:3: in <module>
    from muneem import db as dbmod
E   ModuleNotFoundError: No module named 'muneem'
_____________________________________________________ ERROR collecting tests/test_eval_cli.py
_____________________________________________________
ImportError while importing test module 'C:\Users\avidu\Projects\muneem\tests\test_eval_cli.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\test_eval_cli.py:7: in <module>
    from typer.testing import CliRunner
E   ModuleNotFoundError: No module named 'typer'
_____________________________________________________ ERROR collecting tests/test_extract.py
_____________________________________________________
ImportError while importing test module 'C:\Users\avidu\Projects\muneem\tests\test_extract.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\test_extract.py:1: in <module>
    from muneem.extract import _normalize
E   ModuleNotFoundError: No module named 'muneem'
_____________________________________________________ ERROR collecting tests/test_log.py
_____________________________________________________
ImportError while importing test module 'C:\Users\avidu\Projects\muneem\tests\test_log.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\test_log.py:5: in <module>
    from muneem.log import RedactingFilter, get_logger
E   ModuleNotFoundError: No module named 'muneem'
_____________________________________________________ ERROR collecting tests/test_migrations.py
_____________________________________________________
ImportError while importing test module
'C:\Users\avidu\Projects\muneem\tests\test_migrations.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\test_migrations.py:7: in <module>
    from muneem import db as dbmod
E   ModuleNotFoundError: No module named 'muneem'
_____________________________________________________ ERROR collecting tests/test_ocr.py
_____________________________________________________
ImportError while importing test module 'C:\Users\avidu\Projects\muneem\tests\test_ocr.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
```

```
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\test_ocr.py:12: in <module>
    from muneem.ocr import ocr_pdf, tesseract_available
E   ModuleNotFoundError: No module named 'muneem'
_____________________________________________________ ERROR collecting tests/test_offers.py
_____________________________________________________
ImportError while importing test module 'C:\Users\avidu\Projects\muneem\tests\test_offers.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\test_offers.py:1: in <module>
    from muneem.offers import extract_offer
E   ModuleNotFoundError: No module named 'muneem'
_____________________________________________ ERROR collecting tests/test_password_rules.py
_____________________________________________
ImportError while importing test module
'C:\Users\avidu\Projects\muneem\tests\test_password_rules.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\test_password_rules.py:11: in <module>
    from muneem import password_rules as pr
E   ModuleNotFoundError: No module named 'muneem'
_____________________________________________ ERROR collecting tests/test_promo_regression.py
_____________________________________________
ImportError while importing test module
'C:\Users\avidu\Projects\muneem\tests\test_promo_regression.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\test_promo_regression.py:13: in <module>
    from muneem.extract import extract
E   ModuleNotFoundError: No module named 'muneem'
_____________________________________________ ERROR collecting tests/test_provenance.py
_____________________________________________
ImportError while importing test module
'C:\Users\avidu\Projects\muneem\tests\test_provenance.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\test_provenance.py:9: in <module>
    from muneem import db as dbmod
E   ModuleNotFoundError: No module named 'muneem'
_____________________________________________________ ERROR collecting tests/test_recovery.py
_____________________________________________________
ImportError while importing test module 'C:\Users\avidu\Projects\muneem\tests\test_recovery.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\test_recovery.py:6: in <module>
    from typer.testing import CliRunner
E   ModuleNotFoundError: No module named 'typer'
_____________________________________________________ ERROR collecting tests/test_redaction.py
```

```
_____________________________________________________
ImportError while importing test module
'C:\Users\avidu\Projects\muneem\tests\test_redaction.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\test_redaction.py:5: in <module>
    from muneem.redaction import mask_secret, redact_filename, redact_text
E   ModuleNotFoundError: No module named 'muneem'
_____________________________________________ ERROR collecting tests/test_regression_synth.py
_____________________________________________
ImportError while importing test module
'C:\Users\avidu\Projects\muneem\tests\test_regression_synth.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\test_regression_synth.py:12: in <module>
    from muneem import db as dbmod
E   ModuleNotFoundError: No module named 'muneem'
_____________________________________________________ ERROR collecting tests/test_secrets.py
_____________________________________________________
ImportError while importing test module 'C:\Users\avidu\Projects\muneem\tests\test_secrets.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\test_secrets.py:8: in <module>
    import keyring
E   ModuleNotFoundError: No module named 'keyring'
_____________________________________________ ERROR collecting tests/test_serving_view.py
_____________________________________________
ImportError while importing test module
'C:\Users\avidu\Projects\muneem\tests\test_serving_view.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\test_serving_view.py:6: in <module>
    from muneem import db as dbmod
E   ModuleNotFoundError: No module named 'muneem'
_____________________________________________________ ERROR collecting tests/test_smoke.py
_____________________________________________________
ImportError while importing test module 'C:\Users\avidu\Projects\muneem\tests\test_smoke.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\test_smoke.py:1: in <module>
    from muneem import __version__
E   ModuleNotFoundError: No module named 'muneem'
_____________________________________________ ERROR collecting tests/test_statement_unlock.py
_____________________________________________
ImportError while importing test module
'C:\Users\avidu\Projects\muneem\tests\test_statement_unlock.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
```

```
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\test_statement_unlock.py:13: in <module>
    from muneem import statement_unlock
E   ModuleNotFoundError: No module named 'muneem'
_____________________________________________________ ERROR collecting tests/test_unlock.py
_____________________________________________________
ImportError while importing test module 'C:\Users\avidu\Projects\muneem\tests\test_unlock.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\test_unlock.py:7: in <module>
    from muneem import unlock
E   ModuleNotFoundError: No module named 'muneem'
_____________________________________________ ERROR collecting tests/test_verifier_properties.py
_____________________________________________
ImportError while importing test module
'C:\Users\avidu\Projects\muneem\tests\test_verifier_properties.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\AppData\Local\Programs\Python\Python313\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests\test_verifier_properties.py:13: in <module>
    from muneem.parsers.concepts import ConceptRow
E   ModuleNotFoundError: No module named 'muneem'
=========================================================== short test summary info
===========================================================
SKIPPED [1] tests\test_db_crypto.py:22: could not import 'sqlcipher3': No module named
'sqlcipher3'
SKIPPED [1] tests\test_db_crypto_ops.py:11: could not import 'sqlcipher3': No module named
'sqlcipher3'
ERROR tests/ingestion/test_gmail.py
ERROR tests/ingestion/test_providers.py
ERROR tests/parsers/test_au.py
ERROR tests/parsers/test_axis.py
ERROR tests/parsers/test_clustering.py
ERROR tests/parsers/test_dates.py
ERROR tests/parsers/test_engine.py
ERROR tests/parsers/test_hdfc.py
ERROR tests/parsers/test_local_au.py
ERROR tests/parsers/test_local_axis.py
ERROR tests/parsers/test_parse_errors.py
ERROR tests/parsers/test_savings.py
ERROR tests/parsers/test_validation.py
ERROR tests/test_cli.py
ERROR tests/test_db.py
ERROR tests/test_db_invariants.py
ERROR tests/test_errors.py
ERROR tests/test_eval_cli.py
ERROR tests/test_extract.py
ERROR tests/test_log.py
ERROR tests/test_migrations.py
ERROR tests/test_ocr.py
ERROR tests/test_offers.py
ERROR tests/test_password_rules.py
ERROR tests/test_promo_regression.py
ERROR tests/test_provenance.py
ERROR tests/test_recovery.py
ERROR tests/test_redaction.py
ERROR tests/test_regression_synth.py
ERROR tests/test_secrets.py
ERROR tests/test_serving_view.py
ERROR tests/test_smoke.py
```

```
ERROR tests/test_statement_unlock.py
ERROR tests/test_unlock.py
ERROR tests/test_verifier_properties.py
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!! Interrupted: 35 errors during collection
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!
============================================== 2 skipped, 44 deselected, 35 errors in 0.77s
==============================================
""
```

469. assistant (2026-06-10T18:14:40.121Z)

That failure is just the wrong interpreter ? bare `pytest` picked up your global Python 3.13 instead of the project venv (every error is `No module named 'muneem'/'typer'/'sqlcipher3'`, i.e. an environment without the project installed). The correct invocation is through the venv. Running it for you now:

470. user (2026-06-10T18:14:44.294Z)

```
sssssssssssssssss                                                        [100%]
========================== short test summary info ==========================
SKIPPED [7] tests\parsers\test_hdfc.py:232: needs MUNEEM_TEST_HDFC_PW + real statements (never
in CI)
SKIPPED [4] tests\parsers\test_local_au.py:36: needs the AU password (password_rules.yaml or
MUNEEM_TEST_AU_PW)
SKIPPED [7] tests\parsers\test_local_axis.py:34: needs the Axis password (password_rules.yaml or
MUNEEM_TEST_AXIS_PW)
SKIPPED [1] tests\test_unlock.py:63: needs MUNEEM_TEST_HDFC_PW + real statements (never in CI)
19 skipped, 516 deselected in 0.93s
```

471. assistant (2026-06-10T18:14:52.797Z)

Right interpreter now ? but all 19 skip because the tests can't resolve your statement passwords. Let me check exactly where they look:

472. user (2026-06-10T18:14:53.049Z)

```
1       """Local-only checks for the AU Bank parser against real encrypted statements.
2
3       Skipped unless an AU password is available (``password_rules.yaml`` or
``MUNEEM_TEST_AU_PW``); never
4       runs in CI. The parser is **oracle-gated**, so this asserts the safety invariant: it
either stored
5       nothing (a vintage it can't yet read) or everything it stored reconciled ? never
mislabeled rows.
6       Surfaces counts / booleans / parser names only ? no statement content.
7       """
8
9       import os
10      from pathlib import Path
11
12      import pdfplumber
13      import pytest
14
15      from muneem import db as dbmod
16      from muneem.parsers import get_parser
17      from muneem.password_rules import get_password_for_bank
18      from muneem.redaction import redact_filename
19
20      AU_PDFS = sorted(Path("testdata/aubank-statements").glob("*.pdf"))
21
22
23      @pytest.mark.local_statements
24      @pytest.mark.parametrize(
25          "pdf_path",
```

```
26          AU_PDFS,
27          ids=lambda p: redact_filename(p),
28      )
29      def test_local_au_never_stores_unreconciled(pdf_path, tmp_path):
30          pw = (
31              get_password_for_bank("aubank")
32              or get_password_for_bank("au")
33              or os.environ.get("MUNEEM_TEST_AU_PW")
34          )
35          if not pw:
36              pytest.skip("needs the AU password (password_rules.yaml or MUNEEM_TEST_AU_PW)")
37
38          with pdfplumber.open(pdf_path, password=pw) as pdf:
39              text = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(2,
len(pdf.pages))))
40          parser = get_parser(text)
41          if parser is None or type(parser).__name__ != "AUParser":
42              pytest.skip(f"{pdf_path.name}: not routed to AUParser")
43
44          with dbmod.connect(tmp_path / "t.db") as conn:
45              dbmod.init_db(conn)
46              doc_id = dbmod.insert_document(conn, source="file", sha256=pdf_path.name,
status="parsed")
47              n = parser.parse_and_insert(pdf_path, conn, doc_id, password=pw)
48              status = dbmod.get_document_by_sha(conn, pdf_path.name)["status"]
49              stored = dbmod.list_au_transactions(conn)
50
51          assert (n == 0 and stored == []) or status == "reconciled", (
52              f"{pdf_path.name}: stored {n} row(s) with status={status} ? must not persist
unreconciled"
53          )
54
```

473. user (2026-06-10T18:15:04.762Z)

```
  File "<string>", line 1
    from muneem.password_rules import _rules_paths if
hasattr(__import__('muneem.password_rules', fromlist=['x']), '_rules_paths') else None
['Any', 'CONFIG_DIR', 'ConfigError', 'Path', '_load_rules', 'annotations',
'candidate_passwords', 'get_password_for_bank', 'warnings', 'yaml']
```

474. user (2026-06-10T18:15:12.904Z)

```
config dir: C:\Users\avidu\AppData\Local\muneem
exists: False
files: n/a
```

475. user (2026-06-10T18:15:28.311Z)

```
1       """Password rule loader for statement decryption (hybrid strategy support).
2
3       Prefers the user config dir (via platformdirs, e.g. ~/.config/muneem/password_rules.yaml
4       on Linux) for real rules. Falls back to legacy CWD "password_rules.yaml" (common during
5       dev) with a DeprecationWarning. **Never commit real rules or the file itself** ?
`.gitignore`
6       blocks `password_rules.*`.
7
8       See DESIGN.md §8.2, CLAUDE.md security rules, and docs/ingesting-statements.md for the
file
9       format. Entries are either ``<bank>: <password>`` or a top-level ``candidates: [..]``
list.
10      """
11
12      from __future__ import annotations
```

```

    import warnings
    from pathlib import Path
    from typing import Any

    import yaml

    from muneem.config import CONFIG_DIR
    from muneem.errors import ConfigError


    def _load_rules() -> dict[str, Any]:
        """Load the rules dict from the config dir (preferred) or legacy CWD file; ``{}`` if
absent.

        Resolution order:
        1. ``<config_dir>/password_rules.yaml``  (recommended location)
        2. ``./password_rules.yaml``  (legacy, emits DeprecationWarning)
        """
        for path, is_legacy in (
            (CONFIG_DIR / "password_rules.yaml", False),
            (Path("password_rules.yaml"), True),
        ):
            if path.exists():
                if is_legacy:
                    warnings.warn(
                        "Loading password_rules from CWD (./password_rules.yaml) is
deprecated. "
                        f"Move it to {CONFIG_DIR / 'password_rules.yaml'} for future-
proofing. "
                        "See docs and CLAUDE.md (never commit real password material).",
                        DeprecationWarning,
                        stacklevel=3,
                    )
                with path.open(encoding="utf-8") as f:
                    try:
                        data = yaml.safe_load(f)
                    except yaml.YAMLError as exc:
                        # Never echo the file's contents in the error ? they are password
material.
                        raise ConfigError(
                            f"could not parse password rules ({path.name}): invalid YAML"
                        ) from exc
                if data is None:
                    return {}
                if not isinstance(data, dict):
                    raise ConfigError(
                        f"password rules ({path.name}) must be a mapping of <bank>:
<password>"
                    )
                return data
        return {}


    def get_password_for_bank(bank_name: str) -> str | None:
        """Return the password (or derived rule string) for a bank, or ``None``."""
        val = _load_rules().get(bank_name)
        return str(val) if val else None


    def candidate_passwords() -> list[str]:
        """Every configured password to try as an unlock candidate, **bank-agnostic**.

        The ingest path resolves the password *before* it knows the issuer (it needs the
decrypted
```

```
72          text to pick a parser), so it tries all configured passwords. Collects each per-bank
value
73          plus any entries under a top-level ``candidates:`` list; order preserved, de-
duplicated.
74          Quote numeric passwords in the YAML so leading zeros survive.
75          """
76          rules = _load_rules()
77          collected: list[str] = []
78          for key, val in rules.items():
79              if key == "candidates" and isinstance(val, list):
80                  collected.extend(str(v) for v in val if v not in (None, ""))
81              elif isinstance(val, str | int | float):
82                  collected.append(str(val))
83          seen: set[str] = set()
84          unique: list[str] = []
85          for pw in collected:
86              if pw and pw not in seen:
87                  seen.add(pw)
88                  unique.append(pw)
89          return unique
90
```

476. assistant (2026-06-10T18:15:38.256Z)

The mechanism is clear. One more check ? which bank keys each test queries, so the instructions are exact:

477. user (2026-06-10T18:15:39.528Z)

```
tests\parsers\test_hdfc.py:228:    Set MUNEEM_TEST_HDFC_PW; never runs in CI. Surfaces
counts/booleans only.
tests\parsers\test_hdfc.py:230:    pw = os.environ.get("MUNEEM_TEST_HDFC_PW")
tests\parsers\test_hdfc.py:232:        pytest.skip("needs MUNEEM_TEST_HDFC_PW + real statements
(never in CI)")
tests\parsers\test_local_au.py:3:Skipped unless an AU password is available
(``password_rules.yaml`` or ``MUNEEM_TEST_AU_PW``); never
tests\parsers\test_local_au.py:17:from muneem.password_rules import get_password_for_bank
tests\parsers\test_local_au.py:31:        get_password_for_bank("aubank")
tests\parsers\test_local_au.py:32:        or get_password_for_bank("au")
tests\parsers\test_local_au.py:33:        or os.environ.get("MUNEEM_TEST_AU_PW")
tests\parsers\test_local_au.py:36:        pytest.skip("needs the AU password
(password_rules.yaml or MUNEEM_TEST_AU_PW)")
tests\parsers\test_local_axis.py:4:``MUNEEM_TEST_AXIS_PW``); never runs in CI. The savings
parser is **oracle-gated**
tests\parsers\test_local_axis.py:19:from muneem.password_rules import get_password_for_bank
tests\parsers\test_local_axis.py:32:    pw = get_password_for_bank("axis") or
os.environ.get("MUNEEM_TEST_AXIS_PW")
tests\parsers\test_local_axis.py:34:        pytest.skip("needs the Axis password
(password_rules.yaml or MUNEEM_TEST_AXIS_PW)")
tests\test_password_rules.py:39:def test_get_password_for_bank(dirs):
tests\test_password_rules.py:42:    assert pr.get_password_for_bank("hdfc") == "SECRET1"
tests\test_password_rules.py:43:    assert pr.get_password_for_bank("nope") is None
tests\test_password_rules.py:48:    assert pr.get_password_for_bank("x") is None
tests\test_password_rules.py:55:        assert pr.get_password_for_bank("hdfc") == "X"
tests\test_password_rules.py:64:        assert pr.get_password_for_bank("hdfc") == "FROMCONFIG"
tests\test_unlock.py:59:    """Local-only: set MUNEEM_TEST_HDFC_PW and place statements in
testdata/hdfc-statements/."""
tests\test_unlock.py:60:    pw = os.environ.get("MUNEEM_TEST_HDFC_PW")
tests\test_unlock.py:63:        pytest.skip("needs MUNEEM_TEST_HDFC_PW + real statements (never
in CI)")
```

478. user (2026-06-10T18:16:00.111Z)

```
217
218     @pytest.mark.local_statements
219     @pytest.mark.parametrize(
220         "pdf_path",
221         sorted(Path("testdata/hdfc-statements").glob("*.pdf")),
222         ids=lambda p: redact_filename(p),
223     )
224     def test_real_hdfc_statement_validates(pdf_path, tmp_path):
225         """Local-only: parse the real statements and require the parse to agree with
226         HDFC's own SUMMARY (Dr/Cr counts + opening/closing) and reconcile fully.
227
228         Set MUNEEM_TEST_HDFC_PW; never runs in CI. Surfaces counts/booleans only.
229         """
230         pw = os.environ.get("MUNEEM_TEST_HDFC_PW")
231         if not pw:
232             pytest.skip("needs MUNEEM_TEST_HDFC_PW + real statements (never in CI)")
233
234         stmt = hdfc.parse_statement(pdf_path, password=pw)
235         assert stmt.transactions, f"{pdf_path.name}: no transactions parsed"
236         summary = hdfc.parse_summary(pdf_path, password=pw)
237         assert summary is not None, f"{pdf_path.name}: SUMMARY block not found"
```

479. user (2026-06-10T18:16:00.711Z)

```
53          res = unlock.unlock(plain_pdf, dest=tmp_path / "copy.pdf")
54          assert res.ok and res.method == "plaintext" and res.output_path.exists()
55
56
57      @pytest.mark.local_statements
58      def test_real_hdfc_statements_decrypt():
59          """Local-only: set MUNEEM_TEST_HDFC_PW and place statements in testdata/hdfc-
statements/."""
60          pw = os.environ.get("MUNEEM_TEST_HDFC_PW")
61          pdfs = sorted(Path("testdata/hdfc-statements").glob("*.pdf"))
62          if not pw or not pdfs:
63              pytest.skip("needs MUNEEM_TEST_HDFC_PW + real statements (never in CI)")
64          for pdf in pdfs:
65              assert unlock.is_encrypted(pdf)
66              assert unlock.unlock(pdf, candidates=[pw]).ok
67
68
69      def test_plaintext_copy_creates_parent_dirs(plain_pdf, tmp_path):
70          # Parity with write_decrypted: the plaintext branch must also mkdir the destination.
71          dest = tmp_path / "staging" / "deep" / "copy.pdf"
72          res = unlock.unlock(plain_pdf, dest=dest)
```

480. user (2026-06-10T18:16:07.465Z)

```
Switched to a new branch 'test/hdfc-password-rules-parity'
```

481. user (2026-06-10T18:16:11.908Z)

```
The file C:\Users\avidu\Projects\muneem\tests\parsers\test_hdfc.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

482. user (2026-06-10T18:16:16.024Z)

```
The file C:\Users\avidu\Projects\muneem\tests\test_unlock.py has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

483. user (2026-06-10T18:16:30.955Z)

```
.........                                                                [100%]
514 passed, 21 deselected in 4.33s
```

```
SKIPPED [7] tests\parsers\test_hdfc.py:235: needs the HDFC password (password_rules.yaml or
MUNEEM_TEST_HDFC_PW)
SKIPPED [4] tests\parsers\test_local_au.py:36: needs the AU password (password_rules.yaml or
MUNEEM_TEST_AU_PW)
SKIPPED [7] tests\parsers\test_local_axis.py:34: needs the Axis password (password_rules.yaml or
MUNEEM_TEST_AXIS_PW)
SKIPPED [1] tests\test_unlock.py:66: needs the HDFC password (password_rules.yaml or
MUNEEM_TEST_HDFC_PW)
19 skipped, 516 deselected in 0.73s
All checks passed!
81 files already formatted
```

484. user (2026-06-10T18:16:55.233Z)

```
[test/hdfc-password-rules-parity 592c3ae] test: HDFC local tests resolve the password from
password_rules.yaml too
 2 files changed, 12 insertions(+), 6 deletions(-)
 * [new branch]      test/hdfc-password-rules-parity -> test/hdfc-password-rules-
parity
https://github.com/avidullu/muneem/pull/71
From https://github.com/avidullu/muneem
 * branch            main       -> FETCH_HEAD
   ee8a8c2..7b4f3cc  main       -> origin/main
Updating ee8a8c2..7b4f3cc
Fast-forward
 tests/parsers/test_hdfc.py | 9 ++++++---
 tests/test_unlock.py       | 9 ++++++---
 2 files changed, 12 insertions(+), 6 deletions(-)
Your branch is up to date with 'origin/main'.
Already on 'main'
Already up to date.
```

485. user (2026-06-10T18:17:08.881Z)

```
Updated task #21 status
```

486. user (2026-06-10T18:17:22.567Z)

```
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\memory\session-
handoff.md has been updated successfully. (file state is current in your context ? no need to
Read it back)
```

487. assistant (2026-06-10T18:17:43.520Z)

Everything is wrapped up. Here's what happened with your test run and what's now waiting on you:

Why your run failed, and how to run it

Bare `pytest` picked up your **global Python 3.13**, which doesn't have the project installed (every error was `No module named 'muneem'`). Through the venv it collects cleanly ? but then all 19 tests **skip**, and I found the real reason: statement passwords are configured **nowhere** on this machine. `C:\Users\avidu\AppData\Local\muneem\` doesn't exist, and the `MUNEEM_TEST_*_PW` env vars aren't set. Your past ingests worked via the interactive prompt + keychain cache, which the test suite deliberately never touches.

I also found and fixed an inconsistency while setting this up ([#71](https://github.com/avidullu/muneem/pull/71), merged): the HDFC tests read only the env var and ignored `password_rules.yaml`, so the designed persistent mechanism couldn't drive a full run. Now all four local tests resolve identically.

To run it (passwords stay with you ? don't paste them to me):

Option A ? persistent (also feeds `muneem ingest`; git guards block this file): create `C:\Users\avidu\AppData\Local\muneem\password_rules.yaml`:

```yaml
hdfc: "<hdfc pdf password>"
axis: "<axis pdf password>"
aubank: "<au pdf password>"
```
Option B ? one session, nothing persisted:
```powershell
$env:MUNEEM_TEST_HDFC_PW = "..."; $env:MUNEEM_TEST_AXIS_PW = "..."; $env:MUNEEM_TEST_AU_PW =
"..."
```
Then either way:
```powershell
.\.venv\Scripts\python.exe -m pytest -m local_statements -q
```
Expected: **19 passed** (the Axis May-2025 test passes by asserting the quarantine invariant ? it stores nothing). Any *failure* is exactly the signal we wanted: a real vintage tripping the stricter oracle/routing ? report it and I'll fix the summary regex or routing evidence, never weaken the oracle.

Sister repo: both PRs ready for your review

The agent finished: [#15 `fix/sync-data-loss`](https://github.com/avidullu/gmail-attachments-to-gdrive/pull/15) (dirty backfill windows no longer advance `lastSynced`; incremental cursor held on dirty runs; the deleted-message wedge in `getMessageById` fixed; spam/trash parity; a consecutive-dirty-pass counter for visibility; +10 tests) and [#16 `fix/operational-defaults`](https://github.com/avidullu/gmail-attachments-to-gdrive/pull/16) (root-folder probe scoped, `Asia/Kolkata` timezone, summary-recipient extraction; +5 tests). Both green on tests/typecheck/lint/build. Two findings were honestly downgraded in the PR bodies ? the multi-user summary-mail bug was already half-fixed at HEAD, and cap-truncated windows already held the cursor. **Left unmerged on purpose: merging master auto-deploys your live archiver**, so the merge click is yours.

That closes every task: **21 muneem PRs merged today** (#50?#71 minus the two sister PRs), the handoff and CURRENT_STATE §5 carry the decision queue for your fresh look, and the only open loops are yours by design ? the password-backed verification run, the two sister-repo merges, and the strategy decisions (Gmail seam, C?D order, wedge).

488. user (2026-06-10T18:24:53.415Z)

19 pass it is!

489. assistant (2026-06-10T18:25:05.929Z)

Excellent ? that closes the last open risk from the entire hardening pass. The stricter oracle (bookend anchoring, signed Cr/Dr, half-paisa tolerance), the letterhead routing, and ISO dates all hold on your real HDFC/Axis/AU vintages, with May-2025 still correctly quarantining. Let me clear the pending-verification flags from the docs:

490. user (2026-06-10T18:25:10.242Z)

The file C:\Users\avidu\Projects\muneem\docs\CURRENT_STATE.md has been updated successfully. (file state is current in your context ? no need to Read it back)

491. user (2026-06-10T18:25:36.903Z)

```
Switched to a new branch 'docs/local-verification-confirmed'
[docs/local-verification-confirmed 91c7bab] docs: real-data verification confirmed -- 19/19
local_statements pass
 1 file changed, 3 insertions(+), 4 deletions(-)
 * [new branch]      docs/local-verification-confirmed -> docs/local-verification-
confirmed
https://github.com/avidullu/muneem/pull/72
From https://github.com/avidullu/muneem
 * branch            main       -> FETCH_HEAD
```

```
   7b4f3cc..7c00f6b  main       -> origin/main
Updating 7b4f3cc..7c00f6b
Fast-forward
 docs/CURRENT_STATE.md | 7 +++----
 1 file changed, 3 insertions(+), 4 deletions(-)
Your branch is up to date with 'origin/main'.
Already on 'main'
Already up to date.
```

492. user (2026-06-10T18:25:51.360Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\memory\session-handoff.md has been updated successfully. (file state is current in your context ? no need to Read it back)

493. user (2026-06-10T18:26:03.074Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\memory\session-handoff.md has been updated successfully. (file state is current in your context ? no need to Read it back)

494. assistant (2026-06-10T18:26:16.747Z)

That's the full loop closed. The 19/19 pass means every behavioral change from the hardening sprint is now validated against your actual statements ? including the two scariest ones, where a plausible-but-wrong parse used to slip through: the bookend anchoring (a statement missing edge rows can no longer pass) and the signed Cr/Dr balances (an overdraft can no longer reconcile inverted). And May-2025 still quarantining is itself a pass ? the oracle keeps refusing to store the incomplete ledger, which is exactly the contract.

Recorded in [CURRENT_STATE.md](docs/CURRENT_STATE.md) via [#72](https://github.com/avidullu/muneem/pull/72) and the session handoff ? **nothing is pending on muneem anymore**. The scoreboard for the day: the deep review, then 22 merged PRs (#48?#49 from the other session, #50?#72 here), 514 offline tests + 19 real-data tests green, and the ledger now survives every failure mode the review found.

What's left is all yours, in rough order of leverage:

1. **The two sister-repo PRs** ([#15](https://github.com/avidullu/gmail-attachments-to-gdrive/pull/15), [#16](https://github.com/avidullu/gmail-attachments-to-gdrive/pull/16)) ? review and merge when you're ready to deploy; the archiver keeps running with the known data-loss edges until then.
2. **The decision queue** in [CURRENT_STATE §5](docs/CURRENT_STATE.md): the cross-repo Gmail-seam ADR (unblocks `muneem fetch`), the C?D phase order, and the wedge question ? finance is proven and humming, so those three choices are what determine where the next month of effort goes.

495. user (2026-06-10T18:27:58.865Z)

BTW, do you not think that if we try to train an ML model to "visually" process the entire pdf and then semantically understand the different breakdowns (eg Fixed Deposits, Current or Recurring deposits, other accounts, Mutual Fund statements, Credit Card Statements etc.) then the possibilities would be more high fidelity? Similar to vertex AI where I just load a bunch of bank statements and it "tries" to make a model for it. Can't we find an appropriate base model and train it for muneem specific tasks?

496. assistant (2026-06-10T18:29:22.097Z)

Your intuition is half right, and the half that's right is worth acting on ? but the framing needs one inversion, and "Vertex AI" specifically is the one tool muneem can never use.

Where you're right. The project actually agrees with you more than you might remember: the L3 fallback decision *is* the "visually process the PDF" play (DeepSeek-OCR-3B, chosen precisely because vision models read layouts the way humans do), and DEPENDENCIES.md rule 4 explicitly

says we're willing to train our own models. And your **section/semantic-breakdown point is the strongest version of the idea**: classification and segmentation ("this page is an FD summary, this block is the MF transaction section, this is a CC bill") is exactly where ML beats hand-rolled code, because it's a *forgiving* task ? a wrong classification gets caught downstream, unlike a wrong amount. Today's routing is letterhead-substring heuristics; a vision classifier would be strictly better. This matters most for **Phase C**: bank statements are one table with a balance column, but instrument documents (CAS, broker statements, MF CAS with per-folio sections) are heterogeneous, sectioned beasts ? replicating the per-issuer-profile approach there would scale terribly, and that's where I'd genuinely revise the plan toward your instinct: VLM-first extraction for instruments, rather than hand-rolled profiles.

The inversion: fidelity doesn't come from the model. This week proved the point empirically ? your 19/19 run. muneem's trust comes from "extraction proposes, reconciliation disposes": the balance walk + bookends are what make a parse *high fidelity*, regardless of whether a clusterer or a neural net proposed the rows. A trained model improves **coverage** (how many layouts parse at all) and **maintenance cost** (no profile-tuning per issuer redesign), not trust. That's why the L3 design is reconcile-gated, and why a model's output for sections with *no* arithmetic identity (FD interest summaries, reward points) would land in the new `unverified` status ? honest, but no more "high fidelity" than the model's whims. The oracle is the product; models are interchangeable proposal engines in front of it.

Why not Vertex. Vertex Document AI custom extractors mean uploading your entire financial life to Google's training pipeline ? it violates the project's first commandment (DESIGN §5: no cloud processing of sensitive categories), the same reason the Account Aggregator TSPs were rejected. "Like Vertex, but local" is the only admissible form of this idea.

The practical reality check, in order of cheapness:

1. **You probably don't need training at all.** The 2023-era "upload statements, it learns a model" workflow exists because OCR models then were weak. A 2026 open VLM (DeepSeek-OCR-3B, Qwen2.5-VL) **zero-shot prompted** with a target JSON schema, gated by your oracle, is the modern equivalent ? no training data, no training run, MIT/Apache licensed, runs 4-bit on your 2070S. This is exactly what the parked L3 build is.
2. **Training data is your real bottleneck, not the base model.** You own ~17 real statements ? that's an *eval set*, not a training set. Fine-tuning needs hundreds-to-thousands of labeled pages. The policy-compatible path is growing `synthpdf` into a layout-randomizing generator (it just learned multi-page and ruled tables in #70) ? synthetic statements can even go to a rented GPU; real ones can never leave the machine, which also rules out fine-tuning on real data anywhere but the 2070S (where QLoRA on a 3B VLM is possible but slow and miserable).
3. **The sequencing decision already anticipated this debate and its logic got stronger this week.** D.3-before-L3 says: characterize where the deterministic ladder actually fails *before* building its replacement. Current evidence: 16/17 vintages parse deterministically and free; the one failure is a single dropped line. Training a model today would be solving a problem we can't yet name. But run the same D.3 eval against *instrument* documents and I'd expect the opposite verdict ? which is your point.

So my concrete recommendation: keep the bank ladder (proven, free, fast); build the D.3 eval harness; baseline **zero-shot** DeepSeek-OCR/Qwen-VL on it; adopt the vision model first as the **document/section classifier** you described (cheap, forgiving, immediately useful for routing FD/RD/MF/CC sections); make **VLM-first-with-oracle-gating the default plan for Phase C instruments**; and treat fine-tuning as the last resort, triggered only when zero-shot demonstrably fails on the eval ? at which point synthetic-corpus QLoRA is the route, and "which GPU" becomes a real question to bring back to you.

Want me to record this as the resolution sketch for DESIGN §8.3's remaining half (it sharpens "which model, when" into "zero-shot classifier ? Phase-C extractor ? fine-tune-on-evidence"), or leave it as discussion until you've sat with it?

497. user (2026-06-10T18:30:51.950Z)

yes please record this!

498. assistant (2026-06-10T18:31:34.393Z)

Recording it across the decision surfaces. First, the exact current text of each spot:

499. user (2026-06-10T18:31:36.015Z)

```
docs\parser-architecture.md-207-  *Provisional (Step 2): generic detectors run first; the
profile supplies aliases / column-order /
C:\Users\avidu\Projects\muneem\docs\parser-architecture.md-208-  anchors / account patterns as
priors. Revisit once more issuers land.*
docs\parser-architecture.md:209:- **L3 scope** ? *model resolved (DeepSeek-OCR-3B; §4).* Still
open: rare fallback vs routine
C:\Users\avidu\Projects\muneem\docs\parser-architecture.md-210-  cross-check on every statement
(lean **rare / reconcile-fail-only**, given the 2070S budget).
C:\Users\avidu\Projects\muneem\docs\parser-architecture.md-211-- **Tier-2** ? when do
transaction classification + merchant canonicalization become in-scope?
--
docs\ROADMAP.md-320-Carried from DESIGN.md § 8, not yet resolved ? surface and ask when a
milestone forces them:
C:\Users\avidu\Projects\muneem\docs\ROADMAP.md-321-
docs\ROADMAP.md:322:[Omitted long matching line]
C:\Users\avidu\Projects\muneem\docs\ROADMAP.md-323-- **§ 8.5 ? Merchant canonicalization.**
Deferred until real receipt data exposes the mess. Forced when `items`/`merchants` land.
docs\ROADMAP.md-324-- ~~**Encryption-at-rest timing** (D.2) ? SQLCipher now vs. later.~~
**Resolved 2026-05:** SQLCipher adopted now (`sqlcipher3` + keychain key), forced by storing
real numbers; research-backed. See [`docs/encryption-at-rest.md`](./encryption-at-rest.md).
```

500. user (2026-06-10T18:31:36.730Z)

```
<system-reminder>This memory is 10 days old. Memories are point-in-time observations, not live
state ? claims about code behavior or file:line citations may be outdated. Verify against
current code before asserting as fact.</system-reminder>
1       ---
2       name: l3-vision-and-eval-strategy
3       description: L3 local-vision-model choice (DeepSeek-OCR-3B) + the dev GPU constraints +
the golden-eval-set policy for muneem statement parsing
4       metadata:
5         node_type: memory
6         type: project
7         originSessionId: a9fd4426-f234-4e4a-bfb4-1f2c8255c661
8       ---
9
10      Resolved 2026-06 from a deep-research run + a check of the dev machine's GPU. Captures
the **L3
11      vision fallback** model choice and the **golden-eval-set** policy. Detail lives in
12      `docs/parser-architecture.md` (§4, §7, §10a) and `docs/ROADMAP.md` (§ Regression CI).
13
14      **L3 = a *verified fallback*, never the primary, never runtime-cloud.** It fires only
when the
15      deterministic ladder (L0?L2) fails to reconcile; its output is accepted **only if it
reconciles**
16      (self-consistency re-sample on a miss, then human review). Reconciliation is what makes
a
17      probabilistic extractor safe on money ? extraction proposes, reconciliation disposes.
18
19      **Model picks (commercial-safe CODE *and* WEIGHTS):**
20      - **DeepSeek-OCR-3B ? MIT (code + weights), the lead.** Cleanest license, OCR/table-
specialized,
21        ~7 GB BF16 / ~4 GB 4-bit.
22      - **Qwen2.5-VL-7B ? Apache-2.0**, heavier fallback / fine-tune base. ?? **only 7B & 32B
are Apache;
23        the 3B and 72B are research-licensed (non-commercial)** ? pin the 7B.
24      - **PaddleOCR-VL (~1B) ? Apache**, 8 GB-friendly co-pilot for cheap re-sampling.
25      - **Avoid:** LayoutLMv3 (weights CC-BY-NC), GOT-OCR2.0 (research-only), Surya/Marker
(GPL / Open-RAIL-M),
26        MinerU (rides an AGPL dependency ? needs audit), Llama-3.2-Vision (custom community
license).
27
28      **Dev GPU = NVIDIA RTX 2070 Super Max-Q: 8 GB, Turing (sm_75).** Two Turing constraints
```

```
beyond VRAM:
29      **no native bf16** (load fp16 or 4-bit) and **no FlashAttention-2** (use
`attn_implementation="sdpa"`).
30      bitsandbytes 4-bit (nf4) works. ? On this machine the local L3 is **DeepSeek-OCR-3B in
4-bit + SDPA**
31      (PaddleOCR-VL as the tiny co-pilot). **Qwen-7B is NOT a local fit here** (4-bit ~6 GB +
visual tokens
32      OOM in 8 GB) and **fine-tuning a 7B VLM is not an 8 GB job** ? that's a bigger-GPU /
approved-cloud
33      task. It also has an Intel UHD iGPU, so routing the desktop/browser to the iGPU frees
most of the 8 GB.
34
35      **Golden eval sets ? local + synthetic first; cloud documented but gated** (user's
decision):
36      - **Local-golden:** a parse that reconciles + bookends is already a gold label (HDFC
4/4, AU 4/4 today).
37      - **Synthetic-golden:** statements in real issuer layouts with fabricated data ?
committable to
38        `testdata/synthetic/`, CI-able, safe for any model.
39      - **Cloud-golden:** eval-only, **approval-gated per category**, zero-retention provider
(Reducto on-prem /
40        Azure 24h-expiry / Textract no-retention), used as **one voter** cross-checked by
reconciliation ?
41        **never** in the runtime path. No real statement data leaves the machine without
explicit in-session OK.
42      This dovetails with the **Regression CI** plan: synthetic-golden = the cloud-CI layer;
local-golden =
43      the self-hosted real-statement layer. Statements are never committed (git history is
permanent).
44
```

501. assistant (2026-06-10T18:31:47.524Z)

All spots located. Recording the decision ? DESIGN §8.3 first:

502. user (2026-06-10T18:32:03.084Z)

The file C:\Users\avidu\Projects\muneem\DESIGN.md has been updated successfully. (file state is current in your context ? no need to Read it back)

503. user (2026-06-10T18:32:12.410Z)

Switched to a new branch 'docs/ml-adoption-ladder'

504. user (2026-06-10T18:32:12.424Z)

```
318     ## 9. Still-open decisions (do NOT pick silently)
319
320     Carried from DESIGN.md § 8, not yet resolved ? surface and ask when a milestone forces
them:
321
322     - **§ 8.3 ? LLM model for sensitive categories.** *Partially resolved (2026-06):* the
**model is chosen** ? **DeepSeek-OCR-3B (MIT)**, with Qwen2.5-VL-7B (Apache-2.0) as the heavier
fallback, reconcile-gated, 4-bit + SDPA on the dev RTX 2070S (see [`parser-
architecture.md`](./parser-architecture.md) § 4). The **locality rule is unchanged** (local-only
for sensitive categories) and the **build is deferred** until the D.3 eval writeup characterizes
where the deterministic ladder actually fails (feedbackJune2026 P4).
323     - **§ 8.5 ? Merchant canonicalization.** Deferred until real receipt data exposes the
mess. Forced when `items`/`merchants` land.
324     - ~~**Encryption-at-rest timing** (D.2) ? SQLCipher now vs. later.~~ **Resolved
2026-05:** SQLCipher adopted now (`sqlcipher3` + keychain key), forced by storing real numbers;
research-backed. See [`docs/encryption-at-rest.md`](./encryption-at-rest.md).
325     - ~~**Ingestion source ? Open Banking (Account Aggregator) vs PDF parsing.**~~
**Resolved 2026-06:** **PDF-first; AA deferred** as an opt-in complement only (regulatory FIU
```

```
gate + paid cloud-decrypting TSP make it non-viable and a net privacy regression for a single-
user local tool; re-adopt triggers tracked). Research-backed ? see [`docs/research-open-banking-
india.md`](./research-open-banking-india.md) and DESIGN § 8.6. Surfaced a new local-only
candidate: **SMS/email transaction-alert parsing** (freshness complement to PDF, zero third
party).
326     - ~~**Storage shape.**~~ **Resolved 2026-06:** **flat per-issuer source-of-truth tables
+ a derived serving layer; provenance/confidence in a separate `provenance` table** (DESIGN §
8.7, feedbackJune2026 P3). First forward migration adds `provenance`.
327     - **Promo-track LLM** (A.3) ? only reached if deterministic extraction underperforms;
cloud is policy-permitted for promo but still needs a quick okay on which vendor.
```

505. user (2026-06-10T18:32:26.626Z)

```
The file C:\Users\avidu\Projects\muneem\docs\ROADMAP.md has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

506. assistant (2026-06-10T18:32:37.040Z)

Now the ROADMAP §0 phase rows, parser-architecture §11, the memory file, and the handoff:

507. user (2026-06-10T18:32:44.114Z)

```
The file C:\Users\avidu\Projects\muneem\docs\ROADMAP.md has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

508. user (2026-06-10T18:32:47.951Z)

```
The file C:\Users\avidu\Projects\muneem\docs\ROADMAP.md has been updated successfully. (file
state is current in your context ? no need to Read it back)
```

509. user (2026-06-10T18:32:51.712Z)

```
The file C:\Users\avidu\Projects\muneem\docs\parser-architecture.md has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

510. user (2026-06-10T18:33:11.231Z)

```
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\memory\l3-vision-and-
eval-strategy.md has been updated successfully. (file state is current in your context ? no need
to Read it back)
```

511. user (2026-06-10T18:33:43.238Z)

```
514 passed, 21 deselected in 4.53s
[docs/ml-adoption-ladder 60aa8b4] docs: record the ML adoption ladder -- DESIGN 8.3 resolved
direction
 3 files changed, 41 insertions(+), 12 deletions(-)
 * [new branch]      docs/ml-adoption-ladder -> docs/ml-adoption-ladder
https://github.com/avidullu/muneem/pull/73
From https://github.com/avidullu/muneem
 * branch            main       -> FETCH_HEAD
   7c00f6b..2dbec9c  main       -> origin/main
Updating 7c00f6b..2dbec9c
Fast-forward
 DESIGN.md                   | 39 +++++++++++++++++++++++++++++++++++++-------
 docs/ROADMAP.md             |  6 +++---
 docs/parser-architecture.md |  8 ++++++--
 3 files changed, 41 insertions(+), 12 deletions(-)
Your branch is up to date with 'origin/main'.
Already on 'main'
Already up to date.
```

512. assistant (2026-06-10T18:33:54.876Z)

Recorded and merged as [#73](https://github.com/avidullu/muneem/pull/73). One handoff touch to keep memory in sync:

513. user (2026-06-10T18:34:01.725Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\memory\session-handoff.md has been updated successfully. (file state is current in your context ? no need to Read it back)

514. assistant (2026-06-10T18:34:16.480Z)

Recorded everywhere it needs to live, merged as [#73](https://github.com/avidullu/muneem/pull/73):

- **[DESIGN.md §8.3](DESIGN.md)** now carries the full resolved direction: the oracle-not-the-model fidelity principle, the four-rung adoption ladder (zero-shot always first ? section classifier as the first ML surface ? Phase C instruments VLM-first ? fine-tune only on D.3 evidence), the hard constraints (Vertex-style cloud training categorically out; off-machine training on synthetic corpora only; your ~17 real docs are the eval set, never a training set; GPU procurement asks first).
- **[ROADMAP §9](docs/ROADMAP.md)** decision log marks §8.3 resolved-direction, with the honest remainder named: the fine-tune trigger threshold and GPU choice stay open, forced only if zero-shot fails the eval.
- **ROADMAP §0** Phase C row now says instruments are VLM-first by default, and Phase E notes the classifier may land ahead of the full extractor fallback.
- **[parser-architecture §11](docs/parser-architecture.md)** closes its "L3 scope" open question, and the `l3-vision-and-eval-strategy` memory is updated so future sessions recall the ladder without re-deriving it.

The decision's practical effect on sequencing: it makes **Phase D (the D.3 eval harness) even more load-bearing** ? it's now the gate for three things at once (L3 build, the classifier's proof, and any fine-tune trigger), which quietly strengthens the case for running D before C when you settle that ordering question. Suite still green at 514 after the doc changes.

515. user (2026-06-10T18:37:00.786Z)

Check the two background agents (D1 synthetic-corpus worktree agent; sister-repo fixes agent). If complete, review their PRs (merge the muneem D1 PR after review; leave the gmail-repo PRs unmerged for Avi), update the task list and session handoff, and report the final wrap-up.

516. assistant (2026-06-10T18:37:14.207Z)

That wakeup is stale ? both agents completed and were handled earlier:

- **D1 (muneem):** [#70](https://github.com/avidullu/muneem/pull/70) was reviewed (full diff read, CI matrix + local Windows run verified) and **merged**; the agent worktree was cleaned up. All five fixtures shipped, including the mutation-verified dropped-row-quarantine test.
- **Sister repo:** [#15](https://github.com/avidullu/gmail-attachments-to-gdrive/pull/15) (sync data-loss fixes) and [#16](https://github.com/avidullu/gmail-attachments-to-gdrive/pull/16) (operational defaults) are open and **deliberately unmerged** ? they await your review, since merging master auto-deploys the live archiver.
- Task list: all 21 items completed. Handoff: current through [#73](https://github.com/avidullu/muneem/pull/73) (the ML adoption ladder you asked to record), including the 19/19 real-data verification.

Final state: muneem `main` @ `2dbec9c`, 24 PRs merged across the session (#48?#73), 514 offline + 19 real-data tests green. Open items are all yours: the two sister-repo merges, and the strategy decisions queued in [CURRENT_STATE §5](docs/CURRENT_STATE.md) (Gmail-seam ADR, C?D order, wedge). Nothing pending on my side.

517. user (2026-06-10T18:38:09.485Z)

I may have to leave and restart a new session for this. So just checking if session handoffs and code maps etc. are all up to date?

518. assistant (2026-06-10T18:38:22.506Z)

Let me do a final verification sweep of all the resumption surfaces before you go:

519. user (2026-06-10T18:38:23.452Z)

```
1       ---
2       name: session-handoff
3       description: "? Start here ? living handoff for muneem (and its sister repo gmail-
attachments-to-gdrive); current state, next steps, ramp-up kit, key decisions."
4       metadata:
5         node_type: memory
6         type: project
7         originSessionId: 02b9e9c9-3806-45ca-9b24-2ecc35a81efd
8       ---
9
10      # Session handoff ? muneem (+ gmail-attachments-to-gdrive)
11
12      > Last refreshed: 2026-06-10, "civilization plan" essentially complete ? 19 PRs merged
(#50?#68).
13
14      ## 1. You are here
15      - muneem `main` @ **b548fb6** ? local == origin, full gate green (509 tests, cov
~86?87%, mypy/ruff clean, deps pinned). Sister repo `gmail-attachments-to-gdrive` master @
a764030 (agent PRs pending ? see §2). **#69** refreshed CURRENT_STATE.md: post-hardening status
+ §5 rewritten as Avi's decision-oriented next-steps queue (do-now / decide-next / then-build) ?
that doc is now the fresh-look entry point alongside ROADMAP §0.
16      - **The post-review "civilization plan" is done on muneem** (one PR per item, gate
green, squash-merged):
17        - **A ? ledger safety:** #50 rotate-key pending-key protocol + self-healing open · #51
backup destination guard + atomic temp-rename · #52 keychain-failure?absence + never-mint-over-
existing-ledger.
18        - **B ? oracle honesty:** #53 bookend-anchored trust oracle · #54 Cr/Dr is a SIGN on
balances · #55 HDFC quarantine parity + status-tagged listings + `+Cr` fix · #56
`BALANCE_TOLERANCE=0.005` · #57 letterhead-anchored routing · #58 status-aware dedup · #59 Axis
CC `Confidence.UNVERIFIED` floor · #60 ISO date normalization.
19        - **C ? hygiene/supply chain:** #61 snapshot-footgun removal + client_secret guards ·
#62 constraints.txt pinning (CI-verified on the full matrix) · #63 atomic migrations + frozen-
checksum re-verify · #64 small-hardening bundle (key-hex validation, ATTACH quoting,
PAN/card/Aadhaar redaction, logger-discipline test, unlock mkdir, recovery KDF guard).
20        - **D/E ? tests + docs:** #65 weekly OCR cron + redacted pytest ids · #66
docs/recovery.md runbook · #67 stale-docs refresh (parser-architecture precision/§9 superseded,
L3 back-recorded in DESIGN §8.3 + ROADMAP, synthetic-fixture wording, README) · #68 ROADMAP §0 =
single sequencing owner (Phases A?H statuses; C?D swap + missing categories recorded as
OPEN/untriaged).
21      - Earlier same day: #48 (antigravity response + Apps Script deprecation trigger), #49
(schema v6 serving view + `txns list all`); the deep review itself: [[deep-review-2026-06]].
22      - **D1 landed:** #70 (synthetic corpus: Axis savings + ruled Axis-CC through
`extract_tables`, multi-page HDFC, dropped-row-must-quarantine ? mutation-verified against the
bookend anchoring ? and an encrypted PDF through the real CLI unlock). **#71**: HDFC local tests
now resolve passwords from `password_rules.yaml` like AU/Axis (was env-var-only). muneem `main`
@ **7b4f3cc**, 514 tests green.
23      - **Sister repo PRs READY FOR AVI'S REVIEW (unmerged ? merging master auto-deploys the
live archiver):** [#15](https://github.com/avidullu/gmail-attachments-to-gdrive/pull/15)
`fix/sync-data-loss` (dirty backfill windows hold `lastSynced`; incremental historyId held on
dirty runs; `getMessageById` deleted-message wedge fixed; spam/trash parity; consecutive-dirty-
pass counter; +10 tests) and [#16](https://github.com/avidullu/gmail-attachments-to-
gdrive/pull/16) `fix/operational-defaults` (root-folder probe scoped, Asia/Kolkata timezone,
summary-recipient extraction; +5 tests). Agent honestly adjusted two findings: the F3c multi-
user-mail bug was already half-fixed at HEAD, and cap-truncated windows already held the cursor
? both noted in the PR bodies.
24      - **? Real-data verification DONE (2026-06-10): Avi ran `pytest -m local_statements` ?
19/19 pass** under the hardened oracle (HDFC 6/6, AU 4/4, Axis 6/7 with May-25 correctly
quarantining). Recorded in CURRENT_STATE via #72. The whole #50?#68 pass is now confirmed on
real vintages ? nothing pending on muneem.
```

```
25      - **? DESIGN §8.3 resolved-direction recorded (#73, Avi 2026-06-11): the ML adoption
ladder** ? zero-shot before training always (D.3 baselines it) ? first ML surface =
document/section classifier ? **Phase C instruments VLM-first** (oracle-gated) ? fine-tune only
on D.3 evidence (QLoRA; off-machine training on synthetic only; GPU asks first). Vertex-style
cloud training categorically out; banks keep the deterministic ladder primary. See [[l3-vision-
and-eval-strategy]] (updated). Still open within it: fine-tune trigger + GPU choice.
26      - PAT note: pushes 403 ? check the fine-grained PAT. Unreviewed remote branch from
another session: `origin/docs/proposal-opendataloader-pdf-eval`.
27
28      ## 2. Next steps / open threads
29      1. ~~Manual verification~~ **DONE: 19/19 local_statements pass on real vintages (see
§1).**
30      2. Review + merge the two sister-repo PRs ([#15](https://github.com/avidullu/gmail-
attachments-to-gdrive/pull/15) sync data-loss, [#16](https://github.com/avidullu/gmail-
attachments-to-gdrive/pull/16) operational defaults) ? **remember merge = auto-deploy of the
live archiver.** (muneem D1 already merged as #70.)
31      3. Avi reacting to the review wedge question (recommendation: finance stays first
personally; grocery as 2nd/front-door corpus; tax-season pack as the public wedge candidate).
32      4. Cross-repo ADR: who reads Gmail ? blocks Phase F (`muneem fetch`); ROADMAP §0 records
the block.
33      5. Open decisions, untouched by design: log format; self-hosted runner; macOS; C?D phase
swap; instrument-family artifact choice; the untriaged categories in ROADMAP §0.
34
35      ## 3. Ramp-up kit
36      - [[deep-review-2026-06]] ? full review findings (memory dir).
37      - muneem: `CLAUDE.md` ? `DESIGN.md` ? `docs/feedbackJune2026.md` (governing phase plan)
? `docs/CURRENT_STATE.md` (refreshed in #49).
38      - Sister: `docs/VISION.md` ? `docs/ARCHITECTURE.md` (ADR-008 invariant) ? `ROADMAP.md`.
39      - Memory index `MEMORY.md` for standing decisions.
40
41      ## 4. Key decisions (don't relitigate)
42      - Python-only; PDF-first/AA-deferred; flat per-issuer tables + separate provenance table
+ serving view (v6); hand-rolled migrations (no Alembic); SQLCipher + keychain + passphrase
recovery; D.3 eval before L3; licenses MIT/BSD/Apache only (PyMuPDF banned); no cloud LLMs on
sensitive docs; sister repo: <100-user Testing mode, CASA deferred; Apps Script archiver
deprecates when `muneem fetch` ships (#48).
43      - New status vocabulary (this session): documents.status ? reconciled / unreconciled
(quarantined, stores nothing) / **unverified** (family has no oracle by design, e.g. CC ?
persisted but tagged); listings tag anything ? reconciled.
44      - Still open (verified in code): log format; self-hosted runner; macOS stance; cross-
repo Gmail seam ADR; instrument-family artifact choice.
45
```

520. user (2026-06-10T18:38:26.169Z)

```
2dbec9c docs: record the ML adoption ladder -- DESIGN 8.3 resolved direction (#73)
7c00f6b docs: real-data verification confirmed -- 19/19 local_statements pass (#72)
7b4f3cc test: HDFC local tests resolve the password from password_rules.yaml too (#71)
branches:
* main
  worktree-agent-a0a70ddb1d7197b83
```

521. user (2026-06-10T18:38:47.275Z)

```
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\memory\session-
handoff.md has been updated successfully. (file state is current in your context ? no need to
Read it back)
```

522. user (2026-06-10T18:38:52.904Z)

```
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\memory\session-
handoff.md has been updated successfully. (file state is current in your context ? no need to
Read it back)
```

523. user (2026-06-10T18:38:54.575Z)

```
Deleted branch worktree-agent-a0a70ddb1d7197b83 (was 87273c5).
* main
```

524. user (2026-06-10T18:39:10.817Z)

```
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\memory\MEMORY.md has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

525. assistant (2026-06-10T18:39:08.457Z)

One last index touch ? the memory hook for the L3 strategy should mention the new ladder:

526. assistant (2026-06-10T18:39:26.345Z)

All up to date ? you can restart cleanly. Final state of every resumption surface:

Memory (loads automatically in a new session):
- `MEMORY.md` index ? "? Start here" points at the handoff; the L3 hook now carries the ML adoption ladder.
- `session-handoff.md` ? refreshed to right now: main @ `2dbec9c` (#73), the full #48?#73 arc summarized, the 19/19 verification, the §8.3 ladder decision, the two pending sister-repo PRs, and §2's open threads (all yours: sister merges, wedge, Gmail-seam ADR, C?D order, the cheap standing decisions).
- `deep-review-2026-06.md` ? the full findings artifact, unchanged and linked.

Repo docs (the in-repo map):
- [CURRENT_STATE.md](docs/CURRENT_STATE.md) ? capabilities + §5 decision queue, current through #72.
- [ROADMAP §0](docs/ROADMAP.md) ? the single sequencing owner, phase statuses current.
- [DESIGN §8.3](DESIGN.md) ? the ML ladder recorded as of yesterday's discussion.

Working state: both repos clean and pushed (muneem `main` = origin @ `2dbec9c`; sister `master` @ `a764030`), no local branches left (stale worktree branch deleted), no uncommitted anything, gate green (514 offline + 19 real-data tests).

A fresh session reading the handoff's ramp-up kit ? CURRENT_STATE §5 ? ROADMAP §0 ? the deep-review memory ? will be fully current in three documents. Safe to close.

527. user (2026-06-10T18:40:39.636Z)

Gracias Amigo! And next session I will have an explicit "NEXT_STEPS" doc created so even I can ramp up with a single doc!

528. assistant (2026-06-10T18:40:54.610Z)

¡De nada! One last line in the handoff so the next session knows to expect that:

529. user (2026-06-10T18:40:58.357Z)

```
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\memory\session-
handoff.md has been updated successfully. (file state is current in your context ? no need to
Read it back)
```

530. assistant (2026-06-10T18:41:11.024Z)

Recorded as item 0 in the handoff ? the next session will see it first thing and can draft `NEXT_STEPS.md` in a minute, seeded from CURRENT_STATE §5 and ROADMAP §0, one page, pointing at detail rather than duplicating it.

It was a good day's work: one deep review, 24 merged PRs, a ledger that can no longer lose its key, an oracle that can no longer be fooled by edge-dropped rows or inverted overdrafts, and your real statements confirming all of it 19/19. The decisions waiting for you are queued with

their context, and the two sister-repo PRs are sitting safely unmerged until you say deploy.

¡Hasta la próxima sesión! ?